

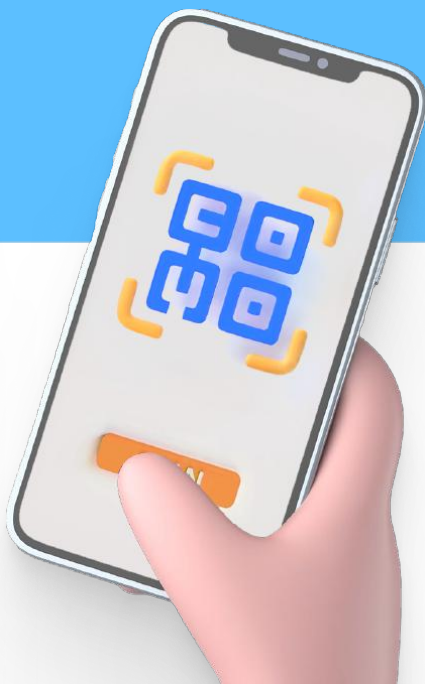


СКЛАДЧИК

КУПЛЕНО НА

SKLADCHIK.ORG

Все самые свежие
курсы по лучшей цене!



O'REILLY®

Apache Iceberg

Полное руководство

Функциональность, производительность
и масштабируемость хранилищ
в озерах данных



Т. Ширан, Дж. Хьюз, А. Мерсед



Томер Ширан, Джейсон Хьюз и Алекс Мерсед

Apache Iceberg. Полное руководство

Tomer Shiran, Jason Hughes, and Alex Merced

Apache Iceberg: The Definitive Guide

**Data Lakehouse Functionality, Performance,
and Scalability on the Data Lake**

Forewords by Gerrit Kazmaier, Raghu Ramakrishnan, and Rick Sears

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Томер Ширан, Джейсон Хьюз и Алекс Мерсед

Apache Iceberg. Полное руководство

**Функциональность, производительность
и масштабируемость хранилищ в озерах данных**

Предисловия Геррита Казмайера, Рагху Рамакришнана и Рика Сирса



УДК 004.62Apache Iceberg
ББК 16.2
Ш64

Ш64 **Ширан Т., Хьюз Дж., Мерсед А.**
Apache Iceberg. Полное руководство / пер. с англ. А. Н. Киселева. – М.: Books.kz, 2024. – 368 с.: ил.

ISBN 978-5-93700-289-1

Традиционные архитектурные шаблоны хранения данных сильно ограничены. Чтобы использовать их, приходится применять довольно дорогостоящие процессы ETL для загрузки данных в каждый инструмент, открывающий доступ к функциям хранилища данных. Отсутствие гибкости в этих шаблонах вынуждает замыкаться на некотором наборе инструментов и форматов, что вызывает дрейф данных.

Apache Iceberg предлагает высокую производительность, масштабируемость и экономичность – главные преимущества, свойственные открытым озерным хранилищам данных. Следуя урокам в этой книге и используя этот высокопроизводительный формат с открытым исходным кодом, вы сможете организовать поддержку интерактивным инструментам, пакетной и потоковой аналитике и машинному обучению.

Издание адресовано специалистам, занимающимся обработкой и анализом данных, а также администраторам, обслуживающим озера данных.

УДК 004.62Apache Iceberg
ББК 16.2

Authorized Russian translation of the English edition of Apache Iceberg: The Definitive Guide, ISBN 9781098148621.

This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

ISBN 978-1-098-14862-1 (англ.)
ISBN 978-5-93700-289-1 (рус.)

© 2024 O'Reilly Media Inc.
© Перевод, оформление, издание,
Books.kz, 2024

Содержание

<i>Положительные отзывы к книге</i>	12
<i>Предисловие Геррита Казмайера</i>	13
<i>Предисловие Рагху Рамакришнана</i>	14
<i>Предисловие Рика Сирса</i>	15
<i>Вступление</i>	16
<i>Благодарности</i>	21
<i>Об авторах</i>	22
<i>Колофон</i>	23

Часть I. ОСНОВЫ APACHE ICEBERG	24
---	----

Глава 1. Введение в Apache Iceberg	25
---	----

Краткая история	25
Основные компоненты системы для рабочих нагрузок OLAP	26
Все вместе	29
Хранилища данных	29
Краткая история	30
Плюсы и минусы хранилищ данных	30
Озера данных	32
Краткая история	33
Плюсы и минусы озер данных	34
Где должна производиться аналитическая обработка – в озере или в хранилище данных?	36
Озерное хранилище данных	37
Что такое формат таблиц?	39
Hive: оригинальный формат таблиц	40
Современные форматы таблиц озер данных	42
Что такое Apache Iceberg?	43
Как появился Apache Iceberg	43
Архитектура Apache Iceberg	45
Ключевые особенности Apache Iceberg	46
Заключение	50

Глава 2. Архитектура Apache Iceberg	51
--	----

Слой данных	52
Файлы данных	52
Файлы удаления	54
Слой метаданных	56
Файлы манифестов	57

Списки манифестов.....	58
Файлы метаданных	60
Puffin-файлы.....	62
Каталог.....	63
Заключение	64

Глава 3. Жизненный цикл запросов на запись и чтение

Написание запросов в Apache Iceberg.....	68
Создание таблицы	68
Запрос вставки	70
Запрос слияния	74
Запросы на чтение в Apache Iceberg.....	78
Запрос SELECT	79
Запрос информации о состоянии в прошлом	82
Заключение	86

Глава 4. Оптимизация производительности таблиц Iceberg.....

Уплотнение	87
Практическая реализация уплотнения	89
Стратегии уплотнения	94
Автоматизация уплотнения.....	96
Сортировка.....	97
Z-упорядочение.....	101
Секционирование	103
Скрытое секционирование	105
Эволюция разделов	106
Другие соображения по секционированию.....	107
Копирование при записи и слияние при чтении.....	108
Копирование при записи.....	109
Слияние при чтении.....	109
Настройка COW и MOR.....	112
Другие соображения	113
Коллекция метрик	113
Перезапись манифестов	114
Оптимизация хранилища.....	115
Режим распределения записи.....	117
Объектное хранилище	118
Фильтры Блума.....	119
Заключение	120

Глава 5. Каталоги Iceberg.....

Требования к каталогу Iceberg	121
Сравнение каталогов	122
Каталог Nadoop	122
Каталог Hive	124

Каталог AWS Glue	126
Каталог Nessie.....	127
Каталог REST.....	128
Каталог JDBC.....	129
Другие каталоги	130
Миграция каталога	131
С помощью инструмента командной строки для миграции каталога	
Iceberg.....	131
С помощью механизма обработки запросов.....	133
Заключение	136

Часть II. ПРАКТИЧЕСКИЕ ПРИЕМЫ РАБОТЫ С APACHE ICEBERG

137

Глава 6. Apache Spark

138

Настройка.....	138
Настройка Apache Iceberg и Spark	138
Настройка каталогов	140
Запуск Spark со всеми конфигурациями (пример AWS Glue).....	143
Операции на языке определения данных.....	144
CREATE TABLE	145
ALTER TABLE	148
Изменение таблиц с помощью расширений Iceberg Spark SQL.....	151
DROP TABLE	154
Чтение данных.....	154
Запрос выборки всех записей	154
Запрос с фильтрацией записей.....	154
Агрегирующие запросы	155
Оконные функции	156
Запись данных	157
INSERT INTO	157
MERGE INTO.....	158
INSERT OVERWRITE	158
DELETE FROM	160
UPDATE	160
Процедуры обслуживания таблиц Iceberg.....	161
Удаление снимков	161
Перезапись файлов данных	162
Перезапись манифестов	162
Удаление осиротевших файлов	162
Заключение	163

Глава 7. Dremio SQL Query Engine

164

Настройка.....	164
Операции на языке определения данных.....	166
CREATE TABLE	166

ALTER TABLE	168
DROP TABLE	169
Чтение данных.....	169
Использование запроса SELECT	170
Фильтрация записей	170
Агрегирующие запросы	170
Оконные функции	171
Запись данных	171
INSERT INTO	172
COPY INTO.....	172
MERGE INTO.....	172
DELETE.....	173
UPDATE	173
Обслуживание таблиц Iceberg	173
Удаление снимков	174
Перезапись файлов данных	174
Перезапись манифестов	174
Заключение	175
Глава 8. AWS Glue.....	176
Настройка.....	176
Создание базы данных Glue	176
Настройка ETL-задания в Glue.....	177
Создание таблицы с помощью Glue Data Catalog.....	180
Чтение из таблицы.....	180
Вставка данных	181
Заключение	181
Глава 9. Apache Flink.....	182
Настройка.....	182
Предварительные условия.....	182
Запуск кластера Flink и клиента Flink SQL.....	184
Операции на языке определения данных	185
CREATE CATALOG	185
CREATE DATABASE	187
CREATE TABLE	188
ALTER TABLE	189
DROP TABLE	189
Чтение данных.....	189
Пакетное чтение	189
Потоковое чтение	189
Таблицы с метаданными	190
Запись данных	191
INSERT INTO	191
INSERT OVERWRITE	191
UPSERT.....	192

Flink DataStream и Table API	193
Предварительные условия	193
Настройка задания Flink	193
Запуск кластера и сборка пакета	197
Выполнение задания	197
Заключение	198

Часть III. АРАСНЕ ICEBERG НА ПРАКТИКЕ 199

Глава 10. Apache Iceberg в производстве 200

Таблицы метаданных в Apache Iceberg	201
Таблица метаданных history	201
Таблица метаданных metadata_log_entries	203
Таблица метаданных snapshots	204
Таблица метаданных files	206
Таблица метаданных manifests	208
Таблица метаданных partitions	211
Таблица метаданных all_data_files	212
Таблица метаданных all_manifests	215
Таблица метаданных refs	217
Таблица метаданных entries	218
Совместное использование таблиц метаданных	220
Изоляция изменений с помощью ветвления	223
Ветвление и маркировка на уровне таблиц	224
Ветвление и маркировка на уровне каталога	227
Многотабличные транзакции	230
Откат изменений	231
Откат на уровне таблицы	231
Откат на уровне каталога	234
Заключение	235

Глава 11. Поточковая обработка в Apache Iceberg 237

Поточковая обработка с помощью Spark	238
Поточковая передача данных в Iceberg	239
Поточковая передача данных из Iceberg	242
Поточковая обработка с помощью Flink	243
Поточковая передача данных в Iceberg	245
Пример потоковой передачи в Iceberg с помощью Flink	250
Поточковая обработка с помощью Kafka Connect	252
Iceberg Kafka Sink	253
Поточковая обработка с помощью AWS	256
Заключение	259

Глава 12. Управление и безопасность 260

Безопасность файлов данных	261
Защита файлов: практические приемы	261

Распределенная файловая система Hadoop.....	262
Amazon Simple Storage Service	264
Azure Data Lake Storage	268
Google Cloud Storage	271
Управление безопасностью на семантическом уровне	273
Практика применения семантического уровня.....	274
Dremio.....	275
Trino	278
Управление безопасностью на уровне каталога	280
Nessie	281
Tabular	282
AWS Glue и AWS Lake Formation.....	283
Дополнительные соображения по управлению безопасностью.....	285
Заключение	286

Глава 13. Миграция данных в Apache Iceberg.....

Вопросы планирования миграции	288
План миграции на месте в три этапа.....	289
План теневой миграции в четыре этапа	290
Миграция таблиц Hive в Apache Iceberg	290
Процедура snapshot.....	291
Процедура migrate	291
Миграция из Delta Lake в Apache Iceberg.....	292
Миграция из Apache Hudi в Apache Iceberg.....	293
Миграция отдельных файлов в ApacheIceberg	294
Использование процедуры add_files	295
Миграция из Delta Lake или Apache Hudi без сохранения истории	295
Миграция откуда угодно путем копирования данных.....	296
Миграция данных в новую таблицу Iceberg.....	296
Миграция данных в существующую таблицу Iceberg.....	298
Заключение	301

Глава 14. Примеры использования Apache Iceberg

Обеспечение высокого качества данных в Apache Iceberg.....	302
WAP с использованием функции ветвления в Iceberg	303
Запуск аналитических рабочих нагрузок в озере данных.....	310
Помещение исходных данных в озеро	311
Курирование виртуальных витрины и продуктов данных	312
Создание агрегированных представлений для ускорения аналитических панелей	313
Подключение аналитических инструментов к представлению.....	313
Преимущества выполнения аналитических рабочих нагрузок в озере данных	314
Реализация сбора измененных данных с помощью Apache Iceberg	314
Создание таблиц Apache Iceberg.....	315
Применение обновлений из операционных систем.....	318

Создание представления журнала изменений для сбора изменений.....	319
Добавление изменений в агрегированную таблицу	320
Заключение	322

Глава 15. Дополнительные примеры использования

Apache Iceberg..... 323

Рабочие нагрузки реального времени, использующие AWS Kinesis и Apache Iceberg.....	323
Прием потоков с помощью AWS Kinesis.....	325
Преобразование данных и ETL с помощью AWS Glue.....	327
Анализ потока	330
Машинное обучение с Apache Iceberg.....	331
Сценарий 1: надежные конвейеры машинного обучения	332
Сценарий 2: воспроизводимость машинного обучения	334
Сценарий 3: поддержка экспериментов с машинным обучением	338
Работа с историческими данными и медленно меняющимися измерениями	340
Создание таблиц фактов и измерений для подготовки к применению SCD2	342
Обработка изменений в данных клиентов и реализация SCD2	344
Добавление новых данных о продажах в новом месте.....	347

Глава 16. Библиотеки на Java и Python для работы с Iceberg..... 349

Java API.....	349
Создание таблицы	349
Создание схемы	350
Создание спецификации секционирования	351
Чтение таблицы.....	351
Обновление таблицы.....	353
Создание транзакции.....	353
Python API	354
Установка PyIceberg	354
Настройка каталога	354
Загрузка таблицы	355
Создание таблицы	356
Обновление свойств таблицы.....	357
Запрос данных.....	357
Заключение	358

Предметный указатель..... 359

Положительные отзывы к книге

Это фантастическое учебное и справочное руководство по внутреннему устройству Apache Iceberg. Моя команда считает его бесценным.

– *Каашиф Хаймабаккус (Kaashif Hymabaccus),
старший инженер-программист, Bloomberg*

Apache Iceberg постепенно становится фактическим стандартным форматом таблиц для платформ данных следующего поколения, и эта книга поможет изучить его основные идеи и компоненты – все это обязательно потребуется большинству инженеров по данным в ближайшие годы.

– *Махди Карабибен (Mahdi Karabiben),
инженер по данным, Zendesk*

С момента появления озер данных Apache Iceberg начал набирать обороты. Эта книга познакомит вас с основными идеями табличного формата Iceberg, даст вам все необходимое для успешного использования в производстве и будет служить исчерпывающим справочником спустя месяцы после начала работы. Томер, Джейсон и Алекс – молодцы!

– *Макс Шульце (Max Schultze),
заместитель директора по инжинирингу данных, HelloFresh*

Комплексный обзор Apache Iceberg от архитектуры до решений по проектированию и реализации.

– *Симеон Шварц (Simeon Schwarz),
директор по данным и аналитике, OMS National Insurance Company*

Предисловие

Геррита Казмайера

Как человек, глубоко заинтересованный в развитии управления данными, я рад представить эту книгу об Apache Iceberg, особенно ценную в наше время, когда отрасли приходится решать сложнейшие задачи управления данными. Apache Iceberg с его революционным форматом открытых таблиц – это не просто технологическое достижение, это значительный сдвиг в подходе организаций к управлению данными в эпоху искусственного интеллекта.

Участие в жизни сообщества специалистов по обработке данных позволило мне заметить растущее любопытство и спрос на знания об озерах данных и Apache Iceberg. Учитывая это, можно смело утверждать, что эта книга – не просто информационный ресурс, но необходимое руководство для тех, кто хочет понять и начать применять эту технологию в своей работе. Она подробно обсуждает архитектуру Apache Iceberg и описывает варианты и передовой опыт практического применения, служа незаменимым источником знаний для архитекторов данных и инженеров.

Эта книга – больше, чем просто сборник информации; это свидетельство развития в сфере управления данными и маяк для тех, кто пытается разобраться в ее сложностях. Я уверен, что эта книга станет ценным ресурсом для всех, кто решит использовать Apache Iceberg в архитектурах данных.

*– Геррит Казмайер (Gerrit Kazmaier),
вице-президент и генеральный директор
по анализу данных, Google Cloud*

Предисловие

Рагху Рамакришнана

Apache Iceberg – один из ведущих открытых форматов обновляемых таблиц на основе Parquet, которые постепенно превращаются в новый стандарт хранения данных для аналитики. Исторически в реляционных базах данных информация хранилась в виде последовательностей записей, упакованных в физические страницы ради эффективности ввода-вывода. Однако хранение данных по столбцам оказалось гораздо эффективнее в окружениях, где приходится обрабатывать большое количество запросов в единицу времени. Озера данных изначально поддерживали запросы к столбчатым форматам, таким как Parquet, но им также требовалась эффективная поддержка транзакционного обновления для решения традиционных задач хранения данных. Iceberg как формат хранения таблиц приобретает все бóльшую популярность, так как он поддерживает сценарии, что необходимо в окружениях, интенсивно обрабатывающих запросы и одновременно обновляющих и загружающих большие объемы данных.

Эта своевременная книга хорошо описывает и основы Iceberg, и его архитектуру, и способы достижения максимальной производительности в широком спектре рабочих нагрузок, включая SQL-запросы в Apache Spark и Dremio, а также потоковую обработку в Apache Flink. В книге также есть глава, в которой рассматривается применение Iceberg в промышленных окружениях, включая использование таблиц метаданных и таких функций, как ветвление, секционирование и мгновенные снимки (снапшоты), для обработки сложных сценариев в масштабе. Вся эта информация бесценна для читателей, интересующихся особенностями системы Iceberg, разработкой приложений, использующих Iceberg (или систему, основанную на Iceberg).

*– Рагху Рамакришнан,
технический директор по данным, технический партнер Microsoft*

Предисловие Рика Сирса

Данные стали центральной частью современных программных приложений и современных организаций, чья работа основана на использовании данных. Инженеры данных, администраторы данных, аналитики данных и ученые, работающие с данными, входят в число людей в таких организациях, которые хотят еще эффективнее использовать свои данные. Многие из этих специалистов по обработке данных создают свои приложения, управляемые данными, на Amazon Web Services (AWS), часто предпочитая хранить свои данные в озере на базе Amazon Simple Storage Service (S3).

Все они могут пожелать изменять свои данные и манипулировать ими с течением времени, продолжая использовать их в процессе изменения, и, следовательно, создавать свои приложения с поддержкой транзакций в озере данных. Apache Iceberg – ключевая технология, используемая клиентами AWS для создания озер транзакционных данных. Она отличается высоким быстродействием, эффективностью и надежностью, предлагает простую интеграцию с популярными платформами обработки данных на AWS, такими как Apache Spark, Apache Flink, Apache Hive, Presto, Trino, Dremio и др., а также поддерживается сервисами AWS, такими как Amazon EMR, Amazon Redshift, Amazon Athena, AWS Glue и др.

«Apache Iceberg. Полное руководство» фокусируется на практике применения и сценариях, полезных для специалистов по работе с данными, использующих Apache Iceberg, а также содержит практические упражнения, помогающие приобрести навыки использования Iceberg в комплексе с ключевыми технологиями AWS, такими как Amazon EMR и AWS Glue, которые поддерживают оптимизацию и позволяют с помощью Iceberg создавать масштабируемые приложения. Книга охватывает весь спектр знаний, необходимых этим специалистам, от решаемых ими задач до архитектуры приложений, управляемых данными, и лучших практик реального использования в AWS, поэтому читатели смогут не только понять и поэкспериментировать с Apache Iceberg, но и в кратчайшие сроки внедрить эту технологию в производство.

Содержимое для этой книги подбиралось с учетом обсуждений с клиентами AWS и более широким сообществом. Поскольку Apache Iceberg становится все более важным для клиентов AWS, я рад, что они получают этот ценный справочник по созданию, масштабированию и оптимизации приложений с помощью Iceberg.

– Рик Сирс (Rick Sears), генеральный директор Amazon Web Services,
Amazon EMR, Amazon Athena, AWS Lake Formation
и AWS Glue Data Catalog

Вступление

Добро пожаловать в «Apache Iceberg. Полное руководство»! Мы рады, что вы решили отправиться в путешествие по этой технологии вместе с нами. В этом вступлении мы даем краткий обзор книги, объясняем, почему мы ее написали и как извлечь из нее максимальную пользу.

Об этой книге

На страницах этой книги вы узнаете, что такое Apache Iceberg, как он появился, как работает и как использовать его возможности. Эта книга, предназначенная для инженеров, архитекторов, ученых и аналитиков, работающих с большими наборами данных в различных сценариях, от информационных панелей бизнес-аналитики до искусственного интеллекта и машинного обучения, исследует основные концепции, внутреннюю работу и практическое применение Apache Iceberg. К концу книги вы усвоите основы и овладете практическими знаниями, необходимыми для эффективного внедрения Apache Iceberg в свои проекты. Независимо от того, являетесь ли вы новичком или опытным практиком, «Apache Iceberg. Полное руководство» станет вашим надежным спутником в этом занимательном путешествии по Apache Iceberg.

Почему мы написали эту книгу

Когда мы наблюдали за быстрым ростом и распространением экосистемы Apache Iceberg, стало очевидно, что необходимо устранить растущий пробел в знаниях. Первоначально мы начали с обмена идеями через серию публикаций в блогах на платформе Dremio, чтобы передать ценную информацию растущему сообществу Iceberg. Однако вскоре стало ясно, что для удовлетворения растущего спроса на исчерпывающую информацию об Iceberg необходим всеобъемлющий и централизованный ресурс. Это осознание стало главной причиной появления «Apache Iceberg. Полное руководство». Наша цель – дать читателям единый авторитетный источник, устраняющий пробелы в знаниях и дающий возможность отдельным лицам и организациям максимально эффективно использовать Apache Iceberg.

Что вы найдете внутри

В главах, следующих далее, вы узнаете, что такое и как работает Apache Iceberg, как использовать преимущества этого формата в комплексе с различными инструментами, а также познакомитесь с передовыми методами

управления данными в таблицах Apache Iceberg. Вот краткое содержание каждой главы.

Глава 1 «Введение в Apache Iceberg» исследует исторический контекст озер данных и основных идей, лежащих в основе Apache Iceberg.

Глава 2 «Архитектура Apache Iceberg» подробно описывает сложную конструкцию Apache Iceberg и взаимодействия его компонентов.

Глава 3 «Жизненный цикл запросов на запись и чтение» дает пошаговый анализ процесса выполнения транзакций Apache Iceberg с учетом операций обновления, чтения и перемещения запросов во времени.

Глава 4 «Оптимизация производительности таблиц Iceberg» обсуждает вопросы оптимизации производительности таблиц Apache Iceberg с помощью таких методов, как сжатие и сортировка.

Глава 5 «Каталоги Iceberg» подробно объясняет роль каталогов Apache Iceberg и представляет доступные варианты каталогов.

Глава 6 «Apache Spark» приводит практические занятия по использованию Apache Spark для управления таблицами Apache Iceberg и взаимодействия с ними.

Глава 7 «Механизм запросов SQL в Dremio» исследует платформу Dremio с упором на DDL, DML и оптимизацию для таблиц Apache Iceberg.

Глава 8 «AWS Glue» демонстрирует использование каталога AWS Glue и AWS Glue Studio для работы с таблицами Apache Iceberg.

Глава 9 «Apache Flink» содержит практические упражнения по использованию Apache Flink для потоковой обработки данных с помощью таблиц Apache Iceberg.

Глава 10 «Apache Iceberg в промышленной среде» предлагает подробные сведения об управлении качеством данных в промышленной среде, использовании таблиц метаданных для мониторинга работоспособности и использовании управления версиями таблиц и каталогов для различных нужд.

Глава 11 «Потоковая передача с помощью Apache Iceberg» описывает особенности использования таких инструментов, как Apache Spark, Flink и AWS Glue, для потоковой обработки данных в таблицах Iceberg.

Глава 12 «Управление и безопасность» исследует приемы управления и обеспечения безопасности на различных уровнях в таблицах Apache Iceberg, таких как хранилище, семантический уровень и каталоги.

Глава 13 «Миграция на Apache Iceberg» дает рекомендации по преобразованию существующих наборов данных из различных типов файлов и баз данных в таблицы Apache Iceberg.

Глава 14 «Примеры использования Apache Iceberg» приводит примеры использования Apache Iceberg на практике, включая информационные панели бизнес-аналитики и реализацию сбора данных об изменениях.

Как пользоваться этой книгой

Главная цель этой книги – помочь вам лучше понять и получить практические навыки использования Apache Iceberg. Эта книга организована так,

чтобы помочь вам получить полное представление об обсуждаемой технологии и читать главы в том порядке, какой вам больше по душе. Каждая глава полностью самостоятельная и не зависит от других, соответственно, вы сможете погрузиться в конкретные темы или случаи использования без необходимости читать предыдущие главы. Такой подход делает эту книгу как ценным ресурсом для систематического обучения, так и хорошим оперативным справочником.

На протяжении всей книги вам будут встречаться ссылки на фрагменты кода и практические примеры. Чтобы помочь вам в обучении, мы создали специальный репозиторий на GitHub (<https://oreil.ly/supp-guide-apache-iceberg>), организованный по главам, что обеспечивает легкий доступ ко всем необходимым справочным материалам, фрагментам кода и примерам, относящимся к содержанию каждой главы. Репозиторий послужит вам дополнительным инструментом для улучшения вашего обучения и применения идей, обсуждаемых в книге. Чтобы получить еще больше информации, включая дополнительную главу об Iceberg Java/Python API¹ и дополнительный обзор вариантов использования Iceberg², посетите этот репозиторий (https://oreil.ly/apache-ice_more-content).

Независимо от того, как вы будете читать это руководство – от начала до конца или сосредоточитесь на отдельных главах, – эта книга задумана как всеобъемлющий и доступный ресурс по Apache Iceberg, обогащенный практическими примерами и сведениями, доступными через наш репозиторий GitHub.

Отзывы и вопросы

Мы ценим ваши отзывы и вопросы. Если у вас есть какие-либо проблемы, предложения или пожелания, пишите нам по адресу tech-advocacy@dremio.com. Мы также приглашаем вас подписаться на нас в LinkedIn.

Ниже перечислены дополнительные ресурсы, которые помогут вам больше узнать об Apache Iceberg и принять участие в жизни сообщества:

- каталог статей и ресурсов «Apache Iceberg 101» (<https://oreil.ly/hDv4H>);
- документация Apache Iceberg (<https://iceberg.apache.org>);
- репозиторий Apache Iceberg на GitHub (<https://github.com/apache/iceberg>);
- канал Iceberg Slack (как получить приглашение, см. в документации Iceberg);
- список рассылки Iceberg (порядок регистрации см. в документации к Iceberg);
- страница семинаров Apache Iceberg на LinkedIn (<https://oreil.ly/WghwD>);
- каталог блогов Apache Iceberg (<https://iceberg.apache.org/blogs>).

¹ Включена в книгу как глава 16. – Прим. перев.

² Включен в книгу как глава 15. – Прим. перев.

Обозначения и соглашения, принятые в этой книге

В книге действуют следующие типографские соглашения:

Курсив

Курсивом выделены новые термины или важные понятия.

Моноширинный шрифт

Используется для листингов программ, а также в тексте для обозначения таких элементов, как переменные и функции, базы данных, типы данных, переменные среды, операторы и ключевые слова, имена файлов и их расширения.

Моноширинный жирный шрифт

Показывает команды или другой текст, который пользователь должен ввести самостоятельно.

Моноширинный курсив

Показывает текст, который должен быть заменен значениями, введенными пользователем, или значениями, определяемыми контекстом.



Так обозначаются советы.



Так обозначаются примечания общего характера.



Так обозначаются предупреждения и предостережения.

Скачивание исходного кода примеров

Скачать файлы с дополнительной информацией для книг издательства «ДМК Пресс» можно на сайте www.dmkpress.com или www.дмк.рф на странице с описанием соответствующей книги.

Мы высоко ценим, хотя и не требуем, ссылки на наши издания. В ссылке обычно указываются имя автора, название книги, издательство и ISBN, например: «Томер Ширан, Джейсон Хьюз и Алекс Мерсед. Apache Iceberg. Полное руководство. М.: ДМК Пресс, 2025. ISBN 978-5-93700-289-1».

Если вы полагаете, что планируемое использование кода выходит за рамки изложенной выше лицензии, пожалуйста, обратитесь к нам по адресу dmkpress@gmail.com.

Отзывы и пожелания

Мы всегда рады отзывам наших читателей. Расскажите нам, что вы думаете об этой книге – что понравилось или, может быть, не понравилось. Отзывы важны для нас, чтобы выпускать книги, которые будут для вас максимально полезны.

Вы можете написать отзыв прямо на нашем сайте www.dmkpress.com, зайдя на страницу книги, и оставить комментарий в разделе «Отзывы и рецензии».

Также можно послать письмо главному редактору по адресу dmkpress@gmail.com, при этом напишите название книги в теме письма.

Если есть тема, в которой вы квалифицированы, и вы заинтересованы в написании новой книги, заполните форму на нашем сайте по адресу http://dmkpress.com/authors/publish_book/ или напишите в издательство по адресу dmkpress@gmail.com.

Список опечаток

Хотя мы приняли все возможные меры для того, чтобы удостовериться в качестве наших текстов, ошибки все равно случаются. Если вы найдете ошибку в одной из наших книг – возможно, ошибку в тексте или в коде, – мы будем очень благодарны, если вы сообщите нам о ней. Сделав это, вы избавите других читателей от расстройств и поможете нам улучшить последующие версии этой книги.

Если вы найдете какие-либо ошибки в коде, пожалуйста, сообщите о них главному редактору по адресу dmkpress@gmail.com, и мы исправим это в следующих тиражах.

Нарушение авторских прав

Пиратство в интернете по-прежнему остается насущной проблемой. Издательство «ДМК Пресс» очень серьезно относится к вопросам защиты авторских прав и лицензирования. Если вы столкнетесь в интернете с незаконной публикацией какой-либо из наших книг, пожалуйста, пришлите нам ссылку на интернет-ресурс, чтобы мы могли применить санкции.

Ссылку на подозрительные материалы можно прислать по адресу электронной почты dmkpress@gmail.com.

Мы высоко ценим любую помощь по защите наших авторов, благодаря которой мы можем предоставлять вам качественные материалы.

Благодарности

Мы выражаем нашу глубочайшую благодарность компании Dremio и издательству O'Reilly Media за предоставленную возможность написать эту книгу. Особенно мы благодарны нашему редактору O'Reilly – Гэри О'Брайену (Gary O'Brien), который постоянно помогал нам не сбиться с пути в процессе работы над книгой. Спасибо нашим техническим рецензентам, которые требовали от нас ответственности на каждом шагу, следя за тем, чтобы книга была точной и полной: Камрану Али (Kamran Ali), Джаю Балани (Jai Balani), Михалю Ганцарски (Michał Gancarski), Махди Карабибену (Mahdi Karabiben), Кевину Хо (Kevin Kho), Марку Лафоре (Kevin Kho), Максу Шульце (Max Schulze) и Симеону Шварцу (Simeon Schwarz). Отдельное спасибо Дипанкару Мазумдару (Dipankar Mazumdar) за его вклад.

Мы также искренне благодарим наши семьи, проявившие терпение в долгие ночи, когда мы писали и редактировали эту книгу. Наконец, мы хотели бы поблагодарить сообщество Apache Iceberg за создание одного из самых интересных и инновационных проектов в области данных.

Благодарим вас за выбор «Apache Iceberg. Полное руководство». Мы надеемся, что вы найдете эту книгу информативной и приятной. Давайте вместе окунемся в захватывающий мир Apache Iceberg!

Приятного чтения!

Об авторах

Томер Ширан (Tomer Shiran) – основатель и директор по разработке Dremio, открытой платформы для хранения данных, которая позволяет компаниям выполнять аналитику в облаке без затрат, без сложностей и без привязки к конкретным хранилищам данных. Будучи генеральным директором и основателем компании, Томер создал организацию мирового уровня с прибылью более 400 млн долларов и теперь обслуживает сотни крупнейших предприятий мира, в том числе три из списка Fortune. До прихода в Dremio Томер был четвертым сотрудником и вице-президентом компании MapR, пионера в области анализа больших данных. Он также занимал многочисленные должности по управлению продуктами и инженерными разработками в Microsoft и IBM Research, основал несколько веб-сайтов, обслуживающих миллионы пользователей, а также является успешным автором и пропагандистом широкого круга отраслевых тем. Получил степень магистра информатики в университете Карнеги–Меллона и степень бакалавра информатики в Израильском технологическом институте Технион.

Джейсон Хьюз (Jason Hughes) – директор по технической поддержке в Dremio. Ранее в Dremio он работал директором по продукту, техническим директором и старшим архитектором решений. Работает в области технологий обработки данных уже более десяти лет и занимал должности технического руководителя в этой области в Dremio, руководителя предпродажного и послепродажного обслуживания Presto и QueryGrid в Северной и Южной Америке в Teradata, а также принимал участие в разработке, развертывании и сопровождении специальной CRM-системы для нескольких автосалонов. Стремится сделать клиентов и частных лиц успешными и самодостаточными. В свободное от работы время обычно гуляет со своей собакой, играет в хоккей или готовит (когда ему этого хочется). Джейсон живет в Сан-Диего, штат Калифорния.

Алекс Мерсед (Alex Merced) – защитник интересов разработчиков в Dremio. Работал разработчиком и инструктором в таких компаниях, как GenEd Systems, Crossfield Digital, CampusGuard и General Assembly. Увлечен технологиями и публикует статьи на технические темы в блоге, а также видеоролики и подкасты на Datanation и Web Dev 101. Выступал на многих крупных отраслевых конференциях, таких как Data Council, Data Day Texas, Subsurface, OSA Con. Алекс внес свой вклад в различные библиотеки для JavaScript и Python, включая SencilloDB, CoquitoJS, dremio-simple-query и др.

Колофон

Птица, изображенная на обложке книги, – это белошекая крачка (*Chlidonias Hybrida*).

Белошекие крачки – водоплавающие птицы малого и среднего размера с относительно коротким, слегка раздвоенным хвостом. В брачный период оперение брюха и груди темно-серого цвета. На голове черная «шапочка», горло, щеки, подхвостье и испод крыла белые. Молодые птицы имеют черно-белый шахматный рисунок на спине. В остальные периоды птицы имеют более скромный окрас: спина серая, брюхо белое, клюв черный.

Белошекие крачки имеют обширный ареал обитания. Их можно найти в южной Европе и Азии, юго-восточной Африке, на Мадагаскаре и в Австралии. Они живут на пресноводных водно-болотных угодьях, пресноводных болотах, солоноватых озерах, орошаемых пахотных землях, искусственных рыбных прудах и множестве других водно-болотных угодий. Их рацион зависит от среды обитания и состоит из мелкой рыбы, земноводных, мелких ракообразных и насекомых.

В какой-то момент мировая популяция белошеких крачек оценивалась от 300 000 до 1 500 000 особей. Широкая распространенность по всему миру позволяет отнести их к видам, вызывающим наименьшее беспокойство. Однако многие другие животные на обложках книг, выпускаемых издательством O'Reilly, находятся под угрозой исчезновения; все они важны для мира.

Иллюстрация на обложке выполнена Карен Монтгомери (Karen Montgomery) на основе черно-белой гравюры из журнала «British Birds». Дизайн серии разработали Эди Фридман (Edie Freedman), Элли Волкхаузен (Ellie Volckhausen) и Карен Монтгомери (Karen Montgomery).

часть I

ОСНОВЫ АРАСНЕ ICEBERG

В первой части книги рассматриваются основы Apache Iceberg, включая такие темы, как архитектура таблиц Iceberg, жизненный цикл запросов на чтение и запись и каталоги Iceberg. Цель состоит в том, чтобы заложить прочную основу для понимания последующих частей книги.

Глава 1

Введение в Apache Iceberg

Данные – это основной ресурс, из которого организации черпают информацию и идеи, необходимые для принятия важных решений. Независимо от того, используются ли они для анализа тенденций годовых продаж конкретного продукта или для прогнозирования будущих рыночных возможностей, данные определяют направление, следуя которому, организации смогут добиться успеха. Более того, в настоящее время анализ данных – это не просто приятное занятие. Это требование, необходимое для успешной конкуренции на рынке. Учитывая такой огромный спрос на информацию, в организациях предпринимаются большие усилия по накоплению данных, генерируемых различными внутренними системами.

В то же время резко возросла скорость, с которой промышленные и аналитические системы генерируют данные. Увеличение объема данных помогает бизнесу принимать более обоснованные решения, однако оно же влечет острую потребность в платформе хранения и анализа этих данных, чтобы их можно было использовать для создания аналитических продуктов, таких как отчеты бизнес-аналитики и модели машинного обучения для поддержки принятия решений. Архитектура озер данных, которую мы подробно рассмотрим в этой главе, отделяет способы хранения от способов обработки данных, давая большую гибкость. В этой главе вы познакомитесь с историей и эволюцией платформ данных и преимуществами архитектуры озер данных с форматами открытых таблиц Apache Iceberg.

Краткая история

В мире хранения и обработки данных давно и прочно обосновались системы управления реляционными базами данных (СУРБД), позволяющие организациям вести учет всех своих транзакционных данных. Например, представьте, что вы управляете транспортной компанией и хотите хранить ин-

формацию о новых бронированиях, сделанных вашими клиентами. В этом случае каждое новое бронирование будет представлять собой новую запись в СУРБД. Реляционные базы данных, используемые для этой цели, поддерживают определенную технологию обработки данных, называемую *обработкой транзакций в реальном времени* (Online Transaction Processing, OLTP). Примерами СУРБД, оптимизированных для OLTP, могут служить PostgreSQL, MySQL и Microsoft SQL Server. Эти системы разработаны и оптимизированы для быстрого взаимодействия с одной или несколькими записями данных одновременно и являются хорошим выбором для поддержки повседневной деятельности бизнеса.

Но предположим, что вам понадобилось узнать среднюю прибыль, которую вы получили от новых бронирований за предыдущий квартал. В этом случае использование OLTP-оптимизированной СУРБД, хранящей большой объем данных, привело бы к значительным проблемам с производительностью, причины которых перечислены ниже.

- Транзакционные системы ориентированы на вставку, обновление и чтение небольшого подмножества записей в таблице, и для такого способа использования хранение данных в формате последовательностей записей является идеальным. Однако аналитические системы обычно фокусируются на агрегировании определенных столбцов или работе со всеми записями в таблице, что делает столбчатую структуру более выгодной.
- Сочетание транзакционных и аналитических рабочих нагрузок в одной инфраструктуре может привести к конкуренции за ресурсы.
- Транзакционные рабочие нагрузки выигрывают от нормализации данных по нескольким связанным таблицам, которые соединяются при необходимости, тогда как аналитические рабочие нагрузки имеют лучшую производительность, когда данные денормализованы в одну таблицу.

Теперь представьте, что в вашей организации имеется большое количество промышленных систем, которые генерируют огромное количество данных, и ваша аналитическая группа хочет создать информационные панели, агрегирующие данных из разных источников (т. е. баз данных). К сожалению, OLTP-системы не предназначены для обработки сложных агрегатных запросов, включающих большое количество исторических записей. Такие рабочие нагрузки известны как *оперативная аналитическая обработка* (OnLine Analytical Processing, OLAP). Чтобы устранить это ограничение, необходим другой тип системы, оптимизированный для рабочих нагрузок OLAP. Именно эта необходимость побудила к развитию архитектуры озер данных.

Основные компоненты системы для рабочих нагрузок OLAP

Система, предназначенная для оперативной аналитической обработки данных, состоит из набора технологических компонентов (показанных на

рис. 1.1 и описанных далее), которые позволяют поддерживать современные аналитические рабочие нагрузки.

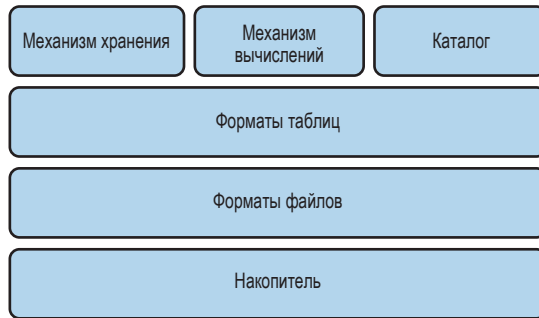


Рис. 1.1 ❖ Технические компоненты поддержки аналитических рабочих нагрузок

Накопитель

Для анализа исторических данных, поступающих из разных источников, необходима система, позволяющая хранить значительные объемы данных. Соответственно, накопитель – это первый компонент, который вам понадобится для создания системы обработки аналитических запросов к большим наборам данных. Существует множество вариантов накопителей, включая локальную файловую систему на дисках с прямым подключением (Direct-Attached Storage, DAS); распределенную файловую систему на наборе узлов, которыми вы управляете, например Hadoop Distributed File System (HDFS); и объектное хранилище, предоставляемое в виде облачной услуги такими провайдерами, как Amazon Simple Storage Service (Amazon S3).

Что касается типов хранилищ, то они могут быть ориентированы на хранение в форме записей или столбцов, а в некоторых системах их можно даже смешивать. В последние годы все более широкое распространение получают столбчатые базы данных, доказавшие свою эффективность при работе с огромными объемами данных.

Форматы файлов

Исходные данные должны храниться в файлах определенного формата. Выбор формата влияет, например, на степень сжатия файлов, структуру данных и производительность рабочей нагрузки.

Форматы файлов обычно делятся на три основные категории: структурированные (CSV), полуструктурированные (JSON) и неструктурированные (обычные текстовые файлы). Структурированные и полуструктурированные форматы могут быть ориентированы на хранение данных по записям или по столбцам. Форматы файлов, ориентированные на записи, хранят все столбцы каждой записи вместе, а форматы, ориентированные на столбцы, хра-

нут вместе каждый столбец записей. Двумя распространенными примерами форматов, ориентированных на записи, являются формат значений, разделенных запятыми (Comma-Separated Values, CSV), и Apache Avro. Примерами столбчатых форматов могут служить Apache Parquet и Apache ORC.

В зависимости от варианта использования некоторые форматы могут быть более выгодными, другие – менее. Например, форматы, ориентированные на записи, лучше подходят при работе с небольшим количеством записей одновременно. Столбчатые форматы, напротив, лучше подходят для работы с большим количеством записей.

Форматы таблиц

Формат таблиц – еще один важный компонент системы, поддерживающей аналитические рабочие нагрузки с агрегирующими запросами к огромному объему данных. Форматы таблиц действуют как слой метаданных поверх форматов файлов и определяют порядок размещения файлов данных в накопителе.

В конечном счете цель форматов таблиц состоит в том, чтобы абстрагировать сложность физической структуры данных и обеспечить возможность манипулирования данными (например, вставку, обновление и удаление) и изменение схемы таблиц. Современные форматы таблиц также обеспечивают гарантии атомарности и согласованности, необходимые для безопасного манипулирования данными.

Механизм хранения

Механизм хранения – это система, отвечающая за фактическое размещение данных в форме, заданной форматами таблиц, и поддержку файлов и структур данных в актуальном состоянии. Механизмы хранения решают такие важные задачи, как физическая оптимизация данных, обслуживание индексов и избавление от старых данных.

Каталог

Имея дело с большим количеством данных из разных источников, важно уметь быстро определить, какие данные могут понадобиться для анализа. Роль каталога – помочь в решении этой задачи путем идентификации наборов данных с применением метаданных. Каталог – это центральное место, куда механизмы вычислений и пользователи могут обратиться, чтобы узнать о существовании таблицы, а также получить дополнительную информацию, такую как имя таблицы, ее схема и местоположение в системе хранения. Некоторые каталоги являются внутренними по отношению к системе, и с ними можно напрямую взаимодействовать через механизмы системы; примерами таких каталогов могут служить Postgres и Snowflake. Другие каталоги, такие как Hive и Project Nessie, могут использоваться в любой системе. Имейте

в виду, что каталоги метаданных – это не то же самое, что каталоги, предназначенные для использования человеком, такие как Colibra, Atlan и внутренний каталог Dremio Software.

Механизм вычислений

Механизм вычислений – последний компонент, необходимый для эффективной работы с огромными объемами данных, находящимися в системе хранения. Роль механизма вычислений в такой системе заключается в запуске рабочих нагрузок для обработки данных. В зависимости от объема данных, сложности вычислений и типа рабочей нагрузки для решения задачи можно использовать один или несколько механизмов вычислений. При работе с большим набором данных и/или выполнении тяжелых вычислений может потребоваться использовать механизм распределенных вычислений в парадигме обработки, называемой массово-параллельной обработкой (Massively Parallel Processing, MPP). Примерами механизмов вычислений MPP могут служить Apache Spark, Snowflake и Dremio.

Все вместе

Для рабочих нагрузок типа OLAP все эти технические компоненты тесно связываются в единую систему, известную как *хранилище данных* (*data warehouse*). Хранилища данных позволяют хранить данные, поступающие из разных источников, и выполнять их анализ. В следующем разделе мы подробно обсудим возможности хранилищ данных, способы интеграции технических компонентов, а также плюсы и минусы использования такой системы.

Хранилища данных

Хранилище данных, или база данных OLAP, – это централизованное хранилище, поддерживающее хранение больших объемов данных, полученных из разных источников, таких как промышленные системы, базы данных приложений и журналы. Схема хранилища данных с его компонентами показана на рис. 1.2.

Хранилище данных объединяет все технические компоненты в единую систему. Иначе говоря, все данные хранятся в конкретных форматах файлов и таблиц в конкретной системе хранения. Управление этими данными осуществляется исключительно механизмом хранения хранилища данных, они регистрируются в каталоге и могут быть доступны только пользователю или аналитическим механизмам через механизм вычислений.

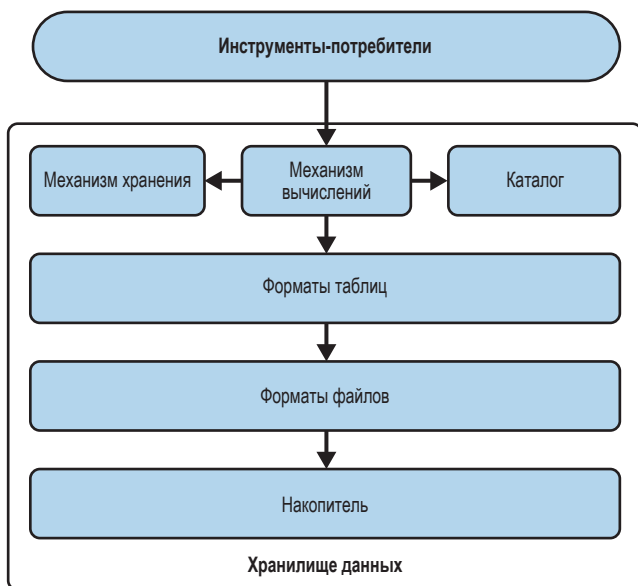


Рис. 1.2 ❖ Технические компоненты хранилища данных

Краткая история

Примерно до 2015 года в большинстве хранилищ данных механизмы хранения и вычислений были тесно связаны и размещались на одних и тех же узлах, потому что большинство из них проектировались как локальные механизмы. Однако это привело к множеству проблем. Масштабирование стало большой проблемой, поскольку объемы данных, а также интенсивность рабочих нагрузок росли быстрыми темпами. В частности, отсутствовала возможность оперативно нарастить вычислительные ресурсы и ресурсы хранения в зависимости от решаемых задач. Если потребности в увеличении объема хранилища росли быстрее, чем вычислительные потребности, то это не имело значения. Вы были вынуждены платить за дополнительную вычислительную мощность, даже если она вам не нужна.

Это привело к созданию следующего поколения хранилищ данных – с размещением в облаке. Эти хранилища данных начали распространяться примерно в 2015 году, когда появились облачные услуги, позволяющие разделить механизмы вычислений и хранения и масштабировать эти ресурсы в соответствии с решаемыми задачами. Они даже позволили отключить компьютер, когда вы его не используете, и при этом ничего не потерять.

Плюсы и минусы хранилищ данных

Хранилища данных, как локальные, так и облачные, позволяют предприятиям быстро разобраться со своими историческими данными, однако есть

области, в которых хранилища данных по-прежнему вызывают проблемы. В табл. 1.1 перечислены плюсы и минусы хранилищ данных.

Таблица 1.1. Плюсы и минусы хранилищ данных

Плюсы	Минусы
Служит единственным источником истины, позволяет хранить данные из разных источников	Хранит данные в системе конкретного поставщика, которую может использовать только механизм вычислений хранилища
Поддерживает запросы к огромным объемам исторических данных, обеспечивая быструю работу аналитических рабочих нагрузок	Дороговизна хранения и вычислений; с увеличением рабочей нагрузки управлять затратами становится все труднее
Обеспечивает эффективное управление данными, гарантирующее их доступность, удобство использования и соответствие политикам безопасности	В основном поддерживаются структурированные данные
Организует структуру данных под конкретные нужды, гарантируя ее оптимальность для запросов	Не позволяет запускать сложные аналитические рабочие нагрузки, такие как машинное обучение
Гарантирует соответствие данных, записанных в таблицу, технической схеме	

Хранилище данных действует как централизованное место хранения данных организаций, поступающих из множества источников, и позволяет потребителям данных, таким как аналитики и инженеры, легко и быстро получать доступ к данным из одного источника, чтобы начать анализ. Кроме того, технологические компоненты, лежащие в основе хранилища данных, дают возможность получать доступ к огромным объемам данных и одновременно выполнять рабочие нагрузки бизнес-анализа.

Хранилища данных сыграли решающую роль в демократизации данных и позволили предприятиям получать историческую информацию из разных источников, но многие из них ограничиваются реляционными рабочими нагрузками. Например, возвращаясь к предыдущему примеру с транспортной компанией, предположим, что вам понадобилось спрогнозировать общий объем продаж в следующем квартале. Для этого вам потребуется построить модель прогнозирования на основе исторических данных. Однако вы не можете сделать этого, используя хранилище данных в качестве вычислительного механизма, так как его технические компоненты не предназначены для решения задач машинного обучения. Поэтому основным реальным вариантом будет перемещение или экспорт данных из хранилища данных на другие платформы, поддерживающие рабочие нагрузки на основе машинного обучения. Это означает необходимость создания нескольких копий данных и конвейеров для их перемещения, что может привести к критическим проблемам, таким как дрейф данных и разрушение модели, когда конвейеры перемещают данные неправильно или непоследовательно.

Еще одно препятствие, мешающее выполнению расширенных аналитических рабочих нагрузок поверх хранилища данных, – поддержка только

структурированных данных. Но другие типы данных, полуструктурированные и неструктурированные, такие как JSON, изображения, текст и т. д., позволяют создавать модели машинного обучения, воплощающие очень интересные идеи. В нашем примере это может быть анализ эмоциональной окраски новых отзывов о бронировании, сделанных в предыдущем квартале. В конечном итоге такого рода анализ влияет на способность организации принимать решения, ориентированные на будущее.

Существуют также специфические проблемы хранилищ данных. На рис. 1.2 можно видеть, что все шесть технических компонентов, образующих хранилище данных, тесно связаны между собой. Особо отметьте, что форматы файлов и таблиц являются *внутренними* для конкретного хранилища данных. Этот шаблон проектирования приводит к *закрытой архитектуре* данных. То есть фактические данные доступны только с использованием вычислительного механизма хранилища данных, который специально спроектирован для взаимодействия с конкретными форматами таблиц и файлов. Такой тип архитектуры заставляет организации серьезно беспокоиться о зависимости: с увеличением рабочих нагрузок и объемов данных, поступающих в хранилище данных, вы оказываетесь прочно привязанными к этой конкретной платформе. А это означает, что ваши аналитические рабочие нагрузки и любые будущие инструменты, которые вы планируете внедрить, могут работать только поверх этого конкретного хранилища данных. Это также мешает вам перейти на другую платформу данных, способную удовлетворить ваши растущие потребности.

Кроме того, хранение данных в хранилище и использование вычислительных механизмов для их обработки сопряжено со значительными затратами. Со временем, по мере увеличения количества рабочих нагрузок и потребления вычислительных ресурсов, затраты только увеличиваются. Помимо финансовых затрат существуют и другие накладные расходы, такие как необходимость создавать и управлять многочисленными конвейерами для перемещения данных из промышленных систем, а также временная задержка получения информации на стороне потребителей данных. Эти проблемы побудили организации искать альтернативные платформы, позволяющие контролировать данные и хранить их в открытых форматах файлов, чтобы последующие приложения бизнес-аналитики и машинного обучения могли работать параллельно со значительно меньшими затратами. Это привело к появлению озер данных.

Озера данных

Хотя хранилища данных предоставляли механизм для анализа структурированных данных, они все же имели несколько проблем:

- они могут хранить только структурированные данные;
- хранение в хранилище данных обычно обходится дороже, чем в локальных кластерах Hadoop или облачных объектных хранилищах;

- хранение и вычисления в традиционных локальных хранилищах данных часто оказываются тесно связанными и не могут масштабироваться по отдельности. Увеличение затрат на хранение влечет увеличение затрат на вычисления, независимо от того, нужна дополнительная вычислительная мощность или нет.

Для устранения этих проблем потребовалось альтернативное решение, удешевляющее хранение данных и избавляющее от необходимости следовать фиксированной схеме. Таким альтернативным решением стало озеро данных.

Краткая история

Первоначально мы использовали Hadoop, платформу распределенных вычислений с открытым исходным кодом, и ее компонент – файловую систему HDFS – для хранения и обработки больших объемов структурированных и неструктурированных данных в кластерах недорогих компьютеров. Но просто хранить все эти данные было недостаточно. Мы также хотели анализировать эти данные.

Экосистема Hadoop включала MapReduce, аналитическую среду, с помощью которой можно было писать аналитические задания на Java и запускать их в кластере Hadoop. Написание заданий MapReduce было сложным делом, и многим аналитикам удобнее писать на SQL, чем на Java, поэтому был создан Hive для преобразования операторов SQL в задания MapReduce.

Для разработки кода на SQL требовался механизм, позволяющий различать файлы в хранилище, которые являются частью определенного набора данных или таблицы. Это привело к рождению формата таблиц Hive, который распознавал каталог и файлы внутри него как таблицу.

Со временем люди перешли от кластеров Hadoop к облачным объектным хранилищам (например, Amazon S3, Minio, Azure Blob Storage), потому что ими было проще управлять и дешевле содержать. Механизм MapReduce тоже вышел из употребления, уступив место другим механизмам распределенных запросов, таким как Apache Spark, Presto и Dremio. Остался только формат таблиц Hive, ставший стандартом для распознавания файлов в хранилище как отдельных таблиц, которые можно анализировать. Однако облачное хранилище требовало больших сетевых затрат для доступа к этим файлам, чего не предполагала архитектура формата Hive и что приводило к избыточным сетевым вызовам из-за зависимости Hive от структуры папок таблицы.

Особенностью озер данных, отличающей их от хранилищ данных, является возможность использовать разные вычислительные механизмы для разных рабочих нагрузок. Это важно, поскольку никогда не существовало универсального вычислительного механизма, который прекрасно справлялся бы с любыми рабочими нагрузками и мог масштабировать вычисления независимо от хранилища данных. Это ограничение заложено в самой природе вычислений, поскольку всегда есть компромиссы, и выбор компромиссов

определяет, для каких целей данная система подходит особенно хорошо, а для каких не очень.

Обратите внимание, что на самом деле в озерах данных нет сервиса, который удовлетворял бы потребности механизма хранения. Как правило, вычислительный механизм решает, как записывать данные, после чего данные никогда не проверяются и не оптимизируются, если только не перезаписываются целые таблицы или разделы, что обычно делается на разовой основе. На рис. 1.3 показано, как компоненты озера данных взаимодействуют друг с другом.

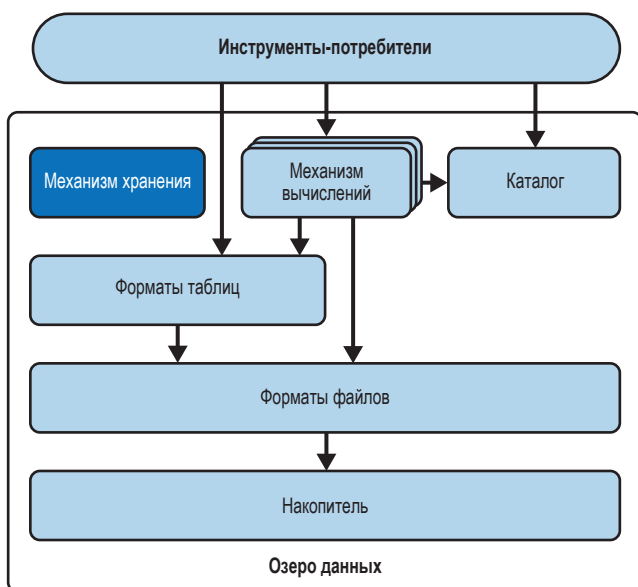


Рис. 1.3 ❖ Технические компоненты озера данных

Плюсы и минусы озер данных

Конечно, нет идеальных архитектурных шаблонов, не идеальны и озера данных. Несмотря на множество преимуществ, они также имеют ряд ограничений. Сначала перечислим преимущества.

Низкие затраты

Стоимость хранения данных и выполнения запросов в озере данных намного ниже, чем в хранилищах данных. Это делает озера данных особенно полезными для хранения аналитических данных.

Хранение данных в открытых форматах

Для хранения данных в озере можно использовать любые форматы файлов, тогда как хранилища данных не дают такой возможности и обычно определяют свой формат, разработанный специально для конкретного

хранилища. Это обеспечивает более полный контроль над данными и позволяет использовать более широкий спектр инструментов, поддерживающих эти форматы.

Поддержка неструктурированных данных

Хранилища данных не могут обрабатывать неструктурированные данные, такие как информация с датчиков, вложения в электронные письма или файлы журналов, поэтому озеро данных остается единственным вариантом, когда возникает необходимость выполнить анализ неструктурированных данных.

А теперь ограничения.

Невысокая производительность

Компоненты озера данных слабо связаны, поэтому многие оптимизации, применяемые в тесно связанных системах, такие как индексы и гарантии ACID (Atomicity, Consistency, Isolation, Durability – атомарность, согласованность, изоляция, долговременность). В принципе, можно собрать компоненты (хранилище, формат файлов, формат таблиц, механизмы) так, чтобы обеспечить производительность, сравнимую с производительностью хранилища данных, но это потребует слишком много усилий и технических средств. По этой причине озера данных не подходят для создания высокопроизводительных аналитических систем, в которых важны производительность и время.

Большое количество настроек

Как упоминалось выше, создание более тесной связи между компонентами озера данных, чтобы добиться уровня оптимизации, сопоставимого с уровнем хранилища данных, потребует значительных усилий по реорганизации. Это приведет к тому, что для настройки всех этих инструментов понадобится нанять множество инженеров, что повлечет еще большее удорожание.

Отсутствие гарантий ACID

Один из особенно заметных недостатков озер данных – отсутствие встроенных гарантий ACID транзакций, которые имеются в традиционных реляционных базах данных. Нередко запись данных в озеро производится по принципу «структурирование при чтении», т. е. проверка схемы и согласованности производится во время обработки данных, а не во время их сохранения. Это может создать проблемы для приложений, требующих строгой целостности транзакций, таких как финансовые системы или приложения, работающие с конфиденциальными данными. Для достижения аналогичных транзакционных гарантий в озере данных обычно необходимо внедрять сложные конвейеры обработки и механизмы координации, что увеличивает затраты на разработку и поддержку. Озера данных отличаются прекрасной масштабируемостью и гибкостью, но могут оказаться не лучшим выбором, когда строгое соответствие ACID является основным требованием.

Все эти плюсы и минусы кратко перечислены в табл. 1.2.

Таблица 1.2. Плюсы и минусы озер данных

Плюсы	Минусы
Низкие затраты	Низкая производительность
Хранение данных в открытых форматах	Отсутствие гарантий ACID
Поддержка неструктурированных данных	Большое количество настроек
Поддержка сценариев с участием машинного обучения	

Где должна производиться аналитическая обработка – в озере или в хранилище данных?

Озера данных прекрасно подходят для хранения любых структурированных и неструктурированных данных, но они имеют существенные недостатки. После запуска конвейера ETL (Extract, Transform, Load – извлечение, преобразование и загрузка) для размещения данных в озере обычно выбирается один из двух вариантов выполнения анализа.

Например, чтобы получить производительность и гибкость хранилища данных, настраивается дополнительный конвейер ETL для создания в хранилище данных копии курируемого подмножества данных, предназначенного для высокоприоритетной аналитики.

Однако это влечет за собой ряд проблем:

- дополнительные затраты на вычисления в конвейере ETL и хранение копии в хранилище данных, где затраты на хранение часто выше;
- также может потребоваться создать дополнительные копии для заполнения киосков данных (data mart) для различных направлений бизнеса, в форме выборок бизнес-аналитики с целью ускорения работы информационных панелей, что повлечет создание сети копий данных, которыми трудно управлять и синхронизировать.

Как вариант для выполнения запросов к озеру данных можно использовать механизмы запросов, такие как Dremio, Presto, Apache Spark, Trino и Apache Impala, поддерживающие озерные рабочие нагрузки. Эти механизмы хорошо подходят для обслуживания рабочих нагрузок, осуществляющих только чтение. Однако из-за ограничений формата таблиц Hive они испытывают сложности при попытке безопасного обновления данных из озера.

Как видите, озера и хранилища данных имеют свои уникальные достоинства и недостатки. Это вызвало необходимость разработки новой архитектуры, которая имела бы как можно больше преимуществ и как можно меньше недостатков. Эта архитектура получила название data lakehouse – озерное хранилище данных.

Озерное хранилище данных

Хранилища данных дают нам производительность и простоту использования, а озера данных – низкие затраты, возможность использования открытых форматов, поддержку неструктурированных данных и многое другое. Попытки преодолеть трудности нередко приводят к появлению успешных инновационных решений, и одно из таких решений – озерное хранилище данных.

Архитектура озерного хранилища данных отделяет механизмы хранения и вычислений от озерной архитектуры и вводит механизмы, обеспечивающие больше функций, подобных функциям хранилищ данных (транзакции ACID, улучшенная производительность, согласованность и т. д.). Эти улучшения обеспечивают форматы таблиц в озере данных, которые устраняют все предыдущие проблемы, связанные с форматом таблиц Hive. Данные хранятся в тех же местах и в тех же форматах, что и в озере данных, для доступа к ним используются те же механизмы запросов. И только формат таблиц, представляющий уровень метаданных / абстракции между механизмом хранения и накопителем, действительно превращает мир данных «только для чтения» в озерное хранилище – «центр моего мира данных», обеспечивая более интеллектуальные взаимодействия между ними (рис. 1.4).

Форматы таблиц создают уровень абстракции поверх хранилища файлов, обеспечивая лучшую согласованность, производительность и гарантии ACID при работе с данными непосредственно в озерном хранилище, что приводит к нескольким ценным предложениям:

Меньше копий = ниже вероятность дрейфа данных.

Благодаря гарантиям ACID и более высокой производительности озерное хранилище может обслуживать рабочие нагрузки, которые используют особенности, присущие хранилищам данных, например возможность обновления данных и выполнения других манипуляций с ними. Переместив данные в озерное хранилище, вы получите более оптимизированную архитектуру с меньшим количеством копий. Уменьшение числа копий снижает затраты на хранение и вычисление при перемещении данных, минимизирует вероятность дрейфа (изменение/разрушение модели в разных версиях одних и тех же данных) и упрощает управление данными.

Быстрая обработка запросов = быстрая аналитика.

Конечная цель всегда состоит в том, чтобы получить выгоду для бизнеса за счет качественного анализа данных. Все остальное – лишь шаги к этой цели. Ускорение обработки запросов позволяет быстрее получать информацию. Озерные хранилища обрабатывают запросы быстрее, чем озера данных, и сопоставимы по этому показателю с хранилищами данных благодаря оптимизации механизма запросов (оптимизаторы и кеширование), формату таблиц (улучшенный обход файлов и планирование запросов с использованием метаданных) и формату файлов (сортировка и сжатие).

Снимки исторических данных = минимизация ущерба от ошибок.

Форматы таблиц озерного хранилища поддерживают создание снимков исторических данных, что дает возможность запрашивать и восстанавливать таблицы до их предыдущих состояний. Вы можете работать со своими данными, не опасаясь, что непреднамеренная ошибка приведет к часам разбирательств, исправления и восстановления данных.

Доступная архитектура = ценность для бизнеса.

Есть два пути увеличения прибыли: увеличить выручку и снизить затраты. Озерные хранилища данных помогут не только увеличить выручку за счет бизнес-аналитики, но и снизить затраты на хранение копий данных и вычисление в конвейерах ETL, создающих эти копии.

Открытая архитектура = душевное спокойствие.

Озерные хранилища данных основываются на открытых форматах, таких как Apache Iceberg (формат таблиц) и Apache Parquet (формат файлов). Эти форматы поддерживаются многими инструментами, что позволяет избежать привязки к какому-то одному поставщику. Привязка к поставщику приводит к увеличению затрат и ограничению круга доступных инструментов, когда данные хранятся в форматах, не поддерживаемых сторонними инструментами. Используя открытые форматы, вы можете быть спокойны, зная, что при работе с данными не ограничены каким-то узким кругом инструментов.

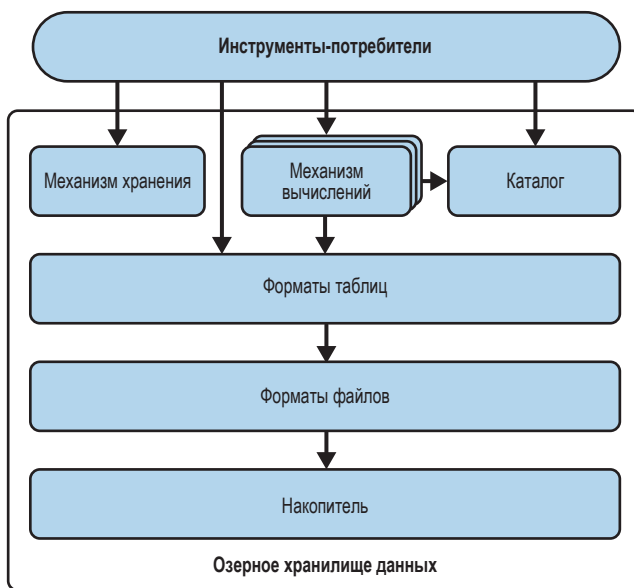


Рис. 1.4 ❖ Технические компоненты озерного хранилища данных

В заключение можно сказать, что благодаря современным инновациям открытых стандартов лучший из миров может работать в озере данных, и этот

архитектурный шаблон называется data lakehouse – озерное хранилище данных. Ключевым компонентом этого вида хранилищ является формат таблиц, обеспечивающий поддержку гарантий и более высокой производительности по сравнению с озерами. Теперь давайте перейдем к формату таблиц Apache Iceberg.

Что такое формат таблиц?

Формат таблиц – это метод структурирования файлов данных для представления их в виде единой «таблицы». С точки зрения пользователя это можно определить как ответ на вопрос «какие данные находятся в этой таблице?».

Этот ответ позволяет нескольким людям и инструментам одновременно взаимодействовать с данными в таблице, независимо от того, читают ли они таблицу или записывают в нее. Основная цель формата таблиц – дать пользователям и инструментам абстракцию таблицы и упростить эффективное взаимодействие с данными.

Форматы таблиц существовали с момента появления таких СУБД, как System R, Multics и Oracle, первыми реализовавших реляционную модель Эдгара Кодда (Edgar Codd), хотя в то время не существовало термина «формат таблиц». В этих системах пользователи могли обращаться к набору данных как к таблице, а ядро базы данных отвечало за управление физическим хранением байтов на диске в виде файлов и работу сложных механизмов, таких как транзакции.

Все взаимодействия с данными в этих СУБД, такие как чтение и запись, управляются механизмом хранения. Никакой другой механизм не может взаимодействовать с файлами напрямую. Детали хранения данных в накопителе абстрагированы, и пользователи считают само собой разумеющимся, что платформа знает, где находятся данные для конкретной таблицы и как получить к ним доступ.

Однако в современном мире больших данных полагаться на единый закрытый механизм для управления доступом к базовым данным стало непрактичным. Данные должны быть доступны различным вычислительным механизмам, оптимизированным для различных вариантов использования, таких как бизнес-аналитика или машинное обучение.

В озере все данные хранятся в виде файлов в некотором решении хранения (например, Amazon S3, Azure Data Lake Storage [ADLS], Google Cloud Storage [GCS]), поэтому одна таблица может состоять из десятков, сотен, тысячи и даже миллионов отдельных файлов. При использовании SQL с нашими любимыми аналитическими инструментами или написании специальных сценариев на таких языках, как Java, Scala, Python и Rust, было бы нежелательно постоянно проверять, какие из файлов находятся в таблице, а какие нет, т. к. это не только утомительно, но и может повлечь несогласованность при различных способах использования данных.

Поэтому было решено создать стандартный для озер данных метод ответа на вопрос «какие данные находятся в этой таблице» (рис. 1.5).

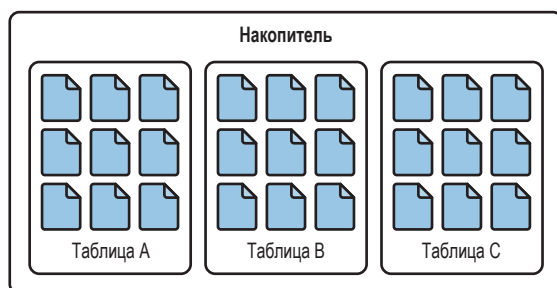


Рис. 1.5 ❖ Файлы данных, организованные в таблицы с помощью формата таблиц

Hive: оригинальный формат таблиц

На первых порах для реализации анализа информации в озерах данных Hadoop использовался фреймворк MapReduce, требовавший от пользователей писать довольно сложные задания на Java. Понимая этот недостаток, в 2009 году в Facebook был разработан фреймворк под названием Hive. Главным преимуществом Hive, упрощающим реализацию анализа в Hadoop, была возможность писать простые запросы SQL вместо заданий MapReduce.

Фреймворк Hive способен принимать инструкции SQL и преобразовывать их в задания MapReduce для последующего выполнения. Но для написания инструкций SQL необходим механизм, позволяющий определить, какие данные в хранилище Hadoop относятся к конкретной таблице. Так появились формат таблиц Hive и механизм определения принадлежности данных к таблицам Hive Metastore.

Формат таблиц Hive основан на определении таблицы как коллекции всех файлов в указанном каталоге (или префиксах в хранилищах объектов), как показано на рис. 1.6. Подкаталоги образуют сегменты таблиц. Пути к каталогам, определяющим таблицы, отслеживаются сервисом *Hive Metastore*, к которому могут обращаться механизмы запросов, чтобы узнать, где искать данные, необходимые для обработки запроса.

Формат таблицы Hive имеет несколько преимуществ.

- Позволяет использовать более эффективные шаблоны выполнения запросов, чем полное сканирование таблицы, в том числе секционирование (разделение данных на основе ключа секционирования) и сегментирование (подход к секционированию или кластеризации/сортировке с использованием хеш-функции для равномерного распределения значений).

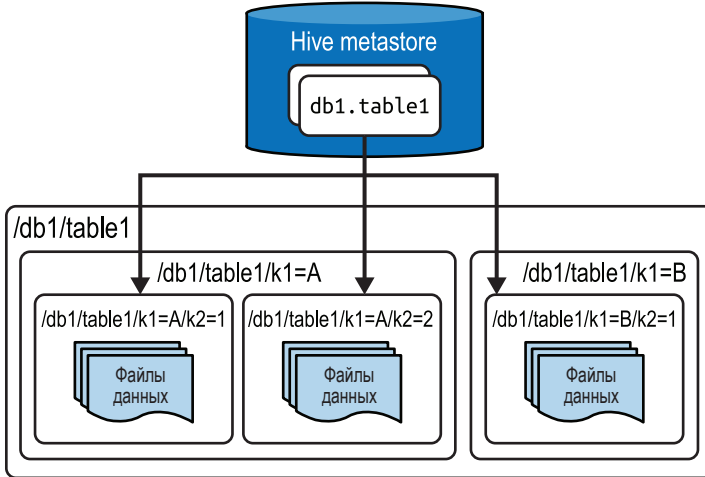


Рис. 1.6 ❖ Организация хранения таблицы в формате Hive

- Не зависит от формата файла, что позволило с течением времени разработать более совершенные форматы файлов, такие как Apache Parquet, и использовать их в таблицах Hive. Также отсутствовала необходимость преобразования данных, перед тем как они станут доступны в таблице Hive (например, Avro, CSV/TSV).
- Посредством атомарных операций замены указанного каталога в Hive Metastore можно атомарно вносить изменения в отдельные разделы таблиц.
- Со временем стал фактическим стандартом для большинства инструментов обработки данных и дающим единый ответ на вопрос «какие данные содержатся в этой таблице?».

Помимо этих весьма значимых преимуществ имело место и множество ограничений, ставших очевидными со временем.

- Изменения на уровне файла неэффективны из-за отсутствия механизма атомарной замены файла, подобного механизму замены каталога. По сути, для атомарной замены одного файла приходилось использовать замену на уровне каталога раздела.
- Несмотря на наличие возможности атомарно заменить раздел, не существовало механизма атомарного обновления нескольких разделов в одной транзакции. Вследствие этого конечные пользователи могут увидеть противоречивые данные между транзакциями, обновляющими несколько разделов.
- На самом деле не существует идеальных механизмов, обеспечивающих одновременное обновление, особенно с помощью инструментов, выходящих за рамки самого Hive.
- Механизм перечисления файлов и каталогов требовал много времени и замедлял запросы. Необходимость читать и перечислять файлы

и каталоги, которые могут не нуждаться в сканировании в запросе, обходится слишком дорого.

- Столбцы разделов часто создавались на основе других столбцов, например столбец месяца получался из отметки времени. Секционирование помогало, только если выполнялась фильтрация по столбцу раздела, но тот, кто фильтровал по столбцу отметки времени, мог не понимать, что нужно также фильтровать и по производному столбцу месяца. Такое непонимание приводило к полному сканированию таблицы, поскольку преимущества секционирования не были использованы.
- Статистика таблиц собиралась с помощью асинхронных заданий, что часто приводило к недоступности таблицы со статистикой. Это затрудняло дальнейшую оптимизацию запросов.
- Поскольку объектное хранилище часто ограничивает запросы одним префиксом (представьте, что префикс объектного хранилища аналогичен каталогу в файловой системе), запросы к таблицам с большим количеством файлов в одном разделе (чтобы все файлы находились в одном префиксе) могут порождать проблемы с производительностью.

Чем больше масштаб наборов данных и вариантов использования, тем больше усугубляются эти проблемы. Это привело к значительным трудностям, требующим нового решения, поэтому были созданы новые форматы таблиц.

Современные форматы таблиц озер данных

В попытках избавиться от ограничений формата таблиц Hive было создано новое поколение форматов таблиц, реализующих разные подходы к решению проблем, описанных выше.

Создатели современных форматов таблиц осознали, что недостатки формата Hive, которые привели к проблемам, заключались в том, что содержимое таблиц определялось содержимым каталогов, а не отдельных файлов. Современные форматы таблиц, такие как Apache Iceberg, Apache Hudi и Delta Lake, использовали другой подход и определяли содержимое таблицы как канонический список, сообщающий, какие *файлы* входят в состав таблицы, а не какие *каталоги*. Этот более детализированный подход к определению «какие данные содержатся в этой таблице?» открыл двери для внедрения таких функций, как ACID-транзакции, путешествия во времени и многое другое.

Все современные форматы таблиц призваны обеспечить ряд основных преимуществ, отсутствующих в Hive:

- поддержка транзакций ACID, которые либо выполняются до конца, либо отменяются. В устаревших форматах, таких как формат таблиц Hive, многие транзакции не имели такой поддержки;
- поддержка безопасности транзакций при одновременном выполнении нескольких операций записи. При попытке одновременно выполнить

две или более операций записи в таблицу в дело вступает механизм, гарантирующий, что модуль, выполняющий запись вторым, будет знать и учитывать результаты, что сделали другие записи, чтобы обеспечить согласованность данных;

- улучшенный сбор статистики таблиц и метаданных, что позволяет механизму запросов эффективнее планировать сканирование и сканировать как можно меньше файлов.

Давайте теперь обратим внимание на Apache Iceberg и узнаем, как он появился.

Что такое Apache Iceberg?

Apache Iceberg – это формат таблиц, созданный в 2017 году Райаном Блю (Ryan Blue) и Дэниелом Уиксом (Daniel Weeks) из Netflix. Его разработка была начата с целью преодолеть проблемы с производительностью, согласованностью и многие другие недостатки формата Hive, перечисленные выше. В 2018 году исходный код проекта был открыт и передан в дар Apache Software Foundation, где им начали заниматься многие другие организации, включая Apple, Dremio, AWS, Tencent, LinkedIn и Stripe. С тех пор в этот проект внесли свой вклад многие организации.

Как появился Apache Iceberg

Разработчики из компании Netflix, создавая формат Apache Iceberg, пришли к выводу, что многие проблемы формата Hive проистекают из одного простого, но фундаментального недостатка: каждая таблица как каталог и подкаталоги, что ограничивает уровень детализации, необходимый для гарантий согласованности, улучшения параллелизма и поддержки некоторых функций, доступных во многих хранилищах данных.

Помня об этом, в Netflix решили создать новый формат таблиц, преследуя несколько целей, перечисленных далее.

Согласованность

Если обновления таблицы затрагивают несколько ее разделов, то конечные пользователи должны видеть данные только в согласованном состоянии. Для этого обновление нескольких разделов таблицы должно выполняться быстро и атомарно. Пользователи должны видеть данные в состоянии либо до обновления, либо после обновления, но не между ними.

Производительность

Так как таблицы в формате Hive определяются содержимым каталогов и подкаталогов, планирование запроса перед его фактическим выполнением может занять слишком много времени. Таблица должна содержать метаданные и не выполнять ненужного перечисления файлов, чтобы уско-

рять не только планирование запроса, но и сами запросы, дабы в процессе их выполнения сканировались только те файлы, которые необходимы для удовлетворения запроса.

Простота использования

Чтобы воспользоваться преимуществами таких методов, как секционирование, конечным пользователям не обязательно знать физическую структуру таблицы. Таблица должна предоставлять пользователям преимущества секционирования на основе интуитивно понятных запросов и не зависеть от фильтрации дополнительных столбцов секционирования, полученных из столбца, по которому уже производится фильтрация (например, фильтрация по столбцу месяца после фильтрации по отметке времени, из которой этот столбец получен).

Развиваемость

Обновление схем таблиц Hive может привести к выполнению небезопасных транзакций, а обновление способа секционирования таблицы может привести к необходимости перезаписи всей таблицы. Должна иметься возможность безопасно и без перезаписи изменять схему таблицы и схему ее секционирования.

Масштабируемость

Все предыдущие цели должны быть достигнуты в петабайтном масштабе данных, характерном для Netflix.

Поэтому было решено создать формат Iceberg, определяющий таблицы как канонический список файлов, а не как список каталогов и подкаталогов. Проект Apache Iceberg – это спецификация или стандарт записи в несколько файлов метаданных, определяющих таблицу в озерном хранилище данных. Для поддержки этого стандарта в Apache Iceberg имеется множество вспомогательных библиотек. Наряду с этими библиотеками в рамках проекта были созданы реализации вычислительных механизмов с открытым исходным кодом, таких как Apache Spark и Apache Flink.

Apache Iceberg стремится обеспечить соответствие стандарту существующих инструментов и предназначен для использования преимуществ имеющихся популярных механизмов хранения и вычислений в надежде, что существующие варианты будут поддерживать работу со стандартом. Цель этого подхода – позволить экосистеме имеющихся инструментов обработки данных обеспечить поддержку таблиц Apache Iceberg и сделать Iceberg стандартом распознавания таблиц в озерах данных. Для этого Apache Iceberg должен стать настолько вездесущим в экосистеме, чтобы превратиться в еще одну деталь реализации, о которой многим пользователям не придется думать. Они просто будут знать, что работают с таблицами, и им не нужно будет задумываться об их фактической организации, независимо от используемого инструмента. Это уже становится реальностью, так как многие инструменты настолько упрощают работу с таблицами Apache Iceberg, что конечным пользователям не требуется понимать базовый формат Iceberg. В конце концов,

благодаря автоматизированной оптимизации таблиц и инструментам ввода даже более технически подкованным пользователям, таким как инженеры по обработке данных, не придется много думать о базовом формате, и они смогут работать со своим озерным хранилищем данных так же, как они работали с хранилищами данных, даже не имея дела непосредственно со слоем хранения.

Архитектура Apache Iceberg

Apache Iceberg отслеживает секционирование таблицы, ее сортировку, схему и многое другое с помощью дерева метаданных, которое позволяет планировать запросы за более короткое время, чем при использовании устаревших шаблонов озер данных. На рис. 1.7 показано, как организовано это дерево метаданных.

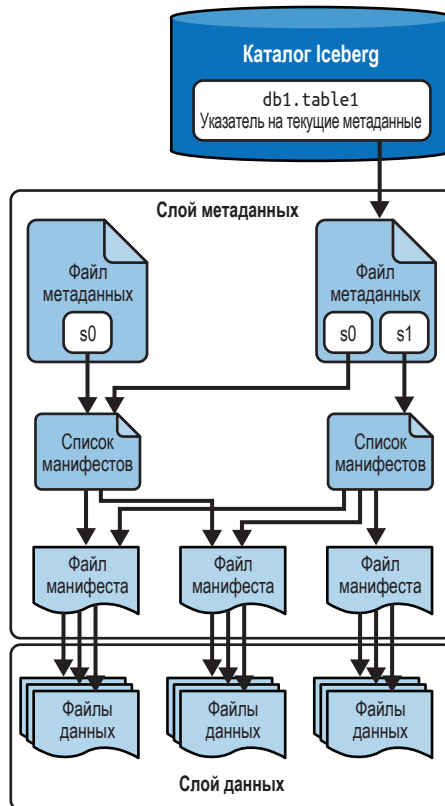


Рис. 1.7 ❖ Архитектура Apache Iceberg

Это дерево метаданных разбивает метаданные таблицы на четыре компонента:

Файл манифеста

Содержит список файлов данных, пути к ним и ключевые метаданные, описывающие эти файлы, что позволяет создавать более эффективные планы выполнения запросов.

Список манифестов

Содержит список файлов манифестов, определяющих один снимок таблицы, вместе со статистикой, что позволяет создавать более эффективные планы выполнения запросов.

Файлы метаданных

Файлы, определяющие структуру таблицы, включая ее схему, схему секционирования и список снимков.

Каталог

Отслеживает местоположение таблицы (подобно Hive Metastore), но отображает имя таблицы не в множество директорий, а в местоположение самого последнего файла с метаданными таблицы. В качестве каталога можно использовать несколько инструментов, в том числе Hive Metastore, и этой теме мы посвятили главу 5.

Все эти файлы подробно рассматриваются в главе 2.

Ключевые особенности Apache Iceberg

Уникальная архитектура Apache Iceberg позволяет постоянно расширять набор функций, выходящих за рамки простого решения задач с помощью Hive, и открывает совершенно новые возможности для озер данных и рабочих нагрузок, использующих их. В этом разделе мы бегло рассмотрим некоторые ключевые особенности Apache Iceberg и более подробно остановимся на них в последующих главах.

Транзакции ACID

Apache Iceberg использует оптимистический подход к управлению параллелизмом, чтобы обеспечить гарантии ACID, даже в случае, когда транзакции выполняются несколькими рабочими нагрузками, осуществляющими чтение и запись. Оптимистический параллелизм предполагает отсутствие конфликтов между транзакциями и проверяет наличие конфликтов только при необходимости, стремясь минимизировать блокировки и повысить производительность. Таким образом, транзакции в озере данных либо фиксируются, либо завершаются неудачей и не могут иметь никаких промежуточных состояний. Пессимистическая модель параллелизма, которая использует блокировки для предотвращения конфликтов между транзакциями, отсутствовала в Apache Iceberg на момент написания этой книги, но может появиться в будущем.

Гарантии параллелизма обеспечиваются каталогом, поскольку обычно этот механизм имеет встроенные гарантии ACID. Именно это позволяет транзакциям Iceberg выполняться атомарно и обеспечивать гарантии корректности. Если бы это было не так, то две разные системы могли бы произвести конфликтующие обновления, что привело бы к потере данных.

Эволюция разделов

Большой головной болью при работе с озерами данных до появления Apache Iceberg была необходимость физической оптимизации таблицы. Слишком часто, когда требовалось изменить секционирование, единственный выход заключался в том, чтобы переписать всю таблицу, но в масштабе эта операция может оказаться слишком дорогостоящей. Альтернативное решение – просто жить с существующей схемой секционирования и пожертвовать повышением производительности, которое могла бы дать более удачная схема секционирования.

С помощью Apache Iceberg можно в любое время обновить способ секционирования таблицы без полной ее перезаписи и всех ее данных. Поскольку секционирование напрямую связано с метаданными, операции, необходимые для внесения изменений в структуру таблицы, выполняются быстро и дешево.

На рис. 1.8 изображена таблица, которая изначально была секционирована по месяцам, а затем выполнено секционирование по дням. Ранее записан-

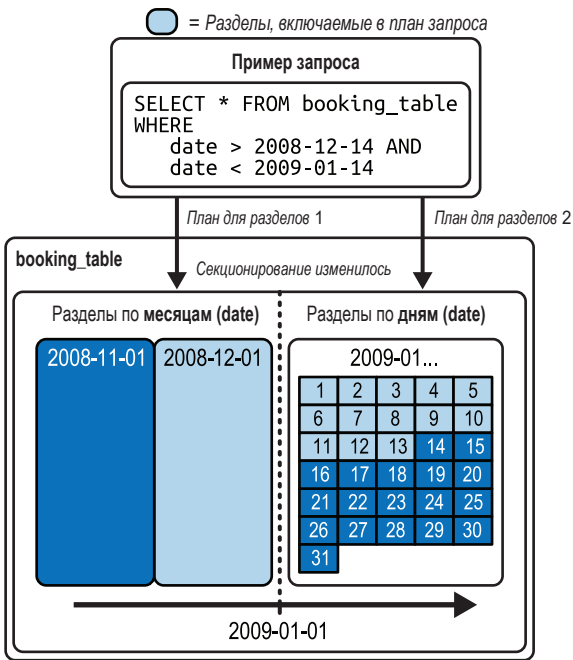


Рис. 1.8 ❖ Эволюция разделов

ные данные остаются в месячных разделах, а новые данные сохраняются в дневных разделах, поэтому, составляя план обработки, механизм запросов отдельно планирует выполнение для каждого из разделов, согласно схеме секционирования.

Скрытое секционирование

Иногда пользователи не знают, как физически секционирована таблица, и, по большому счету, их это не должно волновать. Часто таблица секционирована по некоторому полю временной метки, и пользователь хочет выполнить запрос по этому полю (например, получить средний доход по дням за последние 90 дней). Наиболее интуитивный для пользователя способ – включить фильтр `event_timestamp >= DATE_SUB(CURRENT_DATE, INTERVAL 90 DAY)`. Однако это приведет к полному сканированию таблицы, потому что таблица фактически секционирована по отдельным полям с именами `event_year`, `event_month` и `event_day`. Это объясняется тем, что секционирование по отметке времени приводит к созданию крошечных разделов, поскольку значения имеют степень детализации до секунд, миллисекунд и даже меньше.

Эта проблема решается благодаря особенностям обработки разделов в Apache Iceberg. В Iceberg секционирование учитывается в двух местах: в столбце, на котором основывается физическое секционирование, и при необязательном преобразовании этого значения в таких функциях, как `bucket`, `truncate`, `year`, `month`, `day` и `hour`. Возможность применения преобразования устраняет необходимость создания новых столбцов для секционирования. В результате секционирование становится более интуитивным для запросов, поскольку потребителям не нужно добавлять в свои запросы дополнительные предикаты фильтров по дополнительным столбцам секционирования.

Предположим, что таблица на рис. 1.9 секционирована по дням. Запрос, изображенный на рисунке, приведет к полному сканированию таблицы

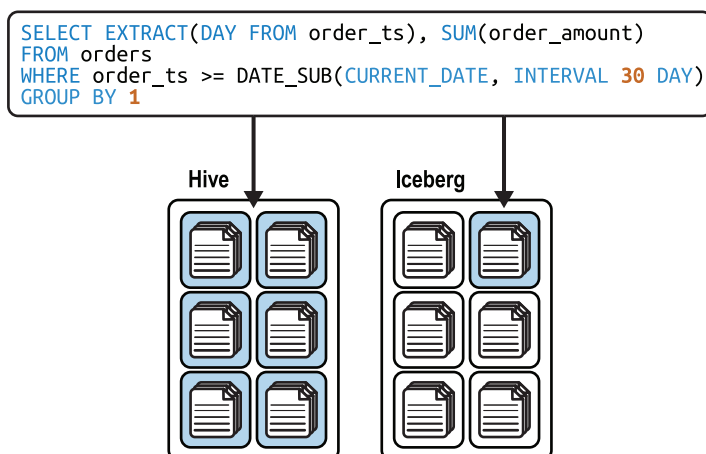


Рис. 1.9 ❖ Преимущества секционирования в Apache Iceberg

в Hive, поскольку для секционирования, вероятно, будет создан еще один столбец «day», тогда как в Iceberg метаданные будут отслеживать секционирование как «преобразованное значение CURRENT_DATE» и использовать секционирование при фильтрации по CURRENT_DATE (подробнее мы обсудим это далее в книге).

Операции с таблицами на уровне записей

Вы можете оптимизировать шаблоны обновления таблицы на уровне записей, приняв одну из двух форм: копирование при записи (copy-on-write, COW) или слияние при чтении (merge-on-read, MOR). При использовании COW изменение любой записи в файле данных приводит к перезаписи всего файла (с изменениями, внесенными в новый файл), даже если обновляется только одна запись. При использовании MOR любые обновления записываются только в новый файл, содержащий изменения в затронутой записи, которые согласовываются при чтении. Это дает ускорение тяжелых обновлений и удалений.

Путешествие во времени

Apache Iceberg предоставляет неизменяемые снимки, благодаря чему доступна информация об историческом состоянии таблицы, что позволяет выполнять запросы о состоянии таблицы в определенный момент времени в прошлом, т. е. выполнять то, что обычно называют *путешествием во времени*. Это может помочь в таких ситуациях, как составление отчетов на конец квартала без необходимости дублировать данные из таблицы в отдельном месте или воспроизводить выходные данные модели машинного обучения (Machine Learning, ML) на определенный момент времени (рис. 1.10).

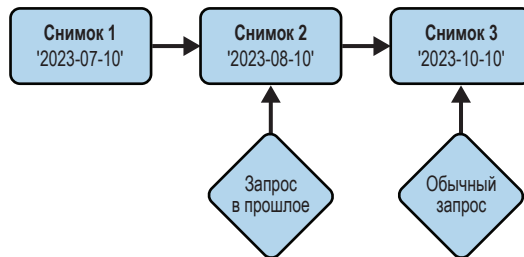


Рис. 1.10 ❖ Запрос состояния таблицы в прошлом

Откат версии

Изоляция моментальных снимков в Iceberg позволяет не только запрашивать данные в том виде, в каком они есть, но также состояние таблицы в любом из предыдущих снимков. Поэтому чтобы исправить ошибки, достаточно лишь откатиться назад (рис. 1.11).

ID	Name	Age	Date
1	Bob	46	Jan. 1st, 2023
2	Josie	65	Jan. 10th, 2023
3	Gene	30	Jan. 20th, 2023
4	Alex	37	Feb. 2nd, 2023
5	Tony	34	Feb. 15th, 2023

→ Откат

ID	Name	Age	Date
1	Bob	46	Jan. 1st, 2023
2	Josie	65	Jan. 10th, 2023
3	Gene	30	Jan. 20th, 2023

Рис. 1.11 ❖ Выполнение отката для перемещения таблицы в состояние, имевшее место в прошлом

Эволюция схемы

Таблицы изменяются при любой операции, такой как добавление или удаление столбца, переименование столбца или изменение типа данных столбца. Независимо от особенностей эволюции таблицы Apache Iceberg предоставляет надежные функции эволюции схемы, например обновление столбца `int` до столбца `long` по мере увеличения значений в столбце.

Заключение

Из этой главы вы узнали, что Apache Iceberg – это формат таблиц в озерном хранилище данных, привносящий множество улучшений, отсутствующих в Hive. Отказываясь от физической структуры файлов и многоуровневого дерева метаданных, Iceberg способен поддерживать транзакции Hive, транзакции ACID, эволюцию схемы, эволюцию разделов и предоставляет множество других функций, обеспечивающих создание озерного хранилища данных. Все это обеспечивается благодаря спецификациям и вспомогательным библиотекам, которые позволяют существующим инструментам обеспечивать поддержку формата открытых таблиц.

В главе 2 мы подробнее поговорим обо всем этом, углубившись в архитектуру Apache Iceberg.

Глава 2

Архитектура Apache Iceberg

В этой главе мы обсудим архитектуру и спецификацию, позволяющие Apache Iceberg решать проблемы, свойственные формату таблиц Hive, и исследуем внутреннее устройство Iceberg. Мы рассмотрим различные структуры Iceberg, а также поговорим о том, что позволяет каждая структура, чтобы вы могли понять, что происходит за кулисами, и могли осознанно проектировать свои озерные хранилища данных на основе Apache Iceberg.

Как упоминалось в главе 1, таблицы Apache Iceberg имеют три разных слоя: слой каталога, слой метаданных и слой данных. На рис. 2.1 показаны различные компоненты, составляющие каждый слой.

В следующих разделах мы подробно рассмотрим каждый из этих компонентов. Поскольку новые концепции легче изучать, основываясь на уже знакомых понятиях, мы пойдем снизу вверх, начав со слоя данных.

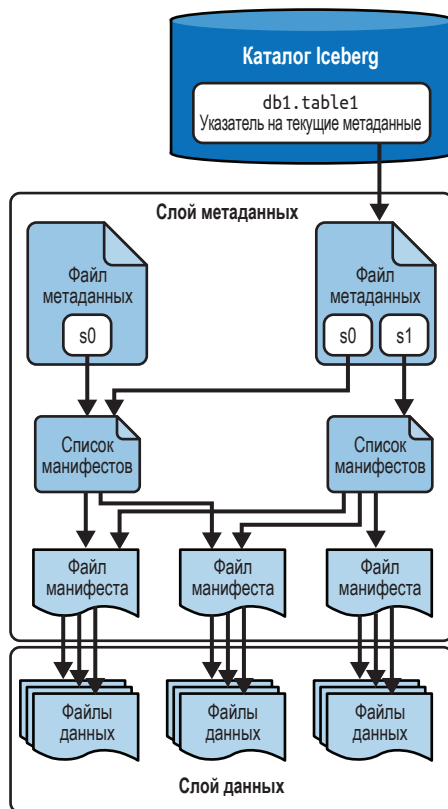


Рис. 2.1 ❖ Архитектура таблиц Apache Iceberg

Слой данных

Слой данных таблицы Apache Iceberg хранит фактические данные таблицы и в основном состоит из файлов самих данных, а также файлов удаления. Слой данных предоставляет данные в ответ на запрос пользователя. За некоторыми исключениями, слой метаданных тоже может возвращать результаты (например, максимальное значение для столбца X), однако чаще всего результаты извлекаются из слоя данных. Файлы слоя данных составляют листья древовидной структуры таблицы Apache Iceberg.

В реальных условиях слой данных поддерживается распределенной файловой системой (например, распределенной файловой системой Hadoop [Hadoop Distributed File System, HDFS]) или чем-то похожим на распределенную файловую систему, например объектным хранилищем (таким как Amazon Simple Storage Service [Amazon S3], Azure Data Lake Storage [ADLS], Google Cloud Storage [GCS]). Это позволяет строить архитектуры озерных хранилищ данных и получать выгоду от этих чрезвычайно масштабируемых и недорогих систем хранения.

Файлы данных

Файлы данных хранят сами данные. Apache Iceberg не зависит от формата этих файлов и в настоящее время поддерживает Apache Parquet, Apache ORC и Apache Avro. Это важно по следующим причинам:

- многие организации используют для хранения данных несколько форматов файлов, потому что разные группы могут или могли самостоятельно выбирать, какой формат использовать. Это также верно и для компаний, которые на некотором этапе своего развития выбрали другой формат;
- возможность выбора формата, который лучше подходит для конкретной рабочей нагрузки. Например, Parquet можно использовать для крупномасштабной оперативной аналитической обработки (OLAP), а Avro – для хранения таблиц потоковой аналитики с малой задержкой;
- гарантии независимости организации от формата файлов. Если будет разработан новый формат файла, который лучше подходит для набора рабочих нагрузок, то организация сможет использовать его для хранения таблиц Apache Iceberg.

Но, несмотря на независимость Apache Iceberg от формата файлов, в реальном мире чаще всего используется формат Apache Parquet, потому что его столбчатая структура обеспечивает значительный прирост производительности для рабочих нагрузок OLAP по сравнению с форматами файлов на основе записей, и он стал фактическим стандартом в отрасли, в том смысле, что практически каждый механизм и инструмент поддерживают Parquet. Его столбчатая структура закладывает основу для таких возможностей, влияющих на производительность, как деление одного файла на разделы несколь-

кими способами для повышения параллелизма, вычисление статистик для каждой из точек разделения и улучшенное сжатие, что помогает уменьшить хранимый объем и увеличить пропускную способность чтения.

На рис. 2.2 показано, как файл Parquet хранит набор записей («Группа записей 0» на рисунке), которые затем разбиваются так, что все значения из записей для данного столбца сохраняются вместе («Столбец А» на рисунке). Далее все значения для данного столбца разбиваются на подмножества значений из записей для данного столбца, которые называются страницами («Страница 0» на рисунке). Каждый из этих уровней может независимо и параллельно читаться механизмами и инструментами. Кроме того, Parquet

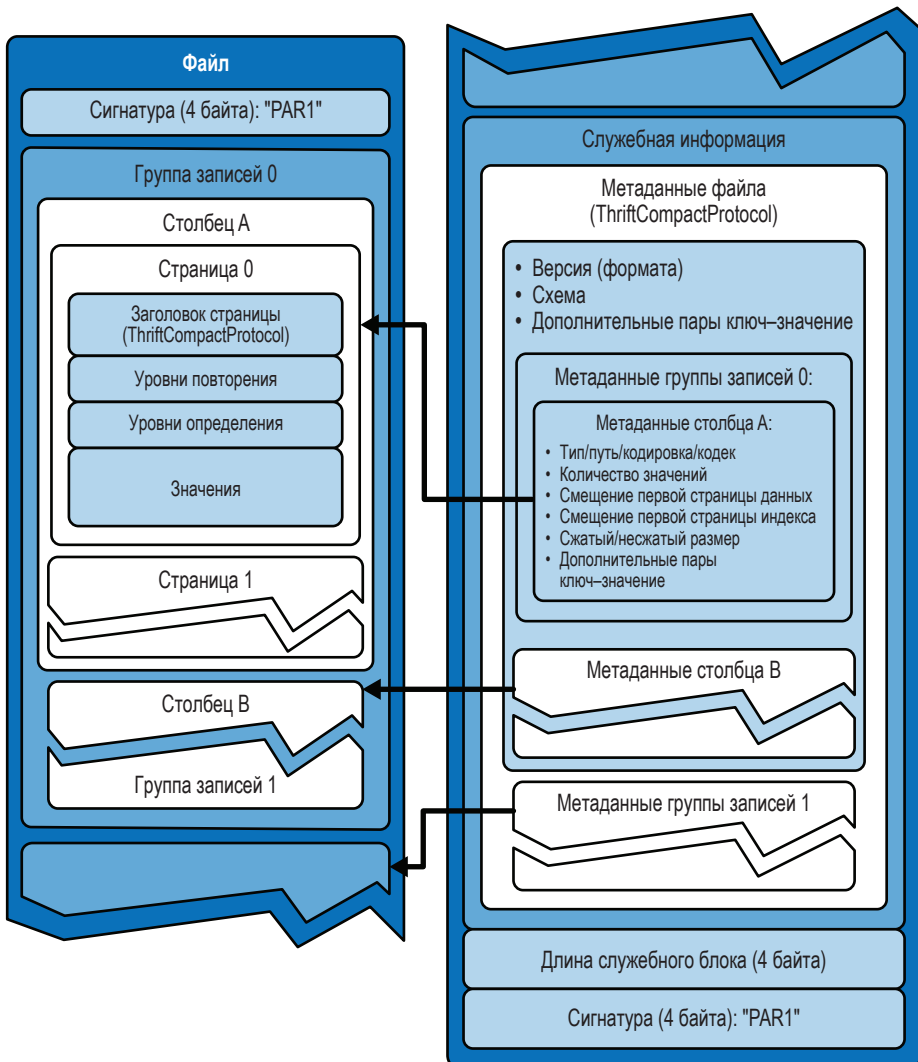


Рис. 2.2 ❖ Архитектура файла Parquet

хранит статистику (например, минимальные и максимальные значения для данного столбца в заданной группе записей), которая позволяет механизмам и инструментам решать, нужно ли читать все данные или можно пропустить группы записей, которые не подходят под условия запроса.

Файлы удаления

Файлы удаления хранят информацию об удаленных записях с данными. Поскольку хранилище озера данных рекомендуется рассматривать как неизменяемое, оно не поддерживает обновление записей прямо в файле. Вместо этого производится запись в новый файл, который может быть копией старого файла с внесенными изменениями (так называемое *копирование при записи* [copy-on-write, COW]) или содержать только изменения. В последнем случае файлы читаются механизмами обработки запросов и затем объединяются (так называемое *слияние при чтении* [merge-on-read, MOR]). Файлы удаления в Iceberg поддерживают стратегию MOR для операций изменения и удаления таблиц, т. е. файлы удаления применимы только к таблицам MOR (причину этого мы подробно объясним в главе 4). Обратите внимание, что файлы удаления поддерживаются только в формате Iceberg v2, который на момент написания этой книги поддерживался почти всеми инструментами Iceberg. На рис. 2.3 представлена упрощенная диаграмма, показывающая файлы данных таблицы MOR до и после операции удаления.

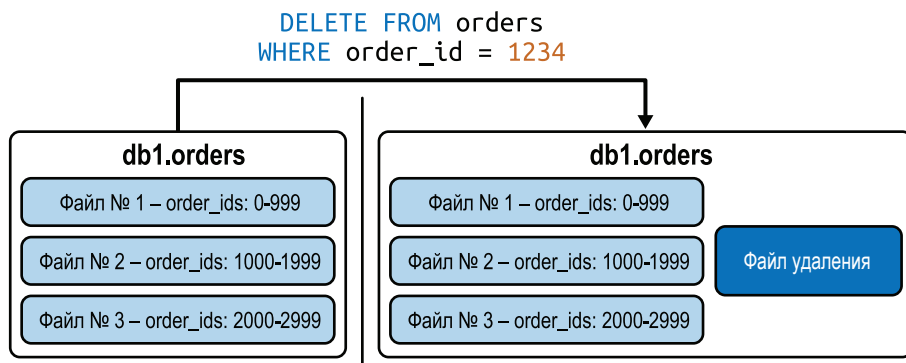


Рис. 2.3 ❖ Диаграмма, показывающая таблицу MOR до и после выполнения операции DELETE

Существует два способа идентификации записи, которую нужно удалить из логического набора данных, возвращаемого механизмом чтения: по ее точному положению в наборе данных или по значениям одного или нескольких полей. Соответственно, существует два типа файлов удаления. Файлы первого типа называют файлами позиционного удаления, а второго типа – файлами удаления по равенству.

Эти два подхода имеют свои плюсы и минусы и, следовательно, применимы в разных ситуациях. Мы подробно поговорим об этих ситуациях в главе 4, когда будем обсуждать COW и MOR, а пока приведем обобщенное описание.

Файлы позиционного удаления

Файлы позиционного удаления содержат информацию о том, какие записи были логически удалены. Сверяясь с этими файлами, механизм чтения данных удаляет их из своего представления таблицы, определяя точные позиции записей в таблице. Позиция определяется как путь к конкретному файлу, содержащему запись, и номер строки в этом файле.

На рис. 2.4 показано удаление строки с идентификатором `order_id`, равным 1234. Если предположить, что данные в файле отсортированы по возрастанию `order_id`, то искомая запись находится в файле № 2 в строке № 234 (обратите внимание: нумерация строк начинается с нуля, поэтому строка № 0 в файле № 2 – это запись с `order_id = 1000`, соответственно, строка № 234 в файле № 2 – это запись с `order_id = 1234`).

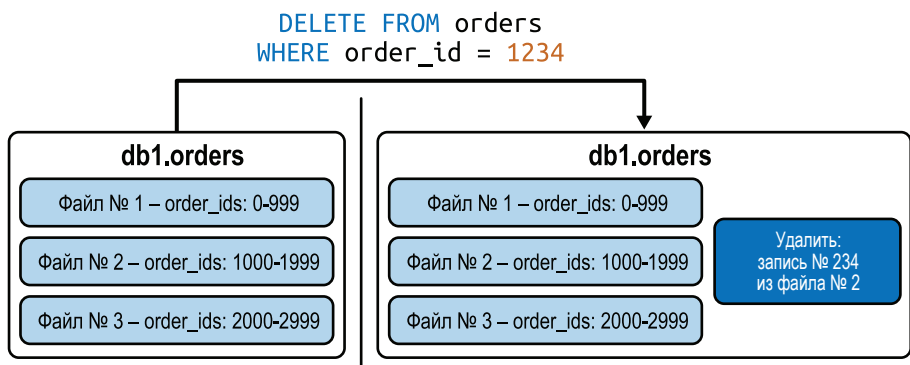


Рис. 2.4 ❖ Диаграмма, показывающая таблицу MOR, настроенную на позиционное удаление, до и после выполнения операции DELETE

Файлы удаления по равенству

Файлы удаления по равенству тоже содержат информацию о том, какие записи были логически удалены. Сверяясь с этими файлами, механизм чтения данных удаляет их из своего представления таблицы, сравнивая значения одного или нескольких полей. Этот тип файлов удаления лучше всего подходит в случаях, когда каждая запись в таблице имеет уникальный идентификатор (он же первичный ключ), однозначно идентифицирующий ее (представьте, например, операцию «удалить запись со значением `order_id = 1234`»). Однако, используя данный подход, можно также удалять сразу несколько записей (примером может служить операция «удалить все записи с `interaction_customer_id = 5678`»).

На рис. 2.5 показано удаление записи с идентификатором `order_id = 1234` с использованием файла удаления по равенству. Механизм записывает файл удаления, который говорит «удалить все записи с `order_id = 1234`», и этого правила затем придерживаются все механизмы чтения. Обратите внимание, что, в отличие от файлов позиционного удаления, здесь нет ссылки на местоположение записей в таблице.

Также отметьте, что может возникнуть ситуация, когда файл удаления по равенству предписывает удалить запись с некоторым значением столбца, а затем в одной из последующих транзакций запись с тем же значением столбца опять будет добавлена в набор данных. В такой ситуации было бы нежелательно, чтобы механизмы чтения удаляли вновь добавленную запись из логического набора данных. В Apache Iceberg эта проблема решается использованием порядковых номеров. Например, в ситуации, изображенной на рис. 2.5, в файле манифеста будет указано, что все файлы данных слева на рис. 2.5 имеют порядковый номер 1. Файл манифеста, соответствующий файлу удаления справа, получит порядковый номер 2. А файл данных, созданный при последующей вставке новой записи с `order_id = 1234`, получит порядковый номер 3. Таким образом, читая таблицу, механизм знает, что файл удаления должен применяться ко всем файлам данных, имеющим порядковый номер меньше 2 (порядковый номер файла удаления), и не должен применяться к файлам данных, имеющим порядковый номер 2 или выше. Такой прием поддерживает правильное состояние таблицы после внесения изменений с течением времени.

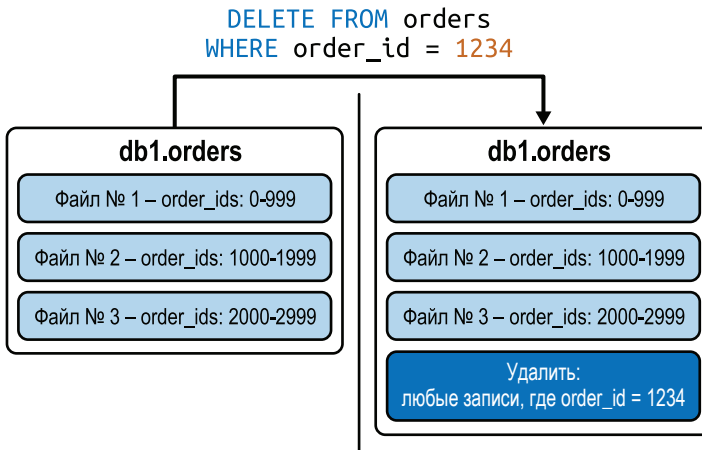


Рис. 2.5 ❖ Диаграмма, показывающая таблицу MOR, настроенную на позиционное удаление, до и после выполнения операции DELETE

Слой метаданных

Слой метаданных является неотъемлемой частью архитектуры и содержит все файлы метаданных таблиц Iceberg. Это древовидная структура, которая

отслеживает файлы данных и метаданные о них, а также операции, которые привели к их созданию. Она состоит из файлов трех типов, хранимых вместе с файлами данных: файлы манифестов, списки манифестов и файлы метаданных. Слой метаданных необходим для эффективного управления большими наборами данных и реализации основных возможностей, таких как путешествия во времени и эволюция схемы.

Файлы манифестов

Файлы манифестов отслеживают файлы в слое данных (т. е. файлы данных и файлы удаления), а также хранят дополнительные сведения и статистику о каждом файле, например минимальные и максимальные значения в столбцах в файлах данных. Как упоминалось в главе 1, основная особенность, помогающая Iceberg решать проблемы, свойственные формату таблиц Hive, – это отслеживание того, какие данные находятся в таблице на уровне файлов. Файлы манифестов – это файлы, обеспечивающие возможность такого отслеживания на конечном уровне дерева метаданных.



Файлы манифестов отслеживают файлы данных и файлы удаления, но для каждого из этих типов файлов используется отдельный набор файлов манифеста (т. е. один файл манифеста будет содержать информацию только о файлах данных или только о файлах удаления), хотя схемы файлов манифеста идентичны.

Каждый файл манифеста содержит информацию о подмножестве файлов данных, такую как сведения о принадлежности к разделам, о количестве записей, а также о нижней и верхней границах значений в столбцах. Эта информация используется для повышения эффективности и производительности обработки запросов. Некоторые из этих статистических данных также хранятся в самих файлах данных, но один файл манифеста хранит статистику для нескольких файлов данных, и это существенно снижает необходимость открытия множества файлов данных и способствует повышению производительности (даже простое чтение служебной информации из множества файлов данных может занять много времени). Этот процесс мы подробно исследуем в главе 3. Статистическая информация записывается во время операции записи в каждое подмножество файлов данных, отслеживаемых файлом манифеста.

Статистики сохраняются небольшими пакетами каждым механизмом для своего подмножества файлов данных, поэтому запись статистик выглядит гораздо проще по сравнению с форматом таблиц Hive, где статистики собираются и сохраняются в ходе длительного и дорогостоящего задания, которое читает весь раздел или таблицу и вычисляет и записывает статистики для всех данных в разделе/таблице. Это объясняется тем, что механизм записи уже обработал все записываемые данные, и поэтому ему проще выполнить сбор статистики при записи. На практике это означает, что задания по сбору статистики при использовании формата таблиц Hive перезапускаются не очень

часто (если вообще запускаются), что приводит к снижению производительности запросов, поскольку механизмы не имеют информации, необходимой для принятия обоснованных решений о том, как выполнить данный запрос. В результате таблицы Iceberg с гораздо большей вероятностью будут содержать актуальную и точную статистику, что помогает механизмам принимать более обоснованные решения и способствует увеличению производительности.

Пример полного содержимого файла манифеста (*Chapter_2/manifest-file.json*) можно найти в репозитории книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg>). Обратите внимание, что в Iceberg файлы манифестов имеют формат Avro, однако для удобства просмотра мы преобразовали содержимое примерного файла из формата Avro в формат JSON.

Списки манифестов

Список манифестов – это снимок таблицы Iceberg в определенный момент времени. Для состояния таблицы на заданный момент времени список манифестов содержит перечень всех файлов манифестов, включая местоположение, разделы, к которым они принадлежат, а также верхние и нижние границы столбцов разделов отслеживаемых файлов данных.

Список манифестов содержит массив структур, каждая из которых отслеживает один файл манифеста. Схема структуры приводится в табл. 2.1, заимствованной из общедоступной документации по Iceberg. Также обратите внимание, что эта информация относится к таблицам Iceberg v2.

Таблица 2.1. Схема файлов манифестов в Iceberg

Всегда имеется?	Имя поля	Тип данных	Описание
Да	manifest_path	string	Путь к файлу манифеста
Да	manifest_length	long	Размер файла манифеста в байтах
Да	partition_spec_id	int	Идентификатор спецификации раздела, использованной при записи в раздел; ссылается на элемент списка partition-specs в метаданных таблицы
Да	content	int, где 0 означает файл данных, а 1 – файл удаления	Тип файла, отслеживаемого манифестом: файл данных или файл удаления
Да	sequence_number	long	Порядковый номер, присваиваемый файлу манифеста при добавлении в таблицу
Да	min_sequence_number	long	Наименьший порядковый номер среди всех существующих файлов данных и файлов удаления в манифесте
Да	added_snapshot_id	long	Идентификатор снимка, в который был добавлен файл манифеста

Таблица 2.1 (окончание)

Всегда имеется?	Имя поля	Тип данных	Описание
Да	added_files_count	int	Количество записей в файле манифеста, которые имеют значение ADDED (1) в поле status
Да	existing_files_count	int	Количество записей в файле манифеста, которые имеют значение EXISTING (0) в поле status
Да	deleted_files_count	int	Количество записей в файле манифеста, которые имеют значение DELETED (2) в поле status
Да	added_rows_count	long	Суммарное количество записей во всех файлах, перечисленных в манифесте, которые имеют значение ADDED в поле status
Да	existing_rows_count	long	Суммарное количество записей во всех файлах, перечисленных в манифесте, которые имеют значение EXISTING в поле status
Да	deleted_rows_count	long	Суммарное количество записей во всех файлах, перечисленных в манифесте, которые имеют значение DELETED в поле status
Нет	partitions	array<field_summary> (см. табл. 2.2)	Список сводок по полям для каждого раздела в спецификации; каждое поле в списке соответствует полю в спецификации раздела в файле манифеста
Нет	key_metadata	binary	Метаданные ключа для шифрования; зависят от конкретной реализации

В табл. 2.2 показана схема структуры `field_summary`, упомянутой в предпоследней строке в табл. 2.1.

Таблица 2.2. Схема `field_summary`

Всегда имеется?	Имя поля	Тип данных	Описание
Да	contains_null	boolean	Содержит ли манифест хотя бы один раздел с пустым значением в поле
Нет	contains_nan	boolean	Содержит ли манифест хотя бы один раздел со значением NaN в поле
Нет	lower_bound	bytes	Нижняя граница для непустых значений и значений, отличных от NaN, в разделе или null, если все значения пустые или равны NaN; значение сериализуется в байты
Нет	upper_bound	bytes	Верхняя граница для непустых значений и значений, отличных от NaN, в разделе или null, если все значения пустые или равны NaN; значение сериализуется в байты

Пример полного содержимого списка манифеста (*Chapter 2/manifest-list.json*) можно найти в репозитории книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg>). Обратите внимание, что в Iceberg списки манифестов имеют формат Avro, однако для удобства просмотра мы преобразовали содержимое примерного файла из формата Avro в формат JSON.

Файлы метаданных

Списки манифестов отслеживаются с помощью файлов метаданных. Файлы метаданных хранят метаданные о состоянии таблиц Iceberg в определенный момент времени. Сюда входит информация о схеме таблицы, сведения о разделах, снимках и о том, какой снимок является текущим.

Каждый раз, когда в таблицу Iceberg вносятся изменения, создается новый файл метаданных, который атомарно регистрируется как последняя версия файла метаданных через каталог, структуру которого мы рассмотрим в следующем разделе. Это гарантирует линейность истории фиксаций таблиц и помогает в таких сценариях, как одновременная запись, то есть когда несколько механизмов записывают данные одновременно. Кроме того, благодаря такому решению механизмы чтения всегда будут видеть последнюю версию таблицы.

Схема файла метаданных подробно описана в табл. 2.3, заимствованной из общедоступной документации по Iceberg.

Таблица 2.3. Схема файла метаданных

Всегда имеется?	Имя поля	Тип данных	Описание
Да	format-version	integer	Целочисленный номер версии формата. В настоящее время может быть 1 или 2. Реализации должны генерировать исключение, если версия таблицы выше поддерживаемой версии. На момент написания этой книги значением по умолчанию было число 2
Да	table-uuid	string	UUID, идентифицирующий таблицу и генерируемый при ее создании. Реализации должны генерировать исключение, если UUID таблицы не совпадает с ожидаемым UUID после обновления метаданных
Да	location	string	Базовое местоположение таблицы. Используется механизмами записи, чтобы определить, где хранятся файлы данных, манифестов и метаданных таблиц
Да	last-sequence-number	long	Наибольший порядковый номер, присвоенный таблице. Это монотонно возрастающее целое число со знаком, отслеживающее порядковые номера снимков таблицы
Да	last-updated-ms	long	Отметка времени в миллисекундах от начала эпохи Unix последнего обновления таблицы. Каждый файл метаданных таблицы должен обновлять это поле непосредственно перед записью

Таблица 2.3 (продолжение)

Всегда имеется?	Имя поля	Тип данных	Описание
Да	last-column-id	integer	Наибольший идентификатор столбца в таблице. Используется, чтобы гарантировать, что при изменении схем новым столбцам всегда будет присваивать неиспользуемый идентификатор
Да	schemas	array	Список схем, хранящихся в виде объектов с идентификатором schema-id
Да	current-schema-id	integer	Идентификатор текущей схемы таблицы
Да	partition-specs	array	Список спецификаций разделов, хранящийся в виде полных объектов спецификаций
Да	default-tspec-id	integer	Идентификатор «текущей» спецификации, которая должна использоваться по умолчанию механизмами записи
Да	last-partition-id	integer	Наибольший присвоенный идентификатор поля раздела по всем спецификациям разделов для таблицы. Используется, чтобы гарантировать присваивание разделам неиспользуемых идентификаторов
Нет	properties	map	Карта свойств таблицы. Используется для управления настройками, влияющими на чтение и запись, и не предназначена для хранения произвольных метаданных. Например, commit.retry.num-retries используется для управления количеством повторных попыток фиксации
Нет	current-snapshot-id	long	Идентификатор текущего снимка таблицы. Должен совпадать с текущим идентификатором ветви main в refs
Нет	snapshots	array	Список действительных снимков. Действительные снимки – это снимки, для которых существуют все файлы данных. Файлы данных должны существовать до тех пор, пока сборщик мусора не удалит последний снимок, в котором они указаны
Нет	snapshot-log	array	Список пар отметок времени и идентификаторов снимков, который кодирует изменения в текущем снимке таблицы. Каждый раз, когда изменяется current-snapshot-id, необходимо добавить новую запись с новыми значениями в полях last-updated-ms и current-snapshot-id. Когда срок действия снимка истекает, список действительных снимков, все записи перед снимком, срок действия которого истек, должны удаляться
Нет	metadata-log	array	Список пар отметок времени и местоположений файлов метаданных, который кодирует изменения в предыдущих файлах метаданных таблицы. Каждый раз, когда создается новый файл метаданных, в список должна добавляться новая запись о местоположении предыдущего файла метаданных. Таблицы можно настроить так, чтобы обеспечить удаление самых старых журнальных записей из журнала метаданных для сохранения фиксированного размера журнала

Таблица 2.3 (окончание)

Всегда имеется?	Имя поля	Тип данных	Описание
Да	sort-orders	array	Список порядков сортировки в виде объектов, описывающих полный порядок сортировки
Да	default-sort-order-id	integer	Идентификатор порядка сортировки по умолчанию для таблицы. Обратите внимание, что это может использоваться при записи, но не используется при чтении, потому что при чтении используются спецификации, хранящиеся в манифестах
Нет	refs	map	Карта ссылок на снимки. В роли ключей карты используются уникальные имена ссылок на снимки таблицы, а в роли значений – объекты ссылок на снимки. Всегда существует ссылка на ветвь main, указывающая на current-snapshot-id, даже если rfhnf refs пустая
Нет	statistics	array	Список (необязательный) со статистиками таблицы

Пример полного содержимого файла метаданных (*Chapter_2/metadata-file.json*) можно найти в репозитории книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg>).

Puffin-файлы

Несмотря на то что в файлах данных и файлах удаления существуют структуры для ускорения взаимодействий с данными в таблице, иногда для увеличения производительности определенных типов запросов бывают нужны более сложные структуры.

Например, представьте, что вам понадобилось узнать количество клиентов, разместивших у вас заказ за последние 30 дней. Этот сценарий, как мы увидим далее, охватывает статистика в файлах данных, а не в файлах метаданных. Вы могли бы использовать эту статистику для некоторого увеличения производительности (например, отсеять данные за последние 30 дней), но вам все равно придется прочитать каждый заказ, размещенный в течение этих 30 дней, и выполнить агрегирование, что может занять некоторое время, в зависимости от таких факторов, как размер записи данных, ресурсы, выделенные механизму чтения, и количество полей в записи.

В таких случаях на помощь можно призвать puffin-файлы. *Puffin-файлы* хранят статистику и индексы данных в таблице, помогающие повысить производительность еще более широкого спектра запросов, подобных вышеупомянутому примеру, чем это позволяет статистика в файлах данных и метаданных.

Файл содержит наборы произвольных последовательностей байтов, называемых *большими двоичными объектами* (binary large object, blob), а также

сопутствующие метаданные, необходимые для анализа этих объектов. Структура puffin-файла показана на рис. 2.6.

Такая организация позволяет использовать статистические и индексные структуры любого типа (например, фильтры Блума), но в настоящее время поддерживается только один тип – Theta Sketch (<https://oreil.ly/YLM8A>) из библиотеки Apache DataSketches. Эта структура позволяет вычислить *приблизительное* количество различных значений столбца в заданном множестве записей и тем самым ускорить вычисления и уменьшить потребление ресурсов, зачастую на порядок. Она может пригодиться в операциях вычисления количества разных значений в столбце (например, количество пользователей по регионам), когда вычисление точного значения требует слишком много ресурсов или времени. Также она может оказаться полезной в сценариях, допускающих приближенные вычисления, особенно когда эти вычисления выполняются многократно.

Каталог

Любой механизм чтения таблиц должен с чего-то начать – куда-то пойти, чтобы узнать, где находится нужная ему таблица. Первым шагом для любого механизма, начинающего взаимодействовать с таблицей, является поиск местоположения файла метаданных, который содержит указатель на текущие метаданные.

Центральным местом, где можно найти указатель на текущие метаданные, является *каталог* Iceberg. Основное требование к каталогу – поддержка атомарных операций изменения указателя на текущие метаданные. Атомарность необходима для того, чтобы в каждый конкретный момент времени все механизмы чтения и записи видели одно и то же состояние таблицы.

Для каждой таблицы в каталоге имеется ссылка, или указатель, на текущий файл метаданных этой таблицы. Например, на рис. 2.1 показаны два файла метаданных. Значение указателя на текущие метаданные таблицы в каталоге – это местоположение файла метаданных.

Поскольку единственные требования к каталогу Iceberg заключаются в том, что он должен хранить указатель на текущие метаданные и гарантировать атомарность его изменения, существует множество различных реализаций, которые могут играть роль каталога Iceberg. Однако разные каталоги по-разному хранят указатель на текущие метаданные. Далее перечислено несколько примеров таких реализаций.



Рис. 2.6 ❖ Структура puffin-файла

- При использовании Amazon S3 в папке с метаданными таблицы создается файл *version-hint.text*, содержащий номер версии текущего файла метаданных. Обратите внимание: каждый раз, используя распределенную файловую систему (или нечто подобное) для хранения указателя на текущие метаданные, используемый каталог фактически называется каталогом *hadoop*.
- При использовании Hive Metastore в записи, соответствующей таблице, имеется свойство *location*, в котором хранится местоположение текущего файла метаданных.
- При использовании Nessie в записи, соответствующей таблице, имеется свойство *metadataLocation*, в котором хранится местоположение текущего файла метаданных.

В предыдущих примерах файлов манифестов, списков манифестов и файлов метаданных мы использовали каталог AWS Glue Catalog. В метаданных Iceberg о таблице, хранящихся в этом каталоге, можно увидеть, что известно о текущем файле метаданных. Следующий запрос вернет подробную информацию о текущем состоянии таблицы, в том числе и о местоположении текущего файла метаданных:

```
SELECT *
FROM my_catalog.iceberg_book.orders.metadata_log_entries
ORDER BY timestamp DESC
LIMIT 1
```

Timestamp	Metadata File	Latest Snapshot ID	Latest Schema ID	Latest Sequence Number
2023-03-21 22:55:31.868	s3://jason-dremio-product-us-west-2/iceberg-book/iceberg_book.db/orders/metadata/00002-509f0747-4dc4-4965-b354-ce5fb747c2f5.metadata.json	8619686881304977663	0	2

То есть чтобы прочитать данные из этой таблицы, то, согласно каталогу Glue, мы должны сделать еще шаг и получить файл метаданных *s3://jason-dremio-product-us-west-2/iceberg-book/iceberg_book.db/orders/metadata/00002-509f0747-4dc4-4965-b354-ce5fb747c2f5.metadata.json*.

Заключение

В этой главе мы познакомились с архитектурой и форматом таблиц Apache Iceberg, которые помогают решать проблемы, свойственные формату таблиц

Hive, и реализовывать такие возможности, как поддержка транзакций ACID в озере данных. Мы рассмотрели три слоя – слой данных, слой метаданных и каталог, – а также структуры соответствующих файлов, которые используются механизмами и инструментами для эффективного доступа к данным и реализации более мощных возможностей, таких как путешествия во времени и эволюция схемы.

В главе 3 мы обсудим жизненный цикл запросов, выполняемых этими механизмами и инструментами, и посмотрим, как используются эти файлы и структуры.

Глава 3

Жизненный цикл запросов на запись и чтение

Формат таблиц Apache Iceberg обеспечивает высокую скорость обработки запросов на запись и чтение, что позволяет запускать рабочие нагрузки оперативной аналитической обработки (OLAP) непосредственно в озере данных. Этому также способствуют особенности организации различных компонентов формата Iceberg. Поэтому очень важно понимать структуру этих компонентов, чтобы писать механизмы запросов, способные эффективно использовать их для быстрого планирования и выполнения запросов. Мы уже обсуждали эти архитектурные компоненты в главе 2. На высоком уровне эти компоненты можно разделить на три слоя, как показано на рис. 3.1.

Давайте кратко рассмотрим, как механизм запросов взаимодействует с этими компонентами при чтении и записи.

Слой каталога

Как отмечалось в главе 2, каталог содержит указатели на текущие метаданные, то есть ссылки на последние файлы метаданных всех таблиц. Независимо от выполняемой операции, чтения или записи, каталог – это первый компонент, с которым взаимодействует механизм запросов. Выполняя чтение, механизм обращается к каталогу, чтобы узнать текущее состояние таблицы, а при записи каталог используется для соблюдения определенной схемы и получения информации о схеме секционирования таблицы.

Слой метаданных

Слой метаданных состоит из трех компонентов: файлов метаданных, списков манифестов и файлов манифестов. Каждый раз, когда механизм запросов записывает что-то в таблицу Iceberg, атомарно создается

новый файл метаданных и определяется как последняя его версия. Это гарантирует линейность истории фиксаций таблицы и помогает в таких сценариях, как одновременная запись (т. е. когда несколько механизмов записывают данные одновременно). Кроме того, во время чтения механизмы запросов всегда будут видеть последнюю версию таблицы. Механизмы запросов взаимодействуют со списками манифестов, чтобы получить информацию о спецификациях разделов, что позволяет пропускать ненужные файлы манифестов для повышения производительности. Наконец, информация из файлов манифеста, такая как верхняя и нижняя границы для определенного столбца, количество пустых значений и данные для конкретного раздела, используется механизмом для уменьшения размеров файлов.

Слой данных

Механизмы запросов используют файлы метаданных для фильтрации и ускорения чтения файлов данных. При выполнении операции записи файлы данных записываются в файловое хранилище, а файлы метаданных создаются и обновляются соответствующим образом.

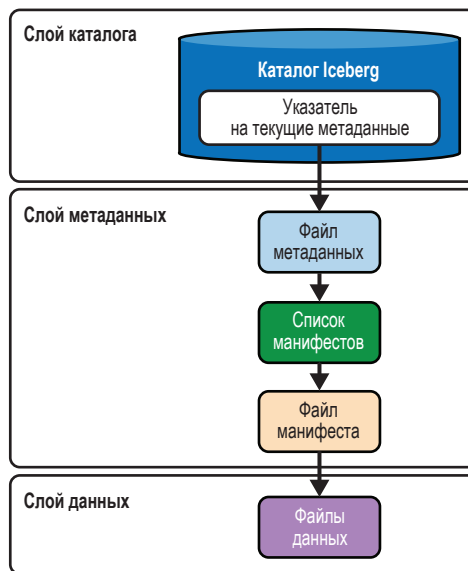


Рис. 3.1 ❖ Компоненты формата Apache Iceberg

В следующих разделах мы расскажем о жизненных циклах различных операций записи и чтения в Apache Iceberg и как каждая операция взаимодействует с только что описанными компонентами. Обратите внимание, что на протяжении всей этой главы мы будем представлять запросы с использованием Spark SQL и механизма запросов Dremio SQL.

Написание запросов в Apache Iceberg

Процесс записи в Apache Iceberg включает в себя ряд шагов, которые позволяют механизмам запросов эффективно вставлять и обновлять данные. Когда инициируется запрос на запись, он отправляется в механизм для анализа. Затем проверяется каталог, чтобы обеспечить согласованность и целостность данных и записать данные в соответствии с установленными стратегиями секционирования. Далее выполняется запись в файлы данных и метаданных. Наконец, файл каталога обновляется с учетом последних метаданных, что позволяет последующим операциям чтения получить доступ к самой последней версии данных. Общий ход процесса показан на рис. 3.2.

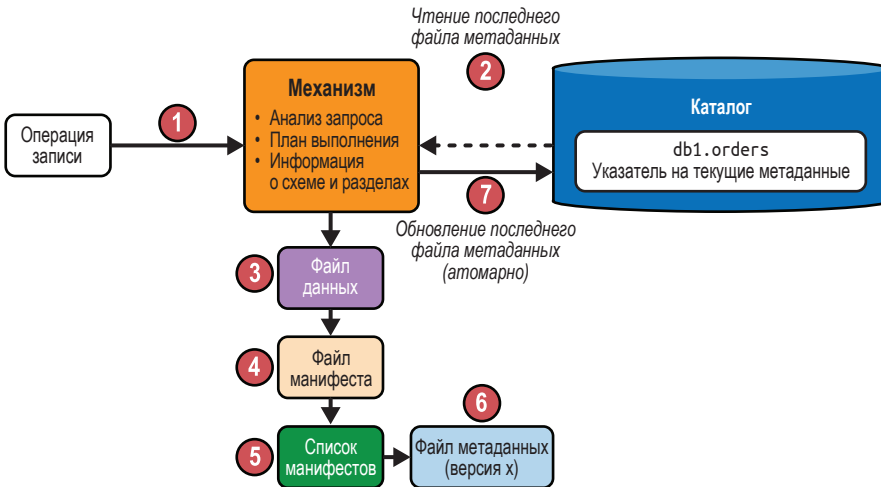


Рис. 3.2 ❖ Общий ход процесса записи в Apache Iceberg

Создание таблицы

Начнем с создания таблицы Iceberg. Запрос в примере ниже создает таблицу `orders` с четырьмя столбцами. Несмотря на сходство синтаксиса Spark и Dremio, в оставшейся части этой главы запросы для них будут показаны отдельно, чтобы вы могли сразу опробовать код в любой из систем и без задержек следовать за нашим повествованием. Код примеров также доступен в репозитории книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg>). Эта таблица секционирована по полю `order_ts` с точностью до часа. Обратите внимание, что в Iceberg не нужно добавлять явный столбец для секционирования таблицы. Это называется *скрытым секционированием*, как отмечалось в главе 1.

```
# Spark SQL
CREATE TABLE orders (
  order_id BIGINT,
```

```

    customer_id BIGINT,
    order_amount DECIMAL(10, 2),
    order_ts TIMESTAMP
)
USING iceberg
PARTITIONED BY (HOUR(order_ts))

# Dremio
CREATE TABLE orders (
    order_id BIGINT,
    customer_id BIGINT,
    order_amount DECIMAL(10, 2),
    order_ts TIMESTAMP
)
PARTITION BY (HOUR(order_ts))

```

Отправка запроса в механизм обработки

Сначала запрос передается механизму обработки запросов для анализа. Затем, поскольку запрос содержит оператор CREATE, механизм приступает к созданию и определению таблицы.

Создание файла метаданных

На этом этапе механизм запросов создает файл метаданных с именем *v1.metadata.json* в файловой системе озера данных, куда будет записана информация о таблице. Общая форма URL выглядит примерно так: *s3://путь/к/хранилищу/db1/table1/metadata/v1.metadata.json*.

На основе информации о пути к таблице */путь/к/хранилищу/db1/table1* механизм создает файл метаданных. Затем определяет схему таблицы *orders* – столбцы и типы данных – и сохраняет ее в файле метаданных. Наконец, присваивает таблице уникальный идентификатор: *table-uuid*. После успешного выполнения запроса файл метаданных *v1.metadata.json* записывается в хранилище файлов озера данных:

```
s3://datalake/db1/orders/metadata/v1.metadata.json
```

Заглянув в файл метаданных, вы увидите схему таблицы вместе со спецификацией секционирования:

```

{
  "table-uuid" : "072db680-d810-49ac-935c-56e901cad686",
  "schema" : {
    "type" : "struct",
    "schema-id" : 0,
    "fields" : [ {
      "id" : 1,
      "name" : "order_id",
      "required" : false,
      "type" : "long"
    } ],
  }
}

```

```

    "id" : 2,
    "name" : "customer_id",
    "required" : false,
    "type" : "long"
  }, {
    "id" : 3,
    "name" : "order_amount",
    "required" : false,
    "type" : "decimal(10, 2)"
  }, {
    "id" : 4,
    "name" : "order_ts",
    "required" : false,
    "type" : "timestampz"
  }
},
"partition-spec" : [ {
  "name" : "order_ts_hour",
  "transform" : "hour",
  "source-id" : 4,
  "field-id" : 1000
} ]
}

```

Это текущее состояние таблицы: мы создали таблицу, но она пока пустая – в ней нет записей. В терминах Iceberg это называется моментальным снимком (подробности см. в главе 2). Здесь важно отметить, что поскольку еще не было добавлено никаких записей, в таблице нет фактических данных, поэтому в вашем озере данных нет файлов данных. Соответственно, снимок не указывает ни на какой список манифестов, и потому нет никаких файлов манифестов.

Обновление каталога для фиксации изменений

Наконец, механизм обновляет в файле каталога *version-hint.text* указатель на текущие метаданные, чтобы он ссылался на *v1.metadata.json* – текущее состояние таблицы. Обратите внимание, что имя файла каталога *version-hint.text* зависит от выбора каталога. Для этого примера мы использовали каталог в файловой системе Hadoop. В главе 5 мы покажем и сравним разные варианты каталогов Iceberg. На рис. 3.3 показана иерархия компонентов Iceberg после создания таблицы.

Запрос вставки

Теперь вставим несколько записей в таблицу и посмотрим, как работает эта операция. Мы создали таблицу *orders* с четырьмя столбцами. Для этого примера добавим в таблицу следующие значения: *order_id* = 123, *customer_id* = 456, *order_amount* = 36.17 и *order_ts* = 07.03.2023 08:10:23. Вот сам запрос:


```
# Spark SQL/Dremio's SQL Query Engine
INSERT INTO orders VALUES (
  123,
  456,
  36.17,
  '2023-03-07 08:10:23'
)
```

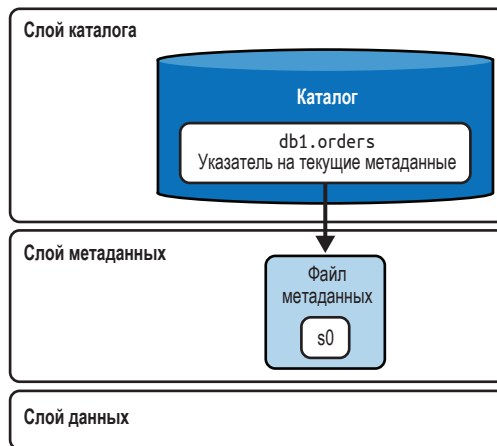


Рис. 3.3 ❖ Компоненты формата Apache Iceberg

Отправка запроса в механизм обработки

Сначала запрос передается механизму обработки запросов для анализа. Затем, поскольку запрос содержит оператор `INSERT`, механизму нужно получить информацию о таблице, в том числе и ее схему, чтобы начать планирование запроса.

Проверка каталога

Сначала механизм запросов обращается к каталогу, чтобы определить местоположение текущего файла метаданных, а затем читает его. Поскольку мы используем каталог `Nadoop`, механизм прочитает файл `/orders/metadata/version-hint.txt` и увидит, что он содержит лишь одно целое число: 1. Учитывая логику организации каталога, механизм понимает, что текущий файл метаданных размещен в `/orders/metadata/v1.metadata.json`, т. е. это тот самый файл, что был создан нашей предыдущей операцией `CREATE TABLE`. Несмотря на то что главным мотивом механизма в данном случае является вставка новых файлов данных, он продолжает взаимодействовать с каталогом по двум причинам:

- механизм должен понимать текущую схему таблицы, чтобы придерживаться ее;
- механизм должен изучить схему секционирования, чтобы соответствующим образом организовать данные во время записи.

Запись в файлы данных и метаданных

После того как механизм исследует схему таблицы и схему секционирования, он приступает к записи в новые файлы данных и соответствующие файлы метаданных. Вот что происходит на этом этапе.

Сначала механизм сохраняет записи в файл данных формата Parquet (этот формат используется по умолчанию, но его можно изменить) на основе схемы почасового секционирования таблицы.

Кроме того, если для таблицы определен порядок сортировки, то записи будут отсортированы перед сохранением в файл данных. Вот как это может выглядеть в файловой системе:

```
s3://datalake/db1/orders/data/order_ts_hour=2023-03-07-08/0_0_0.parquet
```

После сохранения записи в файл данных механизм создает файл манифеста. В файле манифеста содержится информация о пути к фактическому файлу данных. Кроме того, механизм записывает в манифест статистики, такие как верхняя и нижняя границы столбца и количество пустых значений, что может пригодиться механизму запросов для увеличения скорости обработки запросов. Механизм вычисляет эту информацию во время обработки данных, которые собираются записать, поэтому это относительно простая операция, по крайней мере по сравнению с процессом, который должен вычислить те же статистики с нуля. Файл манифеста записывается в систему хранения как файл *.avro*:

```
s3://datalake/db1/orders/metadata/62acb3d7-e992-4cbc-8e41-58809fcacb3e.avro
```

Ниже показано, как выглядит содержимое файла манифеста в формате JSON. Обратите внимание, что это лишь фрагмент полного содержимого с некоторой ключевой информацией, имеющей отношение к нашей теме:

```
{
  "data_file" : {
    "file_path" :
"s3://datalake/db1/orders/data/order_ts_hour=2023-03-07-08/0_0_0.parquet",
    "file_format" : "PARQUET",
    "block_size_in_bytes" : 67108864,
    "null_value_counts" : [],
    "lower_bounds" : {
      "array": [{
        "key": 1,
        "value": 123
      }],
    }
  },
  "upper_bounds" : {
    "array": [{
      "key": 1,
      "value": 123
    }],
  },
}
}
```

Затем механизм создает список манифестов для отслеживания файла манифеста. Если с текущим моментальным снимком связаны уже имеющиеся файлы манифестов, они тоже будут добавлены в новый список манифестов. В файл со списком записывается такая информация, как путь к файлу манифеста, количество добавленных или удаленных файлов данных и записей, а также статистики, касающиеся секционирования, например нижние и верхние границы столбцов раздела. И снова в механизме уже есть вся эта информация, поэтому ему не составляет труда получить эти статистики. Статистики помогают при чтении исключать ненужные файлы манифестов и выполнять запросы быстрее:

```
s3://datalake/db1/orders/metadata/
snap-8333017788700497002-1-4010cc03-5585-458c-9fdc-188de318c3e6.avro
```

Вот фрагмент содержимого файла со списком манифестов:

```
{
  "manifest_path":
"s3://datalake/db1/orders/metadata/62acb3d7-e992-4cbc-8e41-58809fcacb3e.avro",
  "manifest_length": 6152,
  "added_snapshot_id": 8333017788700497002,
  "added_data_files_count": 1,
  "added_rows_count": 1,
  "deleted_rows_count": 0,
  "partitions": {
    "array": [ {
      "contains_null": false,
      "lower_bound": {
        "bytes": "\u0000\u0000\u0000\u0000"
      },
      "upper_bound": {
        "bytes": "\u0000\u0000\u0000\u0000"
      }
    } ]
  }
}
```

Наконец, механизм создает новый файл метаданных *v2.metadata.json* с новым снимком *s1*, учитывая существующий файл метаданных *v1.metadata.json* (бывший текущим ранее), отслеживая при этом предыдущий снимок *s0*. Этот новый файл метаданных включает информацию о списке манифестов, созданном механизмом, идентификатор моментального снимка и сведения об операции. Кроме того, механизм меняет ссылку на вновь созданный список манифестов (или снимок), который теперь является текущим:

```
s3://datalake/db1/orders/metadata/v2.metadata.json
```

Вот как выглядит содержимое (точнее фрагмент) этого нового файла метаданных:

```

"current-snapshot-id" : 8333017788700497002,
"refs" : {
  "main" : {
    "snapshot-id" : 8333017788700497002,
    "type" : "branch"
  }
},
"snapshots" : [ {
  "snapshot-id" : 8333017788700497002,
  "summary" : {
    "operation" : "append",
    "added-data-files" : "1",
    "added-records" : "1",
  },
  "manifest-list" : "s3://datalake/db1/orders/metadata/
snap-8333017788700497002-1-4010cc03-5585-458c-9fdc-188de318c3e6.avro",
} ],

```

Обновление каталога для фиксации изменений

Теперь механизм снова обращается к каталогу, чтобы убедиться, что во время выполнения операции INSERT не было зафиксировано никаких других снимков. Выполняя эту проверку, Iceberg гарантирует отсутствие конфликтов в сценариях одновременной записи данных с нескольких устройств. В любой операции записи Iceberg оптимистично создает файлы метаданных, предполагая, что текущая версия останется неизменной, пока автор записи не зафиксирует ее. По завершении записи механизм выполняет атомарную фиксацию, переключая указатель на файл метаданных таблицы с существующей базовой версии на новую версию *v2.metadata.json*, которая теперь становится текущим файлом метаданных.

Визуальное представление, как выглядит иерархия компонентов Iceberg на этом этапе, показано на рис. 3.4.

Запрос слияния

В следующем примере операций записи выполним операторы UPSERT/MERGE INTO. Такие запросы обычно выполняются для обновления существующей записи, если она имеется в таблице, или вставки новой в противном случае.

Итак, предположим, что есть некая промежуточная таблица с обновлениями *orders_staging*, содержащая две записи: одна с обновлением для существующей записи с *order_id*=123, а другая представляет совершенно новый заказ. Нам нужно, чтобы таблица *orders* обновлялась последними сведениями, поэтому мы обновим *order_amount*, если заказ с указанным *order_id* уже существует в целевой таблице (*orders*). Иначе мы просто вставим новую запись. Вот сам запрос:

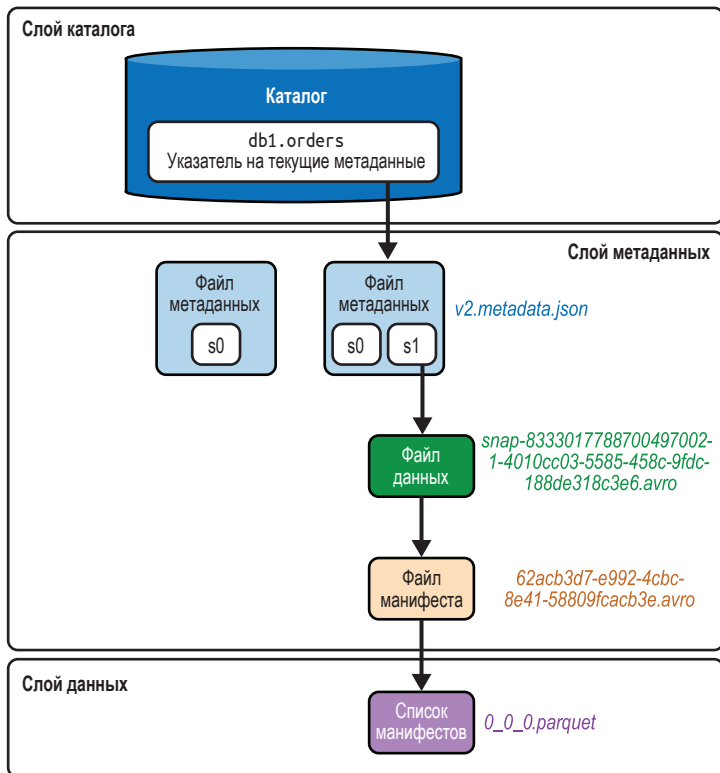


Рис. 3.4 ❖ Иерархия компонентов Iceberg после выполнения оператора INSERT

```
# Spark SQL
MERGE INTO orders o
USING (SELECT * FROM orders_staging) s
ON o.order_id = s.order_id
WHEN MATCHED THEN UPDATE SET order_amount = s.order_amount
WHEN NOT MATCHED THEN INSERT *;

# Dremio
MERGE INTO orders o
USING (SELECT * FROM orders_staging) s
ON o.order_id = s.order_id
WHEN MATCHED THEN UPDATE SET order_amount = s.order_amount
WHEN NOT MATCHED THEN INSERT (order_id, customer_id, order_amount, order_ts)
VALUES (s.order_id, s.customer_id, s.order_amount, s.order_ts)
```

Этот запрос объединит следующие наборы данных:

orders:

order_id	customer_id	order_amount	order_ts
123	456	36.17	2023-03-07 08:10:23

orders_staging:

order_id	customer_id	order_amount	order_ts
123	456	50.5	2023-03-07 08:10:23
124	326	60	2023-01-27 10:05:03

Отправка запроса в механизм обработки

Сначала запрос передается механизму обработки запросов для анализа. В этом случае, поскольку задействованы две таблицы (stage и destination), для планирования запроса механизму необходимы данные из обеих таблиц.

Проверка каталога

Подобно операции INSERT, обсуждавшейся в предыдущем разделе, механизм запросов сначала обращается к каталогу, чтобы определить текущее местоположение файла метаданных, а затем читает его. Поскольку в этом примере используется каталог Hadoop, механизм прочитает файл `/orders/metadata/versionhint.txt` и получит его содержимое – целое число 2. Получив эту информацию и учитывая логику работы каталога, механизм понимает, что файл метаданных находится в пути `/orders/metadata/v2.metadata.json`. Этот файл был создан нашей предыдущей операцией INSERT, поэтому механизм прочитает его. Затем он исследует текущую схему таблицы, чтобы операции записи могли ее придерживаться. Наконец, механизм определяет, как организованы файлы данных, исследовав стратегию секционирования, и начинает создавать новые файлы данных.

Запись в файлы данных и метаданных

Сначала механизм запросов читает данные в память из таблиц `order_staging` и `orders`, чтобы выявить совпадающие записи. Обратите внимание, что процесс READ мы будем рассматривать в следующем разделе. Механизм просматривает поле `order_id` в каждой записи в обеих таблицах и находит совпадающие записи.

Здесь следует сделать одно важное замечание: поскольку механизм выявляет совпадения, то выбор того, что должно отслеживаться в памяти, основывается на двух стратегиях, определяемых свойствами таблицы Iceberg: копирование при записи (COW) и объединение при чтении (MOR).

Эти две стратегии мы подробно рассмотрим в главе 4, а пока отметим, что при использовании стратегии COW при каждом обновлении таблицы Iceberg любые связанные файлы данных с соответствующими записями будут перезаписаны заново. Однако при использовании стратегии MOR существующие файлы данных останутся как есть и будут созданы новые файлы удаления для отслеживания изменений.

В этом примере мы будем предполагать использование стратегии COW. Соответственно, в память будет прочитан файл данных `0_0_0.parquet`, содержа-

щий запись с `order_id = 123` из таблицы `orders`. Затем полю `order_amount` этой записи будет присвоено новое значение из таблицы `order_staging`. Наконец, измененные данные будут записаны в новый файл данных Parquet:

```
s3://datalake/db1/orders/data/order_ts_hour=2023-03-07-08/0_0_1.parquet
```

Обратите внимание, что в этом конкретном примере в таблице `orders` имеется только одна запись. Но даже если бы в этой таблице были другие записи, не соответствующие условию, указанному в запросе, механизм все равно скопировал бы все эти записи и обновил только соответствующие условию, а несоответствующие просто переписал бы в новый файл. Так работает стратегия записи COW. Больше об этих стратегиях записи вы узнаете в главе 4.

Далее механизм переходит к записи в таблице `order_staging`, которая не соответствует условию. Он рассматривает ее как инструкцию `INSERT` и запишет в новый файл данных в другом разделе, потому что значение часа (`order_ts`) в ней отличается от предыдущей записи:

```
s3://datalake/db1/orders/data/order_ts_hour=2023-01-27-10/0_0_0.parquet
```

После записи файлов данных механизм создает новый файл манифеста со ссылками, содержащими пути к этим двум файлам данных. Кроме того, в файл манифеста включаются различные статистические данные о файлах данных, такие как нижняя и верхняя границы столбца и количество значений:

```
s3://datalake/db1/orders/metadata/faf71ac0-3aee-4910-9080-c2e688148066.avro
```

Пример получившегося файла манифеста можно найти в репозитории книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg>).

Затем механизм генерирует новый список манифестов, указывающий на файл манифеста, созданный на предыдущем шаге. Он также добавляет в список все существующие файлы манифестов и записывает его в озеро данных:

```
s3://datalake/db1/orders/metadata/snap-5139476312242609518-1-e22ff753-2738-4d7da810-d65dcc1abe63.avro
```

Изучив список манифестов (см. репозиторий книги на GitHub [<https://oreil.ly/supp-guide-apache-iceberg>]), в нем можно заметить такую информацию, как статистика разделов и количество добавленных и удаленных файлов.

После этого механизм создает новый файл метаданных `v3.metadata.json` с новым снимком `s2` на основе предыдущего текущего файла метаданных `v2.metadata.json` и снимков, `s0` и `s1`, включенных в него (этот файл можно найти в репозитории книги на GitHub [<https://oreil.ly/supp-guide-apache-iceberg>]):

```
s3://datalake/db1/orders/metadata/v3.metadata.json
```

Обновление каталога для фиксации изменений

Наконец, на этом этапе движок выполняет проверку, чтобы убедиться в отсутствии конфликтов записи, а затем обновляет каталог, используя значение

последнего файла метаданных, то есть *v3.metadata.json*. Визуально компоненты Iceberg на этом этапе выполнения операции UPSERT будут выглядеть так, как показано на рис. 3.5.

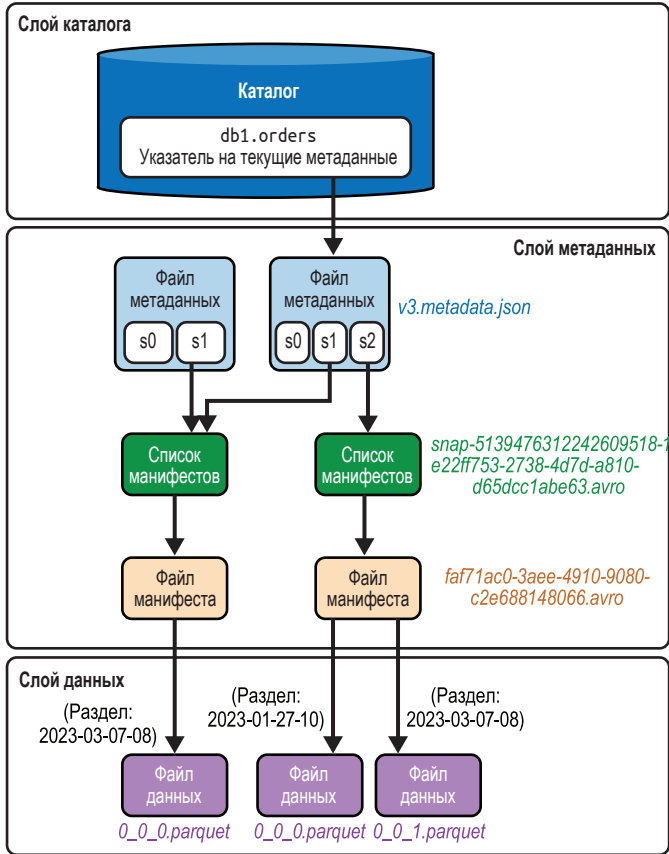


Рис. 3.5 ❖ Иерархия компонентов Iceberg после выполнения MERGE INTO

Запросы на чтение в Apache Iceberg

Чтение данных из таблиц Apache Iceberg следует четко определенной последовательности действий, что позволяет без задержек преобразовывать запросы в полезную информацию. Когда инициируется запрос на чтение, он сначала отправляется в механизм запросов. Тот обращается к каталогу для получения последнего местоположения файла метаданных, содержащего информацию о схеме таблицы и других файлах метаданных, таких как список манифестов, который в конечном итоге ведет к фактическим файлам дан-

ных. В этом процессе используется статистическая информация о столбцах, чтобы минимизировать количество читаемых файлов, что помогает повысить производительность запросов.

Запрос SELECT

В этом разделе мы посмотрим, как работают различные компоненты Apache Iceberg при выполнении запроса на чтение. Вот запрос, который мы запустим:

```
# Spark SQL/Dremio Sonar
SELECT *
FROM orders
WHERE order_ts BETWEEN '2023-01-01' AND '2023-01-31'
```

Отправка запроса в механизм обработки

Сначала запрос передается механизму обработки запросов для анализа. На этом этапе механизм планирует запрос на основе файлов метаданных.

Проверка каталога

Сначала механизм запросов обращается к каталогу, чтобы определить местоположение текущего файла метаданных для таблицы `orders`, а затем читает его. Как обсуждалось в двух предыдущих разделах, механизм будет читать файл `/orders/metadata/version-hint.txt`, поскольку в этом примере используется каталог Hadoop. Этот файл содержит одно целое число: 3. Учитывая логику организации каталога, механизм понимает, что текущий файл метаданных размещен в `/orders/metadata/v3.metadata.json`. Это файл, созданный предыдущей операцией `MERGE INTO`.

Получение информации из файла метаданных

Затем механизм запросов открывает и читает файл метаданных `v3.metadata.json`. Сначала он определяет схему таблицы, чтобы подготовить внутренние структуры в памяти для чтения данных. См. примеры данных в файле `metadata.json` в репозитории книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg>). Затем он изучает схему секционирования таблицы, чтобы понять, как организованы данные. Позже эта информация будет использоваться механизмом запросов для пропуска ненужных файлов данных.

Одна из наиболее важных частей информации, которую механизм получает из файла метаданных, – это идентификатор текущего снимка `current-snapshot-id`. Это то, что означает текущее состояние таблицы. На основе текущего идентификатора моментального снимка механизм определяет путь к файлу списка манифестов, чтобы пройти дальше и просканировать соответствующие файлы.

Получение информации из списка манифеста

Получив путь к файлу со списком манифестов из файла метаданных, механизм запросов читает файл `snap-5139476312242609518-1-e22ff753-2738-4d7d-a810-d65dcc1abe63.avro`, чтобы получить дополнительную информацию (см. примеры в репозитории книги на GitHub [<https://oreil.ly/supp-guide-apache-iceberg>]). Самая важная информация, которую механизм получает из этого файла, – путь к манифесту для каждого моментального снимка в списке манифестов. Эта информация нужна механизму, чтобы получить соответствующие файлы данных.

Список манифестов содержит также важную информацию о разделах, например идентификатор спецификации раздела `partition-spec-id`. Она сообщает механизму схему разделов, использованную для записи конкретного снимка. В настоящее время это поле имеет значение 0 (см. репозиторий книги на GitHub [<https://oreil.ly/supp-guide-apache-iceberg>]), т. е. это единственный раздел, определенный для таблицы.

Существуют также статистические данные для разделов, например нижняя и верхняя границы столбцов в разделах. Эту информацию механизм использует, чтобы определить, какие файлы манифеста следует пропустить. В этом файле также содержатся другие сведения, такие как общее количество добавленных/удаленных файлов данных и количество добавленных/удаленных записей для каждого снимка.

Получение информации из файла манифеста

Затем механизм открывает файл манифеста `faf71ac0-3aee-4910-9080-c2e688148066.avro`, который не был удален (т. е. имеет отношение к запросу). Он читает файл, чтобы получить подробную информацию (см. примеры в репозитории книги на GitHub [<https://oreil.ly/supp-guide-apache-iceberg>]). Сначала механизм запросов сканирует все записи, представляющие файлы данных, отслеживаемых этим файлом манифеста. Он сравнивает значения разделов, которым принадлежит каждый из файлов данных, со значениями в фильтрах запросов.

В запросе мы затребовали вернуть информацию о заказах, размещенных между '01.01.2023' и '31.01.2023'. Поэтому механизм проигнорирует раздел 2023-03-07-08, поскольку он не соответствует диапазону значений в фильтре. Обнаружив подходящий раздел, механизм проверит все записи в нем.

Основываясь на значении раздела, механизм ищет соответствующий файл данных `0_0_0.parquet`. Он также собирает другую статистическую информацию, такую как нижняя и верхняя границы значений в каждом столбце и количество пустых значений, чтобы пропустить ненужные файлы.

Методы оптимизации данных и файлов, такие как секционирование и фильтрация на основе метрик (верхняя/нижняя границы столбцов), доступных по умолчанию в Apache Iceberg, позволяют механизму избегать полного сканирования таблицы, как показано в этом примере, тем самым

обеспечивая увеличение производительности. Наконец, найденная запись возвращается пользователю:

	order_id	customer_id	order_amount	order_ts
1	123	456	36.17	2023-03-07 08:10:23

Визуально весь процесс чтения выглядит так, как показано на рис. 3.6.

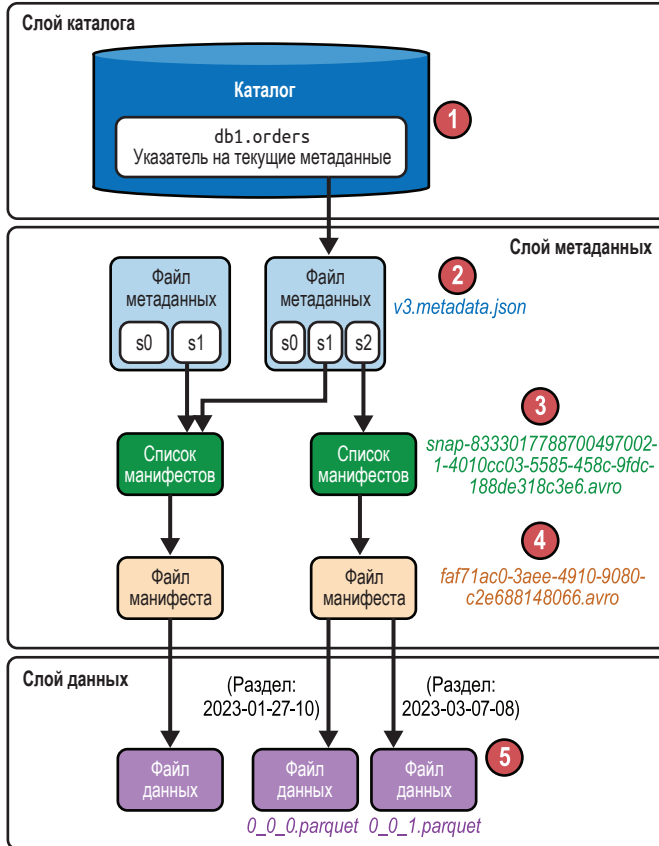


Рис. 3.6 ❖ Схема работы процесса чтения в Apache Iceberg

На рис. 3.6 обратите внимание на следующее:

1. Механизм запросов обращается к каталогу, чтобы получить имя текущего файла метаданных (*v3.metadata.json*).
2. Затем он получает идентификатор текущего снимка (в данном случае s2) и местоположение списка манифестов для этого снимка.
3. Потом из списка манифестов извлекается путь к файлу манифеста.
4. По содержимому файла манифеста механизм определяет путь к файлу данных с учетом фильтра разделов (2023-03-07-08).

5. Наконец, возвращает пользователю соответствующие значения из файла данных.

Запрос информации о состоянии в прошлом

Важной особенностью в мире баз и хранилищ данных является возможность вернуться к определенному состоянию таблицы в прошлом и запросить исторические данные (т. е. данные, которые позже были изменены или удалены). Apache Iceberg привносит аналогичную возможность путешествий во времени в архитектуру озерных хранилищ данных. Это особенно полезно в таких сценариях, как анализ данных за предыдущие кварталы, восстановление случайно удаленных записей или воспроизведение результатов анализа. Apache Iceberg предоставляет два способа выполнения запросов информации о состоянии в прошлом: с использованием отметки времени и с использованием идентификатора снимка.

В этом разделе вы узнаете, как выполнять запросы состояния в прошлом таблицы Apache Iceberg. Для целей демонстрации предположим, что нам нужно вернуться к состоянию, предшествовавшему запросу `MERGE INTO` (т. е. к моменту сразу после выполнения оператора `INSERT`). Итак, учитывая эти предпосылки, первое, с чем мы должны разобраться, – это история таблицы Iceberg. Одна из лучших особенностей формата Apache Iceberg – он позволяет анализировать метаданные таблиц с помощью системных таблиц, называемых *таблицами метаданных*. Подробно о таблицах метаданных мы расскажем в главе 10. Чтобы проанализировать историю таблицы `order`, запросим таблицу метаданных `history` (доступна в репозитории книги на GitHub [<https://oreil.ly/supp-guide-apache-iceberg>]):

```
# Spark SQL
SELECT * FROM catalog.db.orders.history;
# Dremio
SELECT * FROM TABLE (table_history('orders'))
```

Этот запрос вернет список всех транзакций, в которых участвовала данная таблица:

made_current_at	snapshot_id	parent_id	is_current_ancestor
2023-03-06 21:28:35.360	7327164675870333694	null	true
2023-03-07 20:45:08.914	8333017788700497002	7327164675870333694	true
2023-03-09 19:58:40.448	5139476312242609518	8333017788700497002	true

Вот что мы видим в таблице метаданных history:

- первый снимок с идентификатором 7327164675870333694 был создан после выполнения инструкции CREATE;
- второй снимок, 8333017788700497002, был создан после вставки новой записи с помощью инструкции INSERT;
- наконец, наш запрос MERGE INTO создал третий снимок, 5139476312242609518.

Поскольку нам нужна информация о состоянии, предшествующем последней транзакции (т. е. MERGE), мы будем ориентироваться на вторую отметку времени или идентификатор моментального снимка. Выполним такой запрос:

```
# Spark SQL
SELECT * FROM orders
TIMESTAMP AS OF '2023-03-07 20:45:08.914'
# Dremio Sonar
SELECT * FROM orders
AT TIMESTAMP '2023-03-07 20:45:08.914'
```



Если передать в запрос неточное значение отметки времени, то Iceberg отыщет снимок старше указанного значения. Если более старых снимков не существует, то Iceberg выдаст такое исключение:

```
IllegalArgumentException: Cannot find a snapshot older
than 2023-03-06T21:28:35+00:00.
```

При использовании идентификатора снимка запрос будет выглядеть так:

```
# Spark SQL
SELECT *
FROM orders
VERSION AS OF 8333017788700497002
# Dremio
SELECT *
FROM orders
AT SNAPSHOT 8333017788700497002
```

Теперь давайте кратко рассмотрим, что происходит с компонентами Iceberg за кулисами, когда выполняется запрос состояния в прошлом, и как соответствующий файл данных возвращается пользователю.

Отправка запроса в механизм обработки

По аналогии с оператором SELECT запрос сначала передается механизму обработки запросов для анализа. Механизм использует метаданные таблицы для планирования запроса.

Проверка каталога

Сначала механизм запросов обращается к каталогу, чтобы определить местоположение текущего файла метаданных, а затем читает его. Поскольку мы используем каталог Hadoop, механизм прочитает файл `/orders/metadata/`

version-hint.txt и увидит, что он содержит целое число 3. Учитывая логику организации каталога, механизм понимает, что текущий файл метаданных размещен в */orders/metadata/v3.metadata.json*. Наконец, механизм читает этот файл, чтобы выявить схему таблицы, а также стратегию секционирования.

Получение информации из файла метаданных

Затем механизм запросов открывает и читает файл метаданных, чтобы получить информацию о таблице. Текущий файл метаданных содержит ссылки на все снимки, созданные для таблицы Iceberg, если только срок их действия не истек, согласно стратегии обслуживания метаданных (подробнее об этом в главе 4). Из списка снимков механизм выбирает конкретный снимок, соответствующий критерию запроса на основе отметки времени или идентификатора снимка.

Механизм также определяет схему таблицы и схему секционирования, чтобы использовать эту информацию позже для пропуска ненужных файлов. Наконец, он получает путь к списку манифестов для этого конкретного снимка:

```
s3://datalake/db1/orders/metadata/
snap-8333017788700497002-1-4010cc03-5585-458c-9fdc-188de318c3e6.avro
```

Получение информации из файла метаданных

Получив путь к списку манифестов, механизм открывает и читает указанный файл *.avro*, содержащий данные о снимке. Из этого файла механизм извлекает пару важных фрагментов информации:

- путь к файлу манифеста, который содержит ссылки на фактические файлы данных: *s3://datalake/db1/orders/metadata/62acb3d7-e992-4cbc-8e41-58809fcacb3e.avro*;
- количество добавленных/удаленных файлов данных и статистическая информация о разделах.

Получение информации из файла манифеста

Затем механизм читает все файлы манифестов, соответствующие запросу, и получает подробную информацию. Самая важная информация в файле манифеста – путь к файлу с искомыми данными. Механизм просматривает каждый файл данных, указанный в манифесте, и определяет, следует его читать или нет. Помимо пути к файлу данных, механизм также собирает статистическую информацию о столбцах, упоминавшуюся в предыдущих разделах.

Наконец, механизм читает файл данных *0_0_0.parquet*, и пользователю возвращается следующий результат:

order_id	customer_id	order_amount	order_ts
123	456	36.17	2023-03-07 08:10:23 +00:00

Это та запись, которую мы вставили в таблицу перед выполнением запроса `MERGE INTO`. На рис. 3.7 показано визуальное представление процесса.

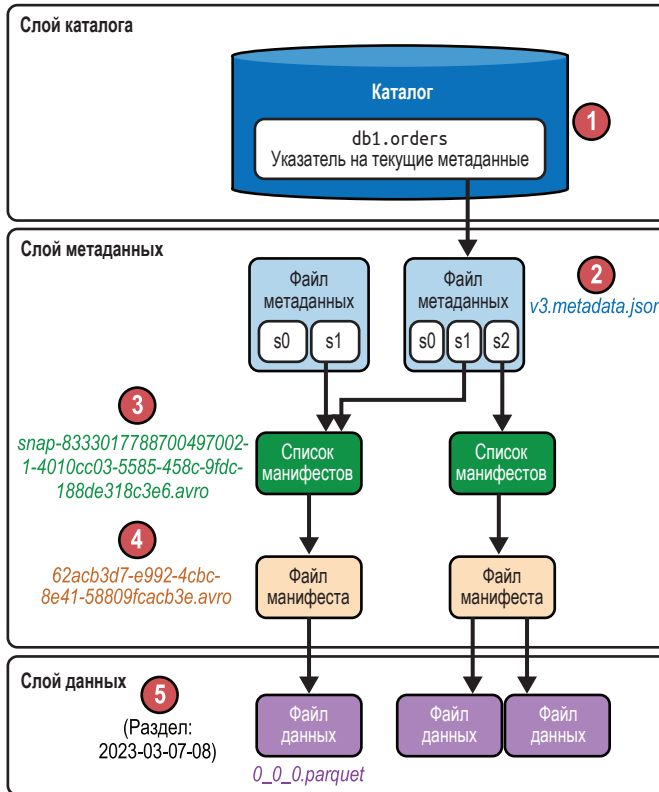


Рис. 3.7 ❖ Как работает запрос на получение информации о прошлом

На рис. 3.7 обратите внимание на следующее:

1. Механизм запросов обращается к каталогу, чтобы получить имя текущего файла метаданных (`v3.metadata.json`).
2. Затем он выбирает снимок (в данном случае `s1`) на основе отметки времени или идентификатора, указанного в запросе, и получает местоположение списка манифестов.
3. Потом из списка манифестов извлекается путь к файлу манифеста.
4. По содержимому файла манифеста механизм определяет путь к файлу данных с учетом фильтра разделов (`2023-03-07-08`).
5. Наконец, возвращает пользователю соответствующие значения из файла данных.

Заключение

В этой главе мы обсудили детали обработки различных запросов на чтение и запись, таких как создание таблицы, вставка и обновление записей, чтобы понять, как разные архитектурные компоненты Apache Iceberg используются вычислительными механизмами.

В главе 4 мы рассмотрим методы оптимизации, доступные в Apache Iceberg, обеспечивающие высокую производительность при чтении и записи данных в таблицы.

Глава 4

Оптимизация производительности таблиц Iceberg

Как было показано в главе 3, таблицы Apache Iceberg имеют слой метаданных, который позволяет механизму запросов создавать более оптимальные планы выполнения запросов. Однако метаданные – это только один из элементов, способствующих оптимизации обработки ваших данных.

В вашем распоряжении имеются различные рычаги оптимизации, включая сокращение количества файлов данных, сортировку данных, секционирование таблиц, обработку изменений на уровне записей, сбор метрик и внешние факторы. Эти рычаги играют важную роль в повышении производительности, и в данной главе мы рассмотрим каждый из них и покажем, как устранить потенциальные замедления. Реализация надежного инструментального мониторинга имеет решающее значение для выявления потребности в оптимизации, включая использование таблиц метаданных Apache Iceberg, которые мы рассмотрим в главе 10.

Уплотнение

Каждая процедура или процесс требует затрат времени, а это означает, что более длинные и сложные запросы требуют больше времени на вычисления. Другими словами, чем больше шагов нужно сделать, чтобы достичь цели, тем больше времени понадобится на это. Чтобы выполнить запрос к таблицам Apache Iceberg, механизм должен открыть и просканировать каждый файл, а затем закрыть файлы по окончании. Чем больше файлов нужно проскани-

ровать, тем больше ресурсов будет затрачено на выполнение запроса. Эта проблема усугубляется в мире потоковой передачи данных или обработки данных «в реальном времени», где данные принимаются по мере их появления и создается множество файлов, в каждом из которых содержится всего несколько записей.

В системах с пакетной обработкой, принимающих информацию сразу за целый день или даже за неделю, есть возможность более эффективно спланировать запись данных в виде более точно организованных файлов. Но даже в пакетных системах можно столкнуться с «проблемой небольших файлов», когда слишком много маленьких файлов влияют на скорость сканирования, потому что приходится выполнять больше операций с файлами и читать гораздо больше метаданных (в каждом файле есть свои метаданные), а при выполнении операций очистки и обслуживания приходится удалять больше файлов. Оба этих сценария показаны на рис. 4.1.

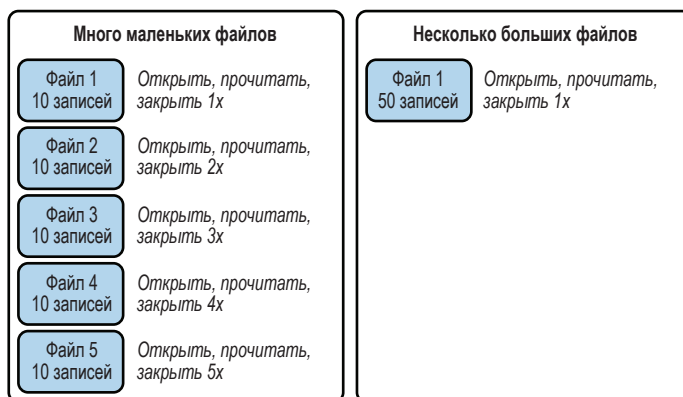


Рис. 4.1 ❖ Множество мелких файлов читаются медленнее, чем тот же объем данных в меньшем количестве файлов большего размера

По сути, когда дело доходит до чтения данных, существуют фиксированные накладные расходы, которых нельзя избежать, и переменные накладные расходы, которые можно сократить, используя различные стратегии. К фиксированным накладным расходам относится чтение конкретных данных, связанных с запросом, – нельзя обработать данные, не прочитав их. К переменным накладным расходам относятся операции с файлами, такие как открытие/закрытие файла, и их можно сократить, используя стратегии, которые мы обсудим в этой главе. После применения этих стратегий вы будете использовать только минимум вычислительных ресурсов, максимально быстро выполняя работу (чем быстрее выполняется работа, тем быстрее освобождаются вычислительные ресурсы кластера и тем дешевле обходится эта работа).

Одно простое решение задачи оптимизации состоит в том, чтобы периодически читать данные из этих небольших файлов и перезаписывать их

в меньшее количество файлов большего размера (аналогично можно переписать файлы манифестов, если их окажется слишком много). Этот процесс называется *уплотнением*, поскольку в ходе его выполнения множество маленьких файлов уплотняется в меньшее их число. Процесс уплотнения показан на рис. 4.2.

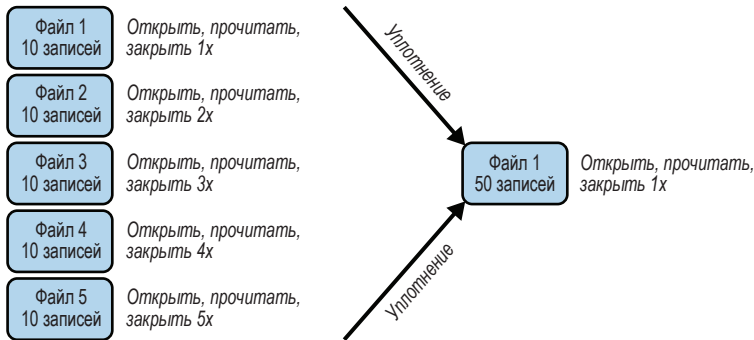


Рис. 4.2 ❖ Процесс уплотнения множества маленьких файлов в меньшее число больших файлов

Практическая реализация уплотнения

Вы могли бы подумать, что, несмотря на кажущуюся простоту, для реализации этого решения потребуется написать массу сложного кода на Java или Python. Но это не так! К счастью, пакет Apache Iceberg Actions включает несколько вспомогательных процедур (этот пакет предназначен для Apache Spark, но может использоваться другими механизмами для реализации своих операций обслуживания). Этот пакет используется изнутри Spark либо из кода на SparkSQL, который демонстрируется на протяжении большей части данной главы, либо из императивного кода, пример которого показан ниже (имейте в виду, что эти операции поддерживают гарантии ACID, что и обычные транзакции Iceberg):

```
Table table = catalog.loadTable("myTable");
SparkActions
    .get()
    .rewriteDataFiles(table)
    .option("rewrite-job-order", "files-desc")
    .execute();
```

Данный фрагмент создает новый экземпляр таблицы, а затем вызывает метод `rewriteDataFiles`, который иницирует операцию уплотнения в Spark. Шаблон построения, используемый `SparkActions`, позволяет объединять методы в цепочку для тонкой настройки задания уплотнения, чтобы выразить не только саму операцию уплотнения, но и порядок ее выполнения.

Существует несколько методов, которые можно добавить в цепочку после вызова `rewriteDataFiles` и `execute`, начинающих задание:

`binPack`

Задаёт стратегию уплотнения `binpack` (обсуждается далее), которая используется по умолчанию и не требует явного выбора.

`Sort`

Задаёт стратегию уплотнения `sort`, согласно которой поля переписываются по одному или по нескольку в порядке приоритета. Более подробно описывается в разделе «Стратегии уплотнения» ниже.

`zOrder`

Задаёт стратегию уплотнения `z-order-sort` – сортировку данных на основе нескольких полей с одинаковым весом, подробнее обсуждается в разделе «Сортировка» ниже.

`filter`

Позволяет передать выражение, используемое для ограничения переписываемых файлов.

`option`

Изменяет один параметр.

`options`

Принимает массив с несколькими параметрами настройки.

Существует несколько возможных параметров настройки задания; вот некоторые из них:

`target-file-size-bytes`

Задаёт предполагаемый размер выходных файлов. По умолчанию используется свойство `write.target.file-size-bytes` таблицы, которое по умолчанию равно 512 Мбайт.

`max-concurrent-file-group-rewrites`

Это максимальное количество групп файлов, записываемых одновременно.

`max-file-group-size-bytes`

Максимальный размер группы файлов – это не один файл. Этот параметр следует использовать при работе с разделами, размер которых превышает объем памяти, доступной рабочему процессу, записывающему определенную группу файлов, чтобы он мог разбить данный раздел на несколько групп файлов для одновременной записи.

`partial-progress-enabled`

Этот параметр позволяет выполнять фиксации во время уплотнения групп файлов. Настраивая данный параметр, можно позволить другим запросам

использовать уже сжатые файлы в ходе выполнения длительного процесса уплотнения.

`partial-progress-max-commits`

Если включен параметр `partial-progress-enabled`, то этот параметр определяет максимальное количество фиксаций, разрешенных для завершения задания.

`rewrite-job-order`

Если включен параметр `partial-progress-enabled`, то этот параметр определяет порядок записи групп файлов, чтобы гарантировать, что первыми будут фиксироваться группы файлов с более высоким приоритетом. Приоритет может основываться на размерах групп по объему в байтах или по количеству файлов в группе (`bytes-asc`, `bytes-desc`, `files-asc`, `files-desc`, `none`).

Размер файла и размер группы записей

Для файлов Apache Parquet существует размер группы записей и размер файла. Размер группы записей – это размер одной группы записей в файле Parquet, при этом каждый файл может иметь несколько групп. Конфигурация таблицы Iceberg по умолчанию допускает размер группы строк 128 Мбайт и размер файла 512 Мбайт (четыре группы записей на файл). Старайтесь обеспечивать кратность этих двух параметров (т. е. размер файла должен делиться на размер группы записей без остатка). Чем меньше количество групп записей, тем меньше размер файла, потому что для меньшего числа групп записей нужно записать меньше метаданных, однако большее число групп строк улучшает производительность предикатов, потому что метаданные группы записей могут включать более детализированные диапазоны, что позволит механизму обработки запросов исключить чтение ненужных групп записей, не содержащих данных, относящихся к текущему запросу.

Другой пример: вы можете увеличить размер файла до 1 Гбайта, но сохранить размер группы записей равным 128 Мбайт (восемь групп на файл). В таком случае механизму потребуется открывать и закрывать меньше файлов. Большинство выполняемых вами запросов будут требовать чтения большей части данных, тем не менее предпочтительнее иметь меньшее количество групп записей, потому что обработка предикатов не ускорит процесс получения всех данных.

Размер группы записей и размер файла можно задать как свойства таблицы (`write.parquet.row-group-size-bytes` и `write.target-file-size-bytes` соответственно), но размер файла можно установить для отдельных заданий уплотнения с помощью `options`.



По мере планирования записи новых файлов в задании уплотнения механизм начинает группировать файлы в группы, которые будут записываться параллельно (это означает, что файлы из разных групп могут записываться одновременно). В заданиях по уплотнению можно настроить параметры, отвечающие за размер групп файлов и количество файлов, записываемых одновременно, чтобы предотвратить проблемы с памятью.

Поэтапное выполнение

Поэтапное выполнение (partial progress) позволяет создавать новые снимки по мере завершения обработки групп файлов и использовать уже сжатые файлы в других запросах. Кроме того, поэтапное выполнение помогает предотвратить ситуации нехватки памяти при выполнении больших заданий уплотнения.

Но не забывайте, что увеличение количества снимков увеличивает объем пространства, занимаемого файлами метаданных в таблице. Однако если вы хотите, чтобы ваши пользователи как можно раньше получили выгоду от уплотнения, то поэтапное выполнение может оказаться полезной функцией. Если вы хотите сбалансировать стоимость дополнительных снимков с преимуществами поэтапного выполнения, то настройте параметр `max-commits`, чтобы ограничить общее количество фиксаций, которые будет выполнять одно задание уплотнения.

Следующий фрагмент кода демонстрирует применение нескольких параметров таблицы:

```
Table table = catalog.loadTable("myTable");
SparkActions
    .get()
    .rewriteDataFiles(table)
    .sort()
    .filter(Expressions.and(
        Expressions.greaterThanOrEqual("date", "2023-01-01"),
        Expressions.lessThanOrEqual("date", "2023-01-31")))
    .option("rewrite-job-order", "files-desc")
    .execute();
```

В предыдущем примере мы реализовали стратегию сортировки, которая по умолчанию использует порядок, указанный в свойствах таблицы. Кроме того, мы добавили фильтр, позволяющий перезаписывать данные только за январь. Важно отметить, что этот фильтр требует создания выражения с использованием внутреннего интерфейса построения выражений Apache Iceberg Expressions. Мы также настроили параметр `rewrite-job-order`, чтобы отдать приоритет большим группам файлов. Это означает, что группа из пяти файлов будет обработана раньше группы из двух файлов.



Библиотека Expressions предназначена для упрощения создания выражений на основе структур метаданных Apache Iceberg. Библиотека предоставляет API для создания и управления выражениями, которые затем можно использовать для фильтрации данных в таблицах и операций чтения. Выражения Iceberg также можно использовать в файлах манифестов для обобщения сведений в каждом файле данных, что позволяет Iceberg пропускать файлы, не содержащие записей, которые могут соответствовать фильтру. Этот механизм важен для масштабируемости архитектуры метаданных Iceberg.

Все это хорошо, но то же самое можно сделать проще с помощью расширений Spark SQL, которые включают процедуры, поддерживающие следующий синтаксис Spark SQL:

```
-- использование позиционных аргументов
CALL catalog.system.procedure(arg1, arg2, arg3)

-- использование именованных аргументов
CALL catalog.system.procedure(argkey1 => argval1, argkey2 => argval2)
```

Вызов процедуры `rewriteDataFiles` в этом синтаксисе будет выглядеть, как показано в примере 4.1.

Пример 4.1 ❖ Использование процедуры `rewrite_data_files` для запуска заданий уплотнения

```
-- вызов процедуры перезаписи файлов данных на SparkSQL
CALL catalog.system.rewrite_data_files(
  table => 'musicians',
  strategy => 'binpack',
  where => 'genre = "rock"',
  options => map(
    'rewrite-job-order', 'bytes-asc',
    'target-file-size-bytes', '1073741824', -- 1GB
    'max-file-group-size-bytes', '10737418240' -- 10GB
  )
)
```

Идея этого сценария состоит в следующем: мы могли передать некоторые данные в нашу таблицу `musicians` и заметили, что для рок-групп было создано множество небольших файлов, поэтому вместо уплотнения всей таблицы, что может занять много времени, мы решили уплотнить только данные, с которыми сопряжена основная проблема. Здесь мы также требуем от Spark отдать приоритет группам файлов с большим размером в байтах и сохранять файлы размером около 1 Гбайта каждый, при этом каждая группа файлов может иметь размер около 10 Гбайт. Результат этих настроек показан на рис. 4.3.



Обратите внимание на использование двойных кавычек в фильтре в примере 4.1. Мы должны заключить фильтр в одинарные кавычки, поэтому использовали в строке двойные кавычки вокруг слова `"rock"`, даже притом что в SQL обычно используются одинарные кавычки. Условие `where` по сути эквивалентно методу `filter`, упоминавшемуся выше. Без этого условия была бы перезаписана вся таблица.

Другие механизмы могут реализовывать свои инструменты уплотнения. Например, Dremio имеет свою функцию управления таблицами Iceberg – команду `OPTIMIZE`, – которая является уникальной реализацией, но соответствующей действию `rewriteDataFiles`:

```
OPTIMIZE TABLE catalog.MyTable
```

Предыдущая команда выполнит простое уплотнение, следуя стратегии `binpack`, и перезапишет все файлы в меньшее количество более оптимальных файлов. Но, как и в случае с процедурой `rewriteDataFiles` в Spark, есть возможность передать более тонкие настройки.

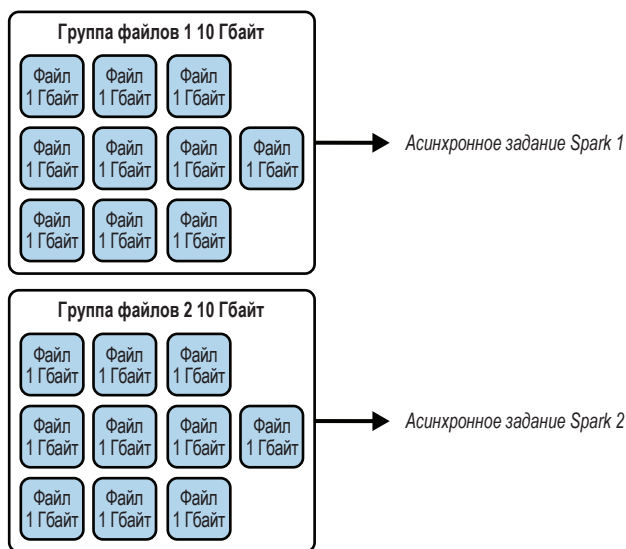


Рис. 4.3 ❖ Результат настройки максимального размера группы файлов и самих файлов равным 10 Гбайт и 1 Гбайт соответственно

Например, следующая команда уплотнит только конкретный раздел:

```
OPTIMIZE TABLE catalog.MyTable
  FOR PARTITIONS sales_year IN (2022, 2023) AND sales_month IN ('JAN', 'FEB',
'MAR')
```

И здесь мы уплотняем файлы, используя определенные параметры размера:

```
OPTIMIZE TABLE catalog.MyTable
  REWRITE DATA (MIN_FILE_SIZE_MB=100, MAX_FILE_SIZE_MB=1000,
TARGET_FILE_SIZE_MB=512)
```

Следующий фрагмент кода переписывает только манифесты:

```
OPTIMIZE TABLE catalog.MyTable
  REWRITE MANIFESTS
```

Как видите, для уплотнения таблиц Apache Iceberg мы с успехом можем использовать и Spark, и Dremio.

Стратегии уплотнения

Как упоминалось выше, существует несколько стратегий уплотнения, которые можно использовать с процедурой `rewriteDataFiles`. Эти стратегии, их достоинства и недостатки обобщены в табл. 4.1. В этом разделе мы обсудим стратегию уплотнения `binpack`, а стандартная сортировка и сортировка `zOrder` будут рассмотрены позже в книге.

Таблица 4.1. Плюсы и минусы стратегий уплотнения

Стратегия	Что делает	Достоинства	Недостатки
Binpack	Объединяет только файлы; глобальная сортировка не выполняется, но выполняется локальная сортировка внутри заданий	Самая быстрая стратегия уплотнения	Данные не кластеризуются
Sort	Последовательно сортирует данные по одному или нескольким полям перед распределением задач (например, сортирует по полю a, затем внутри него сортирует по полю b)	Кластеризация данных по часто запрашиваемым полям может увеличить скорость операций чтения	Уплотнение выполняется медленнее, по сравнению с binpack
zOrder	Сортирует данные по нескольким полям с одинаковым весом перед распределением задач (значения X и Y в этом диапазоне находятся в одной группе; значения в другом диапазоне находятся в другой группе)	Если запросы часто используют фильтры по нескольким полям, то эта стратегия может еще больше сократить время чтения	Еще больше замедляет уплотнение, по сравнению с binpack

Стратегия binpack – это, по сути, чистое уплотнение без применения каких-либо других действий, касающихся организации данных. Из трех стратегий, перечисленных в табл. 4.1, binpack – самая быстрая, потому что просто переписывает содержимое файлов небольшого размера в файл большего, желаемого размера, тогда как стратегии sort и zOrder дополнительно предусматривают сортировку данных перед выделением групп файлов для записи. Их применение дает особенно заметные выгоды, когда есть потоковые данные и необходимо выполнить их уплотнение, чтобы обеспечить скорость чтения, соответствующую соглашениям об уровне обслуживания (SLA).



Если в настройках таблицы Apache Iceberg установлен порядок сортировки, то даже при использовании стратегии binpack этот порядок будет использоваться в рамках одной задачи для (локальной) сортировки данных. Использование стратегий Sort и zOrder позволит отсортировать данные до того, как механизм запросов распределит записи по различным задачам, оптимизируя кластеризацию данных по разным задачам.

При приеме потоковых данных вам может потребоваться выполнить быстрое уплотнение информации, принимаемой каждый час. Это можно организовать как-то так:

```
CALL catalog.system.rewrite_data_files(
  table => 'streamingtable',
  strategy => 'binpack',
  where => 'created_at between "2023-01-26 09:00:00" and "2023-01-26 09:59:59" ',
  options => map(
    'rewrite-job-order', 'bytes-asc',
```

```
'target-file-size-bytes','1073741824',
'max-file-group-size-bytes','10737418240',
'partial-progress-enabled','true'
)
)
```

В этом задании по уплотнению используется стратегия binpack, обеспечивающая более быстрое согласование с требованиями соглашения об уровне обслуживания. Оно специально нацелено на прием данных в течение одного часа, в роли которого может выступать самый последний час. Использование поэтапных фиксаций гарантирует, что при записи групп файлов они будут немедленно фиксироваться и обеспечивать немедленное увеличение производительности для нагрузок чтения. Важно отметить, что этот процесс уплотнения фокусируется исключительно на ранее записанных данных, изолируя их от любых параллельных нагрузок записи, которые могут привести к появлению новых файлов данных.

Применение более быстрой стратегии к ограниченному объему данных может значительно ускорить процесс уплотнения. Конечно, разрешив уплотнение за более продолжительный период, чем один час, можно было бы уплотнить данные еще больше, но тогда следует сбалансировать необходимость быстрого выполнения задания уплотнения с необходимостью оптимизации данных. Например, можно добавить дополнительное задание по уплотнению данных за день в ночное время и задание по уплотнению данных за неделю в выходные дни, чтобы обеспечить еще большую оптимизацию, создавая минимальные помехи другим операциям. Имейте в виду, что при уплотнении всегда учитывается текущая спецификация раздела, поэтому при перезаписи данных, записанных согласно старой спецификации, к ним будут применены новые правила секционирования.

Автоматизация уплотнения

Было бы сложно выполнить все соглашения об уровне обслуживания, если бы требовалось вручную запускать задания по уплотнению, поэтому изучение способов автоматизации процессов может принести реальную выгоду. Вот несколько подходов, которые можно использовать для автоматизации задач уплотнения:

- для отправки нужного запроса SQL в Spark или Dremio после завершения приема данных, в определенные моменты или через определенный интервал можно использовать инструмент оркестрации, такой как Airflow, Dagster, Prefect, Argo или Luigi;
- после того как данные попадут в облачное хранилище объектов, для запуска задания можно использовать бессерверные функции;
- для запуска заданий в соответствующие моменты времени можно также использовать задания cron.

Все эти подходы требуют создания сценариев и развертывания сервисов вручную. Однако существует также класс управляемых сервисов каталога Apache Iceberg, который обеспечивает автоматическое обслуживание таблиц и включает уплотнение. Примерами таких сервисов могут служить Dremio Arctic и Tabular.

Сортировка

Прежде чем углубляться в детали стратегии уплотнения sort, важно понять, что сортировка обеспечивает оптимизацию таблиц.

Сортировка («кластеризация») данных имеет особое значение для запросов: она помогает ограничить количество файлов, которые необходимо просканировать, чтобы получить данные, соответствующие критериям в запросе. Сортировка позволяет сосредоточить данные с похожими значениями в меньшем количестве файлов, что позволяет эффективнее планировать запросы.

Например, предположим, что у нас есть набор данных, представляющий всех игроков команд национальной футбольной лиги в 100 файлах Parquet, которые никак не отсортированы. Если вы затребуете список игроков только из команды «Detroit Lions», то даже если файл со 100 записями содержит только одну запись с информацией об игроке из «Detroit Lions», его придется добавить в план запроса и просканировать. Это означает, что может потребоваться просканировать до 53 файлов (максимальное количество игроков, которое может быть в команде НФЛ). Если отсортировать данные в алфавитном порядке по названиям команд, то все игроки из «Detroit Lions» окажутся примерно в четырех файлах (если 100 файлов разделить на 32 команды НФЛ, то получается примерно 3,125 файла на одну команду), которые, вероятно, будут включать несколько игроков из «Green Bay Packers» и «Denver Broncos». Таким образом, отсортировав данные, можно сократить количество сканируемых файлов с 53 до 4, что, как обсуждалось в разделе «Стратегии уплотнения» выше, значительно повысит скорость обработки запроса. На рис. 4.4 показаны преимущества сканирования отсортированных наборов данных.

Сортированные данные могут быть весьма полезны, если способ сортировки данных основан на типичных шаблонах запросов, таких как в этом примере, когда регулярно запрашиваются данные по конкретным командам. Сортировка данных в Apache Iceberg может осуществляться в разных точках, поэтому старайтесь использовать все эти точки.

Существует два основных способа создания таблиц. Один из них – стандартный оператор CREATE TABLE:

```
-- Spark
CREATE TABLE catalog.nfl_players (
  id bigint ,
  player_name varchar,
  team varchar,
```

```

num_of_touchdowns int,
num_of_yards int,
  player_position varchar,
  player_number int,
)

-- Dremio
CREATE TABLE catalog.nfl_players (
  id bigint ,
  player_name varchar,
  team varchar,
  num_of_touchdowns int,
  num_of_yards int,
  player_position varchar,
  player_number int,
)

```

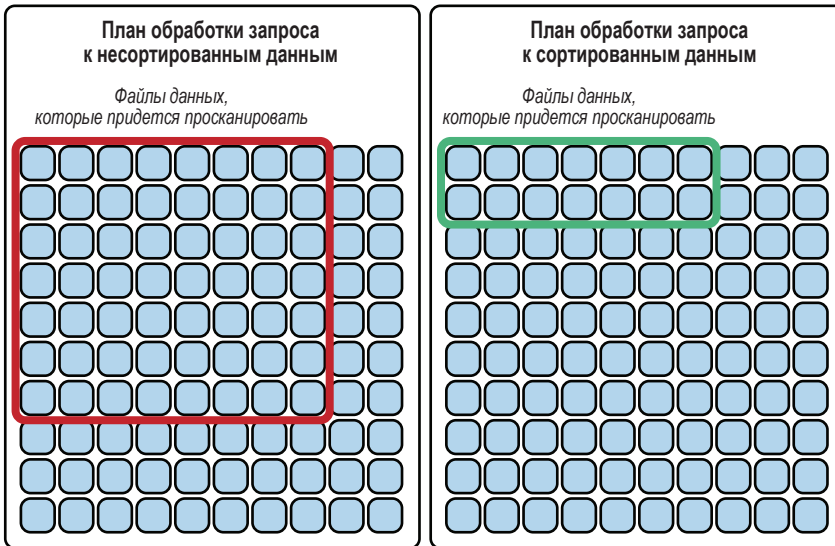


Рис. 4.4 ❖ Сортировка наборов данных уменьшает количество файлов для сканирования

Другой – оператор `CREATE TABLE ... AS SELECT (CTAS)`:

```

-- Spark SQL & Dremio
CREATE TABLE catalog.nfl_players
  AS (SELECT * FROM non_iceberg_teams_table);

```

После создания таблицы обычно устанавливается порядок ее сортировки, который будет использоваться любым механизмом, поддерживающим это свойство, для сортировки данных перед записью, а также стратегией уплотнения `sort`:

```
ALTER TABLE catalog.nfl_teams WRITE ORDERED BY team;
```

При использовании оператора CTAS укажите порядок сортировки в запросе AS:

```
CREATE TABLE catalog.nfl_teams
  AS (SELECT * FROM non_iceberg_teams_table ORDER BY team);

ALTER TABLE catalog.nfl_teams WRITE ORDERED BY team;
```

Оператор ALTER TABLE устанавливает глобальный порядок сортировки, который будет использоваться для всех операций записи в будущем, при условии что механизм записи соблюдает этот порядок. Также порядок сортировки можно указать в операторе INSERT INTO, например:

```
INSERT INTO catalog.nfl_teams
  SELECT *
  FROM staging_table
  ORDER BY team
```

В этом случае данные будут сортироваться по мере их записи, но это не идеальный способ. Так, если в предыдущем примере набор данных НФЛ будет изменяться раз в год при изменении составов команд, может возникнуть множество файлов, содержащих одновременно игроков «Lions» и «Packers», потому что теперь нужно будет записать новый файл с новыми игроками «Lions» за текущий год. Именно здесь в игру вступает стратегия уплотнения sort.

Стратегия уплотнения sort предусматривает сортировку данных по всем файлам, на которые воздействует задание. Так, например, чтобы переписать весь набор данных со всеми игроками, отсортированными по командам глобально, можно выполнить следующую инструкцию:

```
CALL catalog.system.rewrite_data_files(
  table => 'nfl_teams',
  strategy => 'sort',
  sort_order => 'team ASC NULLS LAST'
)
```

Вот описание строки, определяющей порядок сортировки (sort_order):

team

Сортирует данные по полю team.

ASC

Сортировка осуществляется по возрастанию (сортировка по убыванию определяется оператором DESC).

NULL LAST

Требуется поместить всех игроков с пустым значением в поле team в конец, после «Washington Commanders» (NULLS FIRST поместит всех игроков в начало, перед «Arizona Cardinals»).

На рис. 4.5 показан результат такой сортировки.

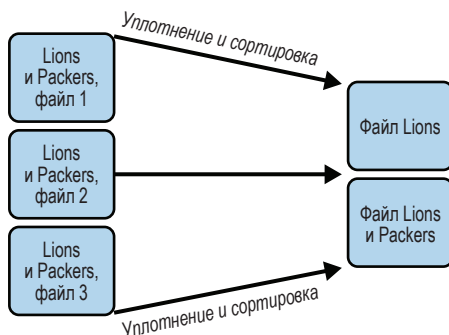


Рис. 4.5 ❖ Уплотнение данных
в меньшее количество файлов с сортировкой

Также есть возможность выполнить сортировку по дополнительным полям. Например, отсортировать данные по командам, а затем внутри каждой команды – в алфавитном порядке по именам игроков. Для этого можно запустить задание со следующими параметрами:

```
CALL catalog.system.rewrite_data_files(
  table => 'nfl_teams',
  strategy => 'sort',
  sort_order => 'team ASC NULLS LAST, name ASC NULLS FIRST'
)
```

Сортировка по команде будет иметь наибольший вес, а по имени – меньший. Вы можете увидеть игроков в таком порядке (рис. 4.6) в файле, где заканчивается состав «Lions» и начинается состав «Packers».

Файл 1 (Lions)	Файл 2 (Lions/Packers)	Файл 3 (Packers)
Starling Thomas V Steven Gilmore Taylor Decker Tom Kennedy Tracy Walker III Tavor Nowaske	Trinity Benson Will Harris Zach Mortonn Aaron Jones AJ Dillon Anders Carlson	Anthony Johnson Jr. Antonio Moultrie Austin Allen B. R. Hatcher Benny Sapp III Bo Melton

Рис. 4.6 ❖ Отсортированный список игроков по файлам

Если конечные пользователи регулярно задают такие вопросы, как «Перечисли игроков “Lions”, имена которых начинаются с буквы А», то такая двойная сортировка еще больше ускорит обработку запросов. Однако если конечные пользователи чаще спрашивают «Перечисли всех игроков НФЛ, чьи имена начинаются с буквы А», то такая сортировка будет не так полезна, поскольку все игроки с именами на букву «А» будут распределены по большему количеству файлов, чем если бы сортировка осуществлялась только по имени. Именно здесь может помочь z-упорядочение.

Итак, чтобы получить максимальную выгоду от стратегии уплотнения с сортировкой, необходимо понимать, какие запросы формируют ваши конечные пользователи, чтобы вы могли отсортировать данные и эффективно отвечать на их вопросы.

Z-упорядочение

Иногда сразу несколько полей оказываются одинаково важными, и в этом случае может пригодиться z-упорядочение, когда данные сортируются по нескольким точкам данных, что дает механизмам больше возможностей сократить количество сканируемых файлов в окончательном плане запроса. Давайте представим, что нам нужно найти элемент Z в сетке 4×4 (рис. 4.7).

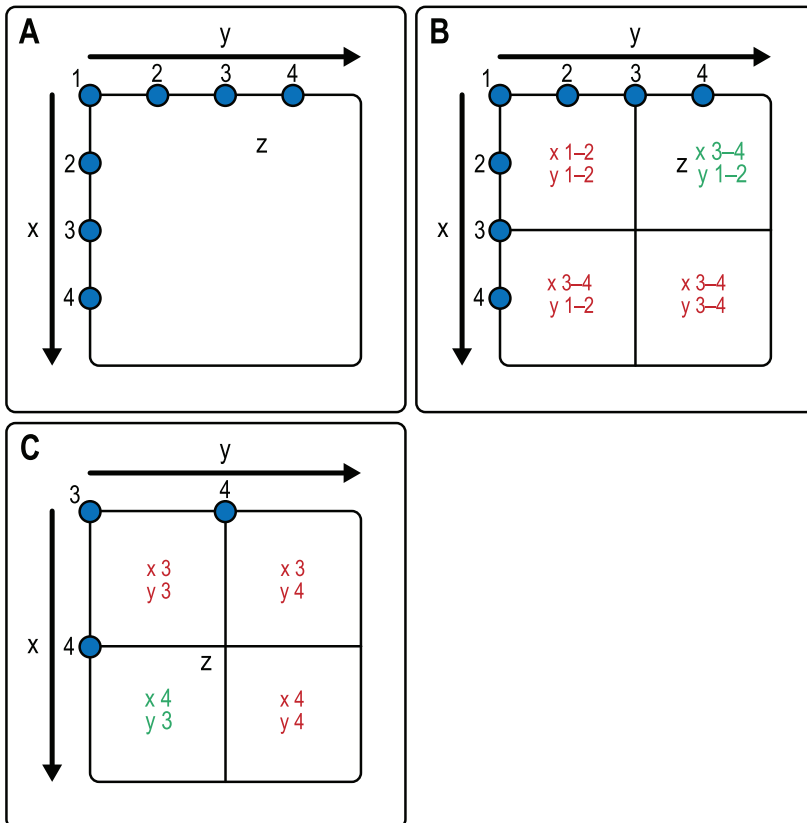


Рис. 4.7 ❖ Основная идея z-упорядочения

Обратимся к рис. 4.7А. Здесь мы имеем значение z, равное 3,5, и нам нужно сузить область поиска в наших данных. Это можно сделать, разбив поле

на четыре квадранта на основе диапазонов значений X и Y, как показано на рис. 4.7В.

Итак, зная, какие данные мы ищем на основе полей, по которым выполнено z-упорядочение, мы сможем избежать сканирования больших областей данных. Затем мы можем взять выбранный квадрант, разбить его еще раз и применить z-упорядочение к данным в этом квадранте, как показано на рис. 4.7С. Поскольку наш поиск основан на нескольких факторах (X и Y), то, применив этот подход, мы могли бы исключить 75 % области поиска.

Аналогично можно сортировать и группировать данные в файлах данных. Например, предположим, что у нас есть набор данных обо всех людях, участвовавших в некотором групповом медицинском исследовании, и нам нужно упорядочить результаты в когорте по возрасту и росту. В этом случае можно применить z-упорядочение. Увидеть результат такого упорядочения можно на рис. 4.8.

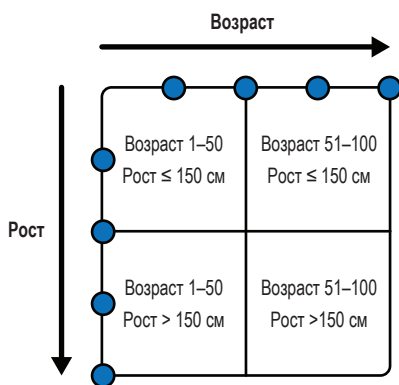


Рис. 4.8 ❖ Z-упорядочение по возрасту и росту

Данные, попадающие в определенный квадрант, будут находиться в одних и тех же файлах данных, что действительно может помочь сократить количество сканируемых файлов при попытке выполнить аналитику по разным возрастным/ростовым группам. Если вы ищете людей с ростом 180 см и старше 60 лет, то сможете сразу исключить файлы данных, принадлежащие другим трем квадрантам.

Это возможно, потому что файлы данных делятся на четыре категории:

- А: файл с записями о людях младше 50 лет и ниже 150 см;
- В: файл с записями о людях в возрасте от 51 до 100 лет и ниже 150 см;
- С: файл с записями о людях в возрасте от 1 до 50 лет и выше 150 см;
- D: файл с записями о людях в возрасте от 51 до 100 лет и выше 150 см.

Зная, что вы ищете людей в возрасте 60 лет и с ростом 180 см, механизм планирования запроса, использующий метаданные Apache Iceberg, сможет исключить из рассмотрения все файлы данных в категориях А, В и С. Имейте в виду, что, даже выполняя поиск только по возрасту, можно заметить выгоду

от группировки, поскольку в этом случае из поиска можно исключить как минимум два из четырех квадрантов.

Для достижения желаемого потребуется выполнить задание по уплотнению:

```
CALL catalog.system.rewrite_data_files(
  table => 'people',
  strategy => 'sort',
  sort_order => 'zorder(age,height)'
)
```

Использование стратегии уплотнения с z-упорядочением не только позволяет сократить количество файлов, хранящих данные, но также гарантирует такое упорядочение данных в этих файлах, которое обеспечит эффективное планирование запросов.

Несмотря на немалую эффективность, сортировка сопряжена с некоторыми проблемами. Во-первых, вновь поступающие данные остаются несортированными до выполнения следующего задания по уплотнению и разбросаны по нескольким файлам. Это происходит потому, что новые данные добавляются в новый файл и потенциально сортируются внутри этого файла, но не в контексте всех предыдущих записей. Во-вторых, файлы по-прежнему могут содержать несколько значений отсортированного поля, что может снижать эффективность запросов, извлекающих данные с определенным значением. Например, в предыдущем примере файлы содержали данные как об игроках «Lions», так и «Packers», и наличие записей об игроках «Packers» снижет эффективность поиска игроков «Lions», потому что эти записи тоже приходится сканировать.

Решить эту проблему можно с помощью секционирования.

Секционирование

Если известно, что определенное поле имеет решающее значение для доступа к данным, то, возможно, вы не захотите ограничиваться только сортировкой и решите задействовать секционирование. Данные в секционированных таблицах не просто упорядочиваются в порядке сортировки, но записи с разными значениями ключевого поля сохраняются в отдельные файлы данных.

Например, если вы работаете в сфере политики, то вам, вероятно, часто придется запрашивать данные об избирателях на основе их партийной принадлежности, что делает это поле хорошим разделителем. Это будет означать, что все избиратели «Синей» партии будут сохранены в свои файлы, а избиратели «Красной», «Желтой» и «Зеленой» партий – в свои. При запросе списка избирателей «Желтой» партии достаточно просканировать только файлы из раздела, выделенного для этой партии (рис. 4.9).

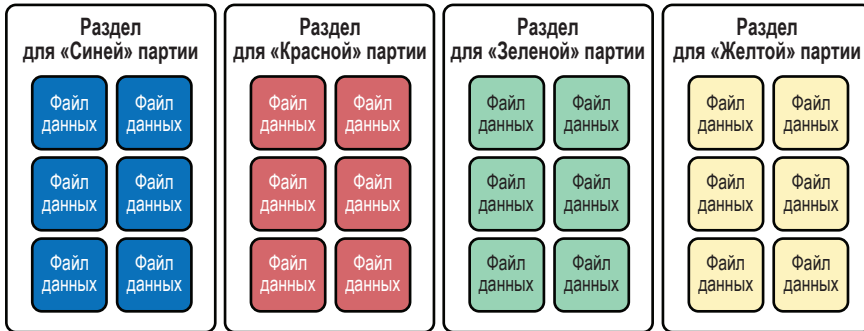


Рис. 4.9 ❖ Секционирование и объединение данных в группы файлов

Традиционно секционирование таблицы по производным значениям определенного поля требовало создания дополнительного поля, которое нужно было поддерживать отдельно, и чтобы пользователи не забывали указывать это отдельное поле в запросе. Например:

- для секционирования по дню, месяцу или году в столбце с отметкой времени требовалось создать дополнительный столбец на основе отметки времени, выражающий год, месяц или день;
- для секционирования по первой букве текстового значения необходимо было создать дополнительный столбец, содержащий только эту букву;
- секционирование в сегменты (заданное количество сегментов для равномерного распределения записей на основе хеш-функции) требовало создания дополнительного столбца, в котором указывалось, к какому сегменту принадлежит запись.

Затем при создании таблицы нужно было задать секционирование на основе производных полей:

```
--Spark SQL
CREATE TABLE MyHiveTable (...) PARTITIONED BY month;
```

После этого вы должны были вручную вычислять производное значение при вставке каждой новой записи:

```
INSERT INTO MyTable (SELECT MONTH(time) AS month, ... FROM data_source);
```

Механизм запросов не знал о связи между исходным и производным полями. Это означало, что выгоду мог бы получить только такой запрос:

```
SELECT * FROM MYTABLE
WHERE time BETWEEN '2022-07-01 00:00:00' AND '2022-07-31 00:00:00'
AND month = 7;
```

Однако часто пользователи не знали о существовании такого столбца. Это означало, что в большинстве случаев запросы, подобные следующему, при-

водили к полному сканированию таблицы, в результате чего на их обработку тратилось гораздо больше времени и потреблялось гораздо больше ресурсов:

```
SELECT * FROM MYTABLE
WHERE time BETWEEN '2022-07-01 00:00:00' AND '2022-07-31 00:00:00';
```

Предыдущий запрос более понятен бизнес-пользователям и аналитикам, которые могут быть не совсем осведомлены о внутреннем устройстве таблицы, но такие запросы приводят ко множеству избыточных полных сканирований таблицы. В подобных случаях можно использовать возможность скрытого секционирования, имеющуюся в Iceberg.

Скрытое секционирование

Apache Iceberg обрабатывает секционирование совершенно иначе, избавляя от многих из этих проблем, используя подход, называемый *скрытым секционированием*.

Прежде всего следует заметить, что Apache Iceberg автоматически отслеживает секционирование. Но в отслеживании полагается не на физическое местоположение файлов, а на диапазон значений в разделах на уровне моментального снимка и манифеста, обеспечивая множество уровней гибкости:

- вместо создания дополнительных столбцов для секционирования можно использовать встроенные преобразования, которые могут применяться механизмами и инструментами при планировании запросов;
- поскольку при использовании этих преобразований вам не нужен дополнительный столбец, в файлах данных сохраняется меньше данных;
- поскольку метаданные помогают механизму узнать о преобразовании исходного столбца, вы можете фильтровать только по исходному столбцу и получить преимущества секционирования.

Это означает, что если создать таблицу, секционированную по месяцам:

```
CREATE TABLE catalog.MyTable (...) PARTITIONED BY months(time) USING iceberg;
```

то следующий запрос получит выгоду от секционирования:

```
SELECT * FROM MYTABLE
WHERE time BETWEEN '2022-07-01 00:00:00' AND '2022-07-31 00:00:00';
```

Как нетрудно заметить, в предыдущем операторе CREATE TABLE к целевому столбцу применяется преобразование в форме функции. При планировании секционирования доступно несколько преобразований:

- year (только год);
- month (месяц и год);
- day (день, месяц и год);

- hour (час, день, месяц и год);
- truncate;
- bucket.

year, month, day и hour преобразуют значения в столбце с отметкой времени. Имейте в виду, что если вы укажете month, секционирование будет выполняться по году и месяцу, а если вы укажете day, то секционирование будет выполнено по году, месяцу и дню, поэтому нет никакой необходимости использовать несколько преобразований для секционирования с разной степенью детализации.

Преобразование truncate секционирует таблицу на основе усеченного значения столбца. Например, если вы решите секционировать таблицу по первой букве в имени человека, то можете создать ее так:

```
CREATE TABLE catalog.MyTable (...)  
PARTITIONED BY truncate(name, 1) USING iceberg;
```

Преобразование bucket идеально подходит для секционирования на основе поля с высокой мощностью (со множеством уникальных значений). Это преобразование использует хеш-функцию для распределения записей по указанному количеству разделов. Так, например, можно секционировать данные избирателей по почтовым индексам, но индексов так много, что в итоге получится слишком много разделов с маленькими файлами данных. В таком случае можно организовать секционирование примерно так:

```
CREATE TABLE catalog.voters (...)  
PARTITIONED BY bucket(24, zip) USING iceberg;
```

В каждый раздел будет включено несколько почтовых индексов, тем не менее когда потребуется отыскать конкретный почтовый индекс, вам не придется выполнять полное сканирование таблицы, достаточно будет просканировать один раздел, включающий искомый почтовый индекс. Таким образом, скрытое секционирование Apache Iceberg дает более выразительный способ реализации типичных шаблонов секционирования, и оно не требует, чтобы конечный пользователь обладал какими-то дополнительными знаниями, кроме привычного для них умения фильтровать по полям.

Эволюция разделов

Еще одна проблема традиционного секционирования, основанного на физической структуре файлов, размещаемых в подкаталогах, заключается в том, что для изменения способа секционирования таблицы требуется переписать ее целиком. Эта проблема неизбежно возникает с развитием данных и шаблонов запросов, что требует переосмысления способов секционирования и сортировки данных.

В Apache Iceberg эта проблема решается с помощью отслеживания метаданных, поскольку в метаданных хранятся не только значения разделов, но

и исторические схемы секционирования, что позволяет развивать схемы секционирования эволюционным путем. То есть если данные в двух разных файлах были записаны с использованием двух разных схем секционирования, то метаданные Iceberg проинформируют механизм чтения, чтобы тот мог создать план со схемой разделов А отдельно от схемы разделов В и затем сформировал общий план сканирования.

Например, предположим, что у нас есть таблица со списком членов сообщества, секционированная по году регистрации участников:

```
CREATE TABLE catalog.members (...)  
PARTITIONED BY years(registration_ts) USING iceberg;
```

Затем, несколько лет спустя, если темпы роста числа членов сообщества начнут бить рекорды по месяцам, то вы сможете изменить таблицу и настроить секционирование следующим образом:

```
ALTER TABLE catalog.members ADD PARTITION FIELD months(registration_ts)
```

Особенность преобразований секционирования в Apache Iceberg, связанных с датами, заключается в том, что для перехода к более детализированному секционированию не требуется удалять менее детализированные правила. Однако если вы используете преобразование bucket или truncate и решаете отказаться от секционирования таблицы по определенному полю, то можете обновить схему секционирования следующим образом:

```
ALTER TABLE catalog.members DROP PARTITION FIELD bucket(24, id);
```

При обновлении схемы секционирования новые правила применяются только к новым данным, поэтому нет необходимости перезаписывать существующие данные. Кроме того, имейте в виду, что любые данные, переписанные процедурой `rewriteDataFiles`, будут перезаписаны с использованием новой схемы секционирования, поэтому если нужно сохранить старые данные в старой схеме, используйте соответствующие фильтры в заданиях по уплотнению.

Другие соображения по секционированию

Предположим, вы переносите таблицу Hive с помощью процедуры миграции (описанной в главе 13). В настоящее время она может быть секционирована по производному столбцу (например, по столбцу месяца, который основан на столбце с отметкой времени в той же таблице), но вы хотите сообщить Apache Iceberg, что теперь должно использоваться встроенное преобразование. Организовать это можно с помощью команды `REPLACE PARTITION`:

```
ALTER TABLE catalog.members REPLACE PARTITION FIELD registration_day  
WITH days(registration_ts) AS day_of_registration;
```

Это не приведет к изменению каких-либо файлов данных, но позволит метаданным отслеживать значения разделов с помощью преобразований Iceberg.

Оптимизировать таблицы можно разными способами. Например, использование секционирования для записи данных с уникальными значениями в уникальные файлы, сортировка данных в этих файлах и последующее уплотнение этих файлов в меньшее количество файлов большего размера обеспечит хорошую производительность таблицы. Причем оптимизация возможна не только для типичных вариантов использования, но также для конкретных случаев, таких как изменение и удаление на уровне записей, для которых можно использовать стратегии копирования при записи и слияния при чтении.

Копирование при записи и слияние при чтении

Еще один фактор, влияющий на скорость рабочих нагрузок, – способ обработки изменений на уровне записей. При добавлении новых данных эти данные записываются в новый файл, но при изменении или удалении существующих записей необходимо учитывать некоторые соображения:

- в озерах данных и, следовательно, в Apache Iceberg файлы данных нельзя изменить. Это дает множество преимуществ, в том числе возможность изолировать снимки (поскольку файлы, на которые ссылаются старые снимки, хранят согласованные данные);
- если, к примеру, изменяется 10 записей, то нет никакой гарантии, что они находятся в одном файле, поэтому может потребоваться перезаписать 10 файлов и каждую запись в них, чтобы обновить лишь 10 записей в новом снимке.

Существует три подхода к обработке изменений на уровне записей, которые мы подробно опишем далее в этом разделе, они перечислены в табл. 4.2.

Таблица 4.2. Стратегии обработки изменений на уровне записей в Apache Iceberg

Стратегия обработки изменений	Скорость чтения	Скорость записи	Лучшая практика
Копирование при записи	Самое быстрое чтение	Самое медленное изменение/удаление	
Слияние при чтении (позиционное удаление)	Быстрое чтение	Быстрое изменение/удаление	Регулярное уплотнение для минимизации стоимости чтения
Слияние при чтении (удаление по равенству)	Медленное чтение	Самое быстрое изменение/удаление	Максимально частое уплотнение для минимизации стоимости чтения

Копирование при записи

Стратегия по умолчанию называется копированием при записи (copy-on-write, COW). При ее использовании файл данных перезаписывается и занимает свое место в новом снимке, даже если изменяется или удаляется всего одна запись. Пример показан на рис. 4.10.

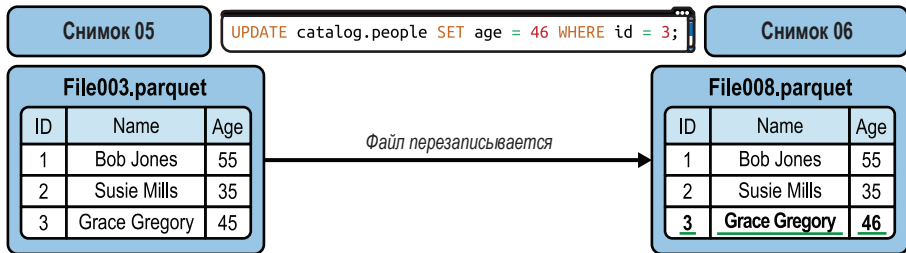


Рис. 4.10 ❖ Результат работы стратегии копирования при записи, когда изменяется одна запись

Эта стратегия идеально подходит для оптимизации операции чтения, потому что при их выполнении не требуется согласовывать прочитанные данные с информацией в файлах удаления. Однако при наличии рабочих нагрузок, которые регулярно изменяют данные на уровне записей, копирование файлов данных целиком может замедлить операции настолько, что будут нарушены соглашения об уровне обслуживания. К плюсам этой стратегии относится максимально быстрое чтение, а к минусам – очень медленное изменение и удаление на уровне записей.

Слияние при чтении

Альтернативой копированию при записи является стратегия объединения при чтении (merge-on-read, MOR), когда не происходит перезапись всего файла данных, а изменения в записях фиксируются в файле удаления.

При удалении записи:

- ссылка на запись добавляется в файл удаления;
- при выполнении операции чтения таблицы содержимое файла данных согласуется с файлом удаления.

При изменении записи:

- ссылка на запись добавляется в файл удаления;
- создается новый файл данных, содержащий только измененную запись;
- при выполнении операции чтения таблицы она игнорирует старую версию записи из-за присутствия ссылки на нее в файле удаления и использует новую версию в новом файле данных.

Пример показан на рис. 4.11.

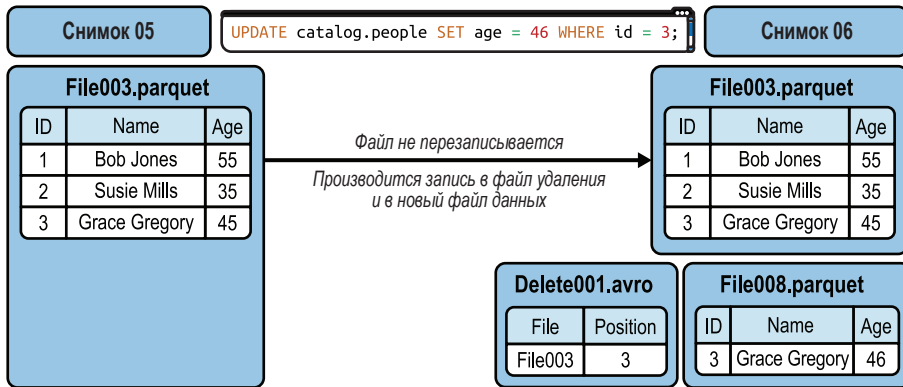


Рис. 4.11 ❖ Результат работы стратегии слияния при чтении, когда изменяется одна запись

Это позволяет избежать необходимости перезаписывать неизменные записи в новые файлы только потому, что они существуют в файле данных с измененной записью, что ускоряет операцию записи. Но за это приходится платить более медленным чтением, потому что механизму обработки запросов придется просканировать файлы удаления, чтобы узнать, какие записи в файлах данных следует игнорировать.

Чтобы минимизировать затраты на чтение, вам придется регулярно запускать задания уплотнения, а для эффективной работы заданий уплотнения необходимо следовать некоторым рекомендациям, с которыми вы познакомились ранее:

- использовать оператор `filter/where`, чтобы уплотнять только файлы, загруженные за последний период времени (час, день);
- использовать режим поэтапного выполнения (`partial progress`) для выполнения фиксаций по мере перезаписи групп файлов, чтобы запросы на чтение могли получить выгоды от уплотнения как можно раньше.

Применяя эти рекомендации, можно ускорить операции записи в тяжелых рабочих нагрузках изменения данных и минимизировать затраты на операции чтения. К плюсам этой стратегии относится довольно быстрое изменение на уровне записей, а к минусам – более медленное чтение из-за необходимости согласования с информацией в файлах удаления.

Применение стратегии MOR с ее файлами удаления позволяет определять, какие записи в существующих файлах данных должны игнорироваться в последующих операциях чтения. Воспользуемся простой аналогией, чтобы помочь вам понять общую концепцию различных типов файлов удаления. (Имейте в виду, что выбор типа файлов удаления обычно определяется в настройках механизма для конкретных случаев использования, а не в настройках таблицы.)

Если имеется большой объем данных и требуется исключить определенную запись, то сделать это можно несколькими способами:

- записи можно отыскивать по их местоположению в наборе данных, это похоже на поиск своего места в кинозале по номеру на билете;
- записи можно отыскивать по их содержанию, как, например, поиск друга в толпе по ярко-красной шляпе.

В первом случае используются так называемые *файлы позиционного удаления*, а во втором – *файлы удаления по равенству*. Оба метода имеют сильные и слабые стороны. Это означает, что в одних ситуациях выгоднее использовать один вариант, а в других – другой. Все дело в том, что лучше всего подходит именно вам!

Рассмотрим эти два типа файлов удаления. Файл позиционного удаления хранит ссылки на записи в конкретных файлах данных, которые следует игнорировать. В следующей таблице показан пример данных в файле позиционного удаления:

Удаленная запись (позиционное удаление)	
Файл	Позиция
001.parquet	0
001.parquet	5
006.parquet	5

Записи, позиции которых указаны в файле удаления, будут игнорироваться при чтении указанных файлов данных. Такой способ требует меньше затрат времени на чтение, потому что имеется конкретная позиция, запись в которой необходимо пропустить, но влечет дополнительные затраты времени на запись, так как при удалении механизм должен узнать местоположение удаленной записи, для чего нужно прочитать файл с удаляемой записью и определить ее позицию.

В файле удаления по равенству указываются значения, которые содержит игнорируемая запись. В следующей таблице показано, как могут располагаться данные в файле удаления по равенству:

Удаленная запись (позиционное удаление)	
Team	State
Yellow	NY
Green	MA

В этом случае не тратится лишнее время на запись, поскольку не нужно открывать и читать файлы для определения целевых значений, но затраты времени на чтение значительно выше из-за отсутствия информации о существовании записей с совпадающими значениями, поэтому при чтении данных необходимо проверять каждую запись на наличие в ней совпадающих значений. Удаление по принципу равенства отлично подходит, когда нужна

максимальная скорость записи, но чтобы уменьшить влияние операций удаления на операции чтения, следует предусмотреть агрессивное уплотнение.

Настройка COW и MOR

Настройка таблицы для обработки изменений на уровне записей с использованием стратегии COW или MOR зависит от:

- свойств таблицы;
- поддержки стратегии MOR механизмом записи в Apache Iceberg.

Следующие свойства таблицы определяют выбор стратегии COW или MOR при обработке конкретной транзакции:

`write.delete.mode`

Подход, используемый для выполнения транзакций удаления.

`write.update.mode`

Подход, используемый для выполнения транзакций изменения.

`write.merge.mode`

Подход, используемый для выполнения транзакций слияния.

Имейте в виду, что для этих и всех других свойств таблицы Apache Iceberg, хотя многие из них являются частью спецификации, соблюдение спецификации зависит от конкретного вычислительного механизма. Вы можете столкнуться с отклонениями в поведении, поэтому узнайте, какие свойства таблиц поддерживаются используемыми у вас механизмами при выполнении конкретных заданий. Разработчики механизмов запросов стремятся обеспечить поддержку всех свойств таблиц Apache Iceberg, но для этого нужны реализации для конкретной архитектуры механизма. Со временем все эти свойства будут реализованы во всех механизмах, чтобы обеспечить одинаковое их поведение.

Поскольку поддержка Apache Iceberg в Apache Spark реализуется в рамках проекта Apache Iceberg, Spark поддерживает все эти свойства, и в Spark их можно установить при создании таблицы:

```
CREATE TABLE catalog.people (
  id int,
  first_name string,
  last_name string
) TBLPROPERTIES (
  'write.delete.mode'='copy-on-write',
  'write.update.mode'='merge-on-read',
  'write.merge.mode'='merge-on-read'
) USING iceberg;
```

Это свойство также можно установить после создания таблицы с помощью оператора `ALTER TABLE`:

```
ALTER TABLE catalog.people SET TBLPROPERTIES (
  'write.delete.mode'='merge-on-read',
  'write.update.mode'='copy-on-write',
  'write.merge.mode'='copy-on-write'
);
```

Как видите, это совсем несложно. Но, работая с механизмами, отличными от Apache Spark, помните, что:

- свойства таблицы могут не поддерживаться. Реализация поддержки зависит от механизма;
- при использовании стратегии MOR убедитесь, что используемые вами механизмы запросов способны читать файлы удаления.

Другие соображения

Помимо файлов данных и особенностей их организации, существует множество других рычагов повышения производительности. Многие из них мы обсудим в следующих разделах.

Коллекция метрик

Как обсуждалось в главе 2, манифест для каждой группы файлов данных содержит метрики для каждого поля в таблице, что помогает быстро определять минимальное/максимальное значение и применять другие оптимизации. В число поддерживаемых метрик на уровне столбца входят:

- счетчики значений, пустых значений и отдельных значений;
- верхние и нижние граничные значения.

Если ваши таблицы очень широкие (т. е. имеют большое количество полей, например более 100), то количество отслеживаемых метрик может отрицательно сказаться на скорости чтения метаданных. К счастью, используя свойства таблицы Apache Iceberg, можно точно указать, для каких столбцов создавать метрики, а для каких нет. Соответственно, вы сможете настроить отслеживание метрик по столбцам, которые часто используются в фильтрах запросов, и не собирать метрики для других столбцов, чтобы они не раздували метаданные.

Настроить уровень сбора метрик для нужных столбцов (необязательно указывать их все) можно в свойствах таблицы, например:

```
ALTER TABLE catalog.db.students SET TBLPROPERTIES (
  'write.metadata.metrics.column.col1'='none',
  'write.metadata.metrics.column.col2'='full',
  'write.metadata.metrics.column.col3'='counts',
  'write.metadata.metrics.column.col4'='truncate(16)',
);
```

Как видите, есть возможность настроить сбор метрик для каждого отдельного столбца в несколько потенциальных значений:

`none`

Не собирать никаких метрик.

`counts`

Собирать только счетчики (значений, отдельных значений, пустых значений).

`truncate(XX)`

Подсчитать и усечь значение до определенного количества символов и с учетом этого усечения определить верхнюю и нижнюю границы. Так, например, строковый столбец можно усечь до 16 символов, и диапазоны его значений в метаданных будут основаны на усеченных значениях.

`full`

Счетчики и границы должны основываться на полном значении.

Нет необходимости явно определять способ определения метрик для каждого столбца, потому что по умолчанию в Iceberg используется значение `truncate(16)`.

Перезапись манифестов

Иногда проблема не в файлах данных, потому что они имеют оптимальный размер и содержат отсортированные данные, а в том, что они записаны в нескольких снимках, поэтому в отдельном манифесте может быть перечислено больше файлов данных, чем нужно. Хотя манифесты являются довольно легковесными файлами, но здесь действует то же правило: чем больше файлов, тем больше затраты на их обработку. Для смягчения этой проблемы существует отдельная процедура `rewriteManifests`, которая перезаписывает только файлы манифестов, чтобы уменьшить их общее количество и расширить списки файлов данных в них:

```
CALL catalog.system.rewrite_manifests('MyTable')
```

Если при выполнении этой операции возникнут проблемы с памятью, то можно попробовать отключить кеширование в Spark, передав второй аргумент `false`. Попытка перезаписать большое количество манифестов при включенном кешировании в Spark может привести к проблемам с отдельными узлами-исполнителями:

```
CALL catalog.system.rewrite_manifests('MyTable', false)
```

Выполнять эту операцию рекомендуется, когда файлы данных имеют оптимальные размеры, а количество файлов манифестов слишком велико. Например, если у вас есть 5 Гбайт данных в одном разделе, хранящихся в 10 файлах данных, но эти файлы перечислены в пяти файлах манифестов,

то нет нужды уплотнять файлы данных, но, вероятно, будет желательно объединить список с 10 файлами в один манифест.

Оптимизация хранилища

При изменении данных в таблице или выполнении заданий уплотнения создаются новые файлы, но старые файлы не удаляются, потому что они связаны с историческими снимками таблицы. Чтобы избежать хранения большого количества ненужных данных, следует периодически объявлять снимки устаревшими. Но имейте в виду, что после этого вы не сможете получить исторические данные из снимка, объявленного устаревшим. В процессе выполнения этой операции все файлы данных, не связанные с действительными снимками, будут удалены.

Вот как можно объявить устаревшими снимки, созданные в определенный момент времени или до него:

```
CALL catalog.system.expire_snapshots('MyTable',
TIMESTAMP '2023-02-01 00:00:00.000', 100)
```

Второй аргумент – это минимальное количество сохраняемых снимков (по умолчанию сохраняются снимки за последние пять дней). В данном случае устаревшими будут объявлены все снимки, созданные в указанный момент времени или до него, но если снимок попадает в число 100 последних снимков, то он не будет объявлен устаревшим.

Также можно объявить устаревшими снимки с определенным идентификатором:

```
CALL catalog.system.expire_snapshots(table => 'MyTable',
snapshot_ids => ARRAY(53))
```

В этом примере устаревшим объявляется снимок с идентификатором 53. Узнать идентификатор снимка можно, заглянув в файл *metadata.json* и воспользовавшись таблицами метаданных, подробно описанными в главе 10. У вас может иметься снимок, в котором случайно оказались конфиденциальные данные, и вы хотите удалить этот единственный снимок, объявив его устаревшим, и очистить файлы данных, созданные в этой транзакции. В таком случае вам поможет операция удаления снимка по его идентификатору. Удаление снимка – это транзакция, поэтому будет создан новый файл *metadata.json* с обновленным списком действительных снимков.

Процедуре `expire_snapshots` можно передать шесть аргументов:

`table`

Таблица, к которой применяется операция

`older_than`

Устаревшими будут признаны все снимки, созданные в этот момент времени или раньше.

`retain_last`

Минимальное количество снимков, которые должны сохраниться.

`snapshot_ids`

Устаревшими будут признаны снимки с указанными идентификаторами.

`max_concurrent_deletes`

Количество потоков, используемых для удаления файлов.

`stream_results`

Если в этом аргументе передать `true`, то удаленные файлы будут отправлены драйверу Spark через раздел Resilient Distributed Dataset (RDD), что может пригодиться для предотвращения проблем с нехваткой памяти при удалении больших файлов.

Еще один фактор для оптимизации хранилища – потерянные файлы. Это файлы и артефакты, которые накапливаются в каталоге данных таблицы, но не отслеживаются в дереве метаданных, так как были созданы заданиями, завершившимися неудачей. Эти файлы не будут удалены вместе с устаревшими снимками, поэтому для решения этой проблемы время от времени следует запускать специальную процедуру, которая проверит каждый файл в каталоге таблицы на его принадлежность активным снимкам. Это может быть очень ресурсоемкий процесс (поэтому не следует выполнять его слишком часто). Удалить потерянные файлы можно следующей командой:

```
CALL catalog.system.remove_orphan_files(table => 'MyTable')
```

Процедуре `remove_orphan_files` можно передать следующие аргументы:

`table`

Таблица, к которой применяется операция.

`older_than`

Процедура удалит только файлы, созданные в этот момент времени или раньше.

`location`

Где искать потерянные файлы; если не указано, то используется местоположение таблицы по умолчанию.

`dry_run`

Если в этом аргументе передать `true`, то вместо удаления файлов процедура просто выведет их список.

`max_concurrent_deletes`

Количество потоков, используемых для удаления файлов.

В большинстве случаев данные таблиц помещаются в местоположение по умолчанию, но иногда в таблицу добавляются внешние файлы с помощью процедуры `addFiles` (описанной в главе 13), и позднее может потребоваться удалить их. Для таких случаев и служит аргумент `location`.

Режим распределения записи

Режим распределения записи требует понимания, как системы массово-параллельной обработки (Massively Parallel Processing, MPP) выполняют запись в файлы. Эти системы распределяют работу между несколькими узлами, каждый из которых выполняет свою работу или решает свою задачу. Под словами «распределение записи» понимается распределение записи данных по разным задачам. Если не установлен конкретный режим распределения записи, то данные будут распределяться произвольно. Первые X записей будут переданы первой задаче, следующие X записей – следующей задаче и т. д.

Все задачи обрабатываются отдельно, поэтому каждая из них создаст хотя бы один файл в каждом разделе, куда она должна сохранить хотя бы одну запись. Таким образом, если у вас есть 10 записей, принадлежащих разделу А, и вы распределили эти записи по 10 задачам, то в этом разделе будет создано 10 файлов по одной записи в каждом, что очень неэффективно.

Было бы лучше, если бы все записи для одного раздела передавались одним и тем же задачам, чтобы те могли записать их в один и тот же файл. В таких случаях можно использовать механизм распределения записи между задачами. Есть три варианта механизмов:

`none`

Не выполняет никакого специального распределения. Обеспечивает самую высокую скорость записи и идеально подходит для предварительно отсортированных данных.

`hash`

Данные по хешу ключа раздела.

`range`

Данные распределяются по диапазонам значений ключа раздела или по порядку сортировки.

При распределении по хешу ключа раздела все записи пропускаются через хеш-функцию и группируются на основе результата. В зависимости от вида хеш-функции в одной группе может оказаться несколько записей. Например, если в ваших данных есть значения 1, 2, 3, 4, 5 и 6, то вы можете передать задачам А, В и С записи со значениями 1 и 4, 2 и 5 и 3 и 6 соответственно. В этом случае будет создано минимальное число файлов в разделах и потребуются меньше последовательных операций записи.

При распределении по диапазону данные сортируются и равномерно распределяются между задачами, поэтому, продолжая предыдущий пример, задача А получит значения 1 и 2, задача В – значения 3 и 4 и задача С – значения 5 и 6. Сортировка выполняется по значению раздела и с учетом значения свойства `SortOrder`, если оно определено в таблице. Другими словами, если свойство `SortOrder` определено, то данные будут сгруппированы в задачи не только по значению раздела, но и в соответствии со значением свойства `SortOrder`. Этот способ распределения идеально подходит для данных, ко-

торые могут выиграть от кластеризации по определенным полям. Однако сортировка данных для последовательного распределения требует больше накладных расходов, чем распределение по хешу ключа раздела.

В таблице также можно задать свойство, управляющее распределением записи при удалении, изменении и слиянии:

```
ALTER TABLE catalog.MyTable SET TBLPROPERTIES (  
    'write.distribution-mode'='hash',  
    'write.delete.distribution-mode'='none',  
    'write.update.distribution-mode'='range',  
    'write.merge.distribution-mode'='hash',  
);
```

В сценариях, когда регулярно обновляется множество записей, но они редко удаляются, может потребоваться использовать разные режимы распределения, поскольку каждый режим может быть более или менее выгодным в зависимости от шаблонов запросов.

Объектное хранилище

Объектное хранилище – уникальный подход к хранению данных. Вместо хранения файлов в дереве каталогов, как в традиционной файловой системе, объектное хранилище распределяет все данные по так называемым *корзинам* (buckets). Каждый файл становится объектом и получает набор сопровождающих его метаданных, которые сообщают самую разнообразную информацию о файле. Такая организация обеспечивает улучшенный параллелизм и устойчивость к ошибкам, потому что появляется возможность реплицировать базовые файлы для регионального или параллельного доступа, тогда как с точки зрения пользователей они просто взаимодействуют с файлом как с простым «объектом».

Чтобы получить файл из объектного хранилища, не нужно просматривать папки. Вместо этого следует использовать API. Взаимодействуя с веб-сайтом, вы используете запросы GET и PUT, аналогично осуществляется доступ к данным в объектном хранилище. Например, вы можете использовать запрос GET, чтобы запросить файл. В этом случае система проверит метаданные, найдет нужный файл, и вы получите свои данные.

Такой подход, ориентированный на API, помогает системе манипулировать вашими данными, например создавать копии в разных местах или одновременно обрабатывать множество запросов. Объектное хранилище, которое предоставляет большинство поставщиков облачных услуг, идеально подходит для озер данных и озерных хранилищ данных, но у него есть одно потенциально узкое место.

Из-за особенностей архитектуры объектных хранилищ и обработки параллельных операций (<https://oreil.ly/jBLXG>) часто имеют место ограничения на количество запросов, которые можно отправить файлам с одним и тем же «префиксом». Поэтому доступ к файлам `/prefix1/fileA.txt` и `/prefix1/fileB.txt`

засчитывается в ограничение, даже притом что это разные файлы. Такое ограничение становится проблемой при работе с разделами, содержащими большое количество файлов, потому что запросы могут повлечь необходимость обращения ко множеству файлов в одном разделе и, соответственно, столкнуться с ограничением, замедляющим выполнение запроса.

Ослабить влияние этого ограничения можно, уменьшив количество файлов в разделе с помощью процедуры уплотнения, но в Apache Iceberg есть другое решение: его независимость от физического местоположения позволяет записывать файлы в один и тот же раздел с разными префиксами.

Эта возможность включается в свойства таблицы следующим образом:

```
ALTER TABLE catalog.MyTable SET TBLPROPERTIES (
    'write.object-storage.enabled'= true
);
```

Это позволит распределить файлы в одном разделе по множеству префиксов, включая хеш, чтобы избежать ограничений.

Итак, вместо

```
s3://bucket/database/table/field=value1/datafile1.parquet
s3://bucket/database/table/field=value1/datafile2.parquet
s3://bucket/database/table/field=value1/datafile3.parquet
```

вы получите:

```
s3://bucket/4809098/database/table/field=value1/datafile1.parquet
s3://bucket/5840329/database/table/field=value1/datafile2.parquet
s3://bucket/2342344/database/table/field=value1/datafile3.parquet
```

Благодаря добавлению хеша в пути к файлам все файлы в одном разделе теперь будут обрабатываться без ограничений, как имеющие разные префиксы.

Фильтры Блума

Фильтр Блума позволяет узнать, присутствует ли значение в наборе данных. Представьте себе набор битов (нули и единицы в двоичном коде), длина которого может устанавливаться по вашему выбору. Далее, добавляя данные в свой набор, вы пропускаете каждое значение через *хеш-функцию*, которая определяет соответствующий бит в битовой линейке, и вы меняете этот бит с 0 на 1. Введенный бит подобен флагу, который говорит: «Значение, которое хешируется в позицию этого бита, может присутствовать в наборе данных».

Например, предположим, что мы пропустили 1000 записей через фильтр Блума, имеющий длину 10 бит. По завершении наш фильтр Блума может выглядеть так:

```
[0,1,1,0,0,1,1,1,1,0]
```

Представим, что мы ищем определенное значение. Пусть это будет значение X. Мы пропускаем X через ту же хеш-функцию, и она указывает нам на бит в третьей позиции в нашей битовой линейке. В этой позиции у нас находится 1. Это означает, что есть вероятность присутствия значения X в наборе данных, поскольку раньше в него было записано значение, которое хешируется в эту позицию. Далее мы сканируем набор данных и убеждаемся, что X действительно присутствует в нем.

Теперь возьмем другое значение. Пусть это будет значение Y. Пропустив Y через нашу хеш-функцию, мы получили четвертую позицию в битовой линейке. В этой позиции в фильтре Блума стоит 0, а это означает, что в номер этой позиции не было хешировано ни одного значения. Соответственно, мы можем с уверенностью сказать, что значение Y отсутствует в наборе данных, и сэкономить время, отказавшись от его сканирования.

Фильтры Блума удобны тем, что помогают избежать ненужного сканирования данных. Чтобы сделать их более точными, можно увеличить чувствительность хеш-функции и размер битовой линейки. Но помните: чем больше данных добавляется, тем большего размера требуется фильтр Блума для сохранения точности и тем больше места ему потребуется для хранения. Как и многое в нашей жизни, важно найти правильный баланс.

Включить запись фильтра Блума для определенного столбца в файлах Parquet (или для файлов ORC) можно в свойствах таблицы:

```
ALTER TABLE catalog.MyTable SET TBLPROPERTIES (
  'write.parquet.bloom-filter-enabled.column.col1'= true,
  'write.parquet.bloom-filter-max-bytes'= 1048576
);
```

После этого механизмы обработки запросов смогут воспользоваться преимуществами фильтра Блума и ускорить чтение, пропуская файлы данных, в которых фильтры Блума четко указывают на отсутствие искомых значений.

Заключение

В этой главе мы рассмотрели различные стратегии оптимизации производительности таблиц Iceberg, такие как уплотнение, сортировка, z-упорядочение, копирование при записи и слияние при чтении, а также скрытое секционирование. Каждая из этих стратегий играет важную роль в повышении эффективности запросов, сокращении времени чтения и записи и оптимальном использовании ресурсов. Понимание и эффективная реализация этих стратегий могут значительно улучшить работу таблиц Apache Iceberg.

В главе 5 мы обратим наше внимание на концепцию каталога Iceberg, который помогает гарантировать переносимость таблиц Iceberg и их доступность для обнаружения нашими инструментами.

Глава 5

Каталоги Iceberg

В этой главе мы углубимся в каталоги Iceberg. Мы уже знаем, что каталог является важнейшим компонентом Iceberg, который обеспечивает согласованность при работе с несколькими устройствами чтения и записи, а также помогает определить, какие таблицы доступны. В этой главе мы рассмотрим:

- требования к каталогу в целом и дополнительные требования, рекомендуемые для использования каталога в промышленном окружении;
- различные реализации каталога, их плюсы и минусы, а также способы настройки Spark для использования каталога;
- в каких ситуациях может понадобиться задуматься о миграции каталогов;
- как мигрировать с одного каталога на другой.

Требования к каталогу Iceberg

Iceberg предоставляет интерфейс каталога (<https://oreil.ly/flAG3>), который требует реализации набора функций, в первую очередь для получения списка существующих таблиц, а также создания, удаления, проверки существования и переименования таблиц.

Этот интерфейс имеет несколько реализаций, включая Hive Metastore, AWS Glue и каталог файловой системы (Hadoop). Помимо требования реализации функций, определяемых интерфейсом, еще одним требованием к реализации каталога Iceberg является отображение пути к таблице (например, `db1.table1`) в путь к файлу метаданных с текущим состоянием таблицы.

Поскольку это общее требование и существует множество реализаций каталогов для разных систем, каждая из которых имеет свои особенности в способах хранения данных, разные каталоги выполняют это отображение по-разному. Например, если в качестве каталога используется файловая система, то в папке с метаданными таблицы имеется файл с именем `version-hint.text`, который хранит номер версии текущего файла метаданных. При ис-

пользовании в роли каталога Hive Metastore запись с информацией о таблице имеет свойство `location` с местоположением текущего файла метаданных.

Это – минимальные требования к реализации каталога Iceberg, но существует различие между минимально возможным и рекомендуемым наборами функций для использования в промышленном окружении. Основное различие заключается в требовании исключить потерю данных при одновременной записи несколькими заданиями в одну и ту же таблицу. На этапе разработки и экспериментирования это требование не является необходимым. Однако в промышленном окружении потеря данных может иметь катастрофические последствия для бизнеса.

Основное требование к каталогу Iceberg, который будет использоваться в промышленном окружении, – он должен поддерживать атомарные операции по обновлению текущего указателя на метаданные. Это необходимо для того, чтобы все читающие и пишущие процессы видели одно и то же состояние таблицы в данный момент времени и чтобы при параллельном выполнении двух операций записи вторая не затирала изменения, внесенные первой, и не приводила к потере данных.

Сравнение каталогов

В этом разделе мы рассмотрим наиболее популярные каталоги Iceberg. Мы расскажем, как каждый из них отображает путь к таблице в местоположение текущего файла метаданных этой таблицы, плюсы и минусы каталога, ситуации, когда можно подумать о применении этого каталога, и как настроить Apache Spark для использования каталога.

Вот основные характеристики каталогов, по которым их можно сравнивать:

- пригодность для использования в промышленном окружении;
- потребность во внешней системе и является ли эта внешняя система автономной или управляемым сервисом;
- совместимость с различными механизмами и инструментами;
- поддержка транзакций с участием нескольких таблиц;
- совместимость с облачными окружениями.

Каталог Hadoop

Самый простой каталог, который можно использовать с Iceberg, – это каталог Hadoop. Его простота обусловлена тем, что он не требует никаких внешних систем или процессов. Несмотря на свое название, каталог Hadoop работает с любой файловой системой (и со всем, что похоже на файловую систему, например с облачными объектными хранилищами), включая распределенную файловую систему Hadoop (HDFS), Amazon Simple Storage Service (Amazon S3), Azure Data Lake Storage (ADLS) и Google Cloud Storage (GCS).

Для отображения пути к таблице в местоположение ее текущего файла метаданных Hadoop извлекает список с содержимым каталога метаданных таблицы и выбирает последний записанный файл, основываясь на отметке времени, которую имеет каждый файл в файловой системе. Однако многие механизмы записывают в папку метаданных таблицы файл с именем *version-hint.txt*, содержащий одно число, которое можно использовать для получения ссылки на текущий файл метаданных, например `v{n}.metadata.json`.

Плюсы и минусы каталога Hadoop

Основное преимущество каталога Hadoop: для его работы не требуется никаких внешних систем и достаточно лишь файловой системы. Это делает каталог Hadoop самым простым выбором для использования с Iceberg.

Однако у каталога Hadoop есть большой недостаток: его не рекомендуется использовать в промышленных целях.

Одна из причин заключается в том, что для предотвращения потери данных при одновременной записи файловая система должна обеспечивать атомарность операции переименования файла/объекта. Некоторые файловые системы и хранилища объектов поддерживают это требование, но не все. Например, ADLS и HDFS обеспечивают атомарность операции переименования и тем самым устраняют риск потери данных при одновременной записи в одну и ту же таблицу, а вот S3 такой гарантии не дает. В S3 для достижения атомарности и предотвращения потери данных во время одновременной записи можно использовать таблицу DynamoDB. Но в такой конфигурации все равно используется внешняя система (к тому же DynamoDB поддерживается как отдельный каталог Iceberg), поэтому нет особого смысла использовать каталог Hadoop.

Вторая причина: когда система настроена на применение каталога Hadoop, она может использовать только один каталог хранилища, поскольку зависит от местоположения списка таблиц в хранилище. Например, единственный сегмент может использоваться, только если вы применяете облачное объектное хранилище, такое как S3 или ADLS. Если вы решите использовать более одного сегмента для набора таблиц, то вам придется настроить отдельный каталог для каждого сегмента.

Третья причина: при выполнении операций, требующих перечисления пространств имен (также известных как базы данных) и/или таблиц, можно столкнуться с ограничениями производительности, особенно если имеется большое количество пространств имен и таблиц. Это связано с тем, что перечисление пространств имен и таблиц производится с помощью операции перечисления, реализованной в файловой системе. В некоторых ситуациях, например при работе в S3 и наличии большого числа пространств имен и/или таблиц, эта операция может выполняться довольно долго.

Последняя причина заключается в невозможности удалить таблицу или ссылку на нее из каталога и при этом сохранить данные. Удаление таблицы без удаления данных позволяет отменить операцию. Например, в Spark SQL, ког-

да используется каталог, отличный от каталога Hadoop, инструкция `DROP TABLE <имя_таблицы>` удалит только ссылку из каталога, но не удалит данные. Чтобы удалить таблицу и данные, если вы совершенно уверены, что они больше никогда не понадобятся, используется инструкция `DROP TABLE <имя_таблицы> PURGE`. Такое невозможно при использовании каталога Hadoop, потому что даже если файл `version-hint.txt` будет удален, таблица все равно останется существовать в определении каталога, поскольку в `warehouse_dir/namespace1/table1/metadata` есть файлы. Текущий файл метаданных по-прежнему можно получить, получив список содержимого каталога метаданных и выбрав последний созданный файл метаданных (основываясь на отметке времени файла в файловой системе).

Варианты использования каталога Hadoop

Учитывая эти плюсы и минусы, можно выделить несколько сценариев, когда применение каталога Hadoop вполне оправдано. Один из них: вы только начинаете осваивать Iceberg и хотите быстро приступить к работе. Поскольку у вас уже есть распределенная файловая система, вы имеете все необходимое для использования Iceberg с каталогом Hadoop; никакая другая система или инфраструктура не нужна. Кроме того, как мы обсудим далее в этой главе, миграция каталогов – довольно простая операция, поэтому вы можете начать работу с каталогом Hadoop и перейти к каталогу промышленного уровня позднее, когда будете готовы к этому. Другой сценарий, когда с успехом можно использовать каталог Hadoop, – это тестирование или разработка, в процессе которых не выполняются одновременные операции записи в одну и ту же таблицу или потеря данных не является проблемой.

Настройка Spark для использования каталога Hadoop

В следующем фрагменте кода показано, как запустить оболочку Spark SQL, в которой можно использовать `my_catalog1` для записи и чтения таблиц Iceberg с использованием каталога Hadoop:

```
spark-sql --packages
org.apache.iceberg:iceberg-spark-runtime-3.3_2.12:0.14.0\
--conf
spark.sql.extensions=org.apache.iceberg.spark.extensions.Iceberg
SparkSessionExtensions \
--conf
spark.sql.catalog.my_catalog1=org.apache.iceberg.spark.SparkCatalog \
--conf spark.sql.catalog.my_catalog1.type=hadoop \
--conf
spark.sql.catalog.my_catalog1.warehouse=<протокол>://<путь>
```

Каталог Hive

Каталог Hive – это популярная реализация каталога Iceberg. Он отображает путь к таблице в текущий файл метаданных, используя свойство таблицы `lo-`

cation в хранилище метаданных Hive. В этом свойстве хранится абсолютный путь в файловой системе, где может находиться текущий файл метаданных таблицы.

Плюсы и минусы каталога Hive

Основное преимущество каталога Hive – его совместимость с широким спектром механизмов и инструментов, поддерживающих подключение к Hive Metastore, даже притом, что они изначально создавались для работы с таблицами в формате Hive. Дополнительным преимуществом является независимость от облака.

Каталог Hive имеет два недостатка. Во-первых, он требует запуска дополнительного сервиса. В отличие от некоторых других реализаций, каталог Hive использует сервис Hive Metastore, который должен быть настроен и управляться пользователем. Однако этот недостаток можно преодолеть, используя управляемые варианты, такие как Amazon EMR. Второй минус – отсутствие поддержки транзакций с участием нескольких таблиц. Транзакции с несколькими таблицами являются ключевой возможностью баз данных, обеспечивающей поддержку согласованности и атомарности для одной или нескольких операций, в которых задействовано несколько таблиц. Их основное достоинство заключается в способности поддерживать согласованность данных в среде, что является ключом к поддержанию качества данных и доверия к ним. Транзакции с участием нескольких таблиц более подробно обсуждаются в главе 10.

Варианты использования каталога Hive

Основной сценарий, когда можно выбрать этот каталог, – если в вашей среде уже работает Hive Metastore и вы планируете сохранить этот экземпляр в течение некоторого времени. Однако имейте в виду, что каталог Hive не поддерживает облачные транзакции.

Настройка Spark для использования каталога Hive

В следующем фрагменте кода показано, как запустить оболочку Spark SQL, в которой можно использовать `my_catalog1` для записи и чтения таблиц Iceberg с помощью каталога Hive:

```
spark-sql --packages
org.apache.iceberg:iceberg-spark-runtime-3.3_2.12:0.14.0\
--conf
spark.sql.extensions=org.apache.iceberg.spark.extensions.Iceberg
SparkSessionExtensions \
--conf
spark.sql.catalog.my_catalog1=org.apache.iceberg.spark.SparkCatalog \
--conf spark.sql.catalog.my_catalog1.type=hive \
--conf
spark.sql.catalog.my_catalog1.uri=thrift://<хост_metastore>:<порт>
```


Каталог AWS Glue

Каталог AWS Glue – еще одна реализация каталогов Iceberg. Она использует каталог данных AWS Glue Data в качестве централизованного хранилища метаданных таблиц Iceberg. AWS Glue отображает путь к таблице в текущий файл метаданных через свойство таблицы `metadata_location`. Значением этого свойства является абсолютный путь к файлу метаданных в файловой системе.

Плюсы и минусы каталога AWS Glue

Использование AWS Glue в качестве каталога Iceberg дает множество преимуществ. Одна из важных особенностей состоит в том, что AWS Glue – это управляемый сервис, что снижает накладные расходы по сравнению с каталогом Hive, который управляет собственным хранилищем метаданных. Еще одно преимущество заключается в тесной интеграции с другими сервисами AWS.

Однако у каталога AWS Glue есть свои недостатки. Как и каталог Hive, он не поддерживает транзакции с участием нескольких таблиц. Более того, он предназначен для использования в экосистеме AWS, а это означает, что использование AWS Glue в мультиоблачной среде может усложнить развертывание.

Варианты использования каталога AWS Glue

Каталог AWS Glue – хороший выбор для тех, кто вкладывает значительные средства в сервисы AWS и не нуждается в мультиоблачных решениях, а также для тех, кому нужно управляемое решение каталога. Он также может помочь быстро начать работу с Iceberg, так как ничего не требует от вас. Но помните: если ваши рабочие нагрузки используют транзакции с участием нескольких таблиц, то выбор этого каталога может оказаться не лучшим решением.

Настройка Spark для использования каталога AWS Glue

В следующем фрагменте кода показано, как запустить оболочку Spark SQL, в которой можно использовать `my_catalog1` для записи и чтения таблиц Iceberg с помощью каталога AWS Glue:

```
spark-sql --packages "org.apache.iceberg:iceberg-sparkruntime-
x.x.x.xx:x.x.x,software.amazon.awssdk:bundle:x.xx.xxx,software.amazon.
awssdk:url-connection-client:x.xx.xxx" \
--conf spark.sql.catalog.my_catalog1=org.apache.iceberg.spark.SparkCatalog \
--conf spark.sql.catalog.my_catalog1.warehouse=s3://<нужно> \
--conf spark.sql.catalog.my_catalog1.catalog-impl=org.apache.iceberg.
aws.glue.GlueCatalog \
--conf spark.sql.catalog.my_catalog1.io-impl=org.apache.iceberg.aws.s3.S3FileIO \
--conf spark.hadoop.fs.s3a.access.key=$AWS_ACCESS_KEY \
--conf spark.hadoop.fs.s3a.secret.key=$AWS_SECRET_ACCESS_KEY
```


Каталог Nessie

Проект Nessie – еще одна реализация каталога Iceberg. В Nessie путь к таблице отображается в текущий файл метаданных с помощью свойства таблицы `metadataLocation`, которое хранится в записи таблицы в Nessie. В этом свойстве хранится абсолютный путь к текущему файлу метаданных таблицы.

Плюсы и минусы каталога Несси

Одно из преимуществ каталога Nessie – он привносит в озера данных Git-подобный интерфейс и реализует концепцию «данные как код», когда данные и связанные с ними метаданные могут версионироваться подобно исходному программному коду. Это особенно ценно для сценариев обработки данных, где решающее значение имеют безопасность изменений, воспроизводимость и отслеживаемость. Еще одно преимущество Nessie – этот каталог позволяет выполнять транзакции с несколькими таблицами. Третье преимущество – независимость от конкретного облачного окружения, поэтому один и тот же каталог можно использовать в мультиоблачной среде.

У проекта Nessie есть два недостатка. Во-первых, не все механизмы и инструменты поддерживают Nessie в качестве каталога. На момент написания этой книги поддержка Nessie имела в Spark, Flink, Dremio, Presto, Trino и PyIceberg. Во-вторых, вам придется управлять инфраструктурой самостоятельно, как и при использовании Hive Metastore. Однако, по аналогии с AWS Glue, Dremio предлагает управляемую версию Nessie (или по крайней мере совместимую, которая выглядит как Hive Metastore).

Варианты использования каталога Nessie

Nessie – хороший выбор на роль каталога Iceberg для тех, кому нужны транзакции с участием нескольких таблиц, желающих использовать парадигму «данные как код» и/или обрести независимость от поставщика облачных услуг.

Настройка Spark для использования каталога Nessie

В следующем фрагменте кода показано, как запустить оболочку Spark SQL, в которой можно использовать `my_catalog1` для записи и чтения таблиц Iceberg с помощью каталога Nessie:

```
spark-sql --packages
"org.apache.iceberg:iceberg-spark-runtime-x.x_x.xx:x.x.x,org.projectnessie:
nessie-spark-extensions-x.x_x.xx:x.xx.x,software.amazon.awssdk:bundle:
x.xx.xxx,software.amazon.awssdk:url-connection-client:x.xx.xxx" \
--conf spark.sql.extensions="org.apache.iceberg.spark.extensions.
IcebergSparkSessionExtensions,
org.projectnessie.spark.extensions.NessieSparkSessionExtensions" \
```

```
--conf spark.sql.catalog.my_catalog1=org.apache.iceberg.spark.SparkCatalog \
--conf spark.sql.catalog.my_catalog1.catalog-impl=org.apache.iceberg.nessie.NessieCatalog \
--conf spark.sql.catalog.my_catalog1.uri=$NESSIE_URI \
--conf spark.sql.catalog.my_catalog1.ref=main \
--conf spark.sql.catalog.my_catalog1.authentication.type=BEARER \
--conf spark.sql.catalog.my_catalog1.authentication.token=$TOKEN \
--conf spark.sql.catalog.my_catalog1.warehouse=<протокол>://<путь> \
--conf spark.sql.catalog.my_catalog1.io-impl=org.apache.iceberg.aws.s3.S3FileIO
```

Каталог REST

Каталог REST, как можно догадаться, основан на философии архитектурного стиля RESTful. Этот подход обеспечивает большую гибкость, потому что, в отличие от большинства других каталогов, каталог REST – это интерфейс, а не конкретная реализация. Сервис, реализующий интерфейс каталога REST, может отображать путь к таблице в текущий файл метаданных любым способом по своему выбору. Он даже может хранить его в любом из других каталогов, упомянутых выше.

Плюсы и минусы каталога REST

У каталога REST есть несколько плюсов. Во-первых, он требует меньше пакетов и зависимостей по сравнению с другими каталогами, что упрощает развертывание и управление. Это объясняется тем, что он использует стандартный протокол HTTP. Еще одно преимущество – большая гибкость, поскольку каталог может представлять любой сервис, способный обрабатывать запросы и ответы RESTful, а хранилище данных сервиса может быть организовано как множество разных объектов. Третье преимущество заключается в том, что каталог REST поддерживает транзакции с несколькими таблицами, обеспечивая поддержку сложных операций. Наконец, он не зависит от конкретного облачного провайдера, поэтому в мультиоблачной среде вы сможете использовать один и тот же каталог, если решите свести к минимуму зависимость от поставщика облачных услуг.

Каталог REST имеет три основных недостатка. Во-первых, необходимо запустить процесс для обработки REST-запросов и возврата ответов. При использовании этого каталога в промышленном окружении также потребуется запустить дополнительный сервис для хранения состояния. Второй недостаток, существовавший на момент написания книги, – отсутствие общедоступной серверной реализации для поддержки конечных точек каталога REST, а это означает, что вам придется написать свою такую реализацию. Альтернативой, в которой отсутствуют эти два недостатка, может служить реализация каталога REST в виде управляемого сервиса, такого как Tabular. Третий недостаток состоит в том, что не все механизмы и инструменты поддерживают каталог REST. На момент написания данной книги каталог REST поддерживался только в Spark, Trino, PyIceberg и Snowflake.

Варианты использования каталога REST

Каталог REST будет хорошим выбором на роль гибкого, настраиваемого решения, которое можно интегрировать с различными серверными хранилищами данных, поддерживает многотабличные транзакции и не зависит от поставщика облачных услуг.

Настройка Spark для использования каталога Rest

В следующем фрагменте кода показано, как запустить оболочку Spark SQL, в которой можно использовать `my_catalog1` для записи и чтения таблиц Iceberg с помощью каталога REST:

```
spark-sql --packages
org.apache.iceberg:iceberg-spark-runtime-3.3_2.12:0.14.0\
--conf
spark.sql.extensions=org.apache.iceberg.spark.extensions.Iceberg
SparkSessionExtensions \
--conf
spark.sql.catalog.my_catalog1=org.apache.iceberg.spark.SparkCatalog \
--conf spark.sql.catalog.my_catalog1.type=rest \
--conf
spark.sql.catalog.my_catalog1.uri=http://<хост>:<порт>
```

Каталог JDBC

Каталог JDBC использует хранилища данных, совместимые с Java Database Connectivity (JDBC), что делает его универсальным выбором, если информация хранится в базе данных с поддержкой JDBC, такой как MySQL или PostgreSQL. Обратите внимание, что база данных, к которой подключается JDBC, должна поддерживать атомарные транзакции.

Каталог JDBC отображает путь таблицы в текущий файл метаданных через свойство таблицы `metadata_location`, хранимое в базе данных, совместимой с JDBC.

Плюсы и минусы каталога JDBC

Использование каталога JDBC имеет множество преимуществ. Первое из них – простота запуска. Если у вас есть хранилище данных, совместимое с JDBC, то, значит, у вас уже есть инфраструктура, необходимая для использования каталога JDBC. Типичными примерами подобных баз данных могут служить MySQL и PostgreSQL. Если такого хранилища нет, то его можно быстро развернуть, воспользовавшись предложениями облачных провайдеров, такими как Amazon Relational Database Service (Amazon RDS) и Azure Database for MySQL/PostgreSQL. Еще одно преимущество состоит в том, что эти базы данных (особенно размещенные в облаке) позволяют обеспечить высо-

кую доступность, даже в случае выхода из строя основного экземпляра базы данных. Третье преимущество – независимость от конкретного облачного окружения, поэтому один и тот же каталог можно использовать в мульти-облачной среде.

У каталога JDBC есть два основных недостатка. Во-первых, он не поддерживает транзакции с участием нескольких таблиц. Во-вторых, требует, чтобы все ваши механизмы развертывались вместе с драйвером JDBC или имели возможность динамически получать его, что увеличивает зависимости от способа развертывания.

Варианты использования каталога JDBC

Каталог JDBC может быть хорошим выбором, если у вас уже есть действующая база данных, совместимая с JDBC, или вы планируете использовать сервис, предлагаемый поставщиком облачных услуг, например Amazon RDS. Он также может считаться хорошим выбором, если ваша среда требует высокой доступности и/или вы хотите оставаться независимыми от типа облака.

Настройка Spark для использования каталога JDBC

В следующем фрагменте кода показано, как запустить оболочку Spark SQL, в которой можно использовать `my_catalog1` для записи и чтения таблиц Iceberg с помощью каталога JDBC:

```
spark-sql --packages org.apache.iceberg:iceberg-spark-runtime-x.x.xx:x.x.x \
--conf spark.sql.catalog.my_catalog1=org.apache.iceberg.spark.SparkCatalog \
--conf spark.sql.catalog.my_catalog1.warehouse=<протокол>://<путь> \
--conf spark.sql.catalog.my_catalog1.catalog-impl=org.apache.iceberg.
jdbc.JdbcCatalog \
--conf spark.sql.catalog.my_catalog1.uri=jdbc:<протокол>://<хост>:<порт>/
<база_данных> \
--conf spark.sql.catalog.my_catalog1.jdbc.user=<имя_пользователя> \
--conf spark.sql.catalog.my_catalog1.jdbc.password=<пароль>
```

Другие каталоги

Обратите внимание, что помимо представленных выше существуют и другие реализации каталогов (например, DynamoDB, Snowflake). Однако мы решили перечислить лишь наиболее распространенные, потому что в целом вариантов очень много, так как в роли каталога Iceberg может выступать практически все, что обеспечивает возможности, упомянутые в разделе «Требования к каталогу Iceberg» в начале главы.

Миграция каталога

Одна замечательная особенность подавляющего большинства таблиц Iceberg в озерных хранилищах данных заключается в том, что миграция из одного экземпляра каталога в другой или из каталога одного типа в другой осуществляется очень просто: вы лишь меняете место отображения пути к таблице в текущий файл метаданных. Однако, несмотря на всю простоту, для любой миграции необходимо разработать план устранения сопутствующих осложнений, таких как задания записи и запуск различных инструментов и приложений в вашей среде.

Простота миграции каталога снижает риск зависимости от поставщиков облачных услуг и обеспечивает безопасность ваших данных в будущем. Если появится лучшее или более экономичное решение, вы сможете перейти на него без особых хлопот.

Миграция каталогов может понадобиться в нескольких ситуациях. Во-первых, после экспериментов и/или оценки работы Iceberg с одним из каталогов, например Hadoop, вы решили использовать в промышленном окружении другой каталог. Вторая ситуация – вам понадобились дополнительные возможности, отсутствующие в вашем текущем каталоге. Третья ситуация – смена местоположения окружения, например из своего старого развертывания в локальном окружении Hadoop с Hive Metastore вы переходите на AWS и хотите использовать AWS Glue. Обратите внимание, что третья ситуация требует дополнительного внимания, так как в этом случае переноситься будет не только каталог, но и данные.

Плюс простоты миграции заключается в возможности продолжать использовать текущий каталог и одновременно с этим зарегистрировать набор таблиц в новом каталоге, сохранить запись в текущем каталоге и провести тестирование в новом каталоге. Просто имейте в виду, что в этой ситуации запись в новом каталоге будет устаревшей, и любые изменения в таблицы будут вноситься с использованием текущего каталога. Не следует вносить какие-либо изменения в таблицу, используя новый каталог, потому что все запросы, использующие текущий каталог, не увидят этих изменений.

Есть два стандартных способа миграции каталогов. Рассмотрим их далее.

С помощью инструмента командной строки для миграции каталога Iceberg

Инструмент командной строки для миграции каталога Iceberg (<https://oreil.ly/iCtUP>) распространяется с открытым исходным кодом в рамках проекта Nessie. Чтобы подчеркнуть, что проект Iceberg фокусируется на спецификации формата таблиц и интеграции механизма, сообщество Apache Iceberg решило сохранить код этого инструмента отдельно от репозитория Iceberg. Он обеспечивает массовую миграцию таблиц Apache Iceberg из одного каталога в другой без копирования данных. Инструмент поддерживает все по-

пулярные каталоги Apache Iceberg, такие как AWS Glue, Nessie, Dremio Arctic, Hadoop, Hive, REST, JDBC, а также любые пользовательские каталоги.

В настоящее время iceberg-catalog-migrator предлагает две основные функции: миграцию и регистрацию. Важно отметить, что ни одна из этих функций не копирует данные. Кроме того, обе передают всю историю таблицы, что позволяет после миграции или регистрации использовать с новым каталогом такие функции, как путешествие во времени. Ниже приводится краткое описание команд `migrate` и `register`:

`migrate`

Команда `migrate` осуществляет массовую миграцию таблиц Iceberg из исходного каталога в новый. После успешного завершения миграции из исходного каталога будут удалены все записи о таблицах, т. е. они фактически будут *перемещены* в новый каталог.

`register`

Команда `register` позволяет включить (т. е. зарегистрировать) таблицы Iceberg, перечисленные в исходном каталоге, в новом каталоге. В отличие от `migrate`, операция `register` не предполагает удаления записи о таблицах из исходного каталога. Поэтому после успешной регистрации ссылки на таблицы продолжат существовать в обоих каталогах. Эта функция особенно полезна для таких задач, как проверочное тестирование перед миграцией или изучение новых каталогов, предлагающих уникальные функции, так как она гарантирует, что исходный каталог и таблицы останутся нетронутыми. При использовании команды `register` важно гарантировать невозможность выполнения операций записи с одной и той же таблицей с использованием нескольких каталогов. Это может привести к потере изменений, данных и повреждению таблицы. Если дать такую гарантию невозможно, то рекомендуется использовать команду `migrate`, которая автоматически исключит таблицу из исходного каталога после регистрации, что позволит избежать подобных проблем. Точно так же избегайте выполнения операций с таблицами после регистрации с использованием исходного каталога.

Обратите внимание, что инструмент командной строки не рекомендуется использовать во время текущих фиксаций таблиц в исходном каталоге. Это необходимо для предотвращения потери изменений и потенциального повреждения таблиц в новом каталоге. Во время миграции инструмент фиксирует определенное состояние таблицы (файл метаданных) и использует это состояние для регистрации в целевом каталоге. Если в это время к таблице применяется фиксация транзакции с использованием исходного каталога, то информация об изменениях не отразится в новом каталоге, что поставит под угрозу целостность данных. Как правило, рекомендуется использовать подход пакетной миграции с использованием регулярного выражения (например, все таблицы в пространстве имен `namespace1` или все таблицы в пространстве имен `namespace1` с именами, начинающимися на букву *a*). Он может быть реализован как часть процессов регулярного обслуживания и во время простоев, когда пользователи не записывают данные и приостанавливают-

ся любые автоматически выполняемые задания. Обратите внимание, что эти задания необходимо будет перенацелить на новый каталог, прежде чем они снова начнут выполняться. Альтернативное решение – использовать промежуточную обертку, которая изначально ссылается на существующий каталог, и настроить задания так, чтобы они использовали эту обертку, убедиться в отсутствии проблем и затем произвести миграцию. По завершении миграции вам не придется менять задания; вместо этого нужно будет лишь перенацелить обертку на новый каталог.

Вот пример использования инструмента командной строки:

```
java -jar iceberg-catalog-migrator-cli-0.2.0.jar migrate \
--source-catalog-type GLUE \
--source-catalog-properties warehouse=s3a://bucket/gluecatalog/,ioimpl=
org.apache.iceberg.aws.s3.S3FileIO \
--source-catalog-hadoop-conf
fs.s3a.secret.key=$AWS_SECRET_ACCESS_KEY,fs.s3a.access.key=$AWS_ACCESS_KEY_ID \
--target-catalog-type NESSIE \
--target-catalog-properties uri=http://localhost:19120/api/v1,ref=main,warehouse=
s3a://bucket/nessie/,io-impl=org.apache.iceberg.aws.s3.S3FileIO \
--identifiers db1.nominees
```

Эта команда перенесет таблицу `db1.nominees` (параметр `--identifiers`) из каталога AWS Glue (параметр `--source-catalog-type`) для учетной записи AWS, соответствующей учетным данным AWS (параметр `--source-catalog-hadoop-conf`), в каталог Nessie (параметр `--target-catalog-type`), работающий на локальном хосте через порт 19120 (значение `uri` в параметре `--target-catalog-properties`).

С помощью механизма обработки запросов

Второй стандартный способ миграции каталогов – использование механизма обработки запросов, такого как Apache Spark. В Spark имеется набор процедур, которые можно использовать из Spark SQL. Вам потребуется настроить как исходный, так и целевой каталог, чтобы сеанс Spark мог использовать эти процедуры.

Например, миграцию из каталога Hadoop в каталог AWS Glue можно выполнить с помощью Spark SQL так:

```
spark-sql --packages "org.apache.iceberg:iceberg-sparkruntime-
x.x.x,software.amazon.awssdk:bundle:x.xx.xxx,software.amazon.
awssdk:url-connection-client:x.xx.xxx" \
--conf
spark.sql.extensions=org.apache.iceberg.spark.extensions.Iceberg
SparkSessionExtensions \
--conf
spark.sql.catalog.source_catalog1=org.apache.iceberg.spark.SparkCatalog \
--conf spark.sql.catalog.source_catalog1.type=hadoop \
--conf spark.sql.catalog.source_catalog1.warehouse=<протокол>://<путь> \
--conf spark.sql.catalog.target_catalog1=org.apache.iceberg.spark.SparkCatalog \
--conf spark.sql.catalog.target_catalog1.warehouse=s3://<путь> \
```



```
--conf spark.sql.catalog.target_catalog1.catalog-impl=org.apache.iceberg.
aws.glue.GlueCatalog \
--conf spark.sql.catalog.target_catalog1.io-impl=org.apache.iceberg.
aws.s3.S3FileIO \
--conf spark.hadoop.fs.s3a.access.key=$AWS_ACCESS_KEY \
--conf spark.hadoop.fs.s3a.secret.key=$AWS_SECRET_ACCESS_KEY
```

Эта команда запустит оболочку Spark SQL, настроенную для миграции таблиц из исходного каталога Hadoop (параметр `--conf spark.sql.catalog.source_catalog1.type`) в заданном исходном местоположении (параметр `--conf spark.sql.catalog.source_catalog1.warehouse`) и целевой каталог AWS Glue (параметр `--conf spark.sql.catalog.target_catalog1.catalogimpl=org.apache.iceberg.aws.glue.GlueCatalog`) для учетной записи AWS с соответствующими учетными данными AWS (параметры `--conf spark.hadoop.fs.s3a.access.key` и `--conf spark.hadoop.fs.s3a.secret.key`).

Обратите внимание, что эта команда просто запустит оболочку Spark SQL, в которой настроены два каталога (`source_catalog1` и `target_catalog1`), что позволит последующим командам использовать эти каталоги, а не выполнять немедленную миграцию таблиц. При переносе таблиц между каталогами следует использовать две основные процедуры, которые мы рассмотрим далее.

register_table()

Процедура `register_table()` в Spark SQL создает облегченную копию исходной таблицы, используя ее файлы данных. Любые изменения, внесенные в таблицу через целевой каталог, будут также отражаться в исходном каталоге, но только пока эти изменения не приводят к физическому удалению файлов данных (например, `expire_snapshot()`). После этого изменения, внесенные через целевой каталог, не будут отражаться в исходном каталоге. Тем не менее вносить изменения в таблицы через целевой каталог в такой ситуации не рекомендуется.

Этот метод может пригодиться для тестирования миграции, когда в ходе тестирования таблицы не изменяются. Его также можно использовать для миграции каталогов, когда желательно, чтобы местоположение файла таблицы в озерном хранилище данных оставалось неизменным.

Обратите внимание, что этот метод передает всю историю таблицы, поэтому по завершении процедуры можно выполнять такие операции с историей, как путешествие во времени и просмотр истории таблицы через системные таблицы.

В табл. 5.1 подробно описаны аргументы, которые принимает процедура `register_table()`.

Таблица 5.1. Аргументы процедуры `register_table()`

Имя аргумента	Обязательный?	Тип	Описание
<code>table</code>	Да	String	Имя целевой таблицы для регистрации
<code>metadata_file</code>	Да	String	Файл метаданных, который будет зарегистрирован как текущий файл метаданных для целевой таблицы

В табл. 5.2 подробно описаны выходные поля, возвращаемые процедурой.

Таблица 5.2. Выходные поля, возвращаемые процедурой `register_table()`

Имя поля	Тип	Описание
<code>current_snapshot_id</code>	Long	Идентификатор текущего снимка вновь зарегистрированной таблицы Iceberg
<code>total_records_count</code>	Long	Общее число записей во вновь зарегистрированной таблице Iceberg
<code>total_data_files_count</code>	Long	Общее число файлов данных, составляющих вновь зарегистрированную таблицу Iceberg

Ниже приводится пример использования процедуры `register_table`, основанный на конфигурации оболочки Spark SQL из предыдущего раздела:

```
CALL target_catalog.system.register_table(
  'target_catalog.db1.table1',
  '/path/to/source_catalog_warehouse/db1/table1/metadata/xxx.json'
)
```

snapshot()

Процедура `snapshot()`, подобно `register_table()`, создает облегченную копию исходной таблицы, используя ее файлы данных. Однако, в отличие от `register_table()`, любые изменения, вносимые через целевой каталог, будут отражаться только на целевой таблице, т. е. никакие изменения в целевой таблице не будут видны в исходной таблице. Точно так же любые изменения, внесенные в исходную таблицу, не будут видны пользователям целевой таблицы.

Этот метод может пригодиться для тестирования миграции, когда в ходе проверки необходимо внести изменения в целевую таблицу, но нежелательно, чтобы эти изменения затронули исходную таблицу. Другим следствием того обстоятельства, что таблица в целевом каталоге не владеет файлами данных, является невозможность применения `expire_snapshots()` к целевой таблице, потому что это повлечет за собой физическое удаление файлов данных, принадлежащих таблице в исходном каталоге. Также этот метод может упростить миграцию в новый каталог, когда требуется изменить местоположение файла таблицы, потому что в таком случае со временем и по мере внесения изменений все больше и больше метаданных таблицы и файлов данных будут оказываться в местоположении целевой таблицы. Кроме того, на более позднем этапе после миграции можно воспользоваться процедурой `rewrite_data_files()`, чтобы ускорить миграцию файлов (например, при миграции из локальной среды в облако или из одного облака в другое).

Обратите внимание, что этот метод перенесет всю историю таблицы, поэтому после применения данной процедуры можно выполнять такие операции с историей, как путешествие во времени и просмотр истории через системные таблицы.

В табл. 5.3 подробно описаны аргументы, которые принимает процедура `snapshot()`.

Таблица 5.3. Аргументы процедуры `snapshot()`

Имя аргумента	Обязательный?	Тип	Описание
<code>source_table</code>	Да	String	Имя таблицы в снимке
<code>table</code>	Да	String	Имя вновь создаваемой таблицы Iceberg
<code>location</code>		String	Местоположение новой таблицы (по умолчанию определяется каталогом)
<code>properties</code>		map<string, string>	Свойства для добавления в новую таблицу

В табл. 5.4 подробно описаны выходные поля, возвращаемые процедурой.

Таблица 5.4. Выходные поля, возвращаемые процедурой `snapshot()`

Имя поля	Тип	Описание
<code>imported_files_count</code>	Long	Общее число файлов, добавленных в новую таблицу

Ниже приводится пример использования процедуры `snapshot()`, основанный на конфигурации оболочки Spark SQL из предыдущего раздела:

```
CALL target_catalog.system.snapshot(
  'source_catalog.db1.table1',
  'target_catalog.db1.table1'
)
```

Заключение

В этой главе мы подробно рассмотрели каталоги Iceberg, изучили их роль в поддержании согласованности при поддержке множества процессов, выполняющих чтение и запись, а также их использование для поиска доступных таблиц в заданной среде.

Мы рассмотрели основные и дополнительные требования, необходимые для развертывания каталога в промышленной среде. Мы также сравнили разные реализации каталога, подробно описав их плюсы и минусы и объяснив, как настроить Spark для каждой из них. Кроме того, мы обсудили миграцию каталогов, включая сценарии, когда имеет смысл рассмотреть возможность миграции каталогов, а также два основных метода ее осуществления.

В главе 6 мы займемся исследованием Apache Iceberg с применением различных инструментов.

ЧАСТЬ II

ПРАКТИЧЕСКИЕ ПРИЕМЫ РАБОТЫ С АРАСНЕ ICEBERG

В этой части книги мы рассмотрим практические аспекты использования Apache Iceberg с некоторыми широко используемыми вычислительными механизмами и автономными API, включая Apache Spark, Dremio SQL Engine, AWS Glue, Apache Flink и PyIceberg. Основное внимание здесь будет уделено подробным объяснениям и примерам кода, чтобы продемонстрировать, как Apache Iceberg работает с разными вычислительными механизмами, чтобы вы могли применять и развивать теоретические идеи, обсуждавшиеся в предыдущих главах.

В репозитории книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg-ch6>) вы найдете инструкции по созданию озерного хранилища данных на своем компьютере с помощью Docker, а также по настройке таких инструментов, как Apache Spark, Apache Flink и Dremio.

Глава 6

Apache Spark

Apache Spark – это очень гибкий механизм распределенных вычислений в сочетании с Apache Iceberg и поддерживающий широкий спектр возможностей. Применение Spark и Iceberg позволяет воспользоваться преимуществами эффективной организации данных в Iceberg. В этой главе мы рассмотрим шаги, которые необходимо выполнить для начала работы с Apache Iceberg и Spark, а также исследуем некоторые важные особенности. К концу этой главы вы научитесь настраивать Apache Iceberg; выполнять различные операции на языке определения данных (Data Definition Language, DDL), такие как CREATE и ALTER, запросы (SELECT) и операции на языке манипулирования данными (Data Manipulation Language, DML), такие как INSERT, UPDATE, DELETE, MERGE; и управлять таблицами Iceberg с помощью различных механизмов обработки.

Настройка

Начнем обсуждение с особенностей настройки таблиц и каталогов Apache Iceberg с помощью Spark в роли вычислительного механизма. Наша задача: познакомиться с основными конфигурационными параметрами, необходимыми для бесперебойной работы с Iceberg и Spark.

Настройка Apache Iceberg и Spark

Для работы с таблицами Apache Iceberg с использованием Apache Spark их необходимо настроить для совместной работы. Есть несколько способов определить эти настройки. Сначала мы покажем, как определить настройки с помощью флагов в Spark Shell или Spark SQL, а затем как сделать то же самое в приложении Python.

Настройка через интерфейс командной строки

Прежде всего нам нужно указать, какие пакеты следует установить и использовать в сеансе Spark. Для этого Spark предоставляет параметр `--packages`, позволяющий загружать указанные пакеты на основе Maven (<https://oreil.ly/cVP-F>) и их зависимости и добавлять их в путь к классам приложения.

Чтобы использовать Iceberg вместе со Spark, нужно добавить параметр `--packages` и указать в нем пакет `iceberg-spark-runtime`. Обычно пакеты указываются в формате `groupId:artifactId:version`. Пакет `iceberg-spark-runtime` содержит классы, которые Spark использует для взаимодействия с таблицами и метаданными Iceberg. Добавляя параметр `--packages`, вы гарантируете, что все необходимые классы будут включены в путь к классам Spark при запуске оболочки или приложения Spark.

Вот команда запуска оболочки Spark с Apache Iceberg:

```
spark-shell --packages org.apache.iceberg:iceberg-spark-runtime-3.3_2.12:1.3.0
```

Эта команда сообщает оболочке Spark, что та должна загрузить пакет `iceberg-spark-runtime` из группы `org.apache.iceberg`, где версия Iceberg – 1.3.0, версия Spark – 3.3, а версия Scala – 2.12. Здесь важно отметить, что версия пакета и версия Scala должны быть совместимы с версией Apache Spark. Совместимость версий можно проверить в официальной документации Iceberg (<https://oreil.ly/JBIYq>).

Аналогично нужно передать те же настройки, если вы решите использовать их в сеансе Spark SQL:

```
spark-sql --packages org.apache.iceberg:iceberg-spark-runtime-3.3_2.12:1.3.0
```

Вместо использования параметра `--packages` можно добавить необходимые JAR-файлы в папку `jars` в каталоге установки Spark. Этот вариант удобно использовать в локальных средах разработки и тестирования, что избавляет от необходимости загружать библиотеки из репозитория Maven. С другой стороны, параметр `--packages` обеспечивает гибкость в управлении версиями.

Этот JAR-файл содержит классы и расширения, необходимые Spark для интерпретации таблиц Iceberg и управления ими. Включение файла JAR можно выполнить из командной строки при запуске оболочки Spark:

```
./bin/spark-shell --jars /путь/к/iceberg-spark-runtime.jar
```

Два подхода, описанных выше, позволяют настроить взаимодействие Spark и Iceberg из командной строки. Однако чтобы выполнить эти настройки в приложении на Python или Scala, пакет необходимо подключить перед созданием сеанса Spark. Давайте рассмотрим, как это сделать в приложении PySpark.

Настройка в коде на Python (PySpark)

Прежде чем начать сеанс PySpark с Apache Iceberg, необходимо установить следующее ПО:

- Java (v8 или v11);
- PySpark;
- Apache Spark (версия зависит от версии Iceberg).

Чтобы запустить сеанс PySpark, включающий все библиотеки, необходимые для работы с Iceberg, вам потребуется использовать объект `SparkSession.builder` из PySpark. С его помощью вы сможете указать необходимые параметры настройки сеанса Spark. Вот фрагмент кода, показывающий, как это сделать:

```
from pyspark.sql import *
from pyspark import SparkConf

# Создать объект конфигурации Spark Configuration
conf = SparkConf()

# Определить настройки
conf.set("spark.jars.packages",
        "org.apache.iceberg:iceberg-spark-runtime-3.3_2.12:1.2.0")

# Создать объект сеанса Spark Session
spark = SparkSession.builder.config(conf=conf).getOrCreate()

## Объект Spark Session можно использовать для выполнения запросов,
## например spark.sql("SELECT * FROM table")
```

Этот код сначала создает объект `SparkConf` и настраивает в нем свойства Spark для приложения. Свойства определяются как пары ключ–значение. В этом примере мы определяем параметр `spark.jars.packages` и указываем в нем необходимый пакет `org.apache.iceberg:iceberg-spark-runtime-3.3_2.12:1.2.0`. Эта конфигурация добавит внешние библиотеки непосредственно в путь к классам. Наконец, код создает экземпляр `SparkSession` с настроенным объектом конфигурации (`conf`).

Настройка каталогов

Следующий важный компонент процесса настройки – каталог. Apache Iceberg поддерживает широкий спектр каталогов для управления метаданными. В главе 5 мы подробно обсудили требования к каталогам и их роль в формате таблиц Apache Iceberg. Если говорить кратко, то каталог – это логическое пространство имен, которое содержит метаданные о таблицах Iceberg и унифицирует представление данных для различных вычислительных механизмов, обеспечивая гарантии надежности и согласованности транзакций. Таким образом, каталог – это первое, что вам нужно настроить для работы с таблицами Iceberg.

Apache Spark предлагает API для добавления каталогов, которые, в свою очередь, используются для загрузки, создания и администрирования таблиц Iceberg. Настройка каталога в Spark включает определение его имени в кон-

фигурации сеанса Spark, в коде PySpark или в командной оболочке Spark. Для этого определяется свойство Spark `spark.sql.catalog.<имя-каталога>` с классом реализации, которое указывает, что должен быть создан каталог с данным именем и управление им должно осуществляться с использованием указанного класса реализации. Например:

```
spark.sql.catalog.my_catalog = org.apache.iceberg.spark.SparkCatalog
```

Эта команда определяет каталог с именем `my_catalog` и с классом реализации `SparkCatalog`, предоставляемой Spark по умолчанию. Класс реализации является основой любого каталога, используемого в Spark, и предоставляет логику для взаимодействия с любым независимым каталогом Apache Iceberg (Nessie, AWS Glue). Прежде чем перейти к другим параметрам настройки, обсудим два встроенных класса реализации каталога для Spark, предоставляемых Apache Iceberg.

Использование *org.apache.iceberg.spark.SparkCatalog*

Первый класс реализации по умолчанию поддерживает использование каталога Hive Metastore или Hadoop (файловая система). Это тип каталога, показанный в предыдущем примере. При настройке для использования Hive Metastore (путем установки типа каталога `hive`) `SparkCatalog` будет хранить метаданные таблиц в Metastore Hive, что позволит вам использовать функции управления метаданными Hive. С другой стороны, если установить тип каталога `hadoop`, то `SparkCatalog` будет использовать каталог в Hadoop или любой другой файловой системе. Вот пример настройки `SparkCatalog` на использование каталога Hive:

```
spark.sql.catalog.hive_catalog = org.apache.iceberg.spark.SparkCatalog
```

```
spark.sql.catalog.hive_catalog.type = hive
```

```
spark.sql.catalog.hive_catalog.uri = thrift://metastore-host:port
```

В этом примере мы настроили `SparkCatalog` с именем `hive_catalog`. Параметру `type` присвоено значение `hive`, чтобы в данном конкретном случае использовать Hive Metastore. Другой параметр, с именем `uri`, требует от Spark применять указанный URI для взаимодействия с таблицами Iceberg в `hive_catalog`. Доступные для настройки свойства каталога перечислены в табл. 6.1.

Таблица 6.1. Свойства каталога

Свойство	Значения	Описание
<code>spark.sql.catalog.имя-каталога.type</code>	Hive, Hadoop, REST	Определяет базовую реализацию каталога Iceberg; не задается, если используется пользовательский каталог

Таблица 6.1 (окончание)

Свойство	Значения	Описание
<code>spark.sql.catalog.имя-каталога.catalog-impl</code>		Класс реализации каталога Iceberg; должно определяться, если свойство <code>type</code> не задано
<code>spark.sql.catalog.имя-каталога.io-impl</code>		Реализация FileIO
<code>spark.sql.catalog.имя-каталога.default-namespace</code>	<code>default</code>	Пространство имен каталога по умолчанию
<code>spark.sql.catalog.имя-каталога.uri</code>	<code>thrift://хост:порт</code>	Hive Metastore URI, REST URI или сервер Nessie/Arctic
<code>spark.sql.catalog.имя-каталога.warehouse</code>	<code>/путь/к/хранилищу</code>	Местоположение хранилища для данных каталога

Использование *org.apache.iceberg.spark.SessionCatalog*

`SessionCatalog` – это узкоспециализированная реализация, обертка вокруг встроенного каталога сеансов Spark, которая добавляет поддержку таблиц Iceberg. Она может оказаться полезной в сценариях, когда в сеансе Spark нужно использовать таблицы Iceberg вместе с таблицами, не относящимися к Iceberg. Здесь все таблицы, не относящиеся к Iceberg, управляются встроенным каталогом Spark, а таблицы Iceberg – классом `SessionCatalog`. Вот пример такой конфигурации:

```
spark.sql.catalog.hive_spark_catalog = org.apache.iceberg.spark.SessionCatalog
```

```
spark.sql.catalog.hive_spark_catalog.type = hive
```

```
spark.sql.catalog.hive_spark_catalog.uri = thrift://localhost:9083
```

Здесь мы настроили `SessionCatalog` с именем `hive_spark_catalog`, который использует хранилище Hive Metastore (доступное по адресу `thrift://localhost:9083`) для хранения метаданных.

Использование пользовательского каталога

Иногда бывает нужно выйти за рамки встроенных реализаций каталога Iceberg, например чтобы интегрироваться с уже существующей платформой, такой как AWS Glue (<https://aws.amazon.com/glue>), или воспользоваться преимуществами парадигмы «данные как код» с современными метакаталогами, такими как Nessie (<https://projectnessie.org>). Для подобных случаев Spark позволяет загрузить собственную реализацию каталога Iceberg. Она должна реализовывать интерфейс `Catalog` в Iceberg и указывается в свойстве `catalog-impl`. Например:

```
spark.sql.catalog.custom_catalog = org.apache.iceberg.spark.SessionCatalog
```

```
spark.sql.catalog.custom_catalog.catalog-impl = com.my.custom.CatalogImpl
```

```
spark.sql.catalog.custom_catalog.my-additional-catalog-config = my-value
```


В этом примере `custom_catalog` – это каталог, использующий `SparkCatalog` из `Iceberg`, но с нестандартной реализацией `com.mu.custom.CatalogImpl`. Свойство `mu-additional-catalog-config` может определять любую дополнительную конфигурацию, необходимую вашей реализации `CatalogImpl`.

В двух предыдущих разделах мы показали, как настроить `Apache Spark` с `Apache Iceberg` и как настроить использование таких каталогов, как `Hive Metastore` и `Hadoop`, для работы с таблицами `Iceberg`. Теперь давайте настроим эти компоненты и приступим к практическим упражнениям.

Запуск Spark со всеми конфигурациями (пример AWS Glue)

Чтобы опробовать примеры, представленные в этой главе, клонируйте репозиторий по адресу <https://oreil.ly/supp-guide-apache-iceberg> и следуйте этим инструкциям:

- в папке *Chapter_6* репозитория прочитайте файл *developer_env.md*, где описывается, как настроить среду разработки;
- после настройки скопируйте все файлы блокнота (они понадобятся для будущих упражнений) в папку */notebooks* в каталоге, где установлена ваша среда.

Если вы предпочитаете использовать локальную установку без привлечения облачной инфраструктуры, то обратитесь к репозиторию книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg-ch6>), где найдете информацию по установке и настройке каталога.

Сначала нужно определить переменные среды `AWS` для взаимодействия с каталогом `Glue`:

```
%env AWS_REGION= region
%env AWS_ACCESS_KEY_ID= key
%env AWS_SECRET_ACCESS_KEY= secret
```

Обязательно замените заполнители `region`, `key` и `secret` фактическими значениями.

```
import pyspark
from pyspark.sql import SparkSession
import os

conf = (
    pyspark.SparkConf()
    .setAppName('app_name')
    .set('spark.jars.packages',
        'org.apache.iceberg:iceberg-spark-runtime-3.3_2.12:1.2.0,software.amazon.
awssdk:bundle:2.17.178,software.amazon.awssdk:url-connection-client:2.17.178')
    .set('spark.sql.extensions',
        'org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions')
    .set('spark.sql.catalog.glue',
        'org.apache.iceberg.spark.SparkCatalog')
```

```
.set('spark.sql.catalog.glue.catalog-impl',
    'org.apache.iceberg.aws.glue.GlueCatalog')
.set('spark.sql.catalog.glue.warehouse', 's3://my-bucket/warehouse/')
.set('spark.sql.catalog.glue.io-impl',
    'org.apache.iceberg.aws.s3.S3FileIO')
)

spark = SparkSession.builder.config(conf=conf).getOrCreate()
```

Этот код импортирует пакеты, необходимые для работы с Iceberg и Spark, вместе с пакетом AWS SDK, который поможет взаимодействовать с сервисами AWS (S3, Glue). Определяет каталог с именем glue, использующий Spark-Catalog в качестве основы, но с собственной реализацией org.apache.iceberg.aws.glue.GlueCatalog. Задает местоположение хранилища для каталога: s3://my-bucket/warehouse/. Сюда вычислительный механизм будет записывать файлы данных и метаданных.

Наконец, настраивается реализация FileIO как org.apache.iceberg.aws.s3.S3FileIO, обеспечивающая чтение и запись данных в AWS S3. После выполнения этого кода приложение Spark будет запущено со всеми необходимыми настройками:

conf	modules				artifacts	
	number	search	downloaded	evicted	number	downloaded
default	35	0	0	1	34	0

```
:: retrieving :: org.apache.spark#spark-submit-parent-18734b47-4777-4ae3-
b1e0-1caa56050c4a
```

```
  confs: [default]
```

```
  0 artifacts copied, 34 already retrieved (0kB/234ms)
```

```
23/08/22 23:48:58 WARN NativeCodeLoader: Unable to load native-hadoop library
for your platform... using builtin-java classes where applicable
```

```
Setting default log level to "WARN".
```

```
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use
setLogLevel(newLevel).
```

```
23/08/22 23:49:07 WARN Utils: Service 'SparkUI' could not bind on port 4040.
```

```
Attempting port 4041.
```

```
Spark Running
```

Операции на языке определения данных

В этом разделе вы познакомитесь с различными операциями на языке определения данных (DDL), доступными в Apache Iceberg с использованием Spark.

В Iceberg операции DDL интуитивно понятны и согласованы со стандартным языком SQL. Apache Spark позволяет выполнять эти операции с помощью SQL API или DataFrame API (`DataFrameWriterV2`) и обеспечивает одинаковые результаты независимо от API. Обратите внимание, что DataFrame API поддерживает не все операции DDL. Мы представим примеры выполнения операций с помощью SQL API, а операции DataFrame API будут демонстрировать там, где это возможно.

CREATE TABLE

Определение данных обычно начинается с создания таблицы Iceberg с помощью команды `CREATE TABLE`. Ниже приводятся примеры выполнения этой операции с помощью Spark SQL API и DataFrame API.

```
Spark SQL:
spark.sql("""
    CREATE TABLE glue.test.employee (
        id INT,
        role STRING,
        department STRING,
        salary FLOAT,
        region STRING)
    USING iceberg
""")
```

Здесь `glue` – это ссылка на экземпляр каталога, `test` – имя базы данных / пространства имен (которая уже должна существовать в каталоге Glue), а `employee` – это имя новой таблицы. Таблица будет создана с пятью столбцами. Оператор `USING iceberg` в SQL API сообщает, что используется формат таблиц Iceberg.

```
DataFrame API:
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

# Определение схемы
schema = StructType([
    StructField("id", IntegerType(), True),
    StructField("role", StringType(), True),
    StructField("department", StringType(), True),
])

# Создать пустой набор данных DataFrame со схемой schema
df = spark.createDataFrame([], schema)

# Записать DataFrame в каталог как новую таблицу
df.writeTo("glue.test.employee").create()
```

Создание таблицы с секционированием

Секционирование – это способ разделения данных на более мелкие, управляемые единицы, позволяющий ускорить выполнение операций за счет ис-

ключения ненужных данных. Когда таблица секционирована, данные физически хранятся в разных каталогах, в зависимости от значений столбцов, по которым выполняется секционирование. Например, если секционировать таблицу с данными о товарах по регионам, то данные для каждого региона будут храниться отдельно, а при запросе данных для конкретного региона читаться и возвращаться будут только соответствующие данные. Мы подробно обсуждали секционирование в главе 4, а теперь покажем, как создать секционированную таблицу Iceberg с помощью Spark.

Spark SQL:

```
spark.sql("""
    CREATE TABLE glue.test.emp_partitioned (
        id INT,
        role STRING,
        department STRING)
    USING iceberg
    PARTITIONED BY (department)
    """)
```

Здесь оператор `PARTITIONED BY (department)` требует секционировать данные по столбцу `department`. В результате данные с разными значениями в этом столбце будут храниться в разных разделах.

DataFrame API:

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType
from pyspark.sql.functions import col
```

Определение схемы

```
schema = StructType([
    StructField("id", IntegerType(), True),
    StructField("role", StringType(), True),
    StructField("department", StringType(), True)
])
```

Создать пустой набор данных DataFrame со схемой schema

```
df = spark.createDataFrame([], schema)
```

Записать DataFrame в каталог как новую таблицу

```
df.writeTo("glue.test.emp_partitioned").partitionedBy(col("department")).create()
```

Apache Iceberg, в отличие от форматов таблиц, таких как Hive, поддерживает также скрытое секционирование, не требуя добавлять явные столбцы для секционирования. Iceberg берет столбец и внутренне преобразует его в значение раздела, сохраняя связанность данных. Это происходит «за кулисами», поэтому пользователям не придется сталкиваться с этим при запросе данных. Подробнее о скрытом секционировании можно прочитать в главе 4.

Оператор `PARTITIONED BY` поддерживает некоторые выражения преобразования для создания скрытых разделов. Вот пример на Spark SQL:

```
spark.sql("""
    CREATE TABLE glue.test.emp_partitioned_month (
```

```

        id INT,
        role STRING,
        department STRING,
        join_date DATE
    )
    USING iceberg
    PARTITIONED BY (months(join_date))
    """
)

```

Этот оператор создаст таблицу Iceberg с именем `emp_partitioned_month`, секционированную по значению месяца в поле `join_date`. При этом Iceberg выполнит внутреннее преобразование столбца `join_date` в `months(join_date)` и будет отслеживать взаимосвязь между ними, избавляя от необходимости создавать дополнительный столбец для секционирования. Соответственно, пользователям не придется беспокоиться о физическом размещении таблицы. Вот другие поддерживаемые преобразования:

- `year(ts)`: секционирование по годам;
- `months(ts)`: секционирование по месяцам;
- `days(ts)` или `date(ts)`: секционирование по дате (день, месяц, год);
- `hours(ts)` или `date_hour(ts)`: секционирование по дате (день, месяц, год) и часам;
- `bucket(N, col)`: секционирование по остатку от деления нацело хеш-значения на количество сегментов `N`;
- `truncate(L, col)`: секционирование по значению, усеченному до длины `L`.

CREATE TABLE...AS SELECT

Инструкция `CREATE TABLE...AS SELECT` (CTAS) позволяет одновременно создать и заполнить новую таблицу записями. Это особенно удобно в сценариях, где нужно создать новую таблицу на основе результатов применения запросов или преобразований к существующим таблицам. Следует отметить, что в контексте Apache Iceberg эта инструкция выполняется атомарно, только если используется класс `SparkCatalog`. Класс `SparkSessionCatalog` поддерживает CTAS, но выполняет ее неатомарно, что может привести к нарушению согласованности при выполнении нескольких операций записи одновременно. Вот пример использования инструкции CTAS в Apache Iceberg с помощью Spark:

```

Spark SQL:
spark.sql("""
    CREATE TABLE glue.test.employee_ctas
    USING iceberg
    AS SELECT * FROM glue.test.sample
    """)

```

Этот код создаст в каталоге `glue.test` новую таблицу `employee_ctas` и заполнит ее данными из существующей таблицы `sample`.

DataFrame API:

```
# Прочитать содержимое существующей таблицы в DataFrame
df_ctas = spark.read.table("glue.test.sample")

# Использовать метод writeTo объекта DataFrame с операцией create,
# чтобы выполнить операцию CTAS
df_ctas.writeTo("glue.test.employee_ctas").create()
```

Обратите внимание, что операция CTAS не наследует настройки секционирования и другие свойства от исходной таблицы – все необходимые свойства и настройки придется определить вручную, используя операторы PARTITIONED BY и TBLPROPERTIES. Например:

Spark SQL:

```
spark.sql("""
    CREATE TABLE glue.test.emp_ctas_partition
    USING iceberg
    PARTITIONED BY (category)
    TBLPROPERTIES (write.format.default='avro')
    AS SELECT *
    FROM glue.test.sample
""")
```

DataFrame API:

```
from pyspark.sql.functions import col

df_new = spark.read.table("glue.test.sample")
df_new.writeTo("glue.test.emp_ctas_partition") \
    .partitionedBy(col("category")) \
    .create()
```

ALTER TABLE

По мере изменения бизнес-требований может потребоваться изменить структуру существующих таблиц. Это может быть простое изменение имен, добавление или удаление столбцов таблицы или что-то более сложное, как, например, изменение свойств, схемы или типов столбцов таблицы. Apache Iceberg обеспечивает поддержку множества разных операций в инструкции ALTER TABLE. Обратите внимание, что эти операции возможны только с использованием Spark SQL и недоступны в DataFrame API.

Рассмотрим некоторые из этих операций.

Переименование таблицы

Иногда бывает нужно изменить схему именования таблиц Iceberg для лучшей организации ресурсов или когда текущее имя таблицы не точно отражает ее содержимое. В таких случаях можно воспользоваться командой ALTER TABLE...RENAME TO:

```
spark.sql("""
    ALTER TABLE glue.test.employee RENAME TO glue.test.emp_renamed
""")
```

В этом примере мы переименовали таблицу `employee` в `emp_renamed` в каталоге `glue.test` (при этом изменилось лишь пространство имен в каталоге, но путь к хранимой таблице остался прежним).

Установка свойств таблицы

Iceberg позволяет устанавливать свойства отдельных таблиц для управления их поведением, например изменять стратегию распределения операций записи для повышения производительности и включения определенных функций. Для этого существует команда `SET TBLPROPERTIES`:

```
spark.sql("""
    ALTER TABLE glue.test.employee SET TBLPROPERTIES ('write.wap.enabled'='true')
""")
```

Эта команда устанавливает в таблице `employee` свойство `write.wap.enabled` со значением `true`, разрешающее промежуточную запись для аудита перед публикацией.

Добавление столбца

Добавить больше полей в существующую таблицу Iceberg можно командой `ALTER TABLE...ADD COLUMN`:

```
spark.sql("""
    ALTER TABLE glue.test.employee ADD COLUMN manager STRING
""")
```

Здесь мы добавили еще один столбец с именем `manager` типа `string` в существующую таблицу `employee`, используя команду `ADD COLUMN`.

Команда позволяет добавить сразу несколько столбцов, перечислив их через запятую:

```
spark.sql("""
    ALTER TABLE glue.test.employee ADD COLUMN details STRING, manager_id INT
""")
```

Чтобы добавить столбцы в определенную позицию, можно использовать команды `FIRST` и `AFTER`:

```
spark.sql("""
    ALTER TABLE glue.test.employee ADD COLUMN new_column bigint AFTER department
""")
```

Этот запрос добавит новый столбец с именем `new_column` в определенную позицию, в данном случае после столбца `department`.

Аналогично следующий запрос добавит новый столбец `first_column` в самую первую позицию в таблице:

```
spark.sql("""
    ALTER TABLE glue.test.employee ADD COLUMN first_column bigint FIRST
""")
```

Переименование столбца

Переименование столбца – это еще одна операция, которая может потребоваться для реализации нового соглашения об именовании или для удовлетворения меняющихся потребностей в данных. Вот пример применения этой команды:

```
spark.sql("""
    ALTER TABLE glue.test.employee RENAME COLUMN role TO title
""")
```

Этот запрос переименует столбец `role` в `title` в таблице `employee` в каталоге `glue.test`.

Изменение столбца

Чтобы изменить такие атрибуты столбца, как тип или допустимость значений `NULL`, добавить комментарии и изменить порядок полей, можно использовать команду `ALTER TABLE...ALTER COLUMN`:

```
spark.sql("""
    ALTER TABLE glue.test.employee ALTER COLUMN id TYPE BIGINT
""")
```

Эта команда изменяет тип `INT` столбца `id` на `BIGINT` в таблице `employee`.

Iceberg позволяет гибко изменять типы столбцов, обеспечивая безопасность обновлений. Безопасность необходима для предотвращения потери или неправильной интерпретации данных. Вот некоторые из рекомендуемых безопасных обновлений:

- изменение целочисленного типа (`int`) на `bigint`;
- изменение типа `float` на `double`;
- изменение типа `decimal(P, S)` на `decimal(P2, S)`, при условии что масштаб `S` не изменяется.

Iceberg также позволяет изменять порядок столбцов с помощью команд `FIRST` и `AFTER`:

```
spark.sql("ALTER TABLE glue.test.employee ALTER COLUMN salary FIRST")
```

Предыдущий запрос перенесет столбец `salary` в самое начало таблицы.

Удаление столбца

Удаление ненужных столбцов из таблицы Iceberg выполняется командой `ALTER TABLE...DROP COLUMN`:

```
spark.sql("""
    ALTER TABLE glue.test.employee DROP COLUMN department
""")
```

Предыдущая команда удалит столбец `department` из таблицы `employee`.

Изменение таблиц с помощью расширений Iceberg Spark SQL

Apache Iceberg имеет модуль расширения Spark, реализующий дополнительные операции, недоступные в стандартном SQL. (Краткое описание этих операций приводится в табл. 6.2.) Например, расширения позволяют запускать различные процедуры Spark для очистки метаданных в Iceberg с помощью оператора `CALL` или изменять схему существующей таблицы с помощью операторов в инструкции `ALTER`. Чтобы использовать эти расширения, необходимо добавить свойство `extensions` в интерфейс командной строки Spark или в код. Вот пример установки этого свойства в коде PySpark:

```
import pyspark
from pyspark.sql import SparkSession
import os

conf = (
    pyspark.SparkConf()
        .setAppName('app_name')

    # Это свойство позволит добавлять и использовать любые расширения
        .set('spark.sql.extensions',
            'org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions')
)
spark = SparkSession.builder.config(conf=conf).getOrCreate()
```

Давайте познакомимся с некоторыми из этих расширений и посмотрим, как их использовать в инструкции `ALTER`.

Добавление/удаление/замена раздела

С помощью расширений Spark SQL Iceberg позволяет изменять схему и указывать дополнительные поля секционирования (которые будут учитываться в будущих операциях записи), а также удалять или заменять имеющиеся поля секционирования. Следующий запрос добавит в таблицу `employee` новое поле

секционирования `region`. После этого данные в таблице будут секционироваться на основе значений в столбце `region`, что ускорит выборку данных для конкретных регионов:

```
spark.sql("""
    ALTER TABLE glue.test.employee ADD PARTITION FIELD region
""")
```

Следующий запрос прекратит секционирование таблицы `employee` по полю `department`. Однако прекращение секционирования не повлияет на существующий столбец или схему таблицы:

```
spark.sql("""
    ALTER TABLE glue.test.employee DROP PARTITION FIELD department
""")
```

Заменить одно поле секционирования другим можно командой `ALTER TABLE...REPLACE PARTITION FIELD`:

```
spark.sql("""
    ALTER TABLE glue.test.employee REPLACE PARTITION FIELD region WITH department
""")
```

В этом примере секционирование по полю `region` заменяется секционированием по полю `department`.



Операции добавления, удаления или замены поля секционирования влияют только на метаданные, т. е. никакие существующие файлы данных не будут изменены. Текущие данные останутся в старом разделе (если применимо), а новые будут записываться в соответствии с новой стратегией секционирования. Однако процедуры обслуживания таблиц будут переписывать файлы, руководствуясь текущей схемой секционирования.

Установка порядка сортировки

В таблицах Iceberg можно настроить порядок сортировки, руководствуясь которым, вычислительный механизм будет автоматически сортировать данные, записываемые в таблицу. Установить порядок сортировки можно командой `ALTER TABLE...WRITE ORDERED BY`.

Вот пример сортировки таблицы `employee` по полю `id` в порядке возрастания:

```
spark.sql("""
    ALTER TABLE glue.test.employee WRITE ORDERED BY id ASC
""")
```

Настройка распределения операций записи

Iceberg позволяет настроить распределение записываемых данных между процессами, предлагая команду `ALTER TABLE...WRITE DISTRIBUTED BY PARTITION`.

Она гарантирует, что каждый раздел данных будет обрабатываться одним записывающим процессом. Такая стратегия способствует предотвращению неравномерного распределения данных между процессами записи. Реализация этой функции по умолчанию использует распределение по хешу.

Вот как применить эту настройку к таблице:

```
spark.sql("""
    ALTER TABLE glue.test.employee WRITE DISTRIBUTED BY PARTITION
""")
```

После выполнения данного запроса запись в каждый раздел таблицы employee будет выполняться отдельным процессом.

Установка/сброс полей идентификаторов

Назначить определенным полям роль идентификатора (поле идентификатора позволяет механизму заметить, что две записи относятся к одному и тому же объекту) или отозвать ее можно с помощью команды ALTER TABLE...SET/DROP IDENTIFIER FIELDS:

```
spark.sql("""
    ALTER TABLE glue.test.employee SET IDENTIFIER FIELDS id
""")
```

В предыдущем примере полю id в таблице employee назначается роль идентификатора, уникально идентифицирующего каждую запись.

Следующий запрос показывает, как отозвать роль идентификатора у поля:

```
spark.sql("""
    ALTER TABLE glue.test.employee DROP IDENTIFIER FIELDS id
""")
```

В табл. 6.2 приводится краткое описание операций ALTER, которые были рассмотрены в этом разделе, и ситуаций, когда расширения SQL могут быть необходимы, а где нет.

Таблица 6.2. Операции, поддерживаемые с использованием расширений SQL и без них

Операция	Доступна без расширений SQL	Доступна в расширениях SQL
Переименование таблиц	Да	Нет
Настройка свойств таблиц	Да	Нет
Добавление/переименование/изменение/удаление столбцов	Да	Нет
Добавление/удаление/замена столбцов секционирования	Нет	Да
Настройка порядка записи	Нет	Да
Настройка распределения операций записи	Нет	Да
Установка/сброс полей идентификаторов	Нет	Да

DROP TABLE

Последняя команда DDL в Iceberg, которую мы рассмотрим, – DROP. Инструкция DROP позволяет удалить существующую таблицу из каталога. Важно отметить, что до версии Iceberg v0.14 выполнение команды DROP TABLE приводило к удалению таблицы из каталога вместе с ее метаданными и содержимым. Однако начиная с версии 0.14 это поведение было изменено: теперь таблица удаляется только из каталога, а ее содержимое сохраняется.

Следующий запрос удалит таблицу `employee` из `glue.test`, но ее содержимое останется нетронутым:

```
spark.sql("DROP TABLE glue.test.employee").
```

Удалить таблицу вместе с ее содержимым можно с помощью команды `DROP TABLE...PURGE`:

```
spark.sql("DROP TABLE glue.test.employee PURGE")
```

Чтение данных

В этом разделе мы сосредоточимся на чтении данных из таблиц Apache Iceberg с помощью Spark. Сначала мы посмотрим, как запрашивать и исследовать данные посредством Spark SQL, а затем обсудим использование `DataFrameWriterV2` для тех же целей.

Запрос выборки всех записей

Команда `SELECT *` извлекает все записи из существующей таблицы Iceberg.

Spark SQL:

```
spark.sql("SELECT * FROM glue.test.employee").show()
```

Предыдущий запрос выбирает все записи из таблицы `employee` в каталоге `glue` в пространстве имен `test`.

DataFrame API:

```
df_emp = spark.table("glue.test.employee")
```

Здесь метод вызывает метод `spark.table()`, который загружает таблицу `employee` в `DataFrame`.

Запрос с фильтрацией записей

Чтобы отфильтровать данные на основе определенных критериев, можно использовать условные запросы. Например:

```

Spark SQL:
spark.sql("SELECT * FROM glue.test.employee
          WHERE department = 'Marketing'").show()
DataFrame API:
df_emp = spark.table("glue.test.employee")

# Фильтрация данных
filtered_df = df_emp.filter(df_emp['department'] == 'Marketing')

filtered_df.show()

```

Агрегирующие запросы

Агрегирующие запросы позволяют выполнять вычисления с группами записей. Используя агрегирующие функции, такие как SUM, AVG, MAX, MIN и COUNT, можно эффективно извлекать ценную информацию и статистику из набора данных. Далее приводятся несколько примеров.

Подсчет записей

Подсчитать количество записей в таблице Iceberg можно с помощью метода COUNT() в Spark SQL и DataFrame API. Следующие запросы вернут количество всех записей в таблице glue.test.employee.

```

Spark SQL:
spark.sql("SELECT COUNT(*) FROM glue.test.employee").show()
DataFrame API:
df_emp = spark.table("glue.test.employee")
print(df_emp.count())

```

Вычисление среднего значения

Метод AVG() вычисляет среднее значение по всем записям в таблице Iceberg. Следующие запросы вернут среднюю зарплату (поле salary) по всем записям в таблице glue.test.employee.

```

Spark SQL:
spark.sql("SELECT AVG(salary) FROM glue.test.employee").show()
DataFrame API:
df_emp = spark.table("glue.test.employee")
df_emp.agg({'salary': 'avg'}).show()

```

Вычисление суммы значений

SUM() вычисляет общую сумму значений во всех записях таблицы. Следующие запросы вернут сумму всех зарплат (поле salary) по всем записям в таблице glue.test.employee.

```

Spark SQL:
spark.sql("SELECT SUM(salary) FROM glue.test.employee").show()

```

DataFrame API:

```
df_emp = spark.table("glue.test.employee")
df_emp.agg({'salary': 'sum'}).show()
```

Поиск максимума

Следующие запросы группируют сотрудников по категориям, а затем определяют максимальную зарплату в каждой категории.

Spark SQL:

```
spark.sql("SELECT category, MAX(salary)
          FROM glue.test.employee GROUP BY category").show()
```

DataFrame API:

```
df_emp = spark.table("glue.test.employee")
df_emp.groupBy("category").max("salary").show()
```

Оконные функции

Оконные функции очень полезны для выполнения вычислений над набором записей, связанных с текущей записью. В отличие от агрегирующих функций, оконные функции не группируют строки в единственное значение. Обычно они используются для таких задач, как ранжирование элементов и расчет скользящих средних. Вот пример использования оконных функций в Spark.

Spark SQL:

```
spark.sql("""
SELECT * , RANK() OVER (PARTITION BY department ORDER BY salary DESC) as rank
FROM glue.test.employee
""").show()
```

В предыдущем запросе инструкция `RANK() OVER (PARTITION BY department ORDER BY salary DESC)` создает окно, которое группирует записи по столбцу `department`, и упорядочивает их по зарплате в порядке убывания. Каждому разделу `RANK()` присваивает уникальный ранг, начиная с 1.

Вот как выглядит результат:

```
id department salary region rank
2 Marketing  10000  NA      1
6 Marketing  5000   EMEA   2
5 Product   25000  NA      1
4 Product   17000  EMEA   2
3 Sales      8000   APAC   1
```

DataFrame API:

```
from pyspark.sql import Window
from pyspark.sql.functions import row_number
```

```
df_emp = spark.table("glue.test.employee")
```

```
windowSpec = Window.partitionBy(df_emp['department']).orderBy(df_emp['salary'].desc())
```

```
df_emp = df_emp.withColumn("row_number", row_number().over(windowSpec))
df_emp.toPandas()
```

В предыдущем запросе инструкция `Window.partitionBy(df['department']).orderBy(df['salary'].desc())` создает спецификацию окна, которое группирует записи по столбцу `department` и упорядочивает их по зарплате в порядке убывания. Затем к этой спецификации применяется функция `row_number().over(windowSpec)` для создания нового столбца `row_number`. Функция `withColumn` добавляет этот новый столбец в `DataFrame`.

Вот результат:

```
id department salary region row_number
2 Marketing  10000  NA      1
6 Marketing   5000  EMEA    2
5 Product   25000  NA      1
4 Product   17000  EMEA    2
3 Sales      8000  APAC    1
```

Запись данных

Apache Iceberg обеспечивает атомарность и гарантии целостности при записи в озера данных, делая эти операции безопасными. В этом разделе мы рассмотрим операции `INSERT INTO`, `MERGE INTO`, `INSERT OVERWRITE`, `DELETE` и `UPDATE`, чтобы дать вам практический опыт применения этих важных функций управления данными. Мы будем использовать как Spark SQL, так и `DataFrameWriterV2 API`, где это применимо. Обратите внимание, что `DataFrameWriterV2 API` поддерживает не все эти операции.

INSERT INTO

`INSERT INTO` позволяет вставлять новые записи в таблицу Iceberg.

Spark SQL:

```
spark.sql("INSERT INTO glue.test.employee VALUES (1, 'Software Engineer',
'Engineering', 25000, 'NA'), (2, 'Director', 'Sales', 22000, 'EMEA')")
```

DataFrame API:

```
from pyspark.sql import Row
```

```
# Создать DataFrame со значениями
```

```
data = [Row(id=1, role='Software Engineer', department='Engineering',
            salary=25000, region='NA'),
        Row(id=2, role='Director', department='Sales', salary=22000,
            region='EMEA')]
```

```
df = spark.createDataFrame(data)
df.writeTo("glue.test.employee").append()
```

Оба этих запроса вставят две записи в таблицу `employee`.

MERGE INTO

`MERGE INTO` обновляет существующую запись в зависимости от выполнения определенного условия. Если условие не выполняется, то в таблицу вставляется новая запись.

Spark SQL:

```
spark.sql("""
MERGE INTO glue.test.employee AS target
USING (SELECT * FROM employee_updates) AS source
ON target.id = source.id
WHEN MATCHED AND source.role = 'Manager' AND source.salary > 100000 THEN
    UPDATE SET target.salary = source.salary
WHEN NOT MATCHED THEN
    INSERT *
""")
```

Предыдущий запрос сначала сравнивает каждую запись в таблице `employee_updates` с каждой записью в таблице `employee`, используя выражение `target.id = source.id`. Если совпадение найдено, роль в промежуточной таблице имеет значение `Manager` и зарплата больше `100000`, то в поле `salary` таблицы `target` записывается значение этого же поля из таблицы `source`. Если условие `target.id = source.id` не выполняется, то в таблицу `target` вставляется новая запись.

INSERT OVERWRITE

Инструкция `INSERT OVERWRITE` используется для замены данных в таблице или разделе Iceberg результатом запроса. Apache Spark предоставляет для этой операции два режима перезаписи: статический и динамический (по умолчанию используется статический режим). Рассмотрим оба режима подробнее.

Статическая перезапись

В статическом режиме перезаписи Spark преобразует предложение `PARTITION` в фильтр (предикат) для определения разделов, которые следует перезаписать. Если запустить запрос без оператора `PARTITION`, он выполнит замену во всех разделах. Следующий запрос перезапишет данными из таблицы `employee_source` только раздел `EMEA` таблицы `employee`. Обратите внимание, что этот режим не заменяет скрытые разделы, потому что оператор `PARTITION` может ссылаться только на столбцы таблицы.

Spark SQL:

```
spark.sql("""
INSERT OVERWRITE glue.test.employees
```



```

PARTITION (region = 'EMEA')
SELECT *
FROM employee_source
WHERE region = 'EMEA'
""")
DataFrame API:
from pyspark.sql.functions import col

# Прочитать таблицу-источник
source_df = spark.read.table("glue.test.employee")

# Отфильтровать записи, оставив только относящиеся к региону 'EMEA'
filtered_df = source_df.filter(col("region") == 'EMEA')

# Перезаписать таблицу
filtered_df.writeTo("glue.test.employee").overwrite(col("region") == 'EMEA')

```

Динамическая перезапись

Чтобы включить режим динамической перезаписи, нужно задать конфигурационное свойство `spark.sql.sources.partitionOverwriteMode=dynamic`. В этом режиме заменяются все разделы, соответствующие записям, которые возвращает запрос `SELECT`:

```
spark.conf.set("spark.sql.sources.partitionOverwriteMode", "dynamic")
```

Следующий запрос заменит любой раздел в таблице `employee`, соответствующий данным, полученным запросом `SELECT`. Поскольку таблица `employee_source` фильтруется по региону `EMEA`, в таблице `employee` будет перезаписан только раздел, соответствующий региону `EMEA`.

Spark SQL:

```

spark.sql("""
INSERT OVERWRITE glue.test.employee
SELECT * FROM employee_source
WHERE region = 'EMEA'
""")
DataFrame API:
from pyspark.sql.functions import col

# Установить режим динамической перезаписи
spark.conf.set("spark.sql.sources.partitionOverwriteMode", "dynamic")

# Прочитать данные из таблицы-источника
source_df = spark.read.table("glue.test.employee")

# Отфильтровать записи, оставив только принадлежащие региону 'EMEA'
filtered_df = source_df.filter(col("region") == 'EMEA')

# Перезаписать данные в таблице Iceberg
filtered_df.writeTo("glue.test.employee").overwritePartitions()

```

Режим динамической перезаписи обычно рекомендуется использовать для записи в таблицы `Iceberg`, потому что он обеспечивает детальный конт-

роль над тем, какие разделы будут перезаписаны, в зависимости от результата запроса.

DELETE FROM

DELETE FROM позволяет удалять из таблицы Iceberg записи, соответствующие заданному условию. Следующий запрос удалит все записи из таблицы employee со значением id меньше 3.

Spark SQL:

```
spark.sql("DELETE FROM glue.test.employee WHERE id < 3")
```

Apache Iceberg поддерживает два типа удаления в зависимости от условия фильтра. Если условие фильтра соответствует целым разделам таблицы, то удаляются только метаданные. Это очень эффективная операция, потому что не затрагивает файлы данных. С другой стороны, если условие удаления соответствует отдельным записям в таблице, то Iceberg перезапишет затронутые файлы данных.

UPDATE

UPDATE позволяет изменять существующие записи в таблице, соответствующие заданному условию.

Spark SQL:

```
spark.sql("""
UPDATE glue.test.employee
SET region = 'APAC', salary = 6000
WHERE id = 6
""")
```

Этот запрос обновит поля region и salary сотрудника с id=6 в таблице employee.

Вот еще один пример:

```
spark.sql("""
UPDATE glue.test.employee AS e
SET region = 'NA'
WHERE EXISTS (SELECT id FROM emp_history WHERE emp_NA.id = e.id)
""")
```

Этот запрос обновит поле region для всех сотрудников, чей идентификатор присутствует в таблице emp_NA. Здесь показано, как можно использовать подзапросы в командах UPDATE, чтобы производить обновления на основе условий, в которых упоминается несколько таблиц.

Процедуры обслуживания таблиц Iceberg

Управление файлами данных и метаданных в озере имеет первостепенное значение, потому что объем данных со временем растет. В Iceberg файлы метаданных лежат в основе многих критически важных операций, таких как доступ к исторической информации и оптимизация запросов. Однако с увеличением количества файлов данных увеличивается и количество файлов метаданных. Кроме того, задания приема на основе потоковой передачи могут привести к созданию множества маленьких файлов, особенно если данные записываются небольшими порциями по мере их поступления. Поэтому в любой организации важно иметь стратегию по удалению ненужных файлов метаданных и уплотнению маленьких файлов данных в более крупные для повышения производительности чтения.

Apache Spark предоставляет простые процедуры для обслуживания таблиц Iceberg. В этом разделе мы рассмотрим некоторые из них. Более подробную информацию вы найдете в главе 4.

Удаление снимков

Любое изменение данных в Iceberg, будь то вставка, изменение или удаление, создает новый снимок. Эти снимки сохраняются для поддержки таких возможностей, как доступ к историческим данным (путешествие во времени). Однако со временем может накопиться много снимков, в том числе и совершенно ненужных. Процедура `expire_snapshots` в Spark помогает удалить старые ненужные снимки вместе с их файлами данных.

Эта процедура удаляет списки манифестов, манифесты, файлы данных и файлы, которые однозначно связаны с устаревшими снимками, и гарантирует сохранность файлов, которые все еще используются активными снимками.

Вот пример вызова этой процедуры:

```
# Общая процедура
CALL catalog_name.system.expire_snapshots(table, older_than, retain_last)

spark.sql("CALL glue.system.expire_snapshots('test.employees',
date_sub(current_date(), 90), 50)")
```

Эта процедура удалит из таблицы `employee` снимки старше 90 дней, но сохранит не меньше 50 последних снимков.

Перезапись файлов данных

Количество файлов данных в таблице напрямую влияет на производительность запросов, потому что открытие, чтение и закрытие каждого файла данных, вовлеченного в обработку запроса, сопряжено с немалыми затратами, и обработка большого количества маленьких файлов может потребовать ненужных накладных расходов на обработку файлов метаданных. Процедура `rewrite_data_files` в Spark позволяет уплотнить небольшие файлы в более крупные и смягчить данную проблему.

Эту процедуру также можно использовать для реорганизации файлов данных с применением таких стратегий, как сортировка и z-упорядочение, чтобы оптимизировать их для наиболее частых запросов. Эта тема тоже рассматривалась в главе 4.

Вот пример вызова данной процедуры:

```
# Общая процедура
CALL catalog_name.system.rewrite_data_files(table, strategy, sort_order, options)

spark.sql("CALL glue.system.rewrite_data_files('test.employee')")
```

Эта процедура перезапишет файлы данных, объединит маленькие файлы данных, составляющие таблицу `employee`, с применением стратегии уплотнения `binpack`, используемой по умолчанию, и разобьет особенно крупные файлы с учетом размера записи по умолчанию.

Перезапись манифестов

Файлы манифестов в таблицах Iceberg играют решающую роль в оптимизации планирования сканирования, так как они хранят статистическую информацию о файлах данных. Процедура `rewrite_manifests` позволяет перезаписывать манифесты параллельно и тем самым повышать скорость и эффективность сканирования данных. Ниже приводится пример процедуры перезаписи манифестов таблицы `employee`:

```
# Общая процедура
CALL catalog_name.system.rewrite_manifests(table)

spark.sql("CALL test.system.rewrite_manifests('test.employee')")
```

Удаление осиротевших файлов

Со временем некоторые файлы данных в таблице Iceberg могут «осиротеть», т. е. из файлов метаданных могут быть исключены все ссылки на них. Эти файлы занимают место в хранилище и могут вызвать проблемы несогласованности. Удалить такие файлы поможет процедура `remove_orphan_files`.

Вот пример вызова этой процедуры:

```
# Общая процедура  
CALL catalog_name.system.remove_orphan_files(table, older_than, dry_run)  
  
CALL glue.system.remove_orphan_files(table => 'test.employee', dry_run => true)
```

Эта процедура позволяет выполнить пробный прогон и ознакомиться со списком «осиротевших» файлов, которые будут удалены.

Заклучение

В данной главе мы рассмотрели методы взаимодействия с таблицами Iceberg, доступные в Apache Spark. Познакомились с различными аспектами этих взаимодействий, такими как первоначальная настройка, создание и изменение структур с помощью DDL, выполнение анализа, запись данных и выполнение операций обслуживания таблиц.

В главе 7 мы поговорим о независимой природе Apache Iceberg от конкретного механизма и покажем, как выполнять те же самые операции с помощью другого механизма – Dremio SQL Engine.

Глава 7

Dremio SQL Query Engine

Dremio SQL Query Engine – механизм обработки запросов SQL – является частью платформы Dremio Lakehouse и широко используется для поддержки различных аналитических сервисов с малой задержкой, которые выполняют как специальные SQL-запросы, так и запросы бизнес-аналитики (Business Intelligence, BI) к озеру данных. Dremio позволяет запрашивать данные из нескольких источников и, соответственно, обеспечивает объединение запросов и единое представление данных без их перемещения или копирования. Все это делается с помощью механизма векторизованных запросов, который позволяет Dremio быстро получать результаты запросов даже из чрезвычайно больших наборов данных. В сочетании с возможностями формата таблиц Apache Iceberg он дает мощную комбинацию инструментов для управления наборами данных и выполнения запросов с повышенной производительностью.

В этой главе мы поможем вам освоить работу с Dremio и Iceberg.

Настройка

Платформа Dremio Lakehouse предлагается как в локальном, так и в облачном вариантах. В примерах этой главы будет использоваться Dremio Cloud. Как обсуждалось в главе 6, первым шагом к началу работы с таблицами Iceberg является определение конфигурации каталога. Чтобы настроить каталог Iceberg на платформе Dremio Lakehouse, нужно добавить новый источник, перейдя в раздел **Sources** (Источники) в интерфейсе Dremio, и выбрать **Add Data Source** (Добавить источник данных), как показано на рис. 7.1.

Здесь можно добавить новый источник, сделав выбор между AWS Glue, Amazon Simple Storage Service (Amazon S3) и реляционными базами данных.

После этого Dremio будет использовать каталог Iceberg, соответствующий этому источнику данных. В табл. 7.1 перечислены типы каталогов Iceberg, соответствующие источникам Dremio.

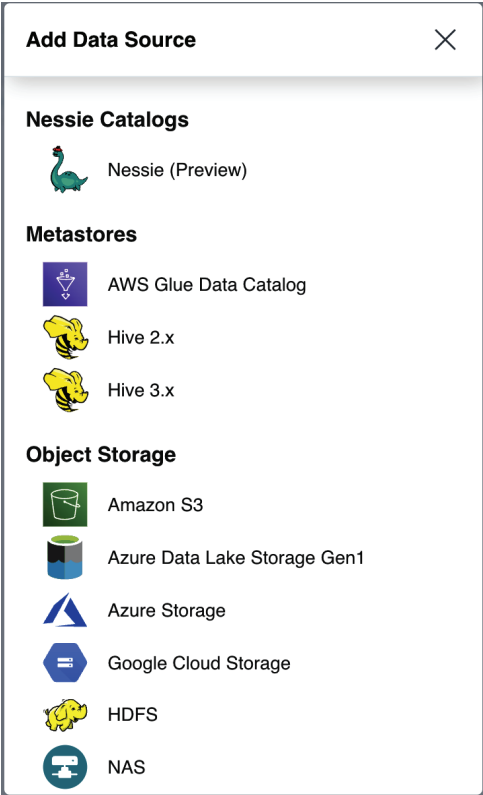


Рис. 7.1 ❖ Источники, доступные для подключения в Dremio

Таблица 7.1. Соответствия между каталогами Apache Iceberg и источниками данных Dremio

Тип источника Dremio	Каталог Iceberg
AWS Glue	Каталог AWS Glue
Amazon Simple Storage Service, Azure Data Lake Storage, Hadoop Distributed File System, Google Cloud Storage	Каталог Hadoop
Arctic/Nessie	Каталог Nessie

После успешной настройки Dremio автоматически подключится к соответствующему каталогу Iceberg, и вы сможете приступить к созданию таблицы Iceberg (если вам потребуется указать конкретное местоположение хранилища для каталогов таблиц, то обратитесь к описанию вашего варианта использования в документации Dremio).

Операции на языке определения данных

В этом разделе вы познакомитесь с различными операциями на языке определения данных (DDL), доступными в Dremio SQL Query Engine для работы с таблицами Iceberg.

CREATE TABLE

Создать таблицу Iceberg можно с помощью оператора `CREATE TABLE`. Мы обсудим несколько вариантов этой команды, чтобы дать вам представление о доступных возможностях.

Начнем с создания простой таблицы:

```
CREATE TABLE employee (ID int, role varchar, department varchar, salary float,
region varchar)
```

Эта инструкция создаст таблицу `employee` с пятью столбцами и сохранит ее в указанном источнике, в данном случае в Amazon S3.

CREATE TABLE...AS SELECT

Как и другие вычислительные механизмы, такие как Spark, Dremio предоставляет оператор `CREATE TABLE...AS SELECT` для создания таблицы Iceberg из существующей таблицы или файла данных:

```
CREATE TABLE employee AS SELECT * FROM mySource."myFolder"."empData.csv"
```

Здесь предполагается, что файл данных существует в источнике `mySource`, Dremio выполнит запрос и создаст таблицу с именем `employee`, используя схему и записи из набора данных, полученных в ответ на запрос `SELECT`.

Создание таблицы с секционированием и сортировкой

Создавая таблицы, их можно сразу оптимизировать для определенных типов запросов, используя операторы `PARTITION BY` и `LOCALSORT BY`. Оператор `PARTITION BY` секционирует таблицу по определенным столбцам, а оператор `LOCALSORT BY` сортирует каждый файл Parquet фрагмента по указанному столбцу для ускорения поиска.

В следующем примере создается таблица `emp_partitioned`, секционированная по полю `department` и отсортированная по полю `id`. В этом случае наибольший выигрыш получают запросы, которые часто фильтруют данные по полю `department` и сортируют или фильтруют по полю `id`:


```
CREATE TABLE emp_partitioned (id int, role varchar, department varchar)
PARTITION BY (department)
LOCALSORT BY (id)
```

Таблицы Iceberg также поддерживают более продвинутые стратегии секционирования с применением функций преобразования секционирования, таких как `year()`, `month()`, `day()`, `hour()`, `bucket()` и `truncate()`. Эти преобразования позволяют секционировать данные на основе полей времени, хеш-значений или усеченных значений, обеспечивая большую гибкость и возможность оптимизации.

Вот как создать таблицу и секционировать ее по месяцам:

```
CREATE TABLE emp_partitioned_by_month
(id int, role varchar, department varchar, join_date date)
PARTITION BY (month(join_date))
```

Создание таблицы с доступом по строкам и маскирование столбцов

Dremio позволяет создавать таблицы с политиками доступа по строкам и маскирования столбцов, обеспечивая дополнительный уровень безопасности. Политика доступа по строкам определяет, какие записи может просматривать или изменять пользователь, а политика маскирования столбцов маскирует данные в определенных столбцах на основе функции, определяемой пользователем (User Defined Function, UDF).

Вот несколько примеров, в которых предполагается наличие функций UDF `strict_region` и `mask_salary`:

```
-- Создать UDF restrict_region
CREATE FUNCTION restrict_region(region VARCHAR)
RETURNS BOOLEAN
RETURN SELECT CASE
    WHEN query_user()='jdoe@dremio.com' OR is_member('HR') THEN true
    WHEN region = 'North' THEN true
    ELSE false
END;

-- Создать таблицу regional_employee_data с политикой доступа по строкам
CREATE TABLE regional_employee_data (
    id INT,
    role VARCHAR,
    department VARCHAR,
    salary FLOAT,
    region VARCHAR,
    ROW ACCESS POLICY restrict_region(region)
);
```

Предыдущая инструкция создаст таблицу `regional_employee_data`, ограничивающую доступ к определенным регионам.

```
-- Создать UDF mask_salary
CREATE FUNCTION mask_salary(salary VARCHAR)
RETURNS VARCHAR
RETURN SELECT CASE
    WHEN query_user()='jdoe@dremio.com' OR is_member('HR') THEN salary
    ELSE 'XXX-XX'
END;

-- Создать таблицу employee_salaries с политикой маскирования столбца
CREATE TABLE employee_salaries (
    id INT,
    salary VARCHAR MASKING POLICY mask_salary (salary),
    department VARCHAR
);
```

Предыдущий запрос создаст таблицу `employee_salaries`, маскирующую столбец `salary` из соображений безопасности.

ALTER TABLE

Инструкция `ALTER TABLE` позволяет изменить структуру существующей таблицы, включая добавление или удаление столбцов, изменение их типов или переименование, а также установку или снятие политик маскировки и многое другое.

Рассмотрим несколько примеров.

ADD COLUMNS

Допустим, нам нужно добавить столбец `date_of_birth` для хранения даты рождения каждого сотрудника. Для этого можно использовать команду `ADD COLUMNS`:

```
ALTER TABLE employee
ADD COLUMNS (date_of_birth DATE);
```

MODIFY COLUMN

Установить или отключить политику маскировки для определенного столбца можно с помощью команды `MODIFY COLUMN`:

```
-- Установка политики маскировки
ALTER TABLE employee
MODIFY COLUMN ssn_col
SET MASKING POLICY protect_ssn;

-- Отмена политики маскировки
ALTER TABLE employee
```

```
MODIFY COLUMN ssn_col  
UNSET MASKING POLICY;+
```

ALTER COLUMN

Переименовать некоторый столбец в таблице Iceberg можно с помощью команды ALTER COLUMN:

```
ALTER TABLE employee  
ALTER COLUMN role title VARCHAR;
```

Эта инструкция изменит имя столбца role на title в таблице employee.

DROP COLUMN

Если по какой-то причине столбец станет не нужен, то его можно удалить командой DROP COLUMN:

```
ALTER TABLE employee  
DROP COLUMN department;
```

DROP TABLE

Инструкция DROP TABLE позволяет удалить таблицу из источника данных. Однако важно отметить, что эффект этой команды зависит от типа источника данных. Например, если источником данных является корзина Amazon S3, то инструкция DROP TABLE безвозвратно удалит все файлы данных и метаданных и их невозможно будет восстановить. Если источником данных является AWS Glue, то инструкция удалит только ссылку на таблицу из каталога; файлы данных останутся нетронутыми.

Например, удалить таблицу employee можно так:

```
DROP TABLE employee;
```

Инструкция безвозвратно удалит таблицу со всеми ее файлами данных и метаданных, потому что в данном случае источником данных служит Amazon S3.

Чтение данных

Dremio поддерживает стандарт SQL и обеспечивает простой и понятный способ извлечения данных. В этом разделе мы покажем разные способы чтения данных из таблицы Iceberg.

Использование запроса SELECT

Самый простой способ получить данные – использовать запрос SELECT. Эта инструкция может запрашивать всю таблицу или только определенные столбцы. Например, вот как можно получить все данные из таблицы employee:

```
SELECT * FROM employee;
```

Фильтрация записей

Часто бывает нужно получить лишь часть записей, соответствующих определенным критериям. Это можно реализовать с помощью оператора WHERE:

```
SELECT * FROM employee  
WHERE department = 'Engineering';
```

Предыдущий запрос отфильтрует записи, не соответствующие критерию department = 'Engineering'.

Агрегирующие запросы

Агрегирующие запросы позволяют вычислять агрегированные значения, такие как сумма, среднее, максимум и минимум, по значениям в таблице Iceberg. Далее приводятся несколько примеров.

Подсчет записей

Функция COUNT() в Dremio позволяет узнать количество записей в таблице:

```
SELECT role, COUNT(*) as employee_count  
FROM employee  
GROUP BY role;
```

Этот запрос подсчитывает количество сотрудников для каждой уникальной роли.

Вычисление среднего значения

С помощью функции AVG() можно, например, вычислить среднюю зарплату в каждом регионе:

```
SELECT region, AVG(salary) FROM employee GROUP BY region;
```

Здесь оператор GROUP BY группирует данные по регионам, а функция AVG() вычисляет среднее значение в каждой группе.

Вычисление суммы значений

Функция `SUM()` вычисляет сумму для данного столбца. Следующий пример вычисляет суммарную зарплату для каждого отдела:

```
SELECT department, SUM(salary) as total_salary
FROM employee
GROUP BY department;
```

Поиск максимума

Функция `MAX()` отыскивает максимальное значение указанного столбца. Например, вот как можно найти самую высокую зарплату в каждом отделе:

```
SELECT department, MAX(salary) as highest_salary
FROM employee
GROUP BY department;
```

Оконные функции

Оконные функции применяются к набору записей в таблице, связанных с текущей записью, и предлагают простой способ выполнения сложных вычислений.

Например, предположим, что вы решили ранжировать сотрудников в каждом регионе по величине зарплаты. В Dremio для этого можно использовать функцию `RANK()`:

```
SELECT salary, region,
RANK() OVER (PARTITION BY region ORDER BY salary DESC) AS salary_rank
FROM company_data.employee;
```

Здесь функция `RANK()` применяется к каждому разделу данных, и затем ранги зарплат упорядочиваются в каждом регионе. В вывод добавляется новый столбец `salary_rank`, в котором возвращается ранг зарплаты каждого сотрудника в его регионе.

Вот получившийся результат:

```
salary region salary_rank
28000  EMEA    1
16000  EMEA    2
30000  NA       1
18000  NA       2
```

Запись данных

Dremio поддерживает множество способов манипулирования данными: от вставки новых данных до изменения или удаления существующих записей. В этом разделе мы рассмотрим некоторые из этих операций.

INSERT INTO

Инструкция `INSERT INTO` добавляет новые записи в таблицу Iceberg. Например, добавим двух новых сотрудников в таблицу `employee`:

```
INSERT INTO employee VALUES
  (7, 'Solution Architect', 'Sales', 15000, 'EMEA'),
  (8, 'Product Manager', 'Product', 28000, 'NA');
```

Эта инструкция добавит в таблицу `employee` две новые записи сотрудников с идентификаторами 7 и 8.

COPY INTO

Еще один простой способ вставить новые данные в таблицу Iceberg – инструкция `COPY INTO`. Она позволяет загружать данные из файлов CSV, JSON или Parquet, хранящихся в разных источниках, и преобразовывать их в таблицы Iceberg. Вот пример использования инструкции `COPY INTO`:

```
COPY INTO employee
FROM '@mySource/myFolder/'
FILE_FORMAT 'csv';
```

Предыдущая инструкция скопирует все файлы CSV, находящиеся в папке *myFolder*, и вставит их содержимое в таблицу `employee`. Источник `@mySource` может быть любым источником данных из числа поддерживаемых Dremio (HDFS, S3, ADLS и т. д.).

Вот пример копирования одного конкретного файла:

```
COPY INTO employee
FROM '@mySource/myFolder/employee_data.csv';
```

Результатом операции `COPY INTO` является количество строк, вставленных в таблицу Iceberg.

MERGE INTO

Когда нужно вставить новые или обновить существующие данные, руководствуясь некоторыми критериями, можете воспользоваться функцией `MERGE`. Эта функция сравнивает записи из двух таблиц (исходной и целевой), используя заданное условие, и выполняет операции `INSERT` или `UPDATE`.

Например, предположим, что у нас есть еще одна таблица с именем `new_employee` с обновленными данными о зарплате существующих сотрудников и с данными о нескольких новых сотрудниках. Нам нужно объединить эту таблицу с таблицей `employee`. Вот как можно это сделать:

```

MERGE INTO employee AS e USING new_employee AS ne ON (e.id = ne.id)
  WHEN MATCHED THEN UPDATE SET salary = ne.salary
  WHEN NOT MATCHED THEN INSERT (id, role, department, salary, region) VALUES
(ne.id, ne.role, ne.department, ne.salary, ne.region);

```

DELETE

Инструкция DELETE удаляет записи, соответствующие заданным условиям. Например, вот как можно удалить всех сотрудников из региона NA в таблице employee:

```
DELETE FROM employee WHERE region = 'NA';
```

UPDATE

Инструкция UPDATE позволяет изменить существующие записи в таблице Iceberg. Например, увеличить зарплату всем сотрудникам в отделе маркетинга на 2000 долларов можно с помощью такой операции UPDATE:

```

UPDATE employee
SET salary = salary + 2000
WHERE department = 'Marketing';

```

Этот запрос изменит значение поля salary в таблице employee, прибавив 2000 долларов к текущей зарплате каждого сотрудника из отдела маркетинга.

Обратите внимание, что Dremio не поддерживает условия соединения в операторах WHERE при использовании UPDATE. Если потребуется использовать условие соединения, то воспользуйтесь инструкцией MERGE.

Обслуживание таблиц Iceberg

Как показало обсуждение в разделе «Процедуры обслуживания таблиц Iceberg» главы 6, обслуживание таблиц Apache Iceberg играет ключевую роль в управлении размером таблиц, повышении производительности запросов и сохранении соответствующих версий данных. Под обслуживанием подразумеваются систематическое администрирование и регулярное удаление ненужных файлов данных и метаданных. Dremio как вычислительный механизм предоставляет пару интуитивно понятных команд SQL для выполнения ряда операций по обслуживанию таблиц. Давайте рассмотрим некоторые из них.

Удаление снимков

Удаление ненужных снимков имеет решающее значение, потому что со временем они могут занимать все больше и больше места. Кроме того, накопление снимков увеличивает объем метаданных таблицы, что может отрицательно сказаться на производительности запросов. Для удаления снимков Dremio предоставляет метод `VACUUM`. Например:

```
VACUUM TABLE 'employee'
  EXPIRE SNAPSHOTS older_than TIMESTAMP '2023-07-10 00:00:00.000' retain_last
  30;
```

Эта инструкция удалит все снимки таблицы `employee_data`, созданные до 2023-07-10 00:00:00.000, оставив нетронутыми не менее 30 самых последних снимков.

Перезапись файлов данных

Dremio позволяет оптимизировать производительность запросов путем перезаписи файлов данных. Этот процесс уплотнения объединяет маленькие файлы в файлы оптимального размера или дробит большие файлы, чтобы уменьшить накладные расходы на обслуживание метаданных и на открытие файлов во время выполнения. По умолчанию Dremio использует стратегию уплотнения `binpack`. Допустим, вы решили перезаписать файлы данных таблицы `employee` и привести их к оптимальному размеру 128 Мбайт. Вот как будет выглядеть SQL-запрос, выполняющий эту операцию:

```
OPTIMIZE TABLE 'employee'
  REWRITE DATA (TARGET_FILE_SIZE_MB=128);
```

Dremio также позволяет перезаписывать файлы данных для конкретных разделов. Например, предположим, что таблица `employee` секционирована по годам. Тогда вы сможете перезаписать файлы данных только для раздела, соответствующего 2023 году, как показано ниже:

```
OPTIMIZE TABLE 'employee'
  FOR PARTITIONS year=2023;
```

Перезапись манифестов

Файлы манифестов используются в Iceberg для хранения информации о данных и играют роль списка файлов данных таблицы. Манифесты чрезвычайно полезны для эффективного планирования и обработки запросов, а также помогают удалять ненужные файлы данных. Однако, как и в случае с другими файлами метаданных, размер манифестов может расти, особенно при при-

еме потоковых данных или при выполнении частых операций DML. Чтобы смягчить эту проблему, Dremio позволяет перезаписывать файлы манифестов, размеры которых превышают установленный порог. Целевой порог по умолчанию для файлов манифестов составляет 8 Мбайт.

Вот пример инструкции, которая перезапишет файлы манифестов для таблицы employee:

```
OPTIMIZE TABLE 'employee'  
  REWRITE MANIFESTS;
```

Заклучение

В этой главе мы рассмотрели методы взаимодействия с таблицами Iceberg, имеющиеся в Dremio, и исследовали некоторые аспекты этих взаимодействий, такие как первоначальная настройка, создание и изменение структур с помощью DDL, выполнение анализа, запись данных и выполнение операций по обслуживанию таблиц.

В главе 8 продолжим говорить о независимой природе Apache Iceberg от конкретного механизма и покажем, как выполнять те же самые операции с помощью AWS Glue.

Глава 8

AWS Glue

AWS Glue – это управляемый сервис, предлагающий оптимизированный способ подготовки и интеграции данных для различных аналитических рабочих нагрузок, таких как бизнес-аналитика (Business Intelligence, BI) и машинное обучение (Machine Learning, ML). Он имеет удобный визуальный интерфейс, упрощающий процесс создания, запуска и управления заданиями. AWS Glue может использоваться как масштабируемый бессерверный каталог данных для управления рабочими процессами. AWS Glue 3.0 и более поздние версии поддерживают формат таблиц Apache Iceberg. Это означает, что Glue можно использовать в комбинации с Iceberg и с его помощью выполнять такие операции, как создание таблиц Iceberg в хранилищах объектов, например в Amazon Simple Storage Service (Amazon S3), чтение и запись данных в таблицы, и просто использовать каталог Glue для хранения таблиц Iceberg.

В этой главе мы расскажем, как настроить AWS Glue для работы с таблицами Apache Iceberg и выполнять различные операции, такие как CREATE, READ и INSERT.

На момент написания этой книги версия AWS Glue 4.0 поддерживала Iceberg v1.0.0, а AWS Glue 3.0 поддерживал Iceberg v0.13.1.

Настройка

Работа инструмента интеграции AWS Glue основана на «заданиях» – единицах работы по перемещению данных из источника (находящегося где угодно) в приемник (в нашем случае – в таблицы Apache Iceberg). Далее мы покажем, какие настройки необходимо выполнить при создании задания с использованием Apache Iceberg в качестве источника или приемника.

Создание базы данных Glue

Первым шагом нужно создать базу данных в каталоге AWS Glue Data, который действует как централизованное хранилище всех метаданных ваших таблиц.

База данных играет роль пространства имен, в котором можно сгруппировать таблицы, помещенные в каталог.

Чтобы создать базу данных, перейдите в консоль AWS Glue и в левой навигационной панели, в разделе **Data Catalog** (Каталог данных), выберите пункт **Databases** (Базы данных). Затем щелкните на ссылке **Add database** (Добавить базу данных) и введите уникальное имя для своей базы данных. В нашем демонстрационном примере мы будем использовать имя «test». Щелкните на **Create Database** (Создать базу данных), чтобы завершить создание.

Настройка ETL-задания в Glue

Для создания нового задания ETL мы используем интерфейс Glue Visual ETL. Перейдите в раздел **Visual ETL** (Визуальный редактор ETL) в панели навигации слева и выберите **Visual with a Blank Canvas** (Визуальный редактор с чистым холстом). Перед вами откроется визуальный редактор, в котором вы сможете приступить к созданию своего задания.

Настройка источника данных

Прежде чем приступить к настройке задания, важно отметить, что таблица Iceberg создается на основе существующего набора данных, который хранится в корзине S3. Для создания таблицы используется команда `CREATE TABLE... AS SELECT (CTAS)`. Итак, первым делом нужно добавить источник S3 на визуальный холст. Для этого щелкните на кнопке **Add Node** (Добавить узел) и из списка выберите **Amazon S3 (source)** (Amazon S3 (источник)).

Далее в разделе **Data Source Properties** (Свойства источника данных) добавьте местоположение корзины S3, где хранится набор данных, и выберите формат данных CSV. Изображения пользовательского интерфейса для справки можно найти в репозитории книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg>).

После добавления узла источника данных перейдите на вкладку **Job details** (Сведения о задании) и задайте настройки, необходимые заданию Glue для подключения к Apache Iceberg. Давайте рассмотрим эти настройки поближе.

Основные свойства

В разделе **Basic Properties** (Основные свойства) необходимо задать несколько ключевых параметров для интеграции задания Glue с Apache Iceberg. Сначала укажите имя задания, например «Iceberg_ETL». Затем выберите соответствующую роль IAM, имеющую необходимые разрешения для операций Glue. Выберите «Glue 4.0» в качестве версии Glue, поскольку она поддерживает Apache Iceberg 1.0.0. Выберите язык сценария «Python 3», на котором будет написан сценарий для задания ETL. Наконец, укажите количество рабочих потоков как «2», чтобы использовать стандартные рабочие потоки для выполнения задания.

Расширенные свойства

В разделе **Advanced Properties** (Расширенные свойства) имеется раздел **Job Parameters** (Параметры задания), где можно указать дополнительные параметры задания в форме пар ключ–значение. Двумя ключевыми параметрами при использовании Apache Iceberg являются `--datalake-formats` и `--conf`.

Присвойте параметру `--datalake-formats` значение `iceberg`, чтобы включить поддержку формата таблиц Iceberg. В параметре `--conf` можно указать несколько пар ключ–значение для настройки каталога Iceberg в сеансе Apache Spark (они покажутся вам знакомыми, потому что мы использовали их в главе 5), в том числе поддержку расширений для Iceberg, реализацию каталога и другие детали.

Вот как будет выглядеть значение параметра `--conf` (обратите внимание, что начальный ключ `--conf` не нужен, поскольку его значение передается в виде параметра; также помните о невидимых разрывах строк, которые могут привести к пропуску конфигураций):

```
--conf spark.sql.extensions=org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions
--conf spark.sql.catalog.glue_catalog=org.apache.iceberg.spark.SparkCatalog
--conf spark.sql.catalog.glue_catalog.warehouse=s3://<каталог-вашего-хранилища>/
--conf spark.sql.catalog.glue_catalog.catalog-impl=org.apache.iceberg.aws.glue.GlueCatalog
--conf spark.sql.catalog.glue_catalog.io-impl=org.apache.iceberg.aws.s3.S3FileIO
```

Наконец, не забудьте вместо `<каталог-вашего-хранилища>` указать путь в S3, куда должны быть записаны ваши таблицы.

После определения настроек можно сохранить задание и перейти на вкладку **Script** (Сценарий). Здесь вы должны увидеть заготовку сценария PySpark для задания:

```
import sys
from awsglue.transforms import *
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.job import Job

args = getResolvedOptions(sys.argv, ["JOB_NAME"])
sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session
job = Job(glueContext)
job.init(args["JOB_NAME"], args)

# Сценарий сгенерирован для узла Amazon S3
AmazonS3_node1689692483975 = glueContext.create_dynamic_frame.from_options(
    format_options={"quoteChar": "'", "withHeader": True, "separator": ","},
```

```

        connection_type="s3",
        format="csv",
        connection_options={
            "paths": ["s3://dm-iceberg/datasets/salesdata.csv"],
            "recurse": True,
        },
        transformation_ctx="AmazonS3_node1689692483975",
    )

job.commit()

```

Обратите внимание, что импортированные данные находятся в `DynamicFrame` – специальной версии `DataFrame` для AWS Glue. В качестве дополнительного шага содержимое этого `DynamicFrame` преобразуется в `Spark DataFrame` вызовом метода `toDF()`. Это совершенно необходимо, потому что объекты `DataFrame` имеют методы, которые понадобятся для преобразования их в представление SQL. Наконец, `DataFrame` превращается во временное представление с помощью метода `createOrReplaceTempView()`:

```

import sys
from awsglue.transforms import *
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.job import Job

args = getResolvedOptions(sys.argv, ["JOB_NAME"])
sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session
job = Job(glueContext)
job.init(args["JOB_NAME"], args)

# Сценарий сгенерирован для узла Amazon S3, добавлен вызов toDF()
# для преобразования DynamicFrame в DataFrame
df = glueContext.create_dynamic_frame.from_options(
    format_options={"quoteChar": "'", "withHeader": True, "separator": ","},
    connection_type="s3",
    format="csv",
    connection_options={
        "paths": ["s3://dm-iceberg/datasets/salesdata.csv"]
    },
    transformation_ctx="AmazonS3_node1689692483975",
).toDF()

# Превратить DataFrame во временное представление SQL
df.createOrReplaceTempView("temp_df")

```

Теперь, определив все необходимые настройки, можно приступить к взаимодействию с Iceberg через Glue. Давайте рассмотрим некоторые из основных операций, чтобы вы могли получить практическое представление о них.

Создание таблицы с помощью Glue Data Catalog

Для создания таблицы Iceberg с помощью каталога Glue можно использовать оператор CTAS. Обратите внимание, что на предыдущем шаге мы уже создали временное представление `temp_df` на основе имеющегося набора данных. Поэтому нам осталось лишь использовать это представление для создания таблицы Iceberg. Перейдите на вкладку **Script** (Сценарий) в задании Glue и добавьте следующий код:

```
df.createOrReplaceTempView("temp_df")
query = f"""
CREATE TABLE glue_catalog.test.employee
USING iceberg
AS SELECT * FROM temp_df
"""
spark.sql(query)
```

Этот код создаст таблицу Iceberg с именем `employee` и сохранит ее в файле `glue_catalog` в базе данных `test`.

Чтение из таблицы

Объект `GlueContext`, позволяющий взаимодействовать с каталогом Glue, имеет метод `create_data_frame.from_catalog()`, который возвращает таблицу из каталога Glue в виде `DataFrame`. Запустите следующий код на вкладке **Script** (Сценарий) задания Glue:

```
from awsglue.context import GlueContext
from pyspark.context import SparkContext

sc = SparkContext()
glueContext = GlueContext(sc)

additional_options = {}

# Использовать метод create_data_frame.from_catalog()
# для чтения таблицы Iceberg
df = glueContext.create_data_frame.from_catalog(
    database="test",
    table_name="employee",
    additional_options=additional_options
)
```

Вставка данных

Для выполнения любых операций с таблицами Apache Iceberg в каталоге Glue можно использовать Spark SQL, но если вам больше нравится Spark DataFrame API, то для записи данных в таблицу используйте метод `GlueContext.write_data_frame.from_catalog()`. Он записывает содержимое DataFrame в целевую таблицу целевой базы данных в каталоге Glue. Но имейте в виду: чтобы воспользоваться этим способом, нужно установить параметр задания `--enable-glue-datacatalog`, разрешающий AWS Glue использовать каталог Glue Data в качестве метастранилища Apache Spark Hive.

Следующий код вставляет данные из `df` типа DataFrame в таблицу `employee` с помощью каталога данных Glue:

```
from awsglue.context import GlueContext
from pyspark.context import SparkContext

sc = SparkContext()
glueContext = GlueContext(sc)

additional_options = {}

# Использовать метод write_data_frame.from_catalog() объекта GlueContext
# для вставки данных в таблицу Iceberg
glueContext.write_data_frame.from_catalog(
    frame=df,
    database="test",
    table_name="employee",
    additional_options=additional_options
)
```

Заклучение

В этой главе мы рассказали, как использовать AWS Glue, чтобы избавиться от большей части сложностей, возникающих при применении Spark для выполнения заданий ETL (развертывание/настройка кластеров, планирование заданий, отправка заданий и т. д.). Благодаря визуальному редактору и интеграции с такими форматами таблиц, как Apache Iceberg, AWS Glue зарекомендовал себя как эффективный инструмент интеграции данных с Apache Iceberg.

В главе 9 мы рассмотрим, как использовать Apache Iceberg с Apache Flink.

Глава 9

Apache Flink

Apache Flink – эффективный фреймворк потоковой обработки данных в пакетном режиме и в режиме реального времени с низкой задержкой. Он поддерживает обработку событий по времени, семантику «точно один раз» и разнообразные механизмы управления окнами. Комбинация Apache Flink и Apache Iceberg имеет целый ряд преимуществ. Возможности Iceberg, такие как изоляция моментальных снимков для чтения и записи, возможность одновременной обработки нескольких операций, ACID-совместимые запросы и инкрементное чтение, позволяют Flink выполнять операции, которые сложно организовать с использованием старых форматов таблиц. Вместе они образуют эффективную и масштабируемую платформу для крупномасштабной обработки данных, особенно в сценариях потоковой передачи.

В этой главе мы углубимся в практическое использование Apache Flink в комбинации с Apache Iceberg. Сначала посмотрим, как настроить Flink SQL Client с каталогом Iceberg для использования в примерах в этой главе, демонстрирующих применение команд DDL, выполнение запросов на чтение и запись. Затем покажем, как выполнять некоторые из этих операций с помощью Flink DataStream и Table API на Java. Все эти примеры вы сможете запустить на своем локальном компьютере, выполнив указанные шаги.

Настройка

Для начала познакомимся с базовыми настройками кластера Flink, которые применяются независимо от того, используется ли стандартный Flink с заданиями, написанными на Java, или PyFlink, который компилирует задания на Python в Java.

Предварительные условия

Вы можете загрузить и распаковать последнюю версию двоичного файла с официального веб-сайта Apache Flink (<https://flink.apache.org/downloads>) или

использовать определенную версию Flink с поддержкой Iceberg. Работая над этой главой, мы использовали Flink 1.16.1. Вот команды, которые установят соответствующие переменные окружения, загрузят двоичный файл и распакут его (предполагается, что Java уже установлена):

```
FLINK_VERSION=1.16.1

SCALA_VERSION=2.12

APACHE_FLINK_URL=https://archive.apache.org/dist/flink/

wget ${APACHE_FLINK_URL}/flink-${FLINK_VERSION}/flink-${FLINK_VERSION}-bin-scala_${SCALA_VERSION}.tgz

tar xzvf flink-${FLINK_VERSION}-bin-scala_${SCALA_VERSION}.tgz
```

Затем загрузите JAR-файл со средой выполнения Iceberg и поместите его в каталог *FLINK_HOME/lib*. Эта библиотека времени выполнения обеспечит интеграцию Iceberg и Flink. Загрузить последнюю версию JAR можно со страницы JAR iceberg-flink-runtime на веб-сайте репозитория Maven (https://oreil.ly/XOW_B). Здесь мы будем использовать iceberg-flink-runtime-1.16. В табл. 9.1 перечислены версии Flink с Iceberg и соответствующие им JAR-файлы среды выполнения, поддерживаемые на момент написания книги.

Таблица 9.1. Версии Flink и поддержка Iceberg

Версия Flink	Поддерживаемые версии Iceberg		Последняя версия JAR-файла среды
	От	До	
1.16	1.1.0	1.3.0	iceberg-flink-runtime-1.16
1.17	1.3.0	1.3.0	iceberg-flink-runtime-1.17

Также понадобятся библиотеки Hadoop Common для организации взаимодействий Flink с файловыми системами, такими как Amazon Simple Storage Service (Amazon S3) и Hadoop Distributed File System (HDFS). В этой главе мы запустим кластер Flink в среде Hadoop и для хранения таблиц Iceberg используем каталог Hadoop. Поэтому вам следует загрузить совместимую версию Hadoop, содержащую эти библиотеки. Однако если вы используете Flink вне Hadoop, то можете загрузить JAR-файл (стабильную версию v2.8.3 (<https://oreil.ly/hEabL>)):

```
APACHE_HADOOP_URL=https://archive.apache.org/dist/hadoop/

HADOOP_VERSION=2.8.5

wget ${APACHE_HADOOP_URL}/common/hadoop-${HADOOP_VERSION}/hadoop-${HADOOP_VERSION}.tar.gz

tar xzvf hadoop-${HADOOP_VERSION}.tar.gz
```

Затем настройте переменную окружения *HADOOP_HOME*, чтобы она ссылалась на загруженную версию Hadoop, и добавьте путь к классам Hadoop в пере-

менные окружения (он будет использоваться кластером Flink при запуске), например так:

```
HADOOP_HOME=`pwd`/hadoop-${HADOOP_VERSION}
export HADOOP_CLASSPATH=`$HADOOP_HOME/bin/hadoop classpath`
```

Вам также могут понадобиться следующие классы:

Классы Hadoop AWS

Они потребуются, только если вы планируете читать или записывать данные именно в Amazon S3. Загрузить их можно из репозитория Maven (<https://oreil.ly/BKwiq>).

Классы для доступа к AWS

Эти классы облегчают взаимодействие с многочисленными сервисами AWS, включая S3, IAM, AWS Lambda, и др. Библиотека Hadoop AWS поддерживает базовые операции с S3, но пакет классов для доступа к AWS позволит решать более сложные задачи, такие как загрузка и выгрузка элементов. Стабильную версию можно найти в репозитории Maven (<https://oreil.ly/xkVum>).

Прежде чем начать создавать кластер Flink, необходимо понять две важные концепции, касающиеся архитектуры Flink:

JobManager

Это оркестратор. Он отвечает за координацию и управление различными действиями в приложении Flink, такими как планирование задач, координация контрольных точек и обработка выполнения кода (ориентированные графы, называемые *JobGraph*).

TaskManager

Отвечает за выполнение задач, назначенных диспетчером заданий JobManager. Имеет набор слотов (наименьшая единица планирования ресурсов), который позволяет запускать несколько заданий в отдельных потоках.

Настройки этих двух компонентов можно определить в соответствии с требованиями в конфигурационном файле *flink-conf.yaml*, находящемся в каталоге *FLINK_HOME/lib*.

Запуск кластера Flink и клиента Flink SQL

Теперь запустим кластер Flink с помощью команды

```
./bin/start-cluster.sh
```

Она запустит кластер Flink на локальном компьютере и выведет примерно следующее:

```
Starting cluster.
Starting standalone session daemon on host Dipankars-MBP.
Starting taskexecutor daemon on host Dipankars-MBP.
```

После запуска кластера Flink можно запустить клиента Flink SQL командой

```
./bin/sql-client.sh embedded
```

Сначала вы увидите текстовую версию логотипа Flink, а затем появится приглашение к вводу Flink SQL>, сообщающее, что вы теперь находитесь в Flink SQL.

Операции на языке определения данных

Flink SQL поддерживает ряд операций DDL для работы с таблицами Apache Iceberg. В этом разделе мы рассмотрим наиболее распространенные из них.

CREATE CATALOG

Начиная работать с таблицами Apache Iceberg, первым делом нужно настроить каталог. Iceberg по умолчанию поставляется с JAR-файлами, содержащими поддержку каталога Hadoop, но в Apache Flink можно также использовать другие каталоги, такие как Hive, REST, AWS Glue и Project Nessie.

Вот как можно создать каталог Iceberg с помощью Flink SQL:

```
CREATE CATALOG <catalog_name> WITH (
  'type'='iceberg',
  'catalog-type'=<значения>
  <ключ>=<значение>
);
```

Существует несколько важных свойств, о которых следует помнить, настраивая каталоги для таблиц Apache Iceberg в Flink SQL. Свойство `type` всегда должно иметь значение `iceberg`. При работе со встроенными каталогами, такими как Hive, Hadoop и REST, следует указать соответствующий тип каталога в свойстве `catalog-type`. Однако если вы используете нестандартный каталог, такой как AWS Glue или Project Nessie, то свойство `catalog-type` следует оставить пустым. Кроме того, если вы используете нестандартный каталог и оставили свойство `catalog-type` пустым, то тогда вы должны определить свойство `catalog-impl`, указав в нем полное имя класса реализации каталога. Эти свойства играют важную роль в настройке и определении поведения вашего каталога Iceberg в среде Flink SQL.

Каталог Hadoop

Iceberg поддерживает каталоги на основе файловых систем, такие как каталог Hadoop в HDFS. Вот пример настройки каталога Hadoop с помощью Flink SQL:

```
CREATE CATALOG local_catalog WITH (
  'type'='iceberg',
  'catalog-type'='hadoop',
  'warehouse'='hdfs://nn:8020/путь/к/хранилищу'
);
```

Эта инструкция создаст каталог Iceberg с именем `local_catalog`. Обязательный параметр `'type' = 'iceberg'` позволяет Flink создать каталог Iceberg. Параметр `'catalog-type'='hadoop'` сообщает Flink, что этот каталог Iceberg является каталогом Hadoop, а это означает, что он будет использовать любой каталог на основе файловой системы для хранения файлов метаданных и данных. Параметр `'warehouse'='hdfs://nn:8020/путь/к/хранилищу'` задает путь в файловой системе HDFS, который Hadoop будет использовать для хранения файлов метаданных и данных. Все вновь создаваемые таблицы будут сохраняться в этом каталоге в HDFS. Если вы решите использовать локальную папку вместо каталога HDFS, то вам следует настроить параметр `'warehouse'='file:///абсолютный/путь/к/хранилищу'`. Другой распространенный вариант – использовать облачное хранилище объектов, настроить параметр `'warehouse'='s3://my-bucket/hadoopcatalog/'`. Обязательно определите свойство `'io-impl' = 'org.apache.iceberg.aws.s3.S3FileIO'` и загрузите необходимые зависимости AWS для взаимодействия с AWS S3 (как описано в главе 5).

Проверить успешность создания каталога можно командой

```
show catalogs;
```

Она должна вывести примерно следующее:

```
[Flink SQL> show catalogs;
catalog name
default_catalog
local_catalog
2 rows in set
```

После создания каталога его можно сделать текущим, выполнив следующую команду:

```
USE CATALOG local_catalog;
```

Каталог Hive

Поскольку по умолчанию Iceberg не включает JAR-файлы с поддержкой Hive, вы должны убедиться в наличии всех необходимых зависимостей в вашей среде Flink и загружать их при запуске SQL-клиента Flink. Скачать последнюю версию JAR можно из репозитория Maven (https://oreil.ly/fR1k_). Скачав файл, его необходимо сделать доступным для клиента Flink SQL. С этой целью можно поместить файлы JAR в каталог `/lib` в папке Flink. После этого можно запустить клиента Flink SQL.

После установки зависимостей и запуска клиента Flink SQL можно создать каталог Iceberg Hive, выполнив следующий запрос:

```
CREATE CATALOG hive_catalog WITH (
  'type'='iceberg',
  'catalog-type'='hive',
  'uri'='thrift://localhost:9083',
  'clients'='5',
  'warehouse'='hdfs://nn:8020/путь/к/хранилищу'
);
```

Важно отметить некоторые ключевые свойства, важные для настройки Hive как каталога таблиц Apache Iceberg. Свойство `'uri'` определяет URI хранилища Hive Metastore, необходимое для подключения к Hive Metastore. Свойство `'clients'`, хотя и необязательное, позволяет указать размер пула Hive Metastore для клиентов (по умолчанию оно получает значение 2). Наконец, свойство `'warehouse'` определяет путь к хранилищу, где будут сохраняться файлы метаданных и данных Iceberg. Все перечисленные параметры играют важную роль в настройке поведения каталога Iceberg при интеграции с Hive в среде Apache Flink SQL.

После создания каталога его можно сделать текущим, выполнив команду

```
USE CATALOG hive_catalog;
```

Нестандартные каталоги

Flink также поддерживает возможность использования нестандартных реализаций каталога Iceberg, предлагая свойство `catalog-impl`. Например:

```
CREATE CATALOG custom_catalog WITH (
  'type'='iceberg',
  'catalog-impl'='com.my.custom.CatalogImpl',
  'my-additional-catalog-config'='my-value'
);
```

Свойство `catalog-impl` определяет имя класса реализации пользовательского каталога. Например, если в качестве каталога используется Nessie, то класс его реализации будет иметь имя `org.apache.iceberg.nessie.NessieCatalog`.

CREATE DATABASE

Flink SQL поставляется с базой данных по умолчанию. Если вы решите создать новую базу данных, то это можно сделать так:

```
CREATE DATABASE iceberg_db;
```

Приведенный выше код создаст новую базу данных с именем `iceberg_db` в текущем активном каталоге. Переключиться на эту новую базу данных можно с помощью инструкции `USE`:

```
USE iceberg_db;
```

CREATE TABLE

Следующий шаг – создание таблицы Iceberg. Вот пример создания таблицы:

```
CREATE TABLE employee (  
    id BIGINT,  
    role STRING,  
    department STRING,  
    salary FLOAT,  
    region STRING  
) WITH (  
    'connector'='iceberg'  
);
```

Этот запрос создаст таблицу `emp` с пятью столбцами. Свойство `'connector' = 'iceberg'` сообщает Flink, что при создании таблицы должен использоваться коннектор Iceberg.

CREATE TABLE...PARTITIONED BY

Чтобы создать секционированную таблицу Iceberg, можно использовать такой запрос:

```
CREATE TABLE emp_partitioned_table (  
    id BIGINT,  
    role STRING,  
) PARTITIONED BY (role) WITH (  
    'connector'='iceberg'  
);
```

Он создаст таблицу `emp_partitioned_table`, секционированную по столбцу `role`.

Важно отметить, что, несмотря на поддержку скрытого секционирования в Iceberg, Flink не поддерживает секционирование с применением функций. Поэтому в Flink DDL отсутствует поддержка скрытых разделов.

CREATE TABLE...LIKE

Иногда требуется создать новую таблицу, отражающую структуру и свойства существующей. Для этой цели в Flink SQL имеется очень удобная команда `CREATE TABLE...LIKE`. Она создает новую таблицу с той же схемой, секционированием и свойствами, что и существующая.

Предположим, у нас есть таблица `emp` с двумя столбцами: `id` и `role`, и нужно создать новую таблицу с той же схемой и свойствами. Для этого можно выполнить такой запрос:

```
CREATE TABLE emp_like LIKE employee;
```

Он создаст новую таблицу `emp_like`, которая будет иметь ту же схему, стратегию секционирования и свойства, что и таблица `emp`. Это быстрый и простой способ дублировать структуру таблицы.

ALTER TABLE

Flink SQL предлагает инструкции для изменения структуры существующей таблицы Iceberg с помощью ALTER TABLE. Рассмотрим пару примеров.

Переименовать таблицу Iceberg можно с помощью следующего запроса:

```
ALTER TABLE employee RENAME TO emp_new;
```

Изменить формат записи по умолчанию для таблицы с именем emp на avro можно так:

```
ALTER TABLE employee SET ('write.format.default'='avro');
```

DROP TABLE

Удалить таблицу из каталога можете с помощью инструкции DROP TABLE.

Чтение данных

Flink SQL поддерживает возможности пакетного и потокового чтения данных из таблиц Iceberg, гибкие режимы выполнения, настраиваемые параметры запросов и проверку различных свойств метаданных, связанных с таблицами. В этом разделе мы рассмотрим ряд операций чтения таблиц Iceberg с использованием Flink SQL.

Пакетное чтение

Чтобы выполнить пакетное чтение с помощью Flink SQL, сначала необходимо установить пакетный режим. Вот как вы можете прочитать все данные из таблицы employee в пакетном режиме:

```
SET execution.runtime-mode = batch;  
SELECT * FROM employee;
```

Этот запрос выполнится как пакетное задание и извлечет сразу весь набор данных.

Потоковое чтение

Для обработки новых данных по мере их появления можно использовать потоковый режим Flink SQL. Например:

```
SET execution.runtime-mode = streaming;
```

```
SELECT * FROM employee  
/*+ OPTIONS('streaming'='true', 'monitorinterval'='1s')*/ ;
```

Здесь оператор `SELECT` начинает чтение всех записей из текущего снимка таблицы `employee`, но не останавливается на этом, а продолжает возвращать вновь прибывающие данные. `/*+ OPTIONS('streaming'='true', 'monitorinterval'='1s')*/` – это подсказка SQL. В Flink подсказки SQL – это дополнительные инструкции внутри оператора SQL, которые предоставляют планировщику SQL дополнительную информацию, помогающую ему изменить план выполнения. В этом случае мы подсказываем, что запрос должен выполняться в потоковом режиме и интервал проверки поступления новых данных должен составлять 1 с, т. е. каждую секунду Flink будет проверять появление изменений или новых данных в таблице Iceberg.

Таблицы с метаданными

Чтобы получить информацию об исторических данных таблицы, деталях моментального снимка и общем состоянии, например узнать количество небольших или осиротевших файлов, можно воспользоваться *таблицами метаданных*, предоставляемыми Iceberg. В Flink SQL эти таблицы доступны под именами, начинающимися с имени основной таблицы, за которым следует символ «\$» и далее имя таблицы метаданных. Давайте взглянем на некоторые из этих таблиц метаданных.

История

Таблица метаданных с историей позволяет увидеть, как эволюционировала ваша таблица с течением времени. Вот как можно получить доступ к этой таблице:

```
SELECT * FROM `catalog`.`database`.`table`$history;
```

В результате вы получите историю таблицы `table` и сможете увидеть, например, были ли отменены какие-либо транзакции.

Журналы метаданных

Журналы метаданных отслеживают все файлы метаданных, включая такую информацию, как `latest_snapshot_id` и `latest_schema_id`, а также `timestamp`. Вот как запросить таблицу журнала метаданных:

```
SELECT * FROM `catalog`.`database`.`table`$metadata_log_entries;
```


Снимки

За информацией о снимках таблицы Iceberg можно обратиться к таблице метаданных снимков. Например:

```
SELECT * FROM `catalog`.`database`.`table`$snapshots;
```

В этой таблице содержится такая информация, как количество добавленных или удаленных записей после каждой операции записи и идентификатор задания Flink.

Существуют также другие доступные таблицы метаданных с информацией о манифестах и разделах. Но мы отложим их обсуждение до главы 10.

Запись данных

Flink SQL предоставляет различные операции записи, такие как `INSERT INTO`, `INSERT OVERWRITE` и `UPSERT`, которые можно использовать как в пакетном, так и в потоковом режиме, хотя и с некоторыми ограничениями. Давайте рассмотрим эти операции.

INSERT INTO

Инструкция `INSERT INTO` добавляет новые данные в таблицу Iceberg. Вот пара примеров:

```
INSERT INTO employee VALUES (1, 'Software Engineer', 'Engineering', 25000, 'NA');
```

```
INSERT INTO employee SELECT id, role from emp_new;
```

Первый запрос вставит одну запись с указанными значениями в таблицу `employee` в выбранном каталоге Iceberg. Второй вставит значения полей `id` и `role`, выбирая их из другой таблицы `emp_new`.

INSERT OVERWRITE

Чтобы заменить данные в таблице результатом запроса, в Flink SQL используется инструкция `INSERT OVERWRITE`. Перезапись в Apache Iceberg является атомарной операцией, поэтому данная инструкция гарантирует сохранение данных в согласованном состоянии.

Вот пример использования `INSERT OVERWRITE` в пакетном задании:

```
INSERT OVERWRITE employee  
VALUES (1, 'Software Tester', 'Engineering', 23000, 'NA');
```

Этот запрос заменит все существующие данные в таблице `emp` указанными значениями.

Iceberg также позволяет перезаписывать определенные разделы выбранными значениями. Представьте, что ваша таблица `employee` секционирована по полю `department` и вам нужно внести некоторые изменения в записи о сотрудниках отдела `Engineering`. В Flink SQL это можно сделать так:

```
INSERT OVERWRITE employee PARTITION(department='Engineering')
SELECT * FROM updated_emp_data WHERE department='Engineering';
```

Этот запрос выберет из таблицы `updated_emp_data` записи о сотрудниках из отдела `Engineering` и перезапишет соответствующий раздел в таблице `employee`.

Обратите внимание, что `INSERT OVERWRITE` поддерживается только в пакетном режиме и не поддерживается в потоковом.

UPSERT

Apache Iceberg поддерживает операцию `UPSERT` – комбинацию `INSERT` и `UPDATE`. Если запись существует, она будет обновлена, а если нет, то просто будет вставлена новая запись. Этим она напоминает операцию `MERGE INTO`, поддерживаемую в Spark и Dremio. Обратите внимание, что для выполнения этой операции необходимо указать первичный ключ в таблице. Есть два способа выполнить `UPSERT`.

Первый: включить режим `UPSERT` в свойствах уровня таблицы, как показано в следующем примере:

```
CREATE TABLE employee (
  `id` INT UNIQUE,
  `role` STRING NOT NULL,
  `department` STRING NOT NULL,
  `salary` FLOAT,
  `region` STRING NOT NULL,
  PRIMARY KEY(`id`) NOT ENFORCED
) WITH ('format-version'='2', 'write.upsert.enabled'='true');
```

Здесь режим `UPSERT` устанавливается при создании таблицы и будет применяться как в пакетном, так и в потоковом режиме, если только свойство не будет переопределено, как показано во втором подходе.

После этого можно выполнить `INSERT INTO`, и, опираясь на значение первичного ключа, таблица Iceberg решит, какую операцию выполнить – обновление или вставку. Например:

```
INSERT INTO employee VALUES (1, 'Director', 'Product', 33000, 'APAC');
```

Второй способ: включить режим `UPSERT`, добавив `upsert-enabled` в параметры операции записи.

При таком подходе режим UPSERT включается для конкретных операций INSERT, что дает бóльшую гибкость. Например:

```
INSERT INTO employee /*+ OPTIONS('upsert-enabled'='true') */
VALUES (3, 'Manager', 'Engineering', 26000, 'NA');
```

Flink DataStream и Table API

В предыдущем разделе мы выполнили пару упражнений, изучая операторы DDL, а также операции чтения и записи данных. В этом разделе мы рассмотрим работу с API-интерфейсами DataStream и Table и программного кода на Java, а также выполним некоторые базовые операции с таблицами Apache Iceberg.

Предварительные условия

Как и прежде, сначала нужно загрузить Apache Flink и все необходимые файлы JAR. Вот список файлов JAR, которые мы будем использовать, со ссылкой для загрузки из репозитория Maven (обратите внимание, что эти версии были стабильными на момент написания главы). Если вы используете Docker-образ *alexmerced/flink-iceberg* для этого упражнения, то все необходимые файлы JAR уже находятся внутри образа:

- *iceberg-flink-runtime-1.16-1.3.0.jar* (<https://oreil.ly/HKUhy>) (среда выполнения Iceberg-Flink);
- *hadoop-common-2.8.3.jar* (<https://oreil.ly/MJyfc>) (классы Hadoop);
- *flink-shaded-hadoop-2-uber-2.8.3-10.0.jar* (<https://oreil.ly/s5TnM>) (классы Hadoop AWS);
- *bundle-2.20.18.jar* (<https://oreil.ly/KerlT>) (классы для доступа к AWS).

У вас также должны быть установлены Java 8+ и Maven. Если вы решите создать локальную среду Flink для следующего упражнения, то обратитесь к репозиторию книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg-ch9>).

Настройка задания Flink

Первым делом нужно создать пустой проект Maven. Для этого выполните следующую команду:

```
mvn archetype:generate
```

В процессе вам будет задано несколько вопросов, большинство из которых можно игнорировать, нажимая **Enter**. Введите только **flink_job** в `artifactID` и **com.my_flink_job** в `groupId`. `artifactID` будет служить именем вашего проекта.

После успешной сборки каталог проекта будет выглядеть примерно так, как показано на рис. 9.1.

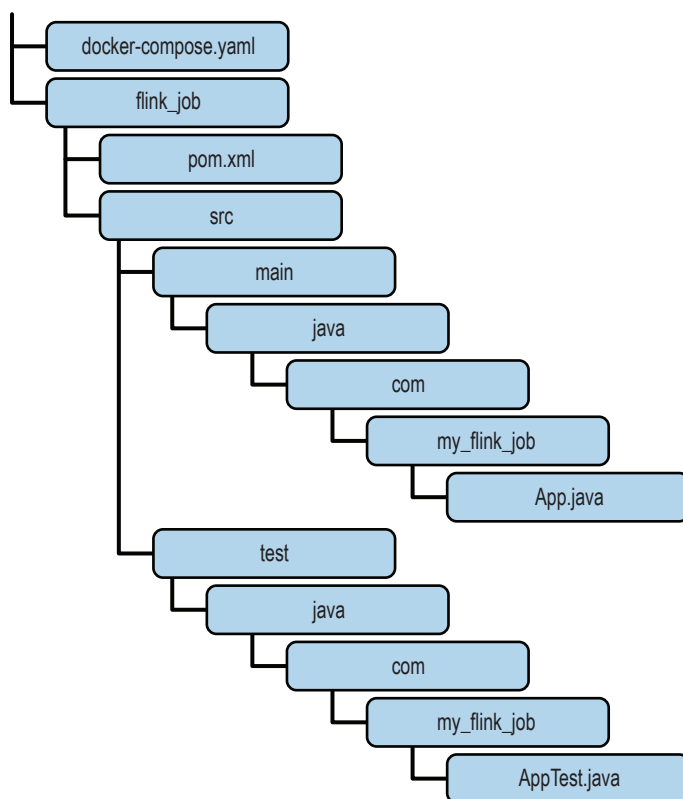


Рис. 9.1 ❖ Содержимое каталога пустого проекта Maven

Обратите внимание на следующие файлы, перечисленные на рис. 9.1:

- в файле *pom.xml* определяются зависимости и плагины проекта;
- файл *App.java* содержит логику приложения;
- файл *AppTest.java* содержит модульные тесты.

Поскольку нашему приложению понадобятся некоторые библиотеки, их нужно импортировать, перечислив в файле *pom.xml*, чтобы фреймворк Maven мог автоматически загрузить и включить их в процесс сборки проекта:

```

<dependencies>
  <dependency>
    <groupId>org.apache.flink</groupId>
    <artifactId>flink-java</artifactId>
    <version>1.16.1</version>
  </dependency>
  <dependency>
    <groupId>org.apache.iceberg</groupId>

```

```

        <artifactId>iceberg-flink-runtime-1.16</artifactId>
        <version>1.3.0</version>
    </dependency>
</dependency>
    <groupId>org.apache.iceberg</groupId>
    <artifactId>iceberg-core</artifactId>
    <version>1.3.0</version>
</dependency>

```

Также необходимо убедиться, что для свойств компилятора Maven установлено значение 1.8 для Java 8 или другое правильное значение, если вы используете другую версию Java:

```

<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <maven.compiler.source>1.8</maven.compiler.source>
  <maven.compiler.target>1.8</maven.compiler.target>
</properties>

```

Теперь создадим класс `EmployeeData` в папке с файлом *App.java*, который будет служить схемой наших данных. Этот класс должен включать несколько методов чтения/записи, как показано ниже:

```

package com.my_flink_job;

public class EmployeeData {
    private Long id;
    private String department;
    private Long salary;

    public EmployeeData() {
    }

    public EmployeeData(Long id, String department, Long salary) {
        this.id = id;
        this.department = department;
        this.salary = salary;
    }

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getDepartment() {
        return department;
    }

    public void setDepartment(String department) {
        this.department = department;
    }
}

```

```
public Long getSalary() {
    return salary;
}

public void setSalary(Long salary) {
    this.salary = salary;
}
}
```

Наконец, напишем логику в *App.java*, которая будет использовать API-интерфейсы Flink Data-Stream и Table для создания каталога Iceberg и таблицы, а также вставки нескольких записей. Вот сокращенная версия кода (полный код вы найдете в репозитории книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg>)):

```
public class App
{
    public static void main(String[] args) throws Exception {

        // настройка окружения
        // ...

        // создать каталог Nessie
        tableEnv.executeSql(
            "CREATE CATALOG iceberg WITH ("
                + "'type'='iceberg',"
                + "'catalog-impl'='org.apache.iceberg.nessie.NessieCatalog',"
                + "'io-impl'='org.apache.iceberg.aws.s3.S3FileIO',"
                + "'uri'='http://catalog:19120/api/v1',"
                + "'authentication.type'='none',"
                + "'ref'='main',"
                + "'client.assume-role.region'='us-east-1',"
                + "'warehouse' = 's3://warehouse',"
                + "'s3.endpoint'='http://{id-address}:9000'"
                + ")");

        // Сделать новый каталог текущим
        tableEnv.useCatalog("iceberg");

        // Создать базу данных в текущем каталоге
        tableEnv.executeSql("CREATE DATABASE IF NOT EXISTS db");

        // создать таблицу
        tableEnv.executeSql(
            "CREATE TABLE IF NOT EXISTS db.employees ("
                + "id BIGINT COMMENT 'unique id',"
                + "department STRING,"
                + "salary BIGINT"
                + ")");

        // Подготовить примеры данных
        // ...
    }
}
```

```
// записать DataStream в таблицу
tableEnv.executeSql(
    "INSERT INTO db.employees SELECT * FROM my_datastream");
}
}
```

Код в примере выше создает окружение `env` типа `StreamExecutionEnvironment` и настраивает задание Flink и среду вычислений. Затем создается экземпляр `StreamTableEnvironment tableEnv`, который позволяет работать с API-интерфейсами `DataStream` и `Table`, а также выполнять преобразования между ними (как показано в коде). Здесь `tableEnv` используется для выполнения операторов SQL, таких как создание каталога, базы данных и таблицы Iceberg.

Последнее, что нужно сделать перед сборкой пакета и запуском задания, – указать IP-адрес в разделе `CREATE CATALOG`. Чтобы получить IP-адрес, запустите кластер. Это необходимо, если вы используете `docker-compose`, как указано в начале упражнения. Если у вас есть сервер Nessie со статическим IP-адресом или доменным именем, то можете просто использовать его в качестве URI. В данном случае мы определяем IP-адрес контейнера Nessie в сети Docker.

Запуск кластера и сборка пакета

Чтобы развернуть среду, просто откройте терминал и запустите команду `docker-compose up`.

Если все настройки выполнены правильно, то вы сможете подключиться к командной оболочке контейнера хранилища командой `docker exec -it storage /bin/bash`, а затем использовать `ifconfig`, чтобы узнать IP-адрес.

Из вывода команды `ifconfig` скопируйте значение `inet` из раздела `eth0`. Это значение вы должны использовать в конфигурации каталога. Вот как будет выглядеть обновленный код:

```
"'s3.endpoint'='http://172.27.0.4:9000'"
```

Наконец, соберите пакет Maven, перейдя в каталог с файлом `pom.xml` и выполнив команду `mvn package`. В результате в каталоге `/target` должен появиться файл `flink_job-1.0-SNAPSHOT.jar`. Этот файл вы будете использовать для отправки задания Flink для выполнения любых операций Iceberg, определенных в вашем коде.

Выполнение задания

Apache Flink предоставляет веб-интерфейс, который можно открыть в браузере по адресу `localhost:8081`. Этот интерфейс позволяет управлять всем, что связано с заданиями Flink, включая отправку заданий, проверку состояния

задания и анализ журналов. Чтобы отправить и запустить задание из пользовательского интерфейса, щелкните на ссылке **Submit New Job** (Отправить новое задание), а затем на **Add New** (Добавить новое) для загрузки файла JAR. В разделе **Entry Class** (Класс с точкой входа) введите `com.my_flink_job.App` (имя вашего пакета) и щелкните на кнопке **Submit** (Отправить).

После выполнения задания в таблице должны появиться записи, которые будут доступны для запроса с помощью любого механизма по вашему выбору.

Заключение

Используя информацию, представленную в этой главе, вы сможете начать пользоваться преимуществами Apache Iceberg в своих заданиях Flink для чтения и записи потоковых и пакетных данных. Вы узнали, как написать базовое задание Flink, познакомились с разными операциями чтения и записи и увидели, как настроить задания для потоковой передачи или пакетной обработки.

ЧАСТЬ III

АРАСНЕ ICEBERG НА ПРАКТИКЕ

Использование Apache Iceberg как части платформы управления данными не ограничивается операциями чтения и записи данных. Нередко возникает необходимость наблюдать за состоянием таблиц с помощью их метаданных, изменять версии данных для изоляции приема и аварийного восстановления, переносить существующие данные в Iceberg и использовать Iceberg в разных сценариях. Цель этой части книги – познакомить вас с инструментами и практиками, которыми вы можете воспользоваться при работе с Iceberg.

Глава 10

Apache Iceberg в производстве

Инженеры данных несут ответственность за сбор, хранение и обработку информации максимально эффективным, надежным и безопасным способом. В промышленных окружениях они должны следовать некоторым передовым методам, чтобы гарантировать точность, согласованность и доступность данных. В этой главе мы обсудим инструменты, которые можно использовать для мониторинга и поддержки таблиц Apache Iceberg в промышленном окружении. Сначала мы обсудим таблицы метаданных в Apache Iceberg. Затем рассмотрим способы обеспечения качества данных, включая ветвление для изоляции приема на уровне таблицы или каталога, управление версиями каталога для выполнения многотабличных транзакций и откат состояния таблицы или каталога, если что-то пойдет не так.

Все методы, обсуждаемые в этой главе, можно применять как реактивно, так и проактивно. Реактивный подход – это реагирование на уже возникшую ситуацию, например перезапись раздела, который стал слишком большим, или откат таблицы, которая получила неверные данные.

Проактивные методы направлены на профилактику и предотвращение подобных проблем и включают, например, мониторинг размеров разделов с использованием таблиц метаданных, а также использование ветвления для изоляции приема, чтобы никакие ошибочные данные не попали в рабочее окружение до того, как начнется выполнение проверки качества.

Старайтесь применять оба подхода для предотвращения возможных проблем и для устранения проблем, которым удалось просочиться. Методы, описанные в этой главе, помогут вам в обоих случаях.

Таблицы метаданных в Apache Iceberg

Одна из самых замечательных особенностей Apache Iceberg заключается в том, что на основе надежных метаданных можно создать несколько таблиц метаданных для мониторинга работоспособности и диагностики возможных узких мест.

Раньше возможности просмотра информации о таблицах данных в таких форматах, как Hive, зависели от конкретного механизма, реализующего специальные команды, такие как `SHOW PARTITIONS`. Поскольку Apache Iceberg предоставляет эти данные в виде самых обычных таблиц, поддерживающих SQL, вы можете использовать всю мощь SQL при работе с этими таблицами, выполнять сортировку, агрегирование, соединение и все остальное, что возможно с помощью SQL, даже путешествия во времени.

```
SELECT * FROM catalog.table.history AS OF VERSION 1059035530770364194
```

Эти таблицы создаются на основе файлов метаданных Apache Iceberg во время выполнения запроса. Давайте сначала рассмотрим существующие таблицы метаданных и их схемы, а затем обсудим способы их использования для мониторинга таблиц данных Apache Iceberg. Имейте в виду, что таблицы метаданных, встроенные в Apache Iceberg, требуют применения немного отличающегося синтаксиса в разных механизмах запросов. В последующих примерах мы будем использовать синтаксис Spark, Dremio и Trino.

Далее мы будем описывать схемы таблиц метаданных, а соответствующие им таблицы данных вы найдете в репозитории книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg>).

Таблица метаданных history

Таблица метаданных `history` хранит эволюцию соответствующей таблицы данных. Каждое из четырех полей в этой таблице предоставляет уникальную информацию об истории таблицы.

Первое поле `made_current_at` представляет точную отметку времени, когда соответствующий снимок стал текущим, по которой можно судить, когда были зафиксированы изменения в таблице.

Поле `snapshot_id` служит уникальным идентификатором для каждого снимка, который позволяет отслеживать определенные снимки в истории таблицы и ссылаться на них.

Поле `parent_id` – это уникальный идентификатор родителя текущего снимка, который фактически отражает происхождение каждого снимка и упрощает отслеживание эволюции таблицы с течением времени.

Наконец, поле `is_current_ancestor` сообщает, является ли снимок предком текущего снимка. Это логическое значение помогает идентифицировать

снимки, являющиеся частью текущего состояния таблицы, и снимки, объявленные недействительными в результате откатов таблицы.

В табл. 10.1 представлена схема таблицы метаданных history.

Таблица 10.1. Схема таблицы метаданных history

Поле	Тип	Пример значения
made_current_at	Timestamp	2023-02-08 03:29:51.215
snapshot_id	Int	5179289226185056830
parent_id	Int или null	5781947345336215154
is_current_ancestor	Boolean	true

Таблицу метаданных history можно использовать для восстановления данных и управления версиями, а также для идентификации откатов таблицы.

С помощью идентификатора снимка (snapshot_id) можно восстановить данные и свести к минимуму потенциальную потерю данных. В случае возникновения каких-либо проблем или ошибок пользователи могут получить более ранние версии данных, обратившись к истории снимков. Вы можете просто запросить таблицу, чтобы получить снимок до сбоя, а затем использовать идентификатор снимка для отката таблицы одним из методов, обсуждаемых далее в этой главе.

В следующем фрагменте показан запрос, получающий snapshot_id всех снимков до 11 июля, которые можно использовать для отката таблицы после инцидента, произошедшего в эту дату. Способы выполнения откатов будут обсуждаться далее в этой книге.

```
SELECT snapshot_id
FROM catalog.table.metadata_log_entries
WHERE made_current_at < '2023-07-11 00:00:00'
ORDER BY made_current_at ASC
```

Данные из этой таблицы можно использовать для идентификации отката таблицы. Это может пригодиться для выяснения, когда были предприняты действия по восстановлению при попытке создать контекст для истории таблицы. Чтобы определить откат, в таблице history следует искать два признака:

- два или более снимков имеют одинаковый parent_id;
- только для одного из этих снимков is_current_ancestor имеет значение true (означающее, что это часть текущей истории таблицы).

Например, на основе информации о таблице, представленной выше, по идентификаторам снимков (296410040247533565 и 2999875608062437345) можно сделать вывод, что в истории таблицы имел место откат. Этот вывод сделан на основании того факта, что снимок 296410040247533565 не является текущим предком и имеет общего родителя со снимком 2999875608062437345.

Следующий фрагмент кода показывает, как запросить все записи из таблицы метаданных history:

```
-- Spark SQL
SELECT * FROM my_catalog.table.history;

-- Dremio
SELECT * FROM TABLE(table_history('catalog.table'))

-- Trino
SELECT * FROM "table$history"
```

Таблица метаданных `metadata_log_entries`

Таблица метаданных `metadata_log_entries` хранит эволюцию таблицы, регистрируя файлы метаданных, созданные во время обновлений. Каждое поле в этой таблице содержит важную информацию о состоянии таблицы в данный момент времени.

Поле `timestamp` хранит точную дату и время обновления метаданных. Эта отметка времени служит маркером состояния таблицы в данный конкретный момент.

Поле `file` указывает местоположение файла данных, соответствующего этой конкретной записи в журнале метаданных. Это местоположение действует как точка отсчета для доступа к фактическим данным, связанным с записью метаданных.

Поле `latest_snapshot_id` хранит идентификатор самого последнего снимка, имевшегося на момент обновления метаданных. Это полезный ориентир для понимания состояния данных на момент обновления метаданных.

Поле `latest_schema_id` хранит идентификатор схемы, использовавшейся при создании записи в журнале метаданных, который определяет контекст о структуре данных на момент обновления метаданных.

Наконец, поле `latest_sequence_number` указывает порядок обновлений метаданных. Это монотонно увеличивающийся счетчик, помогающий отслеживать последовательность изменений метаданных с течением времени.

В табл. 10.2 представлена схема таблицы `metadata_log_entries`.

Таблица 10.2. Схема таблицы `metadata_log_entries`

Поле	Тип	Пример значения
<code>timestamp</code>	Timestamp	2023-07-28 10:43:57.487
<code>file</code>	String	.../v1.metadata.json
<code>latest_snapshot_id</code>	Int	180260833656645300
<code>latest_schema_id</code>	Int	0
<code>latest_sequence_number</code>	Int	1

Таблицу метаданных `metadata_log_entries` можно использовать для поиска последнего снимка с предыдущей схемой. Например, вы могли внести изменения в схему и теперь хотите вернуться к предыдущей схеме. Вам понадобится найти последний снимок, использовавший эту схему, что можно

определить с помощью запроса, который ранжирует снимки по значению `schema_id`, а затем возвращает только снимок с наибольшим рангом:

```
WITH Ranked_Entries AS (
    SELECT
        latest_snapshot_id,
        latest_schema_id,
        timestamp,
        ROW_NUMBER() OVER(PARTITION BY latest_schema_id ORDER BY timestamp
DESC) as row_num
    FROM
        catalog.table.metadata_log_entries
    WHERE
        latest_schema_id IS NOT NULL
)
SELECT
    latest_snapshot_id,
    latest_schema_id,
    timestamp AS latest_timestamp
FROM
    Ranked_Entries
WHERE
    row_num = 1
ORDER BY
    latest_schema_id DESC;
Следующий запрос вернет все записи из этой таблицы:
-- Spark SQL
SELECT * FROM my_catalog.table.metadata_log_entries;
```

Таблица метаданных snapshots

Таблица метаданных snapshots отслеживает версии и истории набора данных. Она хранит метаданные о каждом снимке данной таблицы, предлагая согласованное представление набора данных в определенное время. Подробности о каждом снимке служат исторической записью изменений и отображают состояние набора данных на момент создания снимка. Таблица имеет несколько полей, каждое из которых играет уникальную роль.

Поле `commit_at` определяет точное время создания снимка и связанное с ним состояние данных.

Поле `snapshot_id` – это уникальный идентификатор для каждого снимка. Оно имеет решающее значение для различения снимков и выполнения определенных операций, таких как получение или удаление снимков.

Строковое поле `operation` хранит тип операции, такой как APPEND и OVERWRITE.

Поле `parent_id` хранит идентификатор родительского снимка, определяя контекст происхождения снимков и позволяя восстановить историческую последовательность их создания.

Поле `manifest_list` хранит информацию о файлах, составляющих снимок. Он чем-то похож на каталог, хранящий список всех файлов данных, связанных с данным снимком.

Наконец, поле `summary` содержит статистические метрики снимка, такие как количество добавленных или удаленных файлов, количество записей и другие данные, позволяющие быстро просмотреть содержимое снимка.

В табл. 10.3 представлена схема таблицы метаданных `snapshots`.

Таблица 10.3. Схема таблицы `snapshots`

Поле	Тип	Пример значения
<code>committed_at</code>	Timestamp	2023-02-08 03:29:51.215
<code>snapshot_id</code>	Int	57897183625154
<code>parent_id</code>	Int или null	NULL
<code>operation</code>	String	APPEND
<code>manifest_list</code>	String	.../table/metadata/snap-57897183999154-1.avro
<code>summary</code>	Map/struct	{ added-records -> 400404, total-records -> 3000000, added-data-files -> 300, total-data-files -> 500, spark.app.id -> application_1520379268916_155055 }

Существует масса возможных вариантов использования таблицы метаданных `snapshots`. Один из них – выяснение закономерностей добавления данных в таблицу. Это может быть полезно при планировании емкости или для понимания тенденций увеличения объема данных с течением времени. Вот SQL-запрос, показывающий общее количество записей, добавленных в каждый снимок:

```
SELECT
    committed_at,
    snapshot_id,
    summary['added-records'] AS added_records
FROM
    catalog.table.snapshots;
```

Еще один вариант использования этой таблицы – отслеживание типов и частоты операций, выполняемых с таблицей с течением времени. Это может быть полезно для понимания особенностей рабочей нагрузки и моделей использования таблицы. Вот SQL-запрос, показывающий количество операций каждого типа с течением времени:

```
SELECT
    operation,
    COUNT(*) AS operation_count,
    DATE(committed_at) AS date
FROM
    catalog.table.snapshots
GROUP BY
    operation,
    DATE(committed_at)
```

```
ORDER BY
    date;
```

Таблица метаданных snapshots в Apache Iceberg является ценным ресурсом для управления версиями набора данных, поддержки исторических запросов, а также инкрементной обработки, оптимизации и репликации. Она позволяет эффективно отслеживать изменения, получать доступ к историческим состояниям и эффективно управлять наборами данных.

Следующий SQL-запрос позволит вам увидеть все хранимые в ней данные:

```
-- Spark SQL
SELECT * FROM my_catalog.table.snapshots;
-- Dremio
SELECT * FROM TABLE(table_snapshot('catalog.table'))
-- Trino
SELECT * FROM "table$snapshots"
```

Таблица метаданных files

Таблица метаданных files отображает текущие файлы данных в таблице и предоставляет подробную информацию о каждом, от их местоположения и формата до содержимого и особенностей секционирования.

Поле content описывает тип содержимого файла: 0 означает файл данных, 1 – файл позиционного удаления позиции и 2 – файл удаления по равенству.

Поле file_path указывает точное местоположение каждого файла. Эта информация упрощает доступ к каждому файлу данных, когда это необходимо.

Поле file_format указывает формат файла данных: Parquet, Avro или ORC.

Поле spec_id хранит идентификатор спецификации раздела, которому соответствует файл, и сообщает, как секционируются данные.

Поле partition представляет конкретный раздел файла данных, указывая, как секционированы данные в файле. Это необходимо для оптимизации доступа и повышения производительности запросов.

Поле record_count сообщает количество записей в каждом файле, что дает меру объема данных файла.

Поле file_size_in_bytes сообщает общий размер файла в байтах, а поле columns_sizes – размеры отдельных столбцов.

Поля value_counts, null_value_counts и nan_value_counts сообщают количество значений, отличных от NULL, значений NULL и NaN (Not a Number – не число) соответственно в каждом столбце.

Поля lower_bounds и upper_bounds содержат минимальное и максимальное значения в каждом столбце, предоставляя важную информацию о диапазоне данных в каждом файле.

Поле key_metadata содержит метаданные, характерные для реализации, если таковые существуют.

Поле split_offsets содержит смещения, по которым файл разбивается на более мелкие сегменты для параллельной обработки.

Поля `equality_ids` и `sort_order_id` хранят идентификаторы, относящиеся к файлам по равенству, если таковые существуют, и идентификаторы порядка сортировки таблицы, если имеются.

В табл. 10.4 представлена схема таблицы метаданных `files`.

Таблица 10.4. Схема таблицы `files`

Поле	Тип	Пример значения
<code>content</code>	<code>Int</code>	0
<code>file_path</code>	<code>String</code>	.../table/data/00000-3-8d6d60e8-d427-4809-bcf0-f5d45a4aad96.parquet
<code>file_format</code>	<code>String</code>	PARQUET
<code>spec_id</code>	<code>Int</code>	0
<code>partition</code>	<code>Struct</code>	{1999-01-01, 01}
<code>record_count</code>	<code>Int</code>	1
<code>file_size_in_bytes</code>	<code>Int</code>	597
<code>columns_sizes</code>	<code>Map</code>	[1 -> 90, 2 -> 62]
<code>value_counts</code>	<code>Map</code>	[1 -> 1, 2 -> 1]
<code>null_value_counts</code>	<code>Map</code>	[1 -> 0, 2 -> 0]
<code>nan_value_counts</code>	<code>Map</code>	[]
<code>lower_bounds</code>	<code>Map</code>	[1 -> , 2 -> c]
<code>upper_bounds</code>	<code>Map</code>	[1 -> , 2 -> c]
<code>key_metadata</code>	<code>Binary</code>	null
<code>split_offsets</code>	<code>List</code>	[4]
<code>equality_ids</code>	<code>List</code>	null
<code>sort_order_id</code>	<code>Int</code>	null

Существует множество возможных вариантов использования таблицы метаданных `files`, включая определение необходимости перезаписи раздела, определение разделов, требующих восстановления данных, определение общего размера снимка и получение списка файлов из предыдущего снимка.

Если в разделе много небольших файлов, он может быть хорошим кандидатом на уплотнение для повышения производительности, как обсуждалось в главе 4. Следующий запрос может помочь получить количество файлов в каждом разделе и средний размер файла, чтобы определить разделы, которые нужно перезаписать:

```
SELECT
    partition,
    COUNT(*) AS num_files,
    AVG(file_size_in_bytes) AS avg_file_size
FROM
    catalog.table.files
GROUP BY
    partition
ORDER BY
    num_files DESC,
    avg_file_size ASC
```

Некоторые поля, вероятно, не должны иметь значений NULL в данных. Используя таблицу метаданных файлов, можно идентифицировать разделы или файлы с отсутствующими значениями, с помощью гораздо более быстрой операции, чем сканирование фактических данных. Следующий запрос вернет разделы и имена файлов с отсутствующими данными в третьем столбце:

```
SELECT
    partition, file_path
FROM
    catalog.table.files
WHERE
    null_value_counts['3'] > 0
GROUP BY
    partition
```

Таблицу метаданных files можно также использовать для обобщения размеров файлов и получения общего размера снимка:

```
SELECT sum(file_size_in_bytes) from catalog.table.files;
```

Используя исторический запрос, можете получить список файлов из предыдущего снимка:

```
SELECT file_path, file_size_in_bytes
FROM catalog.table.files
VERSION AS OF <идентификатор_снимка>;
```

Таблица метаданных files в Apache Iceberg предоставляет подробную информацию об отдельных файлах данных, позволяющую детализировать обработку данных, организовать управление схемой, отслеживать происхождение и качество данных. Она служит ценным ресурсом для различных вариантов использования, позволяя пользователям получать более полное понимание данных. Следующий SQL-запрос вернет данные из таблицы files:

```
-- Spark SQL
SELECT * FROM my_catalog.table.files;
-- Dremio
SELECT * FROM TABLE(table_files('catalog.table'))
-- Trino
SELECT * FROM "table$files"
```

Таблица метаданных manifests

В таблице метаданных manifests содержится подробная информация обо всех текущих файлах манифестов. Таблица хранит массу полезной информации, которая помогает понять структуру таблицы и изменения, происходившие с течением времени.

Поле `path` указывает путь к файлу манифеста, обеспечивая быстрый доступ к нему. Поле `length` сообщает размер файла манифеста.

Поле `partition_spec_id` указывает идентификатор спецификации раздела, с которым связан файл манифеста, что полезно для отслеживания изменений в секционированных таблицах. Поле `added_snapshot_id` хранит идентификатор снимка, в который был добавлен этот файл манифеста, создавая связь между снимками и манифестами.

Три поля счетчиков – `added_data_files_count`, `existing_data_files_count` и `deleted_data_files_count` – сообщают количество новых файлов, добавленных в манифест, количество существующих файлов данных, добавленных в предыдущие моментальные снимки, и количество файлов, удаленных из этого манифеста соответственно. Эта тройка полей играет важную роль в понимании эволюции данных.

Поле `partition_summaries` хранит массив структур `field_summary` со статистикой уровня раздела, содержащий следующую информацию: `contains_null`, `contains_nan`, `lower_bound` и `upper_bound`. Эти поля указывают, содержит ли раздел значения `null` или `NaN`, а также сообщают нижнюю и верхнюю границы данных в разделе. Важно отметить, что `contains_nan` может возвращать значение `null`, если информация недоступна из метаданных файла, что обычно происходит при чтении из таблицы V1. В табл. 10.5 приведена схема таблицы метаданных `manifests`.

Таблица 10.5. Схема таблицы `manifests`

Поле	Тип	Пример значения
<code>path</code>	String	<code>.../table/metadata/45b5290b-ee61-4788-b324-b1e2735c0e10-m0.avro</code>
<code>length</code>	Int	4479
<code>partition_spec_id</code>	Int	0
<code>added_snapshot_id</code>	Int	6668963634911763636
<code>added_data_files_count</code>	Int	8
<code>existing_data_files_count</code>	Int	0
<code>deleted_data_files_count</code>	Int	0
<code>partition_summaries</code>	List	<code>[[false,null,2019-05-13,2019-05-15]]</code>

С помощью таблицы метаданных `manifests` пользователи могут, например, отыскивать манифесты, которые необходимо перезаписать, определять среднее количество файлов, добавленных в один снимок, находить снимки, из которых файлы были удалены, и определять, хорошо ли отсортирована таблица.

С помощью следующего запроса можно определить, какие манифесты имеют размер меньше среднего, что может помочь определить, какие манифесты можно уплотнить с помощью `rewrite_manifests`:

```
WITH avg_length AS (
  SELECT AVG(length) as average_manifest_length
```

```
FROM catalog.table.manifests
)

SELECT
    path,
    length
FROM
    catalog.table.manifests
WHERE
    length < (SELECT average_manifest_length FROM avg_length);
```

Возможно, вам будет интересно узнать скорость роста файлов в вашей таблице. Следующий запрос поможет вам узнать, сколько файлов было добавлено в каждый снимок:

```
SELECT
    added_snapshot_id,
    SUM(added_data_files_count) AS total_added_data_files
FROM
    catalog.table.manifests
GROUP BY
    added_snapshot_id;
```

Возможно, также вам пригодится знание закономерностей удаления данных в целях выполнения запросов на очистку личной информации. Знание того, какие снимки были удалены, может помочь отслеживать, какие снимки следует объявить устаревшими для окончательного удаления данных:

```
SELECT
    added_snapshot_id
FROM
    catalog.table.manifests
WHERE
    deleted_data_files_count > 0;
```

Следующий запрос возвращает верхнюю и нижнюю границы из манифеста, что может помочь определить, насколько хорошо отсортированы данные и следует ли их перезаписать для улучшения кластеризации:

```
SELECT path, partition_summaries
FROM db.table.manifests;
```

Таблица метаданных manifests также может помочь в управлении, анализе и оптимизации наборов данных, хранящихся в Apache Iceberg:

```
-- Spark SQL
SELECT * FROM my_catalog.table.manifests;
-- Dremio
SELECT * FROM TABLE(table_manifests('catalog.table'))
-- Spark SQL
SELECT * FROM "table$manifests"
```

Таблица метаданных partitions

Таблица метаданных partitions в Apache Iceberg представляет моментальный снимок секционирования данных в таблице на отдельные непересекающиеся области, известные как *разделы*. Каждая запись представляет определенный раздел в таблице.

Поле `partition` определяет фактические значения раздела, обычно основанные на определенных столбцах данных. Это позволяет организовать данные осмысленным образом и обеспечить эффективную обработку запросов, поскольку данные могут быть получены на основе определенных значений разделов.

Поле `record_count` указывает общее количество записей в данном разделе. Эта метрика может пригодиться для понимания распределения данных по разделам и служить руководством для стратегий оптимизации, таких как выбор другого способа секционирования и перебалансировка.

Поле `file_count` сообщает общее количество файлов данных в разделе, что крайне важно для управления и оптимизации хранилища, поскольку слишком большое количество маленьких файлов может отрицательно сказаться на производительности запросов.

Наконец, поле `spec_id` хранит идентификатор спецификации раздела, использованной для его создания. Спецификации разделов определяют порядок секционирования данных, а идентификатор помогает понять используемую стратегию секционирования.

Стоит отметить, что для несекционированных таблиц таблица метаданных partitions будет иметь одну запись, содержащую только поля `record_count` и `file_count`. Также имеются счетчики удаленных записей и файлов в полях `position_delete_record_count`, `position_delete_file_count`, `equality_delete_record_count` и `equality_delete_file_count` соответственно.

Таблица метаданных partitions предоставляет моментальный снимок текущего состояния разделов, тем не менее важно отметить, что информация в ней не учитывает сведения из файлов удаления. В результате иногда разделы могут отображаться в списке даже после того, как все записи в них будут помечены как удаленные. В табл. 10.6 представлена схема таблицы метаданных partitions.

Таблица 10.6. Схема таблицы partitions

Поле	Тип	Пример значения
<code>partition</code>	List	{20211001, 11}
<code>spec_id</code>	Int	0
<code>record_count</code>	Int	1
<code>file_count</code>	Int	1
<code>position_delete_record_count</code>	Int	0
<code>position_delete_file_count</code>	Int	0
<code>equality_delete_record_count</code>	Int	0
<code>equality_delete_file_count</code>	Int	0

Существует множество вариантов использования таблицы метаданных `partitions`, включая определение количества файлов в разделах, суммирование размера раздела в байтах и определение количества разделов в каждой схеме секционирования. Например, иногда бывает нужно узнать, сколько файлов находится в разделе, поскольку раздел с большим количеством файлов может быть хорошим кандидатом на уплотнение. Следующий запрос поможет в этом:

```
SELECT partition, file_count FROM catalog.table.partitions
```

Помимо количества файлов, можно узнать размер раздела. Если какой-то раздел особенно велик, можно попробовать изменить схему секционирования, чтобы лучше сбалансировать распределение, как показано здесь:

```
SELECT partition, SUM(file_size_in_bytes) AS partition_size FROM catalog.table.files GROUP BY partition
```

Благодаря эволюции со временем могут появиться разные схемы секционирования. Если вам интересно, как разные схемы секционирования влияют на количество разделов, то можете попробовать выполнить следующий запрос:

```
SELECT
    spec_id,
    COUNT(*) as partition_count
FROM
    catalog.table.partitions
GROUP BY
    spec_id;
-- Spark SQL
SELECT * FROM my_catalog.table.partitions;
-- Trino
SELECT * FROM "test_table$partitions"
```

Таблица метаданных `all_data_files`

Таблица метаданных `all_data_files` в Apache Iceberg предоставляет подробные сведения о каждом файле данных во всех действительных снимках таблицы.

Поле `content` сообщает тип файла. Значение 0 указывает, что это файл данных, 1 – файл позиционного удаления и 2 – файл удаления по равенству.

Поле `file_path` содержит строку – полный путь к файлу данных. Обычно эта строка включает местоположение системы хранения (например, `s3://my-bucket/folder/subfolder/myfile.xyz`), имя таблицы и уникальный идентификатор файла.

Поле `file_format` сообщает формат файла данных. В нашем примере это Parquet, но это может быть другой формат, например AVRO или ORC.

Поле `spec_id` хранит идентификатор спецификации секционирования, использованной для создания этого раздела.

Поле `partition` представляет раздел, которому принадлежит файл данных. Обычно это значение основано на схеме секционирования, определенной для таблицы.

Поле `record_count` сообщает общее количество записей в файле, а поле `file_size_in_bytes` – размер файла данных в байтах. Обе метрики необходимы для понимания объема данных и могут использоваться в стратегиях оптимизации запросов.

Поле `columns_sizes` хранит соответствия идентификаторов столбцов и их размеров в байтах.

Поле `value_counts` содержит словарь, отображающий общее количество значений в каждом столбце в файле данных. Аналогично поля `null_value_counts` и `nan_value_counts` предоставляют количество значений `null` и `NaN` в каждом столбце соответственно.

Поля `lower_bounds` и `upper_bounds` представляют собой словари, с минимальными и максимальными значениями в каждом столбце в файле данных. Эти поля играют важную роль в ускорении выполнения запросов.

Поле `key_metadata` содержит метаданные, характерные для реализации.

Поле `split_offsets` хранит информацию о точках разделения в файле. Это массив длинных целых значений, который полезен в сценариях распределенной обработки, где файлы данных можно разделить на более мелкие фрагменты для параллельной обработки.

Поле `equality_ids` относится к удалениям по равенству и помогает идентифицировать записи, удаленные по соответствию некоторому критерию.

Поле `sort_order_id` содержит идентификатор порядка сортировки, использованного для записи в файл данных.

Поле `readable_metrics` – производное. Оно обеспечивает удобочитаемое представление метаданных файла, включая размер столбца, количество значений, количество пустых значений, а также нижнюю и верхнюю границы.

Имейте в виду, что таблица метаданных `all_data_files` может содержать несколько записей для каждого файла данных, так как файл может быть частью нескольких снимков. Эта таблица помогает понять детали состояния и организации данных. В табл. 10.7 представлена схема таблицы метаданных `all_data_files`.

Таблица 10.7. Схема таблицы `all_data_files`

Поле	Тип	Пример значения
<code>content</code>	<code>Int</code>	0
<code>file_path</code>	<code>String</code>	.../dt=20210103/00000-0-26222098-032f-472b-8ea5...
<code>file_format</code>	<code>String</code>	PARQUET
<code>spec_id</code>	<code>Int</code>	0
<code>partition</code>	<code>Struct</code>	{20210102}
<code>record_count</code>	<code>Int</code>	14
<code>file_size_in_bytes</code>	<code>Int</code>	2444

Таблица 10.7 (окончание)

Поле	Тип	Пример значения
columns_sizes	Map	{1 -> 94, 2 -> 17}
value_counts	Map	{1 -> 14, 2 -> 14}
null_value_counts	Map	[1 -> 0, 2 -> 0]
nan_value_counts	Map	{1 -> 0, 2 -> 0}
lower_bounds	Map	{1 -> 1, 2 -> 20210102}
upper_bounds	Map	{1 -> 2, 2 -> 20210102}
key_metadata	Binary	null
split_offsets	List	[4]
equality_ids	List	null
sort_order_id	Int	null
readable_metrics	List	{{48, 2, 0, null, Benjamin, Brandon}}

Таблица метаданных `all_data_files` имеет множество практических применений, включая поиск самой большой таблицы во всех моментальных снимках, определение общего размера файлов во всех снимках и оценку размеров разделов между снимками.

Следующий запрос сначала проверяет наличие отдельных файлов, поскольку в одном и том же файле может быть несколько записей. Затем возвращает пять самых больших файлов из этого списка отдельных файлов:

```
WITH distinct_files AS (
  SELECT DISTINCT file_path, file_size_in_bytes
  FROM catalog.table.all_data_files
)
SELECT file_path, file_size_in_bytes
FROM distinct_files
ORDER BY file_size_in_bytes DESC
LIMIT 5;
```

Узнать общее количество файлов, размеры этих файлов и количество записей во всех снимках можно с помощью следующего запроса:

```
WITH unique_files AS (
  SELECT DISTINCT file_path, record_count, file_size_in_bytes
  FROM catalog.table.all_data_files
)
SELECT COUNT(*) as num_unique_files,
       SUM(record_count) as total_records,
       SUM(file_size_in_bytes) as total_file_size
FROM unique_files;
```

С помощью следующего запроса можно узнать количество файлов, количество записей и общий размер файлов в каждом разделе во всех снимках. Опираясь на эту информацию, можно понять распределение данных по разделам:

```
WITH unique_files AS (
  SELECT DISTINCT file_path, partition, record_count, file_size_in_bytes
```



```

FROM catalog.table.all_data_files
)
SELECT partition,
       COUNT(*) as num_unique_files,
       SUM(record_count) as total_records,
       SUM(file_size_in_bytes) as total_file_size
FROM unique_files
GROUP BY partition;

-- Spark SQL для всех файлов данных
SELECT * FROM my_catalog.table.all_data_files;

```

Таблица метаданных all_manifests

Таблица метаданных all_manifests в Apache Iceberg предоставляет подробную информацию о каждом файле манифеста во всех действительных снимках таблицы.

Поле content обозначает тип файла, по аналогии с таблицей all_data_files. Значение 0 указывает, что манифест отслеживает файлы данных; 1 – файлы удаления.

Поле path содержит строку – полный путь к файлу манифеста. Как и таблица all_data_files, она включает местоположение системы хранения (например, s3://...), имя таблицы и уникальный идентификатор файла.

Поле length хранит размер файла манифеста в байтах. Эта информация может дать представление об объеме метаданных, хранящихся в манифесте.

Поле partition_spec_id содержит идентификатор спецификации секционирования, используемой для записи этого файла манифеста. Он сообщает, как секционированы файлы данных, перечисленные в манифесте.

Поле added_snapshot_id представляет идентификатор снимка при создании манифеста.

Поля added_data_files_count, existing_data_files_count и deleted_data_files_count предоставляют сводную информацию об изменениях в файлах данных, перечисленных в этом файле манифеста. Поля added_delete_files_count, existing_delete_files_count и deleted_delete_files_count предоставляют аналогичные сведения о файлах удаления.

Поле partition_summaries – это массив структур, каждая из которых содержит сводную информацию об определенном разделе в файле манифеста. Каждая структура сообщает, содержит ли раздел значения null или NaN, а также нижнюю и верхнюю границы.

Поле reference_snapshot_id представляет идентификатор снимка, с которым связана эта запись. Вы увидите, что манифест указывается один раз для каждого снимка, для которого он был действителен.

Имейте в виду, что таблица метаданных all_manifests может содержать несколько записей для каждого файла манифеста, потому что файл манифеста может быть частью нескольких снимков. Эта таблица помогает понять со-

стояние и организацию данных на более целостном уровне, чем таблица `all_data_files`. В табл. 10.8 приводится схема таблицы метаданных `all_manifests`.

Таблица 10.8. Схема таблицы `all_manifests`

Поле	Тип	Пример значения
<code>content</code>	Int	0
<code>file_path</code>	String	<code>../metadata/a85f78c5-3222-4b37-b7e4-faf944425d48-m0.avro</code>
<code>length</code>	Int	6367
<code>partition_spec_id</code>	Int	0
<code>added_snapshot_id</code>	Int	6272782676904868561
<code>added_data_files_count</code>	Int	2
<code>existing_data_files_count</code>	Int	0
<code>deleted_data_files_count</code>	Int	0
<code>added_delete_files_count</code>	Int	2
<code>existing_delete_files_count</code>	Int	0
<code>deleted_delete_files_count</code>	Int	0
<code>partition_summaries</code>	List	<code>[{false, false, 20210101, 20210101}]</code>
<code>reference_snapshot_id</code>	Int	6272782676904868561

Таблица метаданных `all_manifests` имеет множество практических применений, включая поиск всех манифестов для определенного снимка, определение тенденции к росту манифестов от снимка к снимку и получение общего размера всех действительных манифестов.

Таблица `all_manifests` может помочь узнать все манифесты для текущего снимка с помощью такого запроса:

```
SELECT *
FROM catalog.table.all_manifests
WHERE reference_snapshot_id = 1059035530770364194;
```

Следующий запрос возвращает общий размер манифеста и количество файлов данных для каждого снимка, что поможет увидеть рост числа файлов и размера манифеста от снимка к снимку:

```
SELECT reference_snapshot_id, SUM(length) as manifests_length,
SUM(added_data_files_count + existing_data_files_count)AS total_data_files
FROM catalog.table.example.all_manifests
GROUP BY reference_snapshot_id;
```

С помощью следующего запроса можно получить суммарный объем всех действительных манифестов:

```
SELECT
  SUM(length) AS total_length
FROM (
  SELECT DISTINCT path, length
  FROM catalog.table.all_manifests
```

```
-- Spark SQL для всех манифестов
SELECT * FROM my_catalog.table.all_manifests;
```

Таблица метаданных refs

Таблица метаданных refs в Apache Iceberg предоставляет список всех именованных ссылок, имеющих в таблице Iceberg. Именованные ссылки можно рассматривать как указатели на конкретные снимки таблицы данных, позволяющие создавать закладки или определять версии состояния таблицы.

Поле `name` представляет уникальный идентификатор именованной ссылки. Именованные ссылки делятся на два типа, и тип ссылки хранится во втором поле – `type`. Поддерживается всего два типа ссылок: `BRANCH` – изменяемая ссылка, которую можно перенацелить на новый снимок, и `TAG` – неизменяемая ссылка, которая после создания всегда указывает на один и тот же снимок.

Поле `max_reference_age_in_ms` хранит максимальное время в миллисекундах, в течение которого ссылка указывала на снимок. Это время измеряется с момента добавления снимка в таблицу. Если возраст снимка превышает эту продолжительность, то он перестанет быть действительным и становится кандидатом на удаление в ближайшей операции обслуживания.

Поле `min_snapshots_to_keep` задает минимальное количество снимков, сохраняемых в истории таблицы. Таблица Iceberg всегда будет поддерживать количество снимков, не менее этого числа, даже если какие-то снимки окажутся старше, чем значение в поле `max_snapshot_age_in_ms`.

Наконец, поле `max_snapshot_age_in_ms` сообщает максимальный возраст в миллисекундах для любого снимка в таблице. Снимки, возраст которых превышает этот срок, могут быть удалены в ходе операций обслуживания, если они не защищены параметром `min_snapshots_to_keep`.

Помните, что таблица метаданных refs помогает изучать и управлять историей снимков таблицы и политикой хранения, что делает ее важной частью системы управления версиями данных и размером таблицы. В табл. 10.9 приводится схема таблицы метаданных refs.

Таблица 10.9. Схема таблицы refs

Поле	Тип	Пример значения
<code>name</code>	String	<code>main</code>
<code>type</code>	String	<code>BRANCH</code>
<code>snapshot_id</code>	Int	<code>4686954189838128572</code>
<code>max_reference_age_in_ms</code>	Int	<code>10</code>
<code>min_snapshots_to_keep</code>	Int	<code>20</code>
<code>max_snapshot_age_in_ms</code>	Int	<code>30</code>

Таблица метаданных refs имеет множество разных применений, включая поиск ссылок, которые могут потерять снимки, на которые они ссылаются, и поиск последнего снимка конкретной ссылки.

Также у вас может возникнуть вопрос, могут ли правила конкретной ветви привести к удалению снимка при его следующем обновлении. Запрос, возвращающий такую информацию, должен помочь отфильтровать ссылки, имеющие правила, ограничивающие максимальное количество снимков:

```
SELECT name, min_snapshots_to_keep, max_snapshot_age_in_ms
FROM catalog.table.refs
WHERE min_snapshots_to_keep IS NOT NULL AND max_snapshot_age_in_ms IS NOT NULL;
```

Следующий запрос вернет вам только идентификатор снимка для каждой ссылки:

```
SELECT name, snapshot_id
FROM catalog.table.refs;

-- Spark SQL
SELECT * FROM my_catalog.table.refs;
```

Таблица метаданных entries

Таблица метаданных entries в Apache Iceberg предоставляет подробные сведения о каждой операции с данными таблицы и файлами удаления во всех снимках. Каждая запись в этой таблице фиксирует операции, затрагивающие множество файлов в определенный момент истории таблицы, что делает ее важным ресурсом для понимания эволюции набора данных.

Поле status – это целое число, указывающее, был ли файл добавлен или удален из снимка. Значение 0 представляет существующий файл, 1 – добавленный файл, а 2 – удаленный файл. Это поле позволяет отслеживать жизненный цикл каждого файла, давая представление об изменениях и модификациях, которые претерпел набор данных с течением времени.

Поле snapshot_id – это уникальный идентификатор снимка, в котором произошла операция. Он позволяет связать каждую файловую операцию с конкретным снимком, что может пригодиться для отслеживания изменений в определенных версиях таблицы.

Поле sequence_number описывает порядок операций. Это глобальный счетчик для всех снимков таблицы, увеличивающийся при каждом внесенном изменении, независимо от характера этого изменения, будь то добавление, изменение или удаление. Зная порядковый номер, можно восстановить точную последовательность операций, приведших к текущему состоянию таблицы.

Поле data_file – это структура, инкапсулирующая сведения о файле, участвующем в операции. Структура включает следующие поля:

file_path

Полный путь к файлу в системе хранения.

file_format

Формат файла, такой как Parquet или AVRO.

`partition`

Информация о разделе, которому принадлежит файл.

`record_count`

Общее количество записей в файле.

`file_size_in_bytes`

Размер файла в байтах.

`column_sizes`

Словарь с размерами каждого столбца в байтах.

`value_counts`

Словарь с общими количествами значений в столбцах.

`null_value_counts`

Словарь с количествами значений null в столбцах.

`nan_value_counts`

Словарь с количествами значений NaN в столбцах.

`lower_bounds` и `upper_bounds`

Словари с минимальными и максимальными значениями в столбцах.

`key_metadata`

Метаданные, характерные для реализации.

`split_offsets`

Информация о точках деления внутри файла.

С помощью таблицы `entries` можно отследить каждую операцию с таблицей. Таким образом ее можно рассматривать как комплексный журнал эволюции данных. В табл. 10.10 представлена схема таблицы метаданных `entries`.

Таблица 10.10. Схема таблицы `entries`

Поле	Тип	Пример значения
<code>status</code>	Int	1
<code>snapshot_id</code>	Int	1059035530770364194
<code>sequence_number</code>	Int	0
<code>data_file</code>	List	{0, s3://...parquet, PARQUET, 0, {A}, 6, 609, {1 -> 83}, {1 -> 6}, {1 -> 0}, {}, {1 -> Adriana}, {1 -> Antonio}, null, null, null, 0}

Таблица метаданных `entries` имеет множество применений, включая выявление файлов, добавленных в конкретный снимок, отслеживание изменений в файлах с течением времени и изменение размера таблицы с течением времени.

Например, следующий запрос вернет все записи, соответствующие снимкам, представляющим добавление файла:

```
SELECT data_file
FROM catalog.table.entries
WHERE snapshot_id = <your_snapshot_id> AND status = 1;
```

Этот запрос вернет все записи для определенного файла, независимо от того, существовал ли он, был ли добавлен или удален:

```
SELECT snapshot_id, sequence_number, status, data_file
FROM catalog.table.entries
WHERE data_file.file_path = '<your_file_path>'
ORDER BY sequence_number ASC;
```

Следующий запрос вернет размеры добавленных файлов для каждого снимка, что поможет увидеть рост размеров разных снимков:

```
SELECT snapshot_id, SUM(data_file.file_size_in_bytes) as total_size_in_bytes
FROM catalog.table.entries
WHERE status = 1
GROUP BY snapshot_id
ORDER BY snapshot_id ASC;

-- Spark SQL
SELECT * FROM my_catalog.table.entries;
```

Совместное использование таблиц метаданных

Соединяя таблицы метаданных, можно извлечь еще более ценную информацию, которая поможет повысить эффективность операций. Давайте рассмотрим несколько примеров соединения таблиц метаданных в разных сценариях использования.

Получение сведений о файлах, добавленных в снимок

Есть возможность оценить объем данных, добавленных в снимок, чтобы убедиться, что добавлено правильное количество записей, увидеть рост объема хранимых файлов в конкретном снимке и многое другое. Получить все метаданные с информацией о файлах в конкретном снимке можно с помощью следующего запроса:

```
SELECT f.*, e.snapshot_id
FROM catalog.table.entries AS e
JOIN catalog.table.files AS f
ON e.data_file.file_path = f.file_path
WHERE e.status = 1 AND e.snapshot_id = <идентификатор_снимка>;
```

Получение подробного обзора жизненного цикла конкретного файла данных

Иногда бывает интересно узнать, когда тот или иной файл был добавлен, удален и существовал, а также состояние табличных операций в каждый мо-

мент времени. Используя следующий запрос, можно получить подробный журнал эволюции конкретного файла данных, что позволит также увидеть, сколько файлов было добавлено и удалено на каждом этапе. Кроме того, используя путь к файлу, можно идентифицировать каждую операцию, в которой участвовал файл, и использовать таблицы `entries` и `manifests` для сбора дополнительной информации об этих операциях:

```
SELECT e.snapshot_id, e.sequence_number, e.status, m.added_snapshot_id,
m.deleted_data_files_count, m.added_data_files_count
FROM catalog.table.entries AS e
JOIN catalog.table.manifests AS m
ON e.snapshot_id = m.added_snapshot_id
WHERE e.data_file.file_path = '<путь_к_файлу>'
ORDER BY e.sequence_number ASC;
```

Отслеживание эволюции таблицы по разделам и снимкам

Иногда бывает нужно увидеть, как изменяются разделы от снимка к снимку, например сколько файлов добавилось. Вот пример запроса, который поможет получить это представление. Вы можете использовать его как основу для оценки количества добавленных файлов и объема файлов, добавленных по разделам:

```
SELECT e.snapshot_id, f.partition, COUNT(*) AS files_added
FROM catalog.table.entries AS e
JOIN catalog.entries.files AS f
ON e.data_file.file_path = f.file_path
WHERE e.status = 1
GROUP BY e.snapshot_id, f.partition;
```

Мониторинг файлов, связанных с конкретной ветвью

При использовании ветвления таблицы может потребоваться отслеживать ее ветви, чтобы понимать потребности в хранении и оптимизации. С помощью следующего запроса вы сможете узнать, какие файлы относятся к текущему снимку конкретной ветви:

```
SELECT r.name as branch_name, f.*
FROM catalog.table.refs AS r
JOIN catalog.table.entries AS e
ON r.snapshot_id = e.snapshot_id
JOIN catalog.table.files AS f
ON e.data_file.file_path = f.file_path
WHERE r.type = 'BRANCH' AND r.name = '<имя_ветви>';
```

Поиск различий между двумя ветвями таблицы

Чтобы выявить файлы, которые не используются совместно двумя ветвями, можно использовать следующий запрос. Он объединяет результаты двух запросов, чтобы выявить уникальные файлы в обеих ветвях:

```
-- файлы, имеющиеся в branch1, но отсутствующие в branch2
SELECT 'branch1' as branch, f.*
FROM catalog.table.refs AS r1
JOIN catalog.table.entries AS e1
ON r1.snapshot_id = e1.snapshot_id
JOIN catalog.table.files AS f
ON e1.data_file.file_path = f.file_path
WHERE r1.type = 'BRANCH' AND r1.name = 'branch1'
AND f.file_path NOT IN (
    SELECT f2.file_path
    FROM catalog.table.refs AS r2
    JOIN catalog.table.entries AS e2
    ON r2.snapshot_id = e2.snapshot_id
    JOIN catalog.table.files AS f2
    ON e2.data_file.file_path = f2.file_path
    WHERE r2.type = 'BRANCH' AND r2.name = 'branch2'
)

UNION ALL

-- файлы, имеющиеся в branch2, но отсутствующие в branch1
SELECT 'branch2' as branch, f.*
FROM catalog.table.refs AS r1
JOIN catalog.table.entries AS e1
ON r1.snapshot_id = e1.snapshot_id
JOIN catalog.table.files AS f
ON e1.data_file.file_path = f.file_path
WHERE r1.type = 'BRANCH' AND r1.name = 'branch2'
AND f.file_path NOT IN (
    SELECT f2.file_path
    FROM catalog.table.refs AS r2
    JOIN catalog.table.entries AS e2
    ON r2.snapshot_id = e2.snapshot_id
    JOIN catalog.table.files AS f2
    ON e2.data_file.file_path = f2.file_path
    WHERE r2.type = 'BRANCH' AND r2.name = 'branch1'
)
```

Определение роста объема хранилища по последнему снимку каждой ветви

Ветки отлично подходят для изоляции и экспериментов, но многие из них, в которые постоянно поступают экспериментальные данные, могут повлечь увеличение затрат на хранение, и поэтому у вас может появиться желание отслеживать их. Следующий запрос позволит узнать, сколько данных было добавлено в текущий снимок каждой ветви:

```
SELECT r.name as branch_name, e.snapshot_id, SUM(f.file_size_in_bytes) as
total_size_in_bytes
FROM catalog.table.refs AS r
JOIN catalog.table.entries AS e
```



```
ON r.snapshot_id = e.snapshot_id
JOIN catalog.table.files AS f
ON e.data_file.file_path = f.file_path
WHERE r.type = 'BRANCH'
GROUP BY r.name, e.snapshot_id
ORDER BY r.name, e.snapshot_id;
```

Используя таблицы метаданных Apache Iceberg, можете повысить надежность отслеживания состояния таблиц данных, своевременно реагировать на снижение производительности и устранять другие проблемы.

Изоляция изменений с помощью ветвления

Возможность изоляции изменений в данных с использованием Git-подобных ветвей может оказаться весьма ценной в современных системах обработки данных, поэтому с каждым годом появляется все больше практиков, начавших применять ее. Ветвление позволяет разделить разные направления работы, дает разработчикам возможность вносить изменения независимо, не мешая работе других разработчиков и не дестабилизируя основную кодовую базу. Это похоже на наличие нескольких параллельных вселенных, где изменения в одной вселенной не влияют на другие. Вы можете экспериментировать, совершать ошибки и учиться, не опасаясь повлиять на более широкую систему.

В контексте таблиц Apache Iceberg существует два способа реализации этой концепции: на уровне таблицы, который является уникальным для Apache Iceberg и не зависит от каталога, и на уровне каталога, при использовании каталога Project Nessie.

Первый способ, изолирующий изменения на уровне таблицы, предполагает создание ветвей для конкретных таблиц. Каждая ветвь хранит полную историю изменений, внесенных в эту таблицу. Этот подход допускает параллельную эволюцию схем, откаты и другие расширенные сценарии использования. Это мощный инструмент для обработки изменений, зависящих от таблицы, но он не позволяет обеспечить целостное представление изменений во всем каталоге данных.

Изоляция изменений на уровне каталога позволяет управлять всем озером данных как единой сущностью, фиксируя изменения в нескольких таблицах в ветви. Используя Nessie, можно сделать снимок всего каталога в определенный момент времени. Такая практика поддерживает более комплексную стратегию управления версиями, давая возможность тестировать способы преобразования данных, отслеживать происхождение данных и поддерживать непротиворечивость данных в нескольких таблицах.

Оба метода имеют свои достоинства и недостатки. Изоляция на уровне таблиц обеспечивает более детальный контроль и гибкость для отдельных

таблиц, но управление ею в крупномасштабной среде данных может быть сопряжено с большими сложностями. Изоляция на уровне каталога обеспечивает комплексное и унифицированное представление изменений, но может быть избыточной в простых сценариях с одной таблицей.

Ценность изоляции изменений в Git-подобной ветви трудно переоценить. Она дает разработчикам свободу экспериментировать и вносить изменения, не опасаясь распространения влияния этих изменений, позволяет управлять версиями и откатывать изменения, а также способствует большей целостности данных и отслеживанию происхождения. Независимо от выбора способа изоляции – на уровне таблицы или каталога – использование Project Nessie будет зависеть от вашего конкретного сценария применения (прием данных, тестирование в промышленной среде, экспериментальные окружения) и сложности самой среды данных.

Ветвление и маркировка на уровне таблиц

В спецификацию Apache Iceberg встроена возможность отслеживать снимки по разным путям (так называемое *ветвление*) или присваивать конкретным снимкам имя (*маркировка*). Эта возможность обеспечивает изоляцию и воспроизводимость при операциях с данными в отдельной таблице.

Ветвление таблицы

Поддержка ветвления таблиц в Apache Iceberg позволяет создавать независимые последовательности снимков, каждый со своим жизненным циклом. Ветвь – это, по сути, именованная ссылка, указывающая на расходящуюся цепочку снимков. Каждая ссылка указывает на голову ветви – самый последний снимок в истории снимков ветви. В каждой ветви также есть настройки максимального возраста снимков и минимального количества снимков, которые должны существовать в ветви.

Рассмотрим сценарий управления данными, в котором имеется конвейер, загружающий данные в существующую таблицу. Прежде чем добавить новые данные в основную таблицу, их желательно изолировать и проверить. Чтобы добиться такой изоляции, можно использовать механизм ветвления Apache Iceberg.

В этом сценарии входные данные можно направить в отдельную ветвь (скажем, "ingestion-validation-branch"), не пересекающуюся с основной таблицей, для последующей проверки. Это легко организовать с помощью Iceberg API для Java, используя операцию `toBranch` во время записи (API состоит из библиотек для Java, входящих в состав проекта Apache Iceberg, которые позволяют выполнять все основные операции с таблицами Iceberg). Этот метод изолирует входные данные, чтобы затем проверить их перед объединением с данными основной таблицы.

Вот фрагмент кода на Java, демонстрирующий этот процесс:

```
// Использование Iceberg API для Java
// Имя ветви определяется как строка
String branch = "ingestion-validation-branch";

// Создать ветвь
table.manageSnapshots()
    // Создать ветвь из конкретного снимка
    .createBranch(branch, 3)
    // Указать, сколько снимков должно храниться в ветви
    .setMinSnapshotsToKeep(branch, 2)
    // Указать максимальный возраст снимков
    .setMaxSnapshotAgeMs(branch, 3600000)
    // Указать максимальный возраст ветви
    .setMaxRefAgeMs(branch, 604800000)
    .commit();

// Записать входные данные в новую ветвь
table.newAppend()
    // Добавить входной файл в ветвь
    .appendFile(INCOMING_FILE)
    // Указать ветвь для выполнения этой операции
    .toBranch(branch)
    .commit();

// Прочитать данные из ветви для проверки
TableScan branchRead = table
    .newScan()
    .useRef(branch);
```

Создание ветви "ingestion-validation-branch" позволяет протестировать новые входные данные, что делает ее ценным инструментом в сценариях обработки данных. После проверки данных в новой ветви основную ветвь можно обновить с помощью операции `fastForward`, переустановив ссылку на нее так, чтобы она указывала на голову ветви "ingestion-validation-branch":

```
// Обновить основную ветвь, внедрив в нее данные из новой ветви
table
    .manageSnapshots()
    // Последнее подтверждение в основной ветви "main" должно соответствовать
    // новой ветви "ingestion-validation-branch"
    .fastForward("main", "ingestion-validation-branch")
    .commit();
```

Таким образом, ветвление в Apache Iceberg обеспечивает эффективную изоляцию, возможность проверки и объединения входных данных, что позволяет гарантировать высокое качество и целостность данных в основной таблице.

Реализовать ту же изоляцию и проверку на SQL можно с помощью оператора `ALTER TABLE`. Сначала нужно создать новую ветвь "ingestion-valida-

tion-branch" в таблице. Для примера предположим, что речь идет о таблице sales_data в каталоге my_catalog и базе данных my_db. Ветвь настраивается для хранения снимков в течение семи дней и всегда хранит не менее двух снимков. Вот сама реализация:

```
-- Создать новую ветвь
ALTER TABLE my_catalog.my_db.sales_data
  CREATE BRANCH ingestion-validation-branch
  RETAIN 7 DAYS
  WITH RETENTION 2 SNAPSHOTS;
```

Затем назначаем вновь созданную ветвь активной для записи с помощью команды SET:

```
-- Назначить новую ветвь активной ветвью для записи
SET spark.wap.branch = 'ingestion-validation-branch';
```

Теперь можно записать входные данные в ветвь "ingestion-validation-branch". Новые данные окажутся изолированными, и их можно проверить и протестировать перед объединением с основными данными. Термин WAP (Write Audit Publish – записать, проверить, опубликовать) – это шаблон, согласно которому данные записываются, проверяются на наличие проблем с качеством, а затем публикуются. Предположим, что мы добавляем некоторые данные в таблицу sales_data:

```
-- Записать входные данные в новую ветвь
INSERT INTO my_catalog.my_db.sales_data (column1, column2, column3)
VALUES (value1, value2, value3), (value4, value5, value6);
```

По окончании проверки нам останется только использовать код на Java, чтобы запустить процедуру fastForward.

Маркировка таблиц

Ветвление позволяет создавать отдельные версии данных, однако в Iceberg также имеется механизм маркировки (tagging). Маркировка в Iceberg позволяет использовать именованные ссылки, указывающие на снимки для упрощения воспроизводимости.

Рассмотрим сценарий управления цепочкой поставок, в котором необходимо воспроизвести состояние таблицы на конец квартала для целей аудита. Маркировка позволяет сохранять важные исторические снимки и затем воспроизводить состояние, имевшее место в определенный момент времени. Метку (tag) для снимка можно создать с помощью операции createTag и указать, как долго должна храниться эта метка:

```
// создать метку
String tag = "end-of-quarter-Q3FY23";
table.manageSnapshots()
  // создать метку для снимка 8
```

```
.createTag(tag, 8)
// указать срок хранения метки
.setMaxRefAgeMs(tag, 1000 * 60 * 60 * 2486400000)
.commit();
```

В этом примере создается метка "end-of-quarter-Q3FY23" для снимка 8, которая должна храниться в течение суток. Чтобы прочесть метку, достаточно передать ее в вызов `useRef` при настройке сканирования таблицы:

```
// Чтение по метке
String tag = "end-of-quarter-Q3FY23";
Table tagRead = table
    .newScan()
    .useRef(tag);
То же самое можно сделать из SQL:
-- Spark SQL
-- Создать метку, хранимую 14 суток
ALTER TABLE catalog.db.closed_invoices
    CREATE TAG 'end-of-quarter-Q3FY23'
    AS OF VERSION 8
    RETAIN 14 DAYS;
```

Ветвление и маркировка вместе образуют мощную комбинацию для управления большими наборами данных. Они позволяют проводить изолированное тестирование, упрощают аудит и дают возможность воспроизводить состояние данных, имевшее место в любой момент времени. Независимо от того, реализуете ли вы требования общего регламента по защите данных (General Data Protection Regulation, GDPR) или преодолеваете сложности управления цепочками поставок, эти функции Apache Iceberg дают гибкость и контроль, необходимые для эффективных операций с данными. Однако при работе со многими таблицами лучше использовать абстракцию на уровне каталога.

Ветвление и маркировка на уровне каталога

Проект Nessie, который часто называют «Git для озер данных», предлагает мощные функции для управления таблицами Apache Iceberg, такие как ветвление и маркировка на уровне каталога. По сути, Nessie обеспечивает более организованный подход к управлению огромными объемами данных, гарантируя при этом целостность и согласованность данных.

Отличительной чертой Nessie является способность постоянно поддерживать согласованное представление данных во всех таблицах внутри каталога. Поддерживая изоляцию и независимую обработку изменений, Nessie гарантирует, что пользователи никогда не столкнутся с неполными изменениями. После внесения всех изменений их можно применить последовательно и атомарно.

Nessie упрощает отслеживание отдельных файлов данных – какие из них используются, а какие можно удалить. Это позволяет нескольким средам, таким как промышленная, обкатки, тестирования и разработки, сосуществовать в одном озере данных без ущерба для целостности промышленных данных.

Важно отметить, что Nessie преследует цель оптимизировать управление данными и исключить ненужное дублирование. Вместо копирования данных Nessie использует систему ссылок на существующие неизменяемые файлы данных. Это позволяет Nessie записывать все изменения в озеро данных в виде фиксаций без дублирования фактических данных.

Одно из важных преимуществ Nessie – поддержка управления версиями на уровне каталога, что гораздо удобнее, по сравнению с управлением версиями отдельных таблиц. При работе с несколькими таблицами управление их версиями по отдельности может превратиться в сложную и обременительную задачу. Управление версиями на уровне каталога, напротив, позволяет эффективно обрабатывать несколько таблиц одновременно, что значительно упрощает процессы управления данными.

Давайте подробнее рассмотрим, как ветвление и маркировка на уровне каталога с использованием Nessie и Apache Iceberg могут улучшить стратегии управления данными.

Ветвление каталога

Ветвление – полезная возможность, предлагаемая Project Nessie при использовании вместе с Apache Iceberg. Она обеспечивает безопасную среду для тестирования новых данных перед их включением в каталог. Создав новую ветвь, можно безопасно принимать и проверять пакеты данных в несколько таблиц без риска появления ошибочных записей в каталоге таблиц.

Представьте, что вы работаете с несколькими большими наборами данных, связанными с текущим проектом, и еженедельно получаете пакеты данных. Вместо прямого добавления этих данных в каталог можно сначала поместить их в отдельную ветвь, например с именем "weekly_ingest_branch". Такой подход позволит проверить данные перед объединением их с основной ветвью каталога.

Вот пример реализации этого рабочего процесса на Spark SQL:

```
-- Создать новую ветвь для еженедельного пакета данных
CREATE BRANCH IF NOT EXISTS weekly_ingest_branch IN catalog;

-- Переключиться на новую ветвь
USE REFERENCE weekly_ingest_branch IN catalog;

-- Принять новые данные в таблицы, принадлежащие ветви
INSERT INTO table_name (...);
```

После проверки данных их можно объединить с основной ветвью:

```
MERGE BRANCH weekly_ingest_branch INTO main IN catalog;
```

Маркировка каталога

Функция маркировки каталога в Project Nessie в сочетании с возможностями Apache Iceberg позволяет маркировать определенные версии данных, отслеживать и воспроизводить состояние данных в разные моменты времени. Это особенно полезно при проведении ежеквартальной аналитики, когда необходимо воспроизвести данные именно такими, какими они были на конец каждого квартала.

Например, предположим, что мы проводим финансовый анализ в конце каждого квартала. Создав такие метки, как `Q1_end_snapshot` и `Q2_end_snapshot`, мы сможем получить состояние данных в конце каждого квартала. Для создания таких квартальных меток можно использовать следующую команду Spark SQL:

```
-- Создать метку для конца первого квартала
CREATE TAG IF NOT EXISTS Q1_end_snapshot IN catalog;
```

Чтобы получить данные в том виде, в котором они были в конце первого квартала, можно переключиться на соответствующую метку и запросить данные:

```
-- Переключиться на снимок, соответствующий концу первого квартала
USE REFERENCE Q1_end_snapshot IN catalog;

-- Запросить данные, соответствующие концу первого квартала
SELECT * FROM table_name;
```

Ветвление и маркировка на уровне каталога дает следующие преимущества:

- обеспечивает безопасное и изолированное тестирование новых пакетов данных;
- обеспечивает простое воспроизведение данных через регулярные интервалы времени, например в конце каждого квартала, что повышает надежность аналитики;
- помогает вести контрольный журнал изменений данных с течением времени;
- помогает создавать разные версии таблицы для разных требований аналитики.

Ветвление и маркировка на уровне каталога – важнейшие функции Apache Iceberg и Project Nessie, обеспечивающие надежный и эффективный способ управления наборами таблиц данных. С помощью этих инструментов можно обеспечить точный и надежный прием данных и простое воспроизведение исторических данных для аналитики.

Многотабличные транзакции

Многотабличные транзакции – фундаментальная концепция баз данных, помогающая поддерживать согласованность и атомарность операций, охватывающих несколько таблиц. Транзакция, выполняющая несколько операций с несколькими таблицами, рассматривается как одна атомарная единица работы. Это означает, что либо все операции в транзакции завершаются успешно, либо в случае сбоя какой-либо операции все внесенные изменения откатываются и база данных остается в согласованном состоянии.

Важность многотабличных транзакций заключается в основном в их способности поддерживать согласованность данных, что является важнейшим аспектом систем управления базами данных. Рассмотрим систему управления цепочкой поставок, в которой размещение заказа включает обновление таблицы `orders` и уменьшение запасов в таблице `inventory`. Если бы эти операции не были частью одной транзакции, сбой при обновлении таблицы `inventory` после обновления таблицы `orders` мог бы привести к несогласованности данных – система отобразила бы размещенный заказ, но объем запасов не отражал бы фактическое состояние дел.

При использовании многотабличных транзакций весь процесс рассматривается как одна атомарная операция. Это означает, что если обновление таблицы `inventory` не удастся, то изменения, внесенные в таблицу `orders`, будут отменены, что обеспечит согласованность данных.

Кроме того, многотабличные транзакции необходимы также для изоляции – еще одного ключевого свойства надежных систем баз данных. Они гарантируют отсутствие взаимовлияния параллельных транзакций друг на друга и, соответственно, появления потенциальных несогласованностей и конфликтов данных.

Таким образом, многотабличные транзакции жизненно важны для поддержания согласованности и изоляции данных. Они позволяют рассматривать несколько операций, возможно с участием разных таблиц, как единую атомарную единицу работы и гарантируют, что либо все операции выполнятся успешно, либо, в случае какого-либо сбоя, все изменения будут отменены. Это особенно важно в сложных системах, таких как управление цепочками поставок, где целостность и согласованность данных имеют первостепенное значение.

Многотабличные транзакции можно реализовать с помощью каталога Project Nessie, применив ветвление на уровне каталога и затем выполнив транзакции, охватывающие несколько таблиц из ветви. Все операции из ветки, независимо от того, какие таблицы, механизмы или пользователи вовлечены в транзакцию, будут изолированы от основной ветви.

После завершения транзакции, если вас устраивает состояние ветви и всех таблиц в ней, вы можете объединить эти изменения с основной ветвью, как показано в следующем коде. Если вас что-то не устраивает, то вы можете просто удалить ветвь, и никто больше не увидит никаких изменений:


```
-- Создать ветвь
CREATE BRANCH IF NOT EXISTS etl IN catalog;

-- Переключиться на вновь созданную ветвь (Spark SQL)
USE REFERENCE ingest IN catalog;

-- Запустить многотабличную транзакцию
INSERT INTO catalog.db.tableA ...;
INSERT INTO catalog.db.tableB ...;

-- По завершении объединить все изменения с основной ветвью
MERGE BRANCH etl INTO main IN catalog;
```

Откат изменений

Откат на уровне каталога и таблицы – это важнейшая концепция управления данными, которая в значительной степени способствует поддержанию высокого качества данных.

Откат на уровне таблицы – это особый метод управления данными, позволяющий отменить изменения в таблице, т. е. «откатить» таблицу до предыдущего состояния. Это особенно полезно в ситуациях, когда в процессе обработки данных произошла ошибка или получились неожиданные результаты. Выполняя откат, можно вернуть данные в заведомо непротиворечивое состояние и тем самым смягчить влияние ошибок или проблемных изменений.

В совокупности откаты на уровне каталога и таблицы обеспечивают защиту от ошибок и изменений, которые могут отрицательно повлиять на качество данных. Такое сочетание упрощает поддержание данных в непротиворечивом и согласованном состоянии, что особенно важно в областях с интенсивным использованием данных, таких как наука о данных, машинное обучение и бизнес-аналитика. Качество данных напрямую влияет на надежность выводов, что делает эти концепции критически важными для принятия точных и надежных решений.

Откат на уровне таблицы

Откаты на уровне таблицы, находящейся в нежелательном состоянии из-за неправильного выполнения задания приема, позволяют отменить произведенные изменения. Эта возможность имеет решающее значение в сценариях, когда произошло ошибочное обновление данных и необходимо вернуть таблицу в предыдущее согласованное состояние. Apache Iceberg имеет четыре процедуры Spark для управления текущим состоянием таблицы: `rollback_to_snapshot`, `rollback_to_timestamp`, `set_current_snapshot` и `cherrypick_snapshot`. Таблицы метаданных – отличный инструмент для определения снимка, к ко-

торому можно вернуться, и для этого можно использовать многие запросы к таблицам метаданных, обсуждавшиеся выше.

rollback_to_snapshot

Apache Iceberg предоставляет процедуру `rollback_to_snapshot` для отката таблицы по идентификатору снимка. Чтобы запустить процедуру `rollback_to_snapshot`, необходимо передать ей два аргумента: имя таблицы и идентификатор снимка, к которому нужно откатиться. Например, вот как можно откатить таблицу `orders` к снимку 12345:

```
spark.sql("CALL catalog.database.rollback_to_snapshot('orders', 12345)")
```

Эта процедура изменит текущее состояние таблицы, приведя его в соответствие с указанным снимком. Метаданные и файлы данных таблицы и ее снимка останутся неизменными, т. е. эта операция является неразрушающей. Она разработана так, чтобы ее можно было безопасно выполнять одновременно с другими операциями в таблице без возникновения любых потенциальных конфликтов.

Для примера рассмотрим ситуацию, когда вы внесли некоторые изменения в таблицу `orders` и поняли, что ошибка в логике обновления привела к появлению неверных значений в некоторых записях. При наличии снимка таблицы, созданного до ошибочного обновления, можно откатить изменения до этого снимка и восстановить правильное состояние данных:

```
# Пусть до ошибочного изменения был создан снимок 12345
spark.sql("CALL catalog.database.rollback_to_snapshot('orders', 12345)")
```

Поведение операций отката можно дополнительно настроить, изменив определенные параметры в Apache Iceberg. Например, можно свойству `rollback_to_snapshot.expire_snapshots.enabled` присвоить значение `false`, чтобы предотвратить автоматическое удаление после отката снимков, возраст которых превысил предельное значение. Также для управления удалением снимков после отката можно установить свойство `rollback_to_snapshot.expire_snapshots.snapshot_age_ms`:

```
spark.sql("""
CALL catalog.database.alter_table_properties(
  'orders',
  map(
    'rollback_to_snapshot.expire_snapshots.enabled', 'false',
    'rollback_to_snapshot.expire_snapshots.snapshot_age_ms', '172800000'
    -- 2 дня в миллисекундах
  )
)
""")
```

Этот параметр гарантирует, что снимки старше одного дня не будут удаляться автоматически после отката, что гарантирует возможность вернуться к этим более старым снимкам, если потребуется.

rollback_to_timestamp

Иногда точный идентификатор снимка неизвестен, но известен момент времени, и именно в таких случаях процедура `rollback_to_timestamp` становится очень полезной. Этой процедуре в аргументах передаются имя таблицы и отметка времени для отката, а она делает недействительными все кешированные планы Spark, которые ссылаются на затронутую таблицу, и тем самым гарантирует, что все последующие операции будут применяться к обновленному состоянию таблицы. В виде результата эта процедура возвращает текущий идентификатор снимка до отката (`previous_snapshot_id`) и новый текущий идентификатор снимка (`current_snapshot_id`).

Рассмотрим сценарий, когда имеется таблица с именем `orders` и нужно откатить ее до состояния, которое было текущим на определенную отметку времени, скажем 01.06.2023 00:00:00. В Apache Spark для этого можно использовать оператор `CALL` с процедурой `rollback_to_timestamp`:

```
// Код на языке Scala

// Использовать процедуру отката таблицы
spark.sql(s"CALL iceberg.system.rollback_to_timestamp('db.orders',
timestamp('2023-06-01 00:00:00'))")
```

Эта инструкция применит процедуру `rollback_to_timestamp` к таблице `orders`, чтобы выполнить откат к снимку, который был текущим на 01.06.2023 00:00:00. Важно отметить, что для выполнения этой операции необходимы соответствующие разрешения.

set_current_snapshot

Процедура `set_current_snapshot` устанавливает идентификатор текущего снимка таблицы. В отличие от процедур отката, она устанавливает для таблицы текущим не снимок из ее истории, а любой произвольный доступный снимок, который может находиться в другой ветви. Это позволяет пользователям переключаться между разными версиями таблицы, даже если они не связаны непосредственно. Процедуре `set_current_snapshot` необходимо передать два аргумента: имя таблицы и идентификатор снимка, который должен быть назначен «текущим». В качестве результата возвращаются те же значения, которые возвращает `rollback_to_timestamp`: идентификатор снимка, который был текущим до изменения, и новый идентификатор текущего снимка.

Предположим, у вас есть таблица с именем `inventory`, и вы хотите назначить ей текущим снимок с определенным идентификатором; например, 123456789. В Apache Spark с Iceberg для этого можно использовать оператор `CALL` с процедурой `set_current_snapshot`:

```
// Код на языке Scala

// Выбрать таблицу
```

```
val tableName = "db.inventory"
```

```
// Указать идентификатор нового текущего снимка  
val snapshotId = 123456789L
```

```
// Использовать процедуру для назначения снимка текущим  
spark.sql(s"CALL iceberg.system.set_current_snapshot('$tableName', $snapshotId)")
```

Эта инструкция применит процедуру `set_current_snapshot` к таблице `inventory`, чтобы назначить для нее текущим снимок с идентификатором 123456789. Имейте в виду, что для выполнения этой операции необходимы соответствующие разрешения.

cherrypick_snapshot

Процедура `cherrypick_snapshot` создает новый снимок, включающий только изменения в метаданных в другом снимке (новые файлы данных не создаются). Процедура `cherrypick_snapshot` принимает два аргумента: имя обновляемой таблицы, а также идентификатор снимка, на основе которого должна обновиться таблица. Она возвращает идентификаторы снимков до и после выполнения процедуры `cherrypick_snapshot`: `source_snapshot_id` и `current_snapshot_id`.

Например, предположим, у вас есть таблица с именем `products` и вы хотите выбрать изменения из снимка с идентификатором 987654321. В Apache Spark с Iceberg для этого можно использовать оператор `CALL` с процедурой `cherrypick_snapshot`:

```
// Код на языке Scala
```

```
// Использовать процедуру, чтобы выбрать изменения из снимка  
spark.sql(s"CALL iceberg.system.cherrypick_snapshot('db.products', 987654321)")
```

Эта инструкция применяет процедуру `cherrypick_snapshot` к таблице `products`, чтобы выбрать изменения из снимка с идентификатором 987654321.

Описанные выше процедуры – это мощные инструменты управления состоянием таблиц Apache Iceberg. Они обеспечивают гибкое управление версиями таблиц, позволяя пользователям откатывать изменения, назначать определенный снимок текущим или выбирать изменения из одного снимка в другой. Все вместе они обеспечивают эффективное управление версиями данных и поддержку анализа истории, которые необходимы в современных рабочих процессах обработки данных и аналитики.

Откат на уровне каталога

Одно из ключевых преимуществ Nessie в роли каталога Apache Iceberg заключается в возможности отката данных на уровне каталога. Подобно системам управления версиями, которые позволяют разработчикам откатывать изменения в кодовой базе до предыдущей версии, Nessie дает возможность

инженерам и специалистам по данным откатить среду данных до некоторого состояния в прошлом. Эта функциональность невероятно ценна в самых разных сценариях.

Например, представьте сценарий, в котором инженер по обработке данных запускает пакетное задание, изменяющее большое количество наборов данных, но позже понимает, что в логику изменения вкралась ошибка. Без такого инструмента, как Nessie, исправление подобной ошибки может потребовать поочередного отката каждой таблицы Iceberg. Однако с помощью Nessie инженер может откатить сразу весь каталог и мгновенно устранить ошибки во всех таблицах.

Откат в Nessie можно выполнить с помощью SQL-запроса в интеграции с такими инструментами, как Apache Hive, Apache Spark и механизм обработки SQL-запросов Dremio. Вот, например, как можно откатить данные с помощью Project Nessie. Сначала нужно найти хеш фиксации, к которой следует откатиться. Это можно сделать с помощью команды `SHOW LOG`:

```
SHOW LOG nessie.main
```

Этот запрос выведет список всех фиксаций в основной ветви вместе с их хешами. Выяснив хеш фиксации, к которой нужно вернуться, можно использовать команду `SET REF`, чтобы установить ссылку в заголовке ветви:

```
SET REF nessie.main TO 'commitHash'
```

Эта команда запишет в заголовок основной ветви хеш выбранной фиксации, выполненной до плохой транзакции, откатив тем самым все изменения, внесенные после нее.

Имейте в виду, что эта команда не вносит необратимых изменений; изменения действуют только в рамках текущего сеанса. Чтобы сделать откат постоянным, вместо `SET REF` следует использовать команду `ASSIGN`:

```
ASSIGN nessie.main AT 'commitHash'
```

Эта команда навсегда откатит основную ветвь до указанной фиксации, и все пользователи будут видеть данные такими, какими они были после нее.

Nessie предлагает мощный инструмент управления данными, позволяющий откатывать изменения на уровне каталога. Это упрощает восстановление после ошибок, облегчает экспериментирование с процедурами преобразования данных и упрощает поддержание согласованности в различных средах. Откат на уровне каталога – отличное решение, когда восстановление данных в промышленной среде должно происходить быстро и во многих таблицах сразу.

Заключение

Возможности, предоставляемые таблицами метаданных Apache Iceberg, ветвлением и маркировкой таблиц Iceberg, многотабличными транзакция-

ми и функцией отката таблиц Iceberg, играют фундаментальную роль в эффективном управлении промышленными данными, позволяя использовать и проактивные, и реактивные подходы. Фактически таблицы метаданных Apache Iceberg, предлагая пользователям полное представление о ландшафте данных и отслеживая изменения в схеме, являются неотъемлемой частью проактивного и реактивного управления данными. Доступность этой важной информации позволяет принимать обоснованные решения, оптимизировать архитектуру данных и предвидеть будущие потребности.

Ветвление и маркировка еще больше способствуют проактивному управлению, предлагая возможность создания безопасных окружений, где можно тестировать изменения или новые функции без опаски повредить промышленные данные и вести разработку. Эти функции также помогают поддерживать управление версиями и облегчают воспроизводимость результатов экспериментов.

Многотабличные транзакции обеспечивают более высокий уровень контроля и согласованности. Они гарантируют выполнение связанных изменений в нескольких таблицах как одной атомарной операции, предотвращая неполные обновления и обеспечивая целостность среды данных.

Наконец, функция отката таблиц Iceberg являет собой важный механизм восстановления в случае ошибок или непредвиденных последствий изменений. Она дает возможность возврата к предыдущему состоянию, что делает ее ценным инструментом снижения рисков и обеспечения целостности данных в стратегии реактивного управления данными.

Все вместе эти функции образуют надежную основу для эффективного управления производственными данными и позволяют сочетать долгосрочные стратегии с гибкими мерами поддержания работоспособности, целостности и эффективности экосистемы данных.

В главе 11 мы рассмотрим способы обработки потоковых данных с помощью Apache Iceberg.

Глава 11

Потоковая обработка в Apache Iceberg

Потоковая обработка данных подразумевает непрерывное получение и обработку данных, часто из разных источников. В роли источников могут выступать, например, файлы журналов, данные с датчиков, информация из каналов социальных сетей и финансовые транзакции. Данные отправляются небольшими порциями (или пакетами), чтобы обеспечить возможность анализа и реагирования в режиме реального времени. Особенность потоковой обработки данных заключается в том, что они находятся в постоянном движении и не имеют начала или конца.

Концепция потоковой обработки данных имеет важное значение в век цифровой информации, когда предприятиям, исследовательским и правительственным учреждениям приходится анализировать и принимать решения на основе самых свежих данных. Например, финансовые учреждения могут использовать потоковые данные для обнаружения мошеннических транзакций по мере их совершения. Аналогично платформы социальных сетей используют потоковые данные для настройки и обновления пользовательских каналов в реальном времени на основе показателей вовлеченности.

Существует несколько причин, почему может потребоваться потоковая передача данных в таблицу Apache Iceberg:

Масштабируемость и производительность

Apache Iceberg обеспечивает эффективное хранение и извлечение информации из больших наборов данных. Процедуры управления файлами позволяют оптимизировать производительность постоянно меняющегося/растущего набора данных, что делает Apache Iceberg отличным выбором для потоковой аналитики.

Эволюция схемы

Поскольку данные со временем меняются, их структура (схема) тоже может нуждаться в развитии. Apache Iceberg позволяет изменять схему, не

прерывая текущих процессов потоковой обработки данных, что упрощает адаптацию к меняющимся требованиям.

Надежность

Apache Iceberg обеспечивает изоляцию снимков, а это означает, что каждая транзакция работает с неизменяющимся снимком таблицы. Эта функция обеспечивает согласованность и надежность данных даже в средах, где одновременно выполняется множество операций.

Путешествие во времени

Iceberg хранит полную историю метаданных таблицы, что позволяет выполнять запросы к историческим данным и предыдущим версиям таблицы.

Передавая потоки данных в таблицы Apache Iceberg, организации могут более эффективно и гибко управлять и анализировать их в реальном времени, адаптируясь к изменениям и поддерживая надежность анализа данных.

Потоковая обработка с помощью Spark

Способность обрабатывать потоковые данные – одно из основных достоинств Apache Spark. Управление обработкой осуществляется с помощью Spark Streaming, компонента Spark, который обеспечивает масштабируемую, высокопроизводительную и отказоустойчивую обработку потоков данных в реальном времени. Вот некоторые ключевые особенности Spark Streaming:

Отказоустойчивость

Отказоустойчивость Spark Streaming обеспечивается встроенными механизмами восстановления. Если узел выходит из строя во время вычислений, то система может быстро восстановиться и продолжить обработку.

Интеграция

Spark Streaming легко интегрируется с другими компонентами Spark, такими как Spark MLlib и Spark SQL, обеспечивая мощные варианты совместного их использования. Например, с помощью Spark можно использовать MLlib для создания модели машинного обучения на большом наборе данных, а затем применить эту модель к действующему потоку данных с помощью Spark Streaming.

Обработка в реальном времени

Spark Streaming может обрабатывать потоки данных в режиме реального времени. Для этого входные данные делятся на пакеты, которые затем обрабатываются механизмом Spark для формирования окончательного потока результатов.

Оконные операции

Spark Streaming поддерживает оконные вычисления, когда преобразования устойчивых распределенных наборов данных (Resilient Distributed

Dataset, RDD) применяются к скользящему окну данных. Это полезно во многих сценариях, например при вычислении тенденций в потоке данных за последние несколько часов.

Высокая пропускная способность

Spark Streaming создавался для обработки больших объемов данных, что позволяет с успехом использовать этот механизм для обработки больших объемов потоковых данных в реальном времени.

Несколько источников данных

Spark Streaming может принимать данные из самых разных источников, включая Kafka, Flume, Kinesis и др.

Подход к обработке данных небольшими пакетами отличает Apache Spark от других механизмов потоковой обработки и делает его привлекательным выбором для обработки данных в реальном времени. Этот подход предполагает обработку данных небольшими дискретными пакетами, а не каждой точки отдельно. Это проектное решение обусловлено поиском компромисса между такими факторами, как задержка, пропускная способность и стоимость.

Разбивая данные на микропакеты, Apache Spark получает ряд преимуществ. Во-первых, этот подход повышает отказоустойчивость, потому что упрощается обработка сбоя в каждом пакете. Во-вторых, облегчает интеграцию с другими компонентами Spark, образующими единую экосистему, способную обрабатывать данные как пакетами, так и в реальном времени, потому что микропакеты обрабатываются непрерывно и очень быстро. Наконец, он обеспечивает высокую пропускную способность, эффективно обрабатывая большие объемы данных.

В целом подход Apache Spark к обработке на основе микропакетов обеспечивает хороший баланс между оперативностью, обработкой больших объемов данных и эффективностью использования ресурсов, что делает его предпочтительным выбором для потоковых приложений и приложений больших данных.

В следующем разделе мы рассмотрим пример использования Apache Spark Streaming для приема данных из Kafka в Apache Iceberg.

Потоковая передача данных в Iceberg

Использование Spark DataSourceV2 API позволяет специалистам по обработке данных читать и записывать таблицы структурированным и масштабируемым образом.

В Spark Structured Streaming – API потоковой передачи Spark на основе SQL – Iceberg полностью совместим с DataFrame API начиная с версии Spark 3. Ключевой особенностью этой интеграции является поддержка обработки инкрементных данных, которая начинается с исторической отметки времени. В контексте потокового чтения из таблицы Iceberg поддерживается только добавление в снимки.

Для потоковой записи данных в таблицу Iceberg используется Spark `DataStreamWriter`. Iceberg поддерживает два режима вывода: добавление, когда в таблицу последовательно добавляются записи из каждого микропакета; и полный режим, когда содержимое таблицы заменяется каждым полученным микропакетом.

При записи в секционированную таблицу Iceberg требует сортировки данных с учетом спецификаций секционирования. Для пакетных запросов рекомендуется явная сортировка, но она может оказаться неприемлемой для потоковой передачи из-за возникающих задержек. Чтобы решить эту проблему, Iceberg предоставляет возможность записи вразброс, которая устраняет необходимость сортировки и тем самым уменьшает задержку. Запись вразброс – это метод оптимизации производительности.

Поддержка потоковых таблиц в Iceberg имеет решающее значение. Поточковые запросы могут быстро создавать новые версии таблиц, генерируя значительный объем метаданных таблиц и множество небольших файлов данных. Для эффективного управления метаданными рекомендуются такие стратегии, как настройка частоты фиксаций, удаление старых снимков, уплотнение файлов данных и перезапись манифестов.

Настройка частоты фиксаций помогает контролировать количество файлов данных, манифестов и снимков, упрощая обслуживание таблиц. Если не принять соответствующие меры, то может произойти быстрое накопление старых снимков, поэтому их необходимо регулярно удалять. Уплотнение файлов данных помогает уменьшить объем метаданных и повысить эффективность запросов, а перезапись манифестов оптимизирует задержку записи и уплотняет небольшие файлы манифестов. Все эти операции по обслуживанию и оптимизации таблиц подробно описаны в главе 4.

Рассмотрим пример потоковой передачи финансовых данных в таблицы Apache Iceberg с помощью Spark Structured Streaming API, который позволяет непрерывно собирать данные, транзакции и обновления портфеля в режиме реального времени. Для примера используем гипотетические финансовые данные. Предположим, что мы читаем данные из темы Kafka и записываем их в таблицу Iceberg.

Сначала используем код на Scala для настройки сеанса Spark:

```
val spark = SparkSession.builder()
    .appName("FinancialDataStreaming")
    .getOrCreate()
```

Предположим, что у нас есть тема Kafka с именем `financialData`, где каждое сообщение представляет собой блок финансовых данных в формате JSON, содержащий следующие поля: `timestamp`, `symbol`, `price` и `volume`. Мы используем метод `readStream` для чтения из Kafka:

```
val df = spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "localhost:9092")
```

```
.option("subscribe", "financialData")
.load()
```

Затем преобразуем поток Kafka в коллекцию DataFrame объектов Stock:

```
val financialData = df.selectExpr("CAST(value AS STRING)").as[String]
.map(Stock.from_json(_))
```

Класс Stock можно определить следующим образом:

```
case class Stock(timestamp: Long, symbol: String, price: Double, volume: Long)
object Stock {
  def from_json(jsonString: String): Stock = {
    val parser = new ObjectMapper()
    parser.registerModule(DefaultScalaModule)
    parser.readValue(jsonString, classOf[Stock])
  }
}
```

Теперь, имея поток данных, мы можем начать добавлять данные в таблицу Iceberg. Используем для этого режим добавления, потому что добавление новых данных не должно затрагивать существующие:

```
val tableIdentifier = "s3://someBucket/financial_data/stock"

financialData.writeStream
  .format("iceberg")
  .outputMode("append")
  .trigger(Trigger.ProcessingTime(1, TimeUnit.MINUTES))
  .option("path", tableIdentifier)
  .option("checkpointLocation", "/tmp/checkpoints")
  .start()
```

Если таблица Iceberg секционирована, скажем по полю `symbol`, то мы должны включить параметр `fanout-enabled`, чтобы исключить необходимость сортировки данных:

```
financialData.writeStream
  .format("iceberg")
  .outputMode("append")
  .trigger(Trigger.ProcessingTime(1, TimeUnit.MINUTES))
  .option("path", tableIdentifier)
  .option("fanout-enabled", "true")
  .option("checkpointLocation", "/tmp/checkpoints")
  .start()
```

Наконец, учитывая быстрый рост объема данных, необходимо позаботиться об обслуживании таблицы, настроив частоту фиксаций, удаление старых снимков, уплотнение файлов данных и переписывание манифестов. Это обеспечит хорошую производительность и управляемость таблицы. Сочетание этих стратегий гарантирует надежность и устойчивость платформы потоковой обработки данных.

Потоковая передача данных из Iceberg

В отличие от приема потоковых данных в таблицы Iceberg, нацеленного на получение данных в реальном времени, потоковая передача из таблиц Iceberg предполагает чтение этих данных для передачи последующим приложениям и аналитическим задачам. Использование Spark Structured Streaming API позволяет эффективно читать и обрабатывать данные, хранящиеся в таблицах Iceberg.

DataSourceV2 API в Spark 3 поддерживает обработку дополнительных данных начиная с исторической отметки времени, как показано в следующем коде:

```
val df = spark.readStream
    .format("iceberg")
    .option("stream-from-timestamp", Long.toString(streamStartTimestamp))
    .load(tableName)
```

Говоря об операциях потокового чтения из таблиц Iceberg, важно отметить, что Iceberg поддерживает только чтение данных из добавленных снимков. Более того, последующие приложения должны быть корректно спроектированы для обработки данных в реальном времени, получаемых из Iceberg. Такие приложения могут решать аналитические задачи в реальном времени, выдавать оповещения или производить дальнейшую обработку информации.

Обмен данными в реальном времени является самым распространенным вариантом использования в современных интегрированных бизнес-средах. Когда обе сущности используют таблицы Iceberg, процесс можно оптимизировать с помощью Spark Structured Streaming. Рассмотрим на примере, как этого можно достичь.

Предположим, вы хотите поделиться своей таблицей Iceberg `our_database.our_table` с партнерами, которые хотят, чтобы данные передавались в их таблицу Iceberg `partner_database.partner_table`. Сначала вы должны настроить сеанс Spark для чтения из своей таблицы:

```
val spark = SparkSession.builder()
    .appName("IcebergDataSharing")
    .getOrCreate()
```

Затем инициировать поток чтения:

```
val ourDF = spark.readStream
    .format("iceberg")
    .option("stream-from-timestamp", Long.toString(streamStartTimestamp))
    .load(sourceTableName)
```

И осуществить запись потока в таблицу Iceberg партнера. Учитывая особенности потоковых данных, запись будет производиться в режиме добавления:

```
ourDF.writeStream
  .format("iceberg")
  .outputMode("append")
  .trigger(Trigger.ProcessingTime(1, TimeUnit.MINUTES))
  .option("path", destinationTableName)
  .option("checkpointLocation", checkpointPath)
  .start()
```

Если таблица Iceberg партнера секционирована, например, по столбцу `region`, то необходимо оптимизировать запись данных:

```
ourDF.writeStream
  .format("iceberg")
  .outputMode("append")
  .trigger(Trigger.ProcessingTime(1, TimeUnit.MINUTES))
  .option("path", destinationTableName)
  .option("fanout-enabled", "true")
  .option("checkpointLocation", checkpointPath)
  .start()
```

Чтобы добиться бесперебойного и эффективного обмена данными с партнерами в режиме реального времени, важно придерживаться определенных практик. Во-первых, огромное значение имеет согласованность схемы: чтобы предотвратить ошибки записи, схема `our_database.our_table` должна совпадать или быть совместимой со схемой `partner_data base.partner_table`.

В случаях, когда партнера интересует только часть данных, следует реализовать фильтрацию данных перед записью в таблицу. Это не только упростит процесс, но и повысит его эффективность. Также должны быть реализованы надежные механизмы обработки ошибок, возникающих при потоковой передаче, чтобы предотвратить потерю или дублирование данных. Наконец, жизненно важно настроить обмен оповещениями при потоковой передаче.

Такой подход поможет следить за эффективностью работы, быстро выявлять потенциальные проблемы и гарантировать своевременное предупреждение заинтересованных сторон в случае проблем. Следование этим практикам поможет укрепить сотрудничество, и обе стороны получают выгоду от доступа к своевременной и точной информации.

Потоковая обработка с помощью Flink

Apache Flink – универсальный механизм с открытым исходным кодом для потоковой и пакетной обработки данных, разработанный Apache Software Foundation. Flink предназначен для обработки больших объемов данных на высоких скоростях, что делает его особенно полезным для обработки и анализа данных в реальном времени.

Flink поддерживает потоковую передачу с высокой пропускной способностью и малой задержкой, а также семантику времени события и «точно

один раз», управление противодавлением и многое другое. Он прекрасно подходит для обработки потоковых и пакетных данных, имеет средства машинного обучения и обработки графов. Вот некоторые ключевые особенности поддержки потоковой обработки данных в Apache Flink:

Обработка с учетом времени события

Поддержка обработки событий с учетом их времени позволяет обрабатывать неупорядоченные данные и предоставлять точные результаты, что очень важно для многих приложений реального времени. Эта особенность особенно полезна в ситуациях, когда порядок событий имеет значение, например в финансовых транзакциях и обработке данных датчиков.

Отказоустойчивость

Как и в Spark, отказоустойчивость в Flink обеспечивается с помощью механизма контрольных точек и точек сохранения, которые предотвращают потерю данных во время обработки и позволяют восстанавливать данные после сбоев.

Обработка противодавления

Flink обрабатывает противодавление, которое возникает, когда данные генерируются быстрее, чем потребляются, обеспечивая стабильность системы даже при большом потоке данных.

Высокая пропускная способность и низкая задержка

Flink создавался для обработки больших объемов данных с низкой задержкой, что делает его хорошим выбором для приложений, осуществляющих анализ данных в реальном времени.

Оконная обработка и обработка сложных событий

Flink обеспечивает гибкое управление окнами в зависимости от времени событий, количества данных, сеанса и т. д. Кроме того, он поддерживает обработку сложных событий (Complex Event Processing, CEP), помогая обнаруживать закономерности в потоках данных.

Интеграция

Flink интегрируется с различными источниками и приемниками данных, включая Kafka, распределенную файловую систему Hadoop (HDFS) и базы данных, такие как Cassandra.

Семантика «точно один раз»

Flink поддерживает семантику однократной обработки, гарантируя, что каждая запись будет обработана точно один раз, и тем самым обеспечивая точные результаты.

Проще говоря, Apache Flink – мощная технология потоковой обработки, прекрасно подходящая для высокоскоростных, точных и масштабируемых решений обработки данных. Сочетание надежности потоковой передачи, отказоустойчивости и интеграции с другими системами делает Apache Flink отличным выбором для обработки данных в реальном времени.

Потоковая передача данных в Iceberg

Библиотеки Apache Iceberg Flink позволяют использовать таблицы Apache Iceberg и как источники, и как приемники данных. Сначала рассмотрим прием данных в таблицы Iceberg, а затем обсудим передачу из Iceberg.

Flink DataStream API и Flink Table API

Фреймворк Apache Flink предлагает два основных API для обработки данных: DataStream API и Table API, каждый из которых отвечает разным потребностям.

DataStream API предназначен для обработки непрерывных, неограниченных потоков данных, таких как данные с датчиков (например, из интернета вещей) или потоки журналов, и предлагает низкоуровневый подход с поддержкой оконных операций и обработкой времени событий. Он идеально подходит для сценариев реального времени, управляемых событиями, и предлагает широкие возможности подключения.

Table API, напротив, ориентирован на работу со структурированными данными ограниченного объема в формате таблиц и лучше всего подходит для пакетной обработки и аналитических задач. Он обеспечивает высокоуровневую абстракцию с SQL-подобным интерфейсом, упрощая выражение задач обработки данных и автоматизируя оптимизацию плана выполнения. Этот API тоже поддерживает обработку времени событий, но возможности обработки сложных событий в нем более ограничены, чем DataStream API.

Выбор между этими API зависит от конкретных требований: DataStream API лучше подходит для приложений реального времени, а Table API – для обработки структурированных данных и аналитических задач.

Потоковое чтение с Flink

Apache Iceberg предлагает надежную поддержку операций потокового и пакетного чтения с использованием Flink DataStream API и Flink Table API. Эта поддержка позволяет легко и просто читать данные из таблиц Iceberg.

В контексте SQL задания Flink могут переключаться между потоковым и пакетным режимами с помощью простых команд. Пакетное чтение может выполняться с помощью инструкции SELECT в пакетном задании после установки пакетного режима выполнения. Для потокового чтения сначала нужно установить потоковый режим выполнения и включить параметры динамической таблицы. Затем можно использовать SELECT с параметром `streaming`, установленным в значение `true`, и заданным интервалом мониторинга.

Рассмотрим пример использования пакетного режима. Сначала установим пакетный режим выполнения, а затем выполним инструкцию SELECT для чтения данных из таблицы `sales_data`:

```
TableEnvironment tableEnv = TableEnvironment
    .create(EnvironmentSettings.newInstance().build());
```

```
// Установить пакетный режим выполнения.
tableEnv.getConfig().getConfiguration()
    .setString("execution.runtime-mode", "batch");

// Выполнить инструкцию SELECT.
TableResult result = tableEnv.executeSql("SELECT * FROM sales_data");
result.print();
```

Вот пример использования потокового режима для потоковой передачи данных. Сначала установим потоковый режим выполнения и включим параметры динамической таблицы, а затем выполним инструкцию SELECT с параметром `streaming`, установленным в значение `true`, и заданным интервалом мониторинга:

```
// Установить потоковый режим выполнения
// и включить параметры динамической таблицы.
tableEnv.getConfig().getConfiguration()
    .setString("execution.runtime-mode", "streaming");
tableEnv.getConfig().getConfiguration()
    .setBoolean("table.dynamic-table-options.enabled", true);

// Выполнить инструкцию SELECT с параметром потоковой передачи
// и интервалом мониторинга, равным 1 с.
TableResult result = tableEnv.executeSql(
    "SELECT * FROM sales_data /*+ OPTIONS('streaming'='true',
    'monitor-interval'='1s')*/");
result.print();
```

Iceberg даже поддерживает чтение инкрементальных данных, начиная с исторического снимка с заданным идентификатором. Пользователи могут манипулировать различными параметрами Flink SQL для настройки потоковой передачи, такими как параметр `start-snapshot-id`, использование которого показано в следующем коде. С помощью шаблона «Подсказки» (Hints), реализованного во многих программных фреймворках, можно отправлять указания через комментарии, которые обрабатываются фреймворком перед выполнением кода:

```
TableEnvironment tableEnv = TableEnvironment
    .create(EnvironmentSettings.newInstance().build());

// Установить потоковый режим выполнения
// и включить параметры динамической таблицы.
tableEnv.getConfig().getConfiguration()
    .setString("execution.runtime-mode", "streaming");
tableEnv.getConfig().getConfiguration()
    .setBoolean("table.dynamic-tableoptions.enabled", true);

// Указать идентификатор снимка, с которого должно начаться чтение.
String startSnapshotId = "3821550127947089987"; // Замените
```



```
// своим фактическим идентификатором снимка.

// Выполнить инструкцию SELECT с параметром потоковой передачи
// и интервалом мониторинга, равным 1 с.
// Начать чтение инкрементальных данных из
// снимка с указанным идентификатором.
TableResult result = tableEnv.executeSql(
    "SELECT * FROM sales_data /*+ OPTIONS('streaming'='true',
      'monitor-interval'='1s', 'start-snapshot-id'='"
      + startSnapshotId + "')*/");
result.print();
```

В контексте DataStream API Iceberg поддерживает чтение как в пакетном, так и в потоковом режиме, что мы проиллюстрируем в следующих примерах. Для настройки потокового чтения выполняются аналогичные шаги, но с параметром `streaming`, установленным в значение `true`, и с идентификатором снимка в параметре `startSnapshotId`.

Для пакетных данных:

```
StreamExecutionEnvironment env = StreamExecutionEnvironment
    .createLocalEnvironment();
TableLoader tableLoader = TableLoader.fromHadoopTable(
    "s3://someBucket/retail/sales_data");

IcebergSource<RowData> source = IcebergSource.forRowData()
    .tableLoader(tableLoader)
    .assignerFactory(new SimpleSplitAssignerFactory())
    .build();

DataStream<RowData> batch = env.fromSource(
    source,
    WatermarkStrategy.noWatermarks(),
    "My Iceberg Source",
    TypeInformation.of(RowData.class));

// Вывести все записи.
batch.print();

// выполнить пакетное задание чтения.
env.execute("Iceberg Batch Read");
```

Здесь для пакетного чтения всех записей из таблицы Iceberg используются Java-классы `TableLoader` и `FlinkSource`, а полученные данные выводятся в стандартный вывод.

Для потоковых данных:

```
StreamExecutionEnvironment env = StreamExecutionEnvironment
    .createLocalEnvironment();
TableLoader tableLoader = TableLoader.fromHadoopTable(
    "s3://someBucket/retail/sales_data");
```

```
IcebergSource source = IcebergSource.forRowData()
    .tableLoader(tableLoader)
    .assignerFactory(new SimpleSplitAssignerFactory())
    .streaming(true)
    .streamingStartingStrategy(
        StreamingStartingStrategy.INCREMENTAL_FROM_LATEST_SNAPSHOT)
    .monitorInterval(Duration.ofSeconds(60))
    .build()

DataStream<RowData> stream = env.fromSource(
    source,
    WatermarkStrategy.noWatermarks(),
    "My Iceberg Source",
    TypeInformation.of(RowData.class));

// Вывести все записи
stream.print();

// выполнить потоковое задание чтения.
env.execute("Iceberg Stream Read");
```

Для потокового чтения выполняются аналогичные шаги, но с параметром `streaming`, установленным в значение `true`, а в параметре `startSnapshotId` указывается идентификатор снимка.

Ветви и теги можно читать с помощью `DataStream` API. Для этого нужные параметры настраиваются в методе `branch` или `tag` объекта `FlinkSource.Builder`. Имеется также ряд параметров, которые можно настроить с помощью `IcebergSource` или подсказок в Flink SQL, что позволяет точно контролировать операции чтения:

```
DataStream<RowData> batch = FlinkSource.forRowData()
    .env(env)
    .tableLoader(tableLoader)
    .branch("etl-branch")
    .streaming(false)
    .build();
```

Потоковая запись с Flink

Потоковая запись в Apache Iceberg с помощью `DataStream` API и `Table` API обеспечивает бесперебойную поддержку приема и обработки данных в пакетном и потоковом режимах, что делает эти API универсальным и мощным решением для реализации конвейеров потоковой передачи данных.

Flink SQL API в Iceberg поддерживает операции `INSERT INTO` и `INSERT OVERWRITE`. Первая используется в заданиях потоковой передачи для добавления новых данных в таблицу Iceberg, а вторая – в пакетных заданиях для замены данных в таблице. Iceberg гарантирует атомарное выполнение замены, что делает обновление данных безопасным и надежным.

Как работает INSERT OVERWRITE?

Инструкция `INSERT OVERWRITE` заменяет существующие данные в таблице результатами запроса или новыми данными и обычно используется для пакетной обработки таблиц Iceberg. Эта атомарная операция заполняет таблицу последними данными в таких сценариях, как периодическая загрузка или обновление. В секционированных таблицах она может быть нацелена на определенные разделы. Однако для обработки в реальном времени, при использовании комбинации Flink и Iceberg, предпочтительнее использовать инструкцию `INSERT INTO`, предназначенную для потокового обновления.

`DataStream` API предлагает удобные функции для выполнения различных операций записи при работе с Iceberg. Например, `DataStream<RowData>` и `DataStream<Row>` можно напрямую записать в таблицу Iceberg с помощью `FlinkSink.forRowData()`:

```
StreamExecutionEnvironment env = ...;
DataStream<RowData> input = ...;
Configuration hadoopConf = new Configuration();
TableLoader tableLoader = TableLoader.fromHadoopTable("hdfs://nn:8020/warehouse/
path", hadoopConf);

FlinkSink.forRowData(input)
    .tableLoader(tableLoader)
    .append();

env.execute("Retail Sales Data Example");
```

Для более сложных случаев Iceberg поддерживает перезапись и обновление данных. Чтобы включить режим перезаписи существующих данных, нужно установить флаг `overwrite` в вызове конструктора `FlinkSink`. А режим обновления на основе первичного ключа можно включить либо с помощью свойства таблицы, либо установив флаг `upsert` в вызове конструктора. Вот примеры того и другого:

```
// Перезапись данных
FlinkSink.forRowData(input)
    .tableLoader(tableLoader)
    .overwrite(true)
    .append();

// Обновление данных
FlinkSink.forRowData(input)
    .tableLoader(tableLoader)
    .upsert(true)
    .append();
```

Помимо прямой записи `DataStream`, формат Iceberg поддерживает также запись обобщенных экземпляров `DataStream<T>` с помощью специального преобразователя данных в требуемый формат, такого как `AvroGenericRecordToRowDataMapper`. Например:

```
// Определить свой преобразователь AvroGenericRecordToRowDataMapper
AvroGenericRecordToRowDataMapper mapper = new AvroGenericRecordToRowDataMapper();

// Создать обобщенный экземпляр DataStream<GenericRecord>
// из источника(например, Kafka)
DataStream<GenericRecord> genericDataStream = ... // ваш источник DataStream

// Записать обобщенный DataStream в таблицу Iceberg, используя преобразователь
FlinkSink.builderFor(
    genericDataStream,
    mapper,
    // При необходимости можно указать TypeInfo для RowData
    TypeInfo.of(RowData.class)
)
    .table(tableName)
    .tableLoader(tableLoader)
    .build();

env.execute("Iceberg Generic DataStream Example");
```

Потоковая запись в Apache Iceberg с помощью Apache Flink – это надежный и эффективный способ обработки данных в режиме реального времени, обеспечивающий согласованность данных и атомарность операций даже в средах высокоскоростной потоковой передачи, а представленные примеры кода демонстрируют, как использовать Flink API для пакетной и потоковой записи в таблицы Iceberg.

Пример потоковой передачи в Iceberg с помощью Flink

В этом примере у нас есть две таблицы в каталоге Hive: `retail_sales_data` и `sales_data_summary`. Таблица `retail_sales_data` содержит исходные данные о продажах, такие как дата продажи (`purchase_date`), количество (`quantity`) и цена (`price`). Таблица `sales_data_summary` – это таблица Iceberg, секционированная для хранения ежемесячных агрегированных данных о продажах. Наша цель – выполнять обработку данных каждый час и обновлять таблицу `sales_data_summary` последними агрегированными данными о продажах. Таблица `sales_data_summary` секционирована по столбцу даты по месяцам. Далее мы опишем некоторые фрагменты кода; полный код вы найдете в репозитории книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg>):

```
String createCatalogStatement = "CREATE CATALOG " + catalogName +
    " WITH (\n" +
```

```

        "type'" + catalogType + "',\n" +
        "catalog-type='hive',\n" +
        "uri'" + hiveCatalogUri + "',\n" +
        "warehouse'" + warehousePath + "',\n" +
        "clients'" + hiveClientConfig + "',\n" +
        "property-version='1'\n" +
        ")";

// Зарегистрировать каталог Hive
tEnv.executeSql(createCatalogStatement);
tEnv.useCatalog(catalogName);

// Определить имена таблиц источника и приемника
String sourceTableName = "retail_sales_data";
String destinationTableName = "sales_data_summary";

// Вычислить значение раздела текущего месяца
// (например, '2023-08' для августа 2023)
String currentMonthPartition = getCurrentMonthPartition();

// SQL-запрос для обработки данных за текущий месяц
String sqlQuery = "INSERT OVERWRITE " + destinationTableName +
    " PARTITION (month = '" + currentMonthPartition + "') " +
    "SELECT date, SUM(sales_amount) as total_sales " +
    "FROM " + sourceTableName + " " +
    "WHERE month = '" + currentMonthPartition + "' " +
    "GROUP BY date";

// Выполнить SQL-запрос
tEnv.executeSql(sqlQuery);

// Приостановиться на 1 час перед повторением задания
Thread.sleep(3600000);

```

Это задание предназначено для выполнения последовательности операций. Сначала задание запрашивает агрегированный по дням объем продаж в текущем месяце. Затем выполняет команду `INSERT OVERWRITE`, чтобы записать обновленные данные в сводную таблицу. Этот шаг имеет решающее значение, потому что обновляет данные в разделе, соответствующем текущему месяцу. После записи данных задание приостанавливается на один час. В этот период никакие операции не выполняются. По завершении часового перерыва задание перезапускается и повторяет ту же последовательность операций.

Эту сводную таблицу можно использовать для реализации информационных панелей бизнес-аналитики с данными о продажах, обновляющихся по мере продаж с точностью до часа. Актуальность данных можно повысить, настроив частоту выполнения задания. Если выполнение задания занимает 30 секунд, то частоту можно увеличить до одного раза в 30 секунд и тем самым повысить актуальность сведений на информационных панелях, основанных на сводной таблице (более частые задания могут выполняться быстрее, что позволит использовать еще более короткие интервалы).

Потоковая обработка с помощью Kafka Connect

Apache Kafka Connect – это компонент с открытым исходным кодом для платформы Apache Kafka, реализующий масштабируемый и надежный способ перемещения данных в Kafka и из нее. Тысячи компаний используют Kafka Connect для интеграции своих систем с Apache Kafka для потоковой передачи данных в режиме реального времени.

Вот некоторые ключевые особенности Apache Kafka Connect:

Высокая пропускная способность

Kafka Connect может упростить передачу огромных объемов данных. Этот компонент предназначен для бесперебойной работы с Apache Kafka и эффективной потоковой передачи данных между системами.

Отказоустойчивость

Kafka Connect обеспечивает высокую отказоустойчивость. Он может автоматически управлять сбоями и бесперебойно передавать данные между разными системами и Apache Kafka.

Интеграция в реальном времени

С помощью Kafka Connect можно организовать интеграцию данных в режиме реального времени, позволяющую системам оставаться в актуальном состоянии.

Долговечность

Kafka Connect работает с темами Kafka, хранящими записи на диске и распределяющимися в кластере для отказоустойчивости. Благодаря этому данные останутся согласованными и нетронутыми даже в случае сбоя.

Широкая интеграция

Kafka Connect поддерживает обширную экосистему коннекторов, что упрощает интеграцию с различными системами, базами данных и приложениями. Если вам нужно переслать данные из базы данных в Kafka или из Kafka в облачное хранилище, то вы почти наверняка найдете коннектор, соответствующий вашим потребностям.

Семантика «точно один раз»

Как и Apache Flink, Kafka Connect поддерживает семантику «точно один раз» и гарантирует, что каждая запись будет обработана ровно один раз, и тем самым обеспечивает точность результатов.

Фреймворк коннекторов

Kafka Connect предлагает фреймворк для создания коннекторов, которые можно использовать для потоковой передачи данных между Apache Kafka и другими системами без каких-либо изменений в коде самой Kafka.

Apache Kafka Connect – это надежный инструмент в экосистеме Apache Kafka, обеспечивающий возможность бесшовной интеграции в режиме реального времени. Имеющиеся обширные возможности делают его идеальным выбором для компаний, желающих интегрировать различные системы с Apache Kafka.

Iceberg Kafka Sink

Kafka Sink – это компонент Apache Kafka, позволяющий получать данные из тем Kafka и записывать их во внешние системы или базы данных. Он играет решающую роль в интеграции с потоками данных, обеспечивая бесшовную передачу данных из Kafka в разнообразные хранилища, базы данных или аналитические платформы.

Apache Iceberg Sink Connector – это специализированный принимающий коннектор Kafka, который используется для передачи данных из Kafka в таблицы Iceberg. Он предоставляет различные функции и возможности, что делает его мощным инструментом для интеграции и хранения данных. Коннектор Apache Iceberg Sink Connector обладает множеством привлекательных особенностей:

Координация фиксаций

Централизованные фиксации Iceberg обеспечивают согласованность и атомарность операций записи данных.

Семантика «точно один раз»

Семантика «точно один раз» гарантирует обработку и запись данных в таблицы Iceberg ровно один раз, даже при наличии сбоя.

Запись сразу в несколько таблиц

Запись данных в несколько таблиц Iceberg обеспечивает гибкую возможность распределения данных.

Изменение записей

Поддержка изменения записей позволяет выполнять операции обновления и удаления существующих записей в таблицах Iceberg.

Режим Upsert

Режим Upsert позволяет эффективно обновлять данные в таблицах Iceberg v2 с определенными полями.

Настройка Apache Iceberg Kafka Sink

Коннектор Apache Iceberg Kafka Sink имеет множество параметров для настройки его поведения и обеспечения бесперебойной передачи данных. Давайте рассмотрим некоторые из них поближе:

`iceberg.tables`

Список имен целевых таблиц Iceberg, перечисленных через запятую, куда будут записываться данные из Kafka.

`iceberg.tables.routeField`

Это свойство используется в режиме записи в несколько таблиц и определяет поле, используемое для маршрутизации записей в разные таблицы.

`iceberg.tables.<имя_таблицы>.routeRegex`

Это свойство используется в режиме записи в несколько таблиц и определяет для таблицы с именем `<имя_таблицы>` регулярное выражение, используемое для сопоставления значения `routeField` с конкретной целевой таблицей.

`iceberg.tables.dynamic.enabled`

Установка этого свойства в значение `true` включает динамическую маршрутизацию записей в разные таблицы на основе значения в `routeField`.

`iceberg.tables.cdcField`

Это свойство определяет поле, содержащее коды операций отслеживания измененных данных (Change Data Capture, CDC), например `I` обозначает вставку (`insert`), `U` – обновление (`update`) и `D` – удаление (`delete`).

`iceberg.tables.upsertModeEnabled`

Установка этого свойства в значение `true` включает режим «`upsert`» (обновления или вставки), который обеспечивает эффективное обновление существующих данных путем удаления по равенству и последующего добавления.

`iceberg.control.topic`

Это свойство определяет имя темы, используемой для управления смещениями в коннекторе и фиксации транзакций.

`iceberg.control.group.id`

Это свойство определяет идентификатор группы потребителей, используемый для хранения смещений в группе потребителей, управляемой приемником.

`iceberg.control.commitIntervalMs`

Это свойство определяет интервал между фиксациями в миллисекундах и, соответственно, управляет частотой фиксации транзакций коннектором.

`iceberg.control.commitTimeoutMs`

Это свойство определяет продолжительность ожидания фиксации в миллисекундах. Если фиксация занимает больше времени, то она считается неудачной.

`iceberg.control.commitThread`

Это свойство определяет количество потоков, используемых для фиксации транзакций. Это количество равно удвоенному числу ядер ЦП.

`iceberg.catalog`

Имя каталога, используемого для подключения к хранилищу таблиц Iceberg. По умолчанию используется имя «iceberg».

`iceberg.catalog.*`

Дополнительные свойства, которые передаются при инициализации каталога Iceberg. Разным каталогам, таким как REST, Hive и Hadoop, могут потребоваться разные дополнительные конфигурационные свойства.

`iceberg.kafka.*`

Дополнительные свойства, которые передаются клиенту Kafka для подключения к управляющей теме. Таким способом можно определять свои настройки для клиента Kafka.

Настройка Kafka Connect для связи с Apache Iceberg

Настройка Kafka Connect для связи с Apache Iceberg начинается с установки этого компонента в систему. Получить Kafka можно на официальном сайте Apache Kafka и там же найти инструкции по установке.

Затем нужно создать коннектор Apache Iceberg Sink для Kafka Connect. Можно использовать уже готовый JAR-файл коннектора или собрать его самостоятельно из исходного кода (находится в репозитории книги на GitHub (<https://github.com/tabular-io/iceberg-kafka-connect>)). Если вы решите собрать коннектор самостоятельно, то клонируйте репозиторий Iceberg и выполните следующую команду внутри модуля `kafka-connect-iceberg`:

```
./gradlew -xtest clean build
```

Откройте файл свойств рабочего процесса Kafka Connect (обычно он имеет имя `worker.properties`) и добавьте необходимые свойства клиента Kafka, связанные с вашей конфигурацией Kafka. Например, чтобы включить SSL-связь с брокером Kafka, добавьте следующие свойства:

```
# Свойства рабочего процесса (worker.properties)
bootstrap.servers=kafka-broker1:9092,kafka-broker2:9092
security.protocol=SSL
ssl.truststore.location=/path/to/truststore.jks
ssl.truststore.password=truststore_password
```

В том же файле свойств `worker.properties` можно установить свойства Apache Iceberg Kafka Sink. Например:

```
# Свойства Apache Iceberg Sink
connector.class=io.tabular.iceberg.connect.IcebergSinkConnector
tasks.max=2
topics=events
iceberg.tables=default.events
iceberg.catalog.type=rest
iceberg.catalog.uri=https://localhost
iceberg.catalog.credential=<учетные_данные>
iceberg.catalog.warehouse=<имя_хранилища>
```

Вы можете изменить эти свойства в соответствии с вашими настройками. Обязательно установите правильные значения для `iceberg.catalog.uri`, `iceberg.catalog.credential` и `iceberg.catalog.warehouse`.

Запустите службу Kafka Connect, указав файл `worker.properties` в виде аргумента командной строки:

```
bin/connect-distributed.sh config/worker.properties
```

Kafka Connect можно также запустить в автономном режиме:

```
bin/connect-standalone.sh config/worker.properties connector.properties
```

Обратите внимание, что файл `connector.properties` должен содержать упомянутые ранее свойства Apache Iceberg Sink Connector.

После запуска Kafka Connect автоматически запустит Apache Iceberg Sink Connector и начнет записывать данные из указанных тем в таблицы Iceberg. Загляните в журналы, чтобы убедиться, что коннектор успешно запустился и обрабатывает данные должным образом.

Следите за производительностью Apache Iceberg Sink и проверяйте наличие предупреждений или сообщений об ошибках в журналах. Для этого можно использовать такие инструменты, как Kafka Connect REST API или пользовательский интерфейс Kafka Connect.

Вот и все! Мы настроили Apache Iceberg Sink и клиента Kafka. Коннектор приемника позаботится о записи данных из тем Kafka в указанные таблицы Iceberg, а клиент Kafka будет обслуживать связь с брокером Kafka на основе настроенных свойств. Обязательно загляните в документацию Kafka и Apache Iceberg, чтобы узнать больше о доступных свойствах клиента Kafka и конфигурационных параметрах Apache Iceberg Sink Connector.

Потоковая обработка с помощью AWS

Amazon Web Services (AWS) предлагает набор сервисов потоковой передачи данных в реальном времени, которые позволяют собирать, обрабатывать, анализировать и доставлять потоковые данные приложениям реального времени и аналитическим решениям. Сервисы потоковой передачи данных AWS безопасны, высокодоступны, надежны и полностью управляемы, что упрощает разработчикам создание приложений реального времени.

Экосистема потоковой обработки данных AWS включает следующие компоненты:

Источник

Под источниками подразумеваются тысячи устройств и/или приложений, таких как мобильные устройства, веб-приложения, журналы приложений, датчики интернета вещей, интеллектуальные устройства и игровые приложения, которые производят данные в больших объемах и с большой скоростью.

Приемники потоков

AWS обеспечивает простую интеграцию с более чем 15 сервисами, которые непрерывно собирают данные надежным и безопасным способом.

Хранилище потоков

AWS предлагает такие решения, как Amazon Kinesis Data Streams, Amazon Kinesis Data Firehose и Amazon Managed Streaming для Apache Kafka (Amazon MSK), которые удовлетворят ваши потребности в хранилище с учетом требований к масштабированию, задержке и обработке.

Обработчики потоков

AWS предоставляет ряд сервисов по преобразованию и доставке данных, от стандартных, таких как Amazon Kinesis Data Firehose, до специализированных приложений реального времени и интеграции машинного обучения, основанных на использовании таких сервисов, как Amazon Kinesis Data Analytics и AWS Lambda.

Получатели

AWS обеспечивает потоковую передачу данных в полностью интегрированные озера данных, хранилища данных и аналитические сервисы для дальнейшего анализа или долгосрочного хранения.

Вот некоторые ключевые сервисы AWS для потоковой обработки данных в реальном времени:

Amazon Kinesis Data Streams

Масштабируемая и надежная служба потоковой передачи данных в реальном времени, способная извлекать гигабайты данных каждую секунду из сотен тысяч источников.

Amazon Kinesis Data Firehose

Собирает, преобразует и загружает потоки данных в хранилища данных AWS для оперативного анализа с помощью существующих инструментов бизнес-аналитики.

Amazon Kinesis Data Analytics

Позволяет обрабатывать потоки данных в реальном времени с помощью SQL или Java (с использованием Apache Flink) без необходимости изучать новые языки программирования или фреймворки.

Amazon MSK

Полностью управляемый сервис, упрощающий создание и запуск приложений, использующих Apache Kafka для обработки потоковых данных.

Сервисы потоковой обработки данных AWS поддерживают широкий спектр вариантов использования, включая перемещение и анализ данных в реальном времени, а также обработку потока событий. Это позволяет пользователям анализировать данные сразу после их получения, сохранять их для дальнейшего анализа, принимать решения в режиме реального времени,

фиксировать события и реагировать на них, по мере того как они происходят в нескольких приложениях, а также поддерживать систему учета.

Использование AWS Glue Studio с Apache Iceberg для загрузки данных в таблицы Iceberg – это мощный и эффективный способ управления данными и их обработки в озерах данных. AWS Glue 3.0 и более поздние версии поддерживают Apache Iceberg. Эта интеграция позволяет выполнять операции чтения и записи с таблицами Iceberg в Amazon Simple Storage Service (Amazon S3) и использовать каталог данных AWS Glue для дополнительных операций, включая вставку, обновление, а также чтение и запись средствами Spark.

Одно из ключевых преимуществ использования AWS Glue Studio с Apache Iceberg – поддержка AWS Kinesis Stream и Kafka Stream в качестве источников данных. При необходимости вы без труда сможете импортировать данные из этих потоковых источников в таблицы Iceberg в Amazon S3 для дальнейшей обработки и анализа.

Чтобы включить поддержку Iceberg в AWS Glue, нужно передать заданию параметр `--datalake-formats` со значением `iceberg`. Также необходимо настроить конфигурационные параметры Spark для правильной обработки таблиц Iceberg, в том числе установку параметра `spark.sql.extensions` в значение `org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions` и настройку свойств `glue_catalog`, таких как `warehouse`, `catalog-impl` и `io-impl`. Более подробно эти настройки обсуждаются в главах 5 и 8.

При использовании AWS Glue 3.0 с Iceberg 0.13.1 также необходимо настроить дополнительные параметры для использования диспетчера блокировок Amazon DynamoDB, чтобы гарантировать атомарность транзакций. Версия AWS Glue 4.0 по умолчанию использует оптимистическую блокировку, поэтому не требует настройки блокировки вручную.

Чтобы использовать другую версию Iceberg, которая изначально не поддерживается AWS Glue, можно указать свои JAR-файлы Iceberg в параметре задания `--extra-jars`.

После включения и настройки поддержки Iceberg можно начинать создавать и регистрировать таблицы Iceberg из наборов данных DataFrames в AWS Glue Studio. Например, можно создать таблицу Iceberg из DataFrame и зарегистрировать ее в каталоге данных AWS Glue с помощью SQL-запросов. Аналогично с помощью каталога данных Glue можно читать данные из таблицы Iceberg и отправлять их в Amazon S3, а также выполнять операции вставки для добавления данных в таблицу Iceberg.

Следующий пример показывает, как записать таблицу Iceberg в Amazon S3 и зарегистрировать ее в каталоге данных AWS Glue на языке Python:

```
# Пример: создать таблицу Iceberg из DataFrame
# и зарегистрировать ее в Glue Data Catalog
dataFrame.createOrReplaceTempView("incoming_data")

# Вставить данные в таблицу Iceberg в Glue Catalog
query = f"""
```

```
INSERT INTO glue.db.destination_table SELECT * FROM incoming_data
"""
spark.sql(query)
```

Этот пример создает временное представление для набора данных `data-Frame`. Затем с помощью SQL-запроса создает таблицу Iceberg и регистрирует ее в каталоге данных AWS Glue.

В целом AWS Glue Studio дает простой и эффективный способ загрузки данных в таблицы Iceberg и последующего их анализа. Независимо от того, откуда получены данные, из AWS Kinesis Stream или из Kafka Stream, пользователи смогут использовать мощные функции Iceberg и AWS Glue для их обработки и анализа.

Заклучение

Существует множество инструментов, которые можно использовать для потоковой передачи данных в таблицы Apache Iceberg и потокового же чтения из них, включая Apache Spark, Apache Flink, AWS Kinesis и Kafka Connect. В репозитории книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg>) имеется диаграмма, перечисляющая функции инструментов, которые были рассмотрены в этой главе.

В главе 12 мы обсудим вопросы управления таблицами Apache Iceberg и обеспечения их безопасности.

Глава 12

Управление и безопасность

Организации все чаще используют современные архитектуры озерных хранилищ данных, такие как Apache Iceberg, и получают выгоды от их гибкости, масштабируемости и производительности. Однако эти преимущества порождают новые проблемы, касающиеся безопасности и управления данными.

В данной главе мы погрузимся в многогранный мир управления таблицами Apache Iceberg и обеспечения их безопасности. Apache Iceberg – это в первую очередь стандарт метаданных, которые определяют набор данных, он не имеет ничего, что касается безопасности, кроме некоторых свойств таблиц, с помощью которых производится выбор типа шифрования файлов. Безопасность таблиц Apache Iceberg в первую очередь обеспечивается уровнями хранения, доступа и вычислений, которые используются для работы с ними.

В этой главе вы познакомитесь с тремя важнейшими аспектами защиты озерного хранилища данных:

- защита файлов данных;
- безопасность и управление через семантический уровень;
- безопасность и управление на уровне каталога.

Организации должны использовать комплексный подход к обеспечению безопасности и эффективному управлению таблицами Apache Iceberg. Изучив эти три аспекта – защиту файлов данных, реализацию безопасности и управления через семантический уровень и реализацию безопасности на уровне каталога, – вы будете готовы к преодолению сложностей организации защиты вашего озерного хранилища данных. Итак, давайте узнаем, как защитить наши ресурсы данных и использовать весь потенциал Apache Iceberg.

Имейте в виду, что управление и безопасность – глубокие темы, которым посвящены целые книги. В этой главе мы рассмотрим несколько примеров использования инструментов для защиты таблиц Apache Iceberg, но главная ее цель – общее знакомство, а не предоставление исчерпывающего списка инструментов или подходов.

Безопасность файлов данных

Всякое озерное хранилище хранит множество файлов данных. Защита этих файлов является первой линией защиты от несанкционированного доступа к данным и их утечки. В этом разделе мы перечислим различные инструменты и приемы защиты файлов данных в таблицах Apache Iceberg, от шифрования и контроля доступа до маскировки данных и аудита.

Защита файлов данных на уровне хранилища имеет первостепенное значение, поскольку они составляют основу инфраструктуры данных. В эпоху постоянных киберугроз организации должны защищать свои данные от множества уязвимостей. Спектр потенциальных угроз очень широк: от внешних, таких как попытки взлома и кражи данных, до внутренних, таких как несанкционированный доступ к данным. Заблаговременно устраняя эти уязвимости, организации могут защитить целостность и конфиденциальность своих данных, гарантируя соблюдение правил защиты данных и укрепляя доверие своих клиентов. В этом разделе мы рассмотрим различные платформы хранения и предлагаемые ими разнообразные средства защиты файлов данных, чтобы дать вам знания, опираясь на которые, вы сможете принять обоснованное решение о выборе мер безопасности, наиболее подходящих для вашей организации.

Защита файлов: практические приемы

Обеспечение безопасности файлов данных таблиц Apache Iceberg, независимо от выбранной платформы хранения, требует соблюдения фундаментальных принципов и применения передовых приемов защиты данных. Вот некоторые из них:

Предоставление доступа с минимально возможными привилегиями

Доступ к файлам данных должны иметь только те лица и процессы, которым он требуется для решения их задач. Предоставьте им минимальные разрешения, необходимые для выполнения этих задач.

Шифрование при хранении и передаче

Обеспечьте шифрование файлов данных при хранении и при передаче. Для защиты целостности и конфиденциальности данных используйте механизмы шифрования, предоставляемые платформой хранения.

Строгая аутентификация и управление идентификацией

Внедрите строгие методы идентификации и управления доступом. Используйте многофакторную аутентификацию и политики надежных паролей для проверки личности пользователей.

Аудит и журналирование

Включите функции аудита и журналирования, предлагаемые платформой хранения. Ведите комплексные журналы для мониторинга и расследования фактов доступа к файлам данных и внесения изменений в них.

Политики хранения и удаления данных

Определите политики хранения и удаления для управления жизненным циклом файлов данных. Безопасно удаляйте или архивируйте файлы, которые больше не нужны, чтобы снизить риск.

Непрерывный мониторинг

Внедрите системы непрерывного мониторинга и оповещения для обнаружения инцидентов безопасности и реагирования на них в режиме реального времени.

В контексте обеспечения безопасности на разных уровнях следует учитывать не только их преимущества, но и недостатки. Из положительных сторон уровня файлов данных Apache Iceberg можно назвать возможность точно назначать разрешения и настраивать шифрование для отдельных файлов данных. Этот уровень также поддерживает шифрование хранимых данных, что помогает обеспечить безопасность конфиденциальной информации и строгую изоляцию данных, а также минимизировать риски несанкционированного доступа. К недостаткам можно отнести сложность управления безопасностью на уровне файлов, особенно при увеличении их количества. Кроме того, может потребоваться добавить больше абстракции, чтобы дать конечным пользователям и инструментам унифицированное и упрощенное представление данных. Наконец, масштабирование может оказаться сложной задачей при управлении разрешениями в растущем озере данных, привести к ошибкам и снижению эффективности.

Распределенная файловая система Hadoop

Защита файлов данных в таблицах Apache Iceberg, хранимых в распределенной файловой системе Hadoop (Hadoop Distributed File System, HDFS), требует комплексного подхода. В этом случае решающее значение имеют три компонента.

Списки управления доступом

Списки управления доступом (Access Control List, ACL) в HDFS позволяют точно управлять разрешениями на доступ к определенным файлам и каталогам в хранилище данных Iceberg, не ограничиваясь стандартными разрешениями на чтение, запись и выполнение. Такой уровень детализации гарантирует, что только авторизованные пользователи смогут получить доступ к конфиденциальным данным, что сводит к минимуму риск несанкционированного доступа или утечки данных.

Установить ACL для файла или каталога можно с помощью следующих команд:


```
# Установить ACL, разрешающий чтение и запись конкретному пользователю
hdfs dfs -setfacl -m user:username:rwX /путь/к/вашему/файлу_или_каталогу

# Разрешает чтение определенной группе
hdfs dfs -setfacl -m group:groupname:r-x /путь/к/вашему/файлу_или_каталогу
```

Шифрование

Шифрование имеет первостепенное значение для защиты файлов данных. HDFS предлагает надежные варианты шифрования, включая прозрачное шифрование и зоны шифрования. При прозрачном шифровании шифруются блоки данных на диске, что делает их нечитаемыми для неавторизованных лиц, даже если им удастся завладеть физическим носителем данных. Зоны шифрования позволяют указать определенные каталоги или файлы, требующие шифрования, что дает возможность сосредоточить усилия по шифрованию на наиболее важных данных.

Вот как можно включить прозрачное шифрование для кластера HDFS:

```
# Включить прозрачное шифрование в HDFS
hdfs crypto -createZone -keyName myEncryptionKey -path /путь/к/зоне/шифрования
```

Разрешения

HDFS использует систему разрешений, аналогичную традиционным файловым системам, что позволяет обеспечить единообразное управление доступом к файлам данных. Назначать разрешения на доступ к файлам и/или каталогам для пользователей и групп можно с помощью таких команд, как `chmod` и `chown`. Правильная настройка разрешений гарантирует, что никто посторонний не сможет получить доступ к файлам, тем самым снижая риск неправильного использования данных.

Вот как можно установить разрешения для файла или каталога:

```
# Сменить владельца файла или каталога
hdfs dfs -chown новый_владелец:новая_группа /путь/к/вашему/файлу_или_каталогу

# Дать разрешения на чтение, запись и выполнение только владельцу
hdfs dfs -chmod 700 /путь/к/вашему/файлу_или_каталогу
```

Защита данных Apache Iceberg в HDFS требует комплексного использования этих мер безопасности с учетом ваших конкретных требований. Вы можете усилить защиту хранилища данных, уделив особое внимание ACL для точного управления доступом, шифрованию для обеспечения конфиденциальности данных и разрешениям для структурированного управления правами доступа. Такой комплексный подход гарантирует защищенность данных от несанкционированного доступа и укрепит надежность ваших информационных ресурсов.

Amazon Simple Storage Service

Защита файлов данных таблиц Apache Iceberg в Amazon Web Services (AWS) предполагает использование разнообразных функций безопасности, предлагаемых Amazon Simple Storage Service (Amazon S3). В этом разделе мы покажем, как использовать некоторые из наиболее важных функций безопасности на уровне файлов, которые обычно связаны с шифрованием и управлением доступом.

Шифрование

Amazon S3 предлагает три варианта шифрования на стороне сервера (Server-Side Encryption, SSE) для защиты хранимых данных. Выберите тот, который лучше всего соответствует вашим требованиям.

SSE-S3 (SSE с ключами, управляемыми S3) Этот вариант, когда Amazon S3 автоматически управляет ключами шифрования, прост и подходит для большинства сценариев. Следующая команда включит SSE-S3 для корзины S3:

```
aws s3api put-bucket-encryption --bucket ИМЯ_ВАШЕЙ_КОРЗИНЫ --server-side-encryption-configuration '{"Rules": [{"ApplyServerSideEncryptionByDefault": {"SSEAlgorithm": "AES256"}}}]'
```

SSE-KMS (SSE со службой управления ключами AWS) Этот вариант использует службу управления ключами (Key Management Service, KMS) AWS для расширенного управления ключами. Он идеально подходит для сценариев, требующих соблюдения нормативных положений или строгих политик корпоративной безопасности.

Следующая команда включит SSE-KMS для корзины S3:

```
aws s3api put-bucket-encryption --bucket ИМЯ_ВАШЕЙ_КОРЗИНЫ --server-side-encryption-configuration '{"Rules": [{"ApplyServerSideEncryptionByDefault": {"SSEAlgorithm": "aws:kms", "KMSEMasterKeyId": "ВАШ_КЛЮЧ_KMS"}}}]'
```

SSE-C (SSE с ключами, предоставленными клиентом) SSE-C обеспечивает высочайший уровень контроля. Вы предоставляете свои ключи для шифрования и дешифрования данных, а Amazon S3 не имеет доступа к этим ключам и не хранит их.

Следующая команда включит SSE-C для корзины S3:

```
aws s3api put-bucket-encryption --bucket ИМЯ_ВАШЕЙ_КОРЗИНЫ --server-side-encryption-configuration '{"Rules": [{"ApplyServerSideEncryptionByDefault": {"SSEAlgorithm": "AES256"}}, {"CustomerAlgorithm": "AES256"}]}'
```

Для настройки шифрования таблицы Apache Iceberg в AWS S3 на стороне сервера можно использовать следующие конфигурационные свойства (это свойства таблиц Iceberg, а не настройки AWS):

- `s3.sse.type`

Значение по умолчанию: none.

Описание: определяет тип используемого SSE. Допустимые значения: none, S3, KMS и custom.

○ s3.sse.key

Значение по умолчанию: aws/s3 для типа KMS, в противном случае null. Описание: для типа KMS в это свойство должен быть записан идентификатор ключа KMS или имя ресурса Amazon (ARN). Для других типов, например пользовательского, можно указать свой симметричный ключ AES-256 в формате base-64.

○ s3.sse.md5

Значение по умолчанию: null

Описание: это свойство актуально, если выбран тип SSE custom. В нем должен быть указан дайджест MD5 симметричного ключа в формате base-64.

Политики корзины

Для управления доступом на уровне корзины с помощью политик Amazon S3 выполните следующие действия:

1. Определите политику корзины в формате JSON с конкретными правилами.
2. Прикрепите политику к корзине S3 в AWS Management Console или с помощью клиента командной строки AWS.

Вот пример политики, разрешающей доступ только с определенного IP-адреса:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowSpecificIP",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::ИМЯ_ВАШЕЙ_КОРЗИНЫ/*",
      "Condition": {
        "IpAddress": {
          "aws:SourceIp": "ВАШ_IP_АДРЕС"
        }
      }
    }
  ]
}
```

Прикрепить эту политику к своей корзине S3 с помощью интерфейса командной строки AWS можно так:

```
aws s3api put-bucket-policy --bucket YOUR_BUCKET_NAME --policy '{
  "Version": "2012-10-17",
```

```

"Statement": [
  {
    "Sid": "AllowSpecificIP",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3::ИМЯ_ВАШЕЙ_КОРЗИНЫ/*",
    "Condition": {
      "IpAddress": {
        "aws:SourceIp": "ВАШ_IP_АДРЕС"
      }
    }
  }
]
}'

```

Управление идентификацией и доступом

Чтобы использовать механизм AWS управления идентификацией и доступом (Identity and Access Management, IAM) для создания и управления ролями и разрешениями на доступ к вашим ресурсам в S3, выполните следующие действия:

1. Создайте пользователей, роли или группы IAM в соответствии с потребностями вашей организации.
2. Определите политики, определяющие доступные пользователям действия и ресурсы (например, `s3:GetObject`, `s3:PutObject`).
3. Прикрепите политики к пользователям, ролям или группам IAM, чтобы предоставить доступ к определенным корзинам или объектам S3.

Политики IAM могут быть очень детализированными, что позволяет точно контролировать доступность действий и ресурсов.

Вот пример команды в клиенте командной строки, прикрепляющей политику IAM к конкретному пользователю IAM, которая позволяет ему читать (`s3:GetObject`) и записывать (`s3:PutObject`) содержимое определенной корзины S3:

```

aws iam put-user-policy --user-name ИМЯ_ВАШЕГО_ПОЛЬЗОВАТЕЛЯ_IAM
--policy-name S3AccessPolicy --policy-document '{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3::ИМЯ_ВАШЕЙ_КОРЗИНЫ/*"
    }
  ]
}'

```

В этом примере демонстрируется применение команды `aws iam put-user-policy`, которая прикрепляет политику к конкретному пользователю IAM. Чтобы выполнить ее у себя, замените `ИМЯ_ВАШЕГО_ПОЛЬЗОВАТЕЛЯ_IAM` фактическим именем пользователя IAM, к которому прикрепляется политика. Параметр `--policy-name` позволяет указать имя политики.

Важнейшей частью этого процесса является `--policy-document`. В этом параметре определяются разрешения в виде документа политики в формате JSON. В данном конкретном случае политика предоставляет пользователю IAM право на выполнение действий `s3:GetObject` (чтение) и `s3:PutObject` (запись) с объектами в указанной корзине S3. Корзина S3 и объекты указываются с помощью атрибута `Resource` в виде ARN. Обязательно замените `ИМЯ_ВАШЕЙ_КОРЗИНЫ` в ARN фактическим именем корзины S3, к которой вы хотите предоставить доступ.

Выполняя эту команду, вы даете пользователю IAM разрешение на чтение и запись объектов в указанной корзине S3. Не забудьте подставить фактические имена пользователя и корзины.

Списки управления доступом к объектам

Настройка списков управления доступом к отдельным файлам данных или каталогам в корзине S3 позволяет назначать очень детальные разрешения. В данном разделе мы расскажем, как использовать эту возможность.

Чтобы получить доступ к свойствам объекта, выполните следующие шаги в консоли управления AWS:

- войдите в консоль управления AWS;
- перейдите к сервису S3;
- щелкните на корзине, где находится искомый объект;
- выберите конкретный объект, щелкнув на его имени или перейдя по структуре каталогов;
- выполните необходимые настройки ACL.

Также получить доступ к свойствам объекта можно с помощью интерфейса командной строки AWS. Для этого откройте терминал. Затем используйте команды AWS S3 CLI, чтобы получить доступ к свойствам объекта. Вот практический пример:

```
aws s3api put-object-acl --bucket имя-вашей-корзины --key путь/к/вашему-объекту
--acl private --grant-read id="ID_ВАШЕГО_ПОЛЬЗОВАТЕЛЯ_IAM"
```

В этом сценарии можно использовать команду `aws s3api put-object-acl` и с ее помощью внести изменения в список ACL объекта, хранящегося в корзине AWS S3. В параметре `--bucket` имя-вашей-корзины нужно указать имя корзины S3, в которой находится объект. Параметр `--key` путь/к/вашему-объекту задает точный путь к объекту в корзине. Передача параметра `--acl private` гарантирует, что ACL будет настроен как частный, который по умолчанию открывает доступ к объекту только его владельцу. И наконец, параметр `--grant-read id="ID_ВАШЕГО_ПОЛЬЗОВАТЕЛЯ_IAM"` позволяет дать разрешение на

чение конкретному пользователю IAM. Не забудьте заменить заполнитель "ID_ВАШЕГО_ПОЛЬЗОВАТЕЛЯ_IAM" фактическим идентификатором пользователя IAM или его ARN.

Выполняя эту команду, вы ограничиваете доступ к указанному объекту, разрешая читать его только пользователю IAM с указанным идентификатором. Этот пример можно расширить и предоставить разные разрешения разным пользователям или указать разные объекты IAM.

Управление списками ACL плохо масштабируется, особенно при большом количестве объектов и разнообразии требований к доступу. Поэтому важно иметь четко определенную и документированную стратегию контроля доступа, которая поможет избежать выдачи непреднамеренных разрешений и появления потенциальных угроз безопасности. Также учтите имеющиеся плюсы и минусы использования ролей IAM и списков ACL (<https://oreil.ly/wPnGI>).

Azure Data Lake Storage

Защита файлов данных таблиц Apache Iceberg в Azure Data Lake Storage (ADLS) предполагает использование различных функций безопасности, в первую очередь связанных с шифрованием и управлением доступом. В этом разделе мы расскажем, как воспользоваться этими функциями.

Шифрование в ADLS

Защита файлов данных в ADLS предполагает их шифрование. На выбор доступно шифрование по умолчанию и с использованием ключей, управляемых клиентом. Ниже описаны шаги, включая соответствующие команды CLI, для добавления в ADLS ключей, управляемых клиентом.

Azure Key Vault – это средство для безопасного управления вашими ключами шифрования.

Создать и настроить Azure Key Vault можно на портале Azure с помощью клиента командной строки Azure или Azure PowerShell. Вот пример с использованием клиента командной строки:

```
az keyvault create --name ИмяКошелька --resource-group ГруппаРесурсов
--location Местоположение
```

Замените ИмяКошелька, ГруппаРесурсов и Местоположение актуальными значениями. Затем сгенерируйте новые ключи шифрования в Key Vault или импортируйте существующие:

```
az keyvault key create --vault-name ИмяКошелька --name ИмяКлюча --key RSA
```

Замените ИмяКошелька и ИмяКлюча актуальными значениями.

После настройки Azure Key Vault и создания или импортирования ключей шифрования можно настроить использование этих ключей в ADLS. Убеди-

тес, что учетная запись Azure ADLS или сервис, который вы планируете использовать, имеет необходимые разрешения для доступа к ключам в Azure Key Vault.

Затем свяжите свою учетную запись ADLS с Azure Key Vault, используя портал Azure или Azure CLI:

```
az storage account update --name ИмяУчетнойЗаписиХранилища
--resource-group ГруппаРесурсов --assign-identity [system] --keyvault ИмяКошелька
```

Замените ИмяУчетнойЗаписиХранилища, ГруппаРесурсов и ИмяКошелька актуальными значениями.

Для безопасности только авторизованный персонал должен иметь доступ к Azure Key Vault. Настроить политики доступа к Key Vault можно на портале Azure с помощью клиента командной строки Azure или Azure PowerShell. Вот пример с использованием клиента командной строки:

```
az keyvault set-policy --name ИмяКошелька --resource-group ГруппаРесурсов
--spn ИмяСервисаСубъекта --key-permissions get list
```

Замените ИмяКошелька, ГруппаРесурсов и ИмяСервисаСубъекта актуальными значениями.

Управление доступом на основе ролей

Управление доступом на основе ролей (Role-Based Access Control, RBAC) в Azure позволяет определять, кто может выполнять те или иные действия с вашими ресурсами ADLS. Ниже приводятся инструкции и соответствующие команды Azure CLI для реализации RBAC в Azure ADLS.

Перед настройкой RBAC убедитесь, что ресурсы ADLS созданы и настроены соответствующим образом. Затем можно создать учетную запись ADLS с помощью портала Azure, Azure CLI или Azure PowerShell:

```
az dls account create --name ИмяУчетнойЗаписиADLS --resource-group ГруппаРесурсов
--location Местоположение --tier ИмяУровня --encryption-type УправляемыйСервис
```

Замените ИмяУчетнойЗаписиADLS, ГруппаРесурсов, Местоположение, ИмяУровня и УправляемыйСервис актуальными значениями.

Azure предлагает предопределенные роли для ресурсов ADLS, например «Storage Blob Data Contributor» (поставщик БЛОБ-объектов в хранилище). Однако вы можете создавать свои роли с конкретными разрешениями, адаптированными к потребностям вашей организации.

Получить список встроенных ролей ADLS можно с помощью Azure CLI:

```
az role definition list --resource-type Microsoft.DataLakeStore/accounts
```

Если предопределенные роли не соответствуют вашим требованиям, то создайте свою роль. Определите разрешения, необходимые для вашей роли:

```
az role definition create --role-definition CustomRoleDefinition.json
```

Замените `CustomRoleDefinition.json` именем файла JSON с определением вашей роли.

Определив роли, вы сможете назначить их пользователям, группам или субъектам-сервисам для управления доступом к вашим ресурсам ADLS.

Используйте Azure CLI, чтобы назначить предопределенную роль пользователю, группе или субъекту-сервису:

```
az role assignment create --assignee ИмяСервисаСубъекта
--role "Storage Blob Data Contributor"
--scope /subscriptions/ИдентификаторПодписки/resourceGroups/ГруппаРесурсов/providers/
Microsoft.DataLakeStore/accounts/ИмяУчетнойЗаписиADLS
```

Замените `ИмяСервисаСубъекта`, `ИдентификаторПодписки`, `ГруппаРесурсов` и `ИмяУчетнойЗаписиADLS` актуальными значениями.

Команда назначения своей роли выглядит аналогично. Если вам нужно отозвать доступ, то удалите назначение роли:

```
az role assignment delete --assignee ИмяСервисаСубъекта
--role "Storage Blob Data Contributor"
--scope /subscriptions/ИдентификаторПодписки/resourceGroups/ГруппаРесурсов/providers/
Microsoft.DataLakeStore/accounts/ИмяУчетнойЗаписиADLS
```

Для обеспечения безопасности и соответствия нормативным требованиям регулярно проверяйте разрешения на доступ, назначенные через RBAC. Убедитесь, что пользователи, группы и субъекты-сервисы имеют уровень доступа, соответствующий принципу наименьших привилегий.

Списки управления доступом

Списки управления доступом (ACL) в ADLS обеспечивают детальный контроль над разрешениями отдельных пользователей, групп или субъектов-сервисов как на уровне каталога, так и на уровне файлов. Ниже представлены инструкции и соответствующие команды Azure CLI для настройки списков ACL в ADLS.

Настроить списки ACL можно с помощью портала Azure или Azure CLI. Войдите на портал Azure (<https://portal.azure.com>) и перейдите к своей учетной записи ADLS.

Или откройте терминал и введите команду:

```
az login
```

Используйте следующую команду, чтобы выбрать подписку по умолчанию (если она еще не выбрана):

```
az account set --subscription ИмяПодписки
```

Получив доступ к своей учетной записи ADLS, перейдите в пользовательском интерфейсе портала к определенному каталогу или файлу, для которого требуется установить ACL.

Вот как можно вывести список содержимого каталога или получить информацию о конкретном файле в Azure CLI:

```
az dls fs list --account ИмяУчетнойЗаписиASDL --path /путь/к/вашему_каталогу
```

Получить информацию о файле можно такой командой:

```
az dls fs show --account ИмяУчетнойЗаписиASDL --path /путь/к/вашему_файлу
```

Замените ИмяУчетнойЗаписиASDL, /путь/к/вашему_каталогу и /путь/к/вашему_файлу актуальными значениями.

Вы можете настроить списки ACL, определив права доступа для пользователей, групп или субъектов-сервисов. Для этого на портале Azure перейдите к каталогу или файлу и откройте раздел **Access control (IAM)** (Управление доступом (IAM)) или **Permissions** (Разрешения). Затем добавьте либо измените разрешения для определенных пользователей, групп или субъектов-сервисов.

Установить списки ACL для определенного каталога можно также в Azure CLI:

```
az dls fs access set --account ИмяУчетнойЗаписиASDL --path /путь/к/вашему_каталогу
--acl-spec "user::rwx,group::r--,other::--"
```

Аналогично установить списки ACL для определенного файла можно командой

```
az dls fs access set --account ИмяУчетнойЗаписиASDL --path /путь/к/вашему_файлу
--acl-spec "user::rw-,group::r--,other::--"
```

Замените ИмяУчетнойЗаписиASDL, /путь/к/вашему_каталогу и /путь/к/вашему_файлу актуальными значениями. При необходимости настройте спецификацию ACL (--acl-spec), предоставив или запретив разрешения.

Настраивая списки ACL, следуйте политикам безопасности вашей организации. Регулярно проверяйте настройки ACL для обеспечения безопасности и соответствия требованиям.

Используя представленные команды Azure CLI, вы сможете эффективно настроить списки ACL в ADLS для управления разрешениями на доступ к каталогам и файлам, гарантировать безопасность данных и соответствие политикам организации.

Google Cloud Storage

Облачное хранилище Google (Google Cloud Storage, GCS) предлагает средства безопасности для защиты файлов данных, такие как шифрование, управление идентификацией и доступом, политики корзин и списки управления доступом к объектам. В данном разделе мы приводим инструкции по настройке этих средств безопасности с помощью команд CLI.

Шифрование при хранении и передаче

GCS автоматически шифрует хранящиеся данные с помощью SSE с использованием ключей, которыми управляет Google. Чтобы получить более полный контроль, можно выбрать ключи шифрования, управляемые клиентом (Customer-Managed Encryption Keys, СМЕК).

Для начала создайте облачную связку ключей и ключ KMS:

```
gcloud kms keyrings create имя-связки --location global
gcloud kms keys create ваш-ключ --location global --keyring имя-связки
--purpose encryption
```

Затем свяжите СМЕК с корзиной:

```
gsutil kms authorize -k projects/имя-проекта/locations/global/keyRings/имя-связки/
cryptoKeys/ваш-ключ
gsutil defacl set private gs://имя-вашей-корзины
```

После этого GCS гарантированно будет шифровать данные во время передачи с использованием стандартных протоколов, таких как HTTPS.

Управление идентификацией и доступом

Управление идентификацией и доступом к облаку Google (IAM) позволяет определять, кто имеет доступ к вашим ресурсам GCS, и действия, которые они могут выполнять. Для этого применяются роли IAM.

Вот код, создающий учетную запись сервиса и предоставляющий ей роль с необходимыми разрешениями:

```
gcloud iam service-accounts create учетная-запись-сервиса
gcloud projects add-iam-policy-binding имя-проекта --member serviceAccount:
учетная-запись-сервиса@имя-проекта.iam.gserviceaccount.com --role roles/
storage.objectAdmin
```

Далее можно использовать такой код для аутентификации в учетной записи сервиса с целью доступа к GCS:

```
gcloud auth activate-service-account --key-file=ключ-учетной-записи-сервиса.json
```

Политики корзины

GCS позволяет настраивать управление доступом на уровне корзины для установки правил доступа на основе таких факторов, как IP-адреса, учетные записи или некоторые условия.

Начните с определения политики корзины в формате JSON, указав правила доступа:

```
{
  "bindings": [
    {
```

```

    "role": "roles/storage.objectViewer",
    "members": ["user-email@example.com"]
  }
]
}

```

Затем примените эту политику к вашей корзине:

```
gsutil iam set файл-с-определением-политики.json gs://имя-вашей-корзины
```

Списки управления доступом к объектам

GCS поддерживает списки ACL для управления доступом к отдельным объектам в корзине, указывающие, кто может читать, записывать или удалять определенные объекты.

Следующая команда настроит ACL для конкретного объекта:

```
gsutil acl ch -u user-email@example.com:READ gs://your-bucket-name/your-object
```

Посредством представленных команд вы сможете эффективно использовать средства безопасности хранилища GCS для защиты файлов данных таблиц Apache Iceberg, гарантировать конфиденциальность и целостность данных и управлять доступом в соответствии с требованиями безопасности вашей организации.

Управление безопасностью на семантическом уровне

Семантический уровень – это виртуальная абстракция поверх хранилища данных, обеспечивающая структурированное и удобное представление данных. Он действует как уровень преобразования, позволяющий пользователям и приложениям взаимодействовать с данными интуитивно понятным способом, не требующим знать и понимать схему данных. Этот уровень обеспечивает безопасность за счет централизации управления доступом к данным, определения детальных разрешений и применения правил маскировки и преобразования данных. Он упрощает управление данными, обеспечивает их согласованность, качество и соответствие политикам организации, одновременно ограждая пользователей от сложностей, лежащих в основе данных. Семантический уровень позволяет организациям эффективно управлять и защищать свое озеро данных.

Семантические уровни повышают производительность, упрощая доступ к данным, поэтому они часто содержат функции для предварительного расчета агрегированных метрик, подобных тем, что предоставляются механизмом Dremio. Давайте рассмотрим некоторые семантические решения, которые могут работать с таблицами Apache Iceberg.

Практика применения семантического уровня

Хорошо спроектированный и безопасный семантический уровень является краеугольным камнем эффективного управления данными в озере данных. Вот несколько рекомендаций, которые следует учитывать при создании такого уровня:

Изучите особенности ваших данных

Начните с изучения источников данных в вашем озере данных, определите их типы, владельцев и степень конфиденциальности. Эти знания будут способствовать принятию решений по управлению данными и обеспечению безопасности. Платформы семантических уровней позволяют выполнить классификацию данных и делают ее более явной.

Определите четкие политики управления доступом

Разработайте комплексную политику управления доступом, которая определяет, кто, к каким данным имеет доступ и какие действия может выполнять. Используйте RBAC и управление доступом на основе атрибутов (Attribute-Based Access Control, ABAC), чтобы доступ к конфиденциальным данным имели только авторизованные пользователи.

Реализуйте маскировку данных

Маскировка защищает конфиденциальные данные. Внедрите правила маскировки, чтобы гарантировать сокрытие конфиденциальной информации при попытке получить ее со стороны неавторизованных пользователей. Подумайте о возможности маскировки на уровне столбца для сокрытия конфиденциальных атрибутов.

Изучите порядок распространения данных

Изучите потоки данных внутри вашего семантического уровня. Знание, как перемещаются данные, поможет понять, как преобразуются данные и как они используются, что имеет решающее значение для соответствия требованиям.

Документируйте все

Подробно документируйте политики безопасности и управления доступом к данным и каталогам. Документация необходима для соблюдения требований и устранения неполадок.

Принимая решение о реализации защиты таблиц Apache Iceberg на семантическом уровне, учитывайте плюсы и минусы этого шага. Положительным моментом является унификация доступа к данным, когда семантический уровень выступает в качестве единой точки входа для запросов и обеспечивает стандартизированные политики безопасности и управления данными. Кроме того, семантический уровень обеспечивает абстракцию данных, упрощает доступ к ним для конечных пользователей и инструментов и тем самым облегчает анализ. Однако у него есть свои недостатки, в том числе

риск превратиться в единую точку отказа из-за использования только семантического уровня, что потенциально может повлиять на доступность данных.

Dremio

Семантический уровень Dremio – это надежное решение, предлагающее ряд возможностей для защиты, мониторинга и управления озерами данных.

Отслеживание происхождения виртуальных наборов данных

Семантический уровень Dremio позволяет создавать виртуальные наборы данных, которые помогают скрыть сложность организации исходных данных. Он обеспечивает ясный и понятный способ визуализации происхождения данных, помогая пользователям понять, как на основе исходных данных получают различные виртуальные наборы. Возможность отслеживания происхождения имеет большую ценность для аудита, соблюдения требований и понимания истории преобразования ваших данных (см. пример на рис. 12.1).

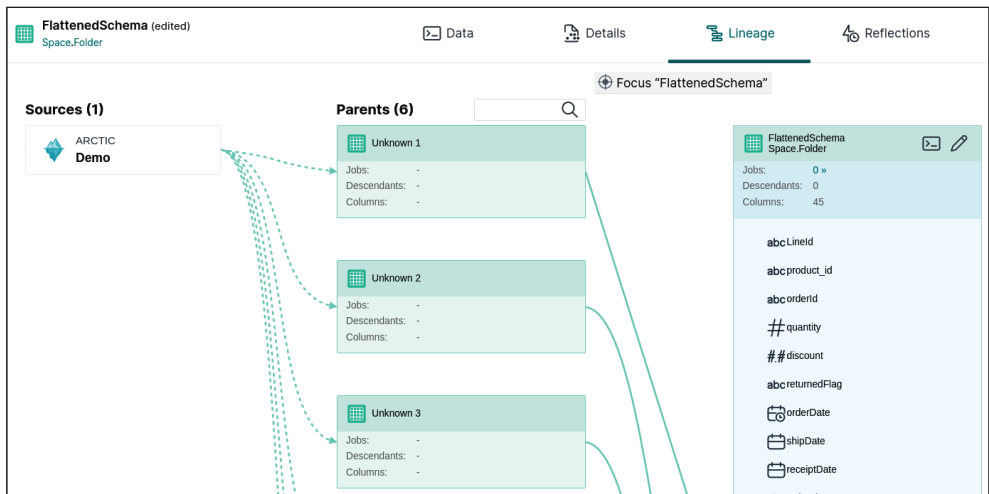


Рис. 12.1 ❖ Просмотр происхождения данных в семантическом уровне Dremio (в репозитории книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg>) доступна версия этого изображения с большим разрешением)

Встроенная вики для документации

Семантический уровень имеет встроенную поддержку вики, упрощающую документирование каталогов, виртуальных наборов данных и папок. До-

кументация имеет большое значение для управления данными, поскольку помогает пользователям понять семантику данных, их происхождение и работу бизнес-логики. Вики в Dremio позволяет создавать и поддерживать обширную документацию, обеспечивая тем самым ясность и прозрачность. На рис. 12.2 показан пример записи в вики для набора данных.

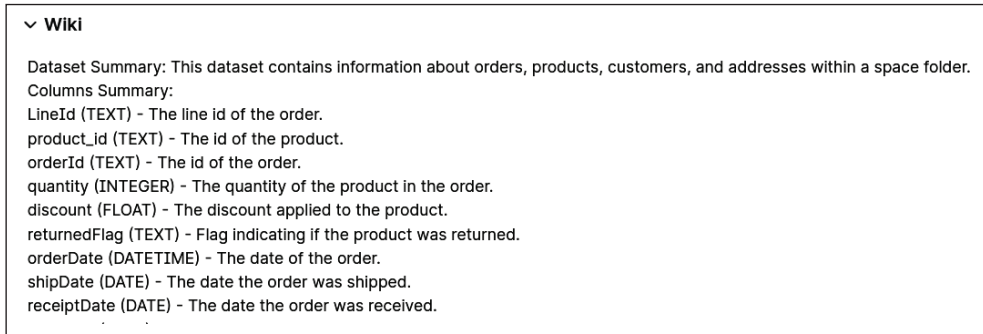


Рис. 12.2 ❖ Вики в Dremio

Правила доступа на основе ролей, столбцов и записей

Семантический уровень Dremio позволяет организовать детальное управление доступом к вашим данным. Определить правила доступа можно на нескольких уровнях.

Управление доступом на основе ролей На уровне RBAC можно создавать роли, представляющие целые категории пользователей. Например, можно создать роль аналитика, менеджера или администратора:

```
CREATE ROLE analyst;
CREATE ROLE manager;
CREATE ROLE admin;
```

Затем указать, какие роли и к каким наборам данных имеют доступ:

```
GRANT SELECT ON VDS_sales_data TO analyst;
GRANT SELECT, UPDATE ON VDS_employee_data TO manager;
```

И присвоить роли отдельным пользователям или их группам:

```
GRANT analyst TO user1;
GRANT manager TO user2;
```

При использовании подхода на основе ролей с данными могут взаимодействовать только авторизованные пользователи, наделенные конкретными ролями. Например:

- user1, которому назначена роль аналитика `analyst`, может читать данные из `VDS_sales_data`;

- user2, которому назначена роль менеджера manager, может читать и записывать данные в VDS_employee_data.

Управление доступом на основе столбцов Поддержка управления доступом на основе столбцов позволяет маскировать содержимое определенных столбцов при выполнении различных условий, например когда к данным обращается пользователь с определенной ролью. Эту особенность можно использовать для сокрытия конфиденциальных данных от тех, кто не должен иметь к ним доступа, но может использовать другие столбцы для аналитики. В Dremio такой способ управления достигается путем применения пользовательских функций (User-Defined Function, UDF), которые проверяют некоторые условия и возвращают только то, что может видеть пользователь:

```
-- Создать функцию mask_salary
CREATE FUNCTION mask_salary(salary VARCHAR)
RETURNS VARCHAR
RETURN SELECT CASE
    WHEN query_user()='user@example.com' OR is_member('HR') THEN salary
    ELSE 'XXX-XX'
END;

-- Создать таблицу employee_salaries с политикой маскировки столбца
CREATE TABLE employee_salaries (
    id INT,
    salary VARCHAR MASKING POLICY mask_salary (salary),
    department VARCHAR
);
```

Управление доступом на уровне записей Доступ к столбцам на уровне записей позволяет маскировать содержимое конкретной записи при выполнении некоторых условий, например когда пользователь наделен определенной ролью. Эту особенность можно использовать для сокрытия записей от тех, кто не должен иметь к ним доступа, но может использовать другие записи для аналитики. В Dremio такой способ управления достигается путем применения пользовательских функций (UDF), которые проверяют условия и возвращают только то, что может видеть пользователь:

```
-- Создать функцию restrict_region
CREATE FUNCTION restrict_region(region VARCHAR)
RETURNS BOOLEAN
RETURN SELECT CASE
    WHEN query_user()='user@example.com' OR is_member('HR') THEN true
    WHEN region = 'North' THEN true
    ELSE false
END;

-- Создать таблицу regional_employee_data с политикой маскировки записей
CREATE TABLE regional_employee_data (
    id INT,
    role VARCHAR,
```

```

department VARCHAR,
salary FLOAT,
region VARCHAR,
ROW ACCESS POLICY restrict_region(region)
);

```

Используя эти механизмы управления доступом, можно гарантировать, что доступ к данным и их обработка будут осуществляться в соответствии с политиками безопасности вашей организации.

Trino

Trino, ранее известный как PrestoSQL, – это распределенный механизм запросов SQL с открытым исходным кодом, предназначенный для высокопроизводительной обработки данных из различных источников данных. Trino позволяет создать семантический уровень с поддержкой слоев логических представлений поверх объединенных источников данных и возможностью управления доступом к этим представлениям.

При создании представлений можно выбрать один из двух режимов безопасности: **DEFINER** и **INVOKER**.

DEFINER

В этом режиме доступ к таблицам, на которые ссылается представление, осуществляется с использованием разрешений владельца (создателя) представления. Этот режим позволяет расширить доступ к базовым таблицам для пользователей, выполняющих запрос к представлению, которые могут не иметь прямого доступа к этим таблицам.

INVOKER

В этом режиме доступ к таблицам, на которые ссылается представление, осуществляется с использованием разрешений пользователя, выполняющего запрос. Представление в данном случае действует как хранимый запрос. Этот режим позволяет гарантировать доступность базовых данных только для тех, кто имеет разрешение на это.

Независимо от выбранного режима безопасности в представлениях всегда можно использовать функцию `current_user`, чтобы идентифицировать пользователя, выполняющего запрос. С ее помощью, например, можно реализовать фильтрацию на уровне записей или другие ограничения доступа к данным в представлениях.

Trino поддерживает RBAC как один из важнейших компонентов модели безопасности, что позволяет организациям управлять доступом к данным, назначая пользователям роли и предоставляя этим ролям определенные привилегии, и тем самым гарантировать, что только авторизованные пользователи смогут выполнять те или иные действия с данными, как определено в политиках организации.

Чтобы начать использовать RBAC в Trino, необходимо создать роли. Роли – это, по сути, именованные группы, которые можно назначать пользователям или другим ролям. Вот пара примеров создания ролей:

```
CREATE ROLE analyst;
CREATE ROLE data_scientist;
```

В предыдущем примере мы создали две роли: аналитика `analyst` и специалиста по данным `data_scientist`.

После создания ролей их можно назначить пользователям или другим ролям с помощью команды `GRANT`:

```
GRANT analyst TO USER alice;
GRANT data_scientist TO USER bob;
```

Здесь мы наделили пользователя `alice` ролью аналитика и пользователя `bob` – и ролью специалиста по данным.

Роли в Trino можно связывать с конкретными разрешениями, которые определяют, какие действия могут выполнять пользователи, наделенные этими ролями. Для связывания ролей с разрешениями доступа к таблицам, схемам или другим объектам можно использовать команду `GRANT`:

```
GRANT SELECT ON orders TO analyst;
GRANT INSERT, SELECT ON customer_data TO data_scientist;
```

В этом примере мы дали роли аналитика разрешение выполнять запросы `SELECT` к таблице `orders`, а роли специалиста по данным – разрешение выполнять операции `INSERT` и `SELECT` в таблице `customer_data`.

Для отзыва разрешений у ролей или ролей у пользователей можно использовать команду `REVOKE`:

```
REVOKE SELECT ON orders FROM analyst;
REVOKE data_scientist FROM USER bob;
```

Перечисленные выше команды позволяют настраивать управление доступом по мере необходимости.

Trino также позволяет указать администраторов ролей, имеющих право предоставлять или отзываться роли. При создании ролей можно использовать оператор `WITH ADMIN`, а при присваивании ролей – оператор `GRANTED BY`:

```
CREATE ROLE admin WITH ADMIN USER admin_user;
GRANT admin TO USER alice GRANTED BY admin_user;
```

В этом примере `admin_user` назначается администратором роли администратора `admin` и может назначить роль администратора пользователю `alice`.

Управление доступом на основе ролей особенно ценно для ограничения доступности конфиденциальных данных. Тщательно определив роли и связанные с ними разрешения, можно гарантировать, что только авторизованные пользователи будут иметь доступ к чувствительной информации.

RBAC в Trino позволяет организовать управление доступом к данным, упростить соблюдение политик безопасности и обеспечить соответствие требованиям вашей организации.

Управление безопасностью на уровне каталога

Каталог данных – это основа архитектуры озера данных. Он управляет метаданными и обеспечивает эффективное выполнение запросов. В этом разделе мы рассмотрим стратегии управления безопасностью каталогов Apache Iceberg, позволяющие сохранить контроль над ресурсами данных.

Некоторые каталоги Apache Iceberg не ограничиваются простым отслеживанием метаданных и дополнительно предлагают надежные средства эффективной защиты таблиц Iceberg. Эти механизмы улучшают защищенность данных и помогают управлять доступом к ним.

Особую ценность этим средствам придает их потенциальная переносимость между различными механизмами запросов. Это означает, что политики безопасности и средства управления доступом, которые определяются в каталоге Iceberg, могут последовательно применяться при использовании разных механизмов запросов, обеспечивая единый уровень безопасности данных независимо от того, как запрашиваются и анализируются данные. Такая переносимость упрощает управление политиками безопасности и сводит к минимуму риск появления брешей в безопасности при переходе между механизмами запросов или интеграции дополнительных инструментов в вашу экосистему. Давайте рассмотрим некоторые из средств безопасности, поддерживаемых отдельными каталогами Apache Iceberg.

Защита таблиц Apache Iceberg на уровне каталога имеет свои достоинства и недостатки. Положительным моментом является централизованное управление метаданными, что позволяет применять согласованные политики на основе метаданных. Средства защиты каталогов также поддерживают детальное управление безопасностью, предоставляя разнообразные разрешения на доступ к таблицам и базам данных. Кроме того, каталоги абстрагируют базовую файловую структуру, упрощая доступ к данным как для пользователей, так и для инструментов. Однако имеются и недостатки, в том числе зависимость метаданных и средств управления доступом от реализации каталога, а это означает, что любые проблемы с каталогом могут повлиять на доступ к данным. Кроме того, управление сложными политиками на уровне каталога может оказаться очень непростой задачей. По мере роста каталога обработка разрешений и метаданных может усложняться, что отрицательно сказывается на масштабируемости.

Nessie

Функция авторизации доступа к метаданным в Nessie имеет решающее значение для организации переносимого управления безопасностью хранилища данных Apache Iceberg. Каталог Nessie в первую очередь управляет метаданными, но он также предлагает мощный уровень авторизации, контролирующий доступ к этим метаданным. Важно отметить, что Nessie не хранит данные непосредственно, а управляет метаданными и местоположением данных. Соответственно, его механизм авторизации в первую очередь фокусируется на управлении доступом к метаданным и гарантирует, что только авторизованные пользователи или роли смогут взаимодействовать с ними.

Механизм авторизации в Nessie сосредоточен на контроле доступа к ссылкам (ветвям и меткам) и путям внутри метаданных. Пользователям и ролям могут быть предоставлены определенные разрешения на выполнение различных операций со ссылками, таких как просмотр, создание, присваивание хеша или удаление ссылок. Аналогично механизм управления доступом применяется к путям, позволяя выполнять такие операции, как создание, удаление и обновление сущностей в пределах конкретной ссылки.

Доступ к историческим данным – ключевая функция Nessie, позволяющая пользователям просматривать состояния данных в разные моменты времени. При этом важно отметить, что авторизация в Nessie в первую очередь распространяется на метаданные, и для предотвращения несанкционированного доступа к данным, хранящимся в озере, необходимо принимать дополнительные меры безопасности. Например, конфиденциальные данные в таблицах следует удалить или замаскировать, а к самому файлу данных нужно применить соответствующие меры ограничения доступа.

Модель управления доступом в Nessie построена на основе ссылок и путей, что обеспечивает довольно детальный контроль над операциями с метаданными. Правила авторизации могут определяться с помощью языка выражений Common Expression Language (CEL), что обеспечивает гибкость определения условий доступа. Эти правила определяются в файле *application.properties* и связаны с конкретными операциями, ролями, ссылками и путями. Например, можно создать правила для предоставления или запрета разрешений на просмотр, создание, удаление или обновление ссылок и сущностей на основе различных условий, включая роли и имена ссылок.

Рассмотрим несколько примеров управления разрешениями для ветвей Nessie через *application.properties*.

Код в этом примере наделяет пользователей с ролями аналитика `analyst` и наблюдателя `viewer` разрешением просматривать ветви и теги:

```
nessie.server.authorization.rules.allow_branch_listing=op=='VIEW_REFERENCE' &&
role in ['analyst', 'viewer']
```

Код в следующем примере наделяет пользователей с ролью `data_admin` правом создавать ветви, соответствующие регулярному выражению `.*prod.*`, т. е. ветви с именами, содержащими слово «prod»:

```
nessie.server.authorization.rules.allow_branch_creation=op=='CREATE_REFERENCE' &&
role=='data_admin' && ref.matches('.*prod.*')
```

Код в следующем примере наделяет пользователей с ролями `data_admin` и `super_admin` правом удалять ветви и метки:

```
nessie.server.authorization.rules.allow_branch_deletion=op=='DELETE_REFERENCE' &&
role in ['data_admin', 'super_admin']
```

Код в следующем примере наделяет пользователей с ролями `analyst` и `viewer` правом читать сущности, путь к которым начинается с `'data/'`:

```
nessie.server.authorization.rules.allow_reading_
entity_value=op=='READ_ENTITY_VALUE' && role
in ['analyst', 'viewer'] && path.startsWith('data/')
```

Код в следующем примере наделяет пользователей с ролью `data_admin` правом удалять сущности, путь к которым начинается с `'data/sensitive'`:

```
nessie.server.authorization.rules.allow Updating_specific_entity=op in ['UPDATE_
ENTITY', 'DELETE_ENTITY'] && role=='data_admin' && path=='data/sensitive'
```

По сути, функция авторизации доступа к метаданным в Nessie играет роль критически важного уровня управления безопасностью озера данных Apache Iceberg, контролируя доступ к метаданным. Правила авторизации помогают гарантировать, что только авторизованные пользователи и роли будут взаимодействовать с метаданными, и тем самым поддерживать целостность и безопасность данных. Однако имейте в виду, что функция авторизации доступа к метаданным должна дополняться более широкими мерами безопасности, чтобы защитить фактические данные, хранящиеся в озере.

Tabular

Tabular – это облачное *автономное хранилище* (уровень управления хранилищем, не зависящий от какого-либо конкретного механизма запросов или вычислений) и платформа автоматизации, предлагающая унифицированное решение для управления аналитическими данными с помощью таблиц Apache Iceberg. Фактически это централизованный узел хранения и каталогизации данных Apache Iceberg, ориентированный на универсальность и эффективность, и фреймворк управления доступом, предназначенный для защиты ресурсов платформы. Давайте перечислим его ключевые элементы.

Tabular использует модель RBAC, в которой разрешения назначаются ролям, а роли – отдельным пользователям или другим ролям.

Роли – это сущности, которые наделяются разрешениями и которые могут назначаться отдельным пользователям и другим ролям. Разрешения, в свою

очередь, определяют уровень доступа к ресурсу, включая LIST, SELECT, UPDATE и др. Пользователи получают совокупный набор привилегий в зависимости от их ролей. В стандартной конфигурации предопределены три системные роли:

ORG_ADMIN (администратор организации)

Отвечает за управление операциями на уровне организации, включая переименование и удаление организации, управление пользователями с ролью SECURITY_ADMIN, а также доступ к информации об использовании и выставлении счетов.

SECURITY_ADMIN

Управляет глобальными разрешениями на доступ к ресурсам, а также создает, отслеживает и управляет пользователями и ролями. Эта роль неявно обладает привилегией безопасности MANAGE GRANTS.

EVERYONE

Автоматически предоставляется всем пользователям. Эта роль несет в себе разрешения на доступ к ресурсам по умолчанию и обычно используется, когда явное управление доступом не требуется.

Пользователи могут определять дополнительные роли, когда не требуется предоставлять специальные разрешения. Эти роли могут наделяться специальными разрешениями для регулирования доступа к ресурсам.

AWS Glue и AWS Lake Formation

Каталог AWS Glue позволяет управлять метаданными хранилища данных в AWS. В дополнение к возможностям управления метаданными каталог AWS Glue предлагает надежные средства безопасности, в основном через сервис AWS Lake Formation, которые позволяют организациям управлять и защищать свои озера данных.

Управление доступом на основе тегов (Tag-Based Access Control, TBAC) – это ценная особенность AWS Lake Formation, дающая возможность управлять доступом к таблицам и данным, хранящимся в каталоге AWS Glue, основываясь на тегах, связанных с этими ресурсами. TBAC позволяет применять детальные политики безопасности, используя теги для ограничения доступа к определенным наборам данных. Ниже описано, как использовать TBAC для управления доступом к таблицам в каталоге AWS Glue.

Определение категорий данных с помощью тегов

Для начала нужно выполнить разделение ресурсов данных по категориям, назначив теги таблицам или другим объектам метаданных в каталоге AWS Glue. Теги могут представлять различные атрибуты данных, например уровни конфиденциальности, владельцев данных или названия проектов. Можно, к примеру, пометить таблицу тегом «Confidential», «Finance» или «ProjectA».

Назначать теги таблице можно в консоли AWS Glue, с помощью клиента командной строки AWS или AWS SDK. Вот пример использования клиента AWS:

```
aws glue update-table --database-name mydatabase --table-input '{
  "Name": "mytable",
  "Parameters": {
    "TAGS": {
      "sensitivity": "Confidential",
      "project": "ProjectA"
    }
  }
}'
```

Определение политик доступа к данным

После назначения тегов таблицам можно с помощью AWS Lake Formation определить политики доступа к данным на основе этих тегов. Политики доступа указывают, опираясь на теги, каким пользователям или группам разрешен или запрещен доступ к таблицам.

Например, вот как можно определить политику, предоставляющую доступ на чтение к таблицам, отмеченным тегом «ProjectA», только членам IAM-группы «ProjectA-Team»:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "lakeformation:GetDataAccess",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "glue:ResourceTag/project": "ProjectA"
        }
      },
      "Principal": {
        "AWS": "arn:aws:iam::123456789012:group/ProjectA-Team"
      }
    }
  ]
}
```

Политики можно создавать с помощью консоли AWS Lake Formation, клиента командной строки AWS или AWS SDK.

Применение политик к наборам данных

После определения политик их можно применить к наборам данных в каталоге AWS Glue. Политики связаны с определенными тегами, поэтому они автоматически будут наследоваться таблицами с совпадающими тегами.

AWS Lake Formation оценивает возможность выполнения запросов, основываясь на этих политиках и тегах. Пользователям или группам, которые соответствуют политикам, будет предоставлен доступ к соответствующим таблицам, а остальным будет отказано.

Мониторинг и аудит доступа

AWS Lake Formation предоставляет инструменты для мониторинга и аудита доступа к ресурсам данных. Для сбора информации о вызовах API, связанных с доступом к данным, можно использовать AWS CloudTrail. Полученная информация позволит вам увидеть, кто обращался к вашим данным и какие действия были им выполнены.

Корректировка политик по мере необходимости

Со временем требования к доступу к данным могут измениться. AWS Lake Formation позволяет легко изменять и обновлять политики и теги по мере необходимости. Вы сможете беспрепятственно адаптировать свои политики с появлением новых категорий данных или требований к доступу.

Использование интеграции с другими сервисами AWS

AWS Lake Formation легко интегрируется с другими сервисами AWS, такими как AWS IAM и AWS KMS, предлагая дополнительные возможности по обеспечению безопасности и шифрованию данных.

Внедрив TBAC в AWS Lake Formation, можно организовать строгий контроль доступа к данным на основе тегов, связанных с ресурсами данных. Такой подход гарантирует, что только авторизованные пользователи и группы смогут получить доступ к конфиденциальным данным, что поможет вам поддерживать уровень безопасности данных, соответствующий требованиям в вашей организации.

Дополнительные соображения по управлению безопасностью

К обеспечению безопасности таблиц Apache Iceberg в озере данных можно подходить на разных уровнях, имеющих свои достоинства и недостатки. В этом разделе, в частности, мы поговорим об обеспечении безопасности на уровне хранилища файлов, семантическом уровне и уровне каталога.

Выбирая, на чем сосредоточить усилия по обеспечению безопасности таблиц Apache Iceberg, учитывайте следующее:

Вариант использования

Разные варианты использования могут получить выгоды от управления безопасностью на разных уровнях. Например, для чрезвычайно конфиденциальных данных может потребоваться реализовать контроль на уровне каталога, тогда как для менее конфиденциальных данных можно ограничиться семантическим уровнем.

Масштабируемость

Подумайте о масштабируемости вашего озера данных. С ростом объема данных управление разрешениями и политиками на уровне хранилища файлов может превратиться в очень сложную задачу.

Производительность

Оцените требования к производительности. Внедрение семантического уровня может помочь оптимизировать производительность запросов, но может и увеличить накладные расходы.

Абстракция данных

Подумайте, насколько абстрактным и упрощенным должен быть доступ к данным для конечных пользователей и инструментов.

Операционные накладные расходы

Оцените операционные накладные расходы, связанные с каждым уровнем управления безопасностью.

Резервирование и аварийное переключение

Подумайте о реализации механизмов резервирования и аварийного переключения для обеспечения доступности и надежности.

Заключение

Не существует универсального подхода к обеспечению безопасности таблиц Apache Iceberg в озере данных. Выбор того или иного уровня (хранилища файлов, семантического или каталога) должен соответствовать конкретным требованиям вашей организации, вариантам использования и приоритетам. Во многих случаях наиболее эффективным способом достижения всеобъемлющей безопасности может быть сочетание этих уровней.

В главе 13 мы поговорим о миграции существующих данных в таблицы Apache Iceberg.

Глава 13

Миграция данных в Apache Iceberg

Организации постоянно ищут инновационные решения для более эффективного управления своими данными. Apache Iceberg стал мощной платформой для озер данных, предлагая высокопроизводительный формат таблиц, работающий как таблицы в системах управления реляционными базами данных (СУРБД). В этой главе рассматривается процесс миграции данных для использования преимуществ Apache Iceberg.

Зачем может понадобиться миграция данных в Apache Iceberg?

Отсутствует хранилище данных или используется формат таблиц Hive

Apache Iceberg снабдит данные в вашем озере транзакциями ACID, возможностью эволюции схем/разделов, доступом к историческим данным и многими другими возможностями, превратив ваше озеро данных в озерное хранилище, дающее гибкость озер данных и производительность/функциональность хранилищ данных.

Iceberg предлагает уникальные преимущества по сравнению с другими форматами таблиц

К уникальным особенностям Apache Iceberg относятся открытая спецификация, библиотеки с открытым исходным кодом, прозрачное управление проектами, разнообразие возможностей управления проектами, отсутствие привязки к поставщику и разнообразие экосистем.

Переход на Apache Iceberg обещает более оптимизированную архитектуру данных, но сам процесс миграции, как и любой другой переход, может быть сложным и трудоемким. Миграция предполагает адаптацию существующих структур данных, изменение конвейеров приема данных и обновление процессов обработки данных. Более того, организациям могут потребоваться рефакторинг существующих моделей данных и реорганизация данных в форматы, совместимые с Iceberg. Миграция также требует решения проблем

обратной совместимости и обеспечения беспрепятственной интеграции существующих данных в новую архитектуру. В этой главе мы рассмотрим практические приемы и стратегии эффективного преодоления этих проблем. Мы разделили главу на несколько разделов, каждый из которых посвящен конкретному аспекту процесса миграции.

Вопросы планирования миграции

Миграция данных требует тщательного планирования и соблюдения правил, чтобы обеспечить надежность процесса. В этом разделе мы расскажем вам о ключевых факторах, которые следует учитывать, прежде чем приступить к миграции. Мы обсудим основные приемы, распространенные ошибки и структурированный подход к миграции данных. Вот некоторые ключевые соображения:

Адаптация структур данных

Прежде чем переносить данные в Apache Iceberg, оцените существующие структуры данных и адаптируйте их, приведя в соответствие с форматом таблиц Iceberg. Для этого может потребоваться реструктурировать данные, переименовать столбцы или привести типы данных в соответствии с требованиями Iceberg. Совместимость данных имеет решающее значение для успеха миграции. Apache Iceberg – достаточно гибкий формат в отношении типов данных, но когда дело доходит до сложных типов, Iceberg предлагает списки, структуры и словари. Так, для данных JSON, которые могут иметь поля своих собственных типов на других платформах, вам придется решить, нужно ли преобразовать их в словари или сохранить в строковом виде.

Адаптация конвейеров

Обновите конвейеры данных, чтобы обеспечить поддержку записи данных в таблицы Iceberg. Для этого может понадобиться изменить процессы ETL и обеспечить правильное секционирование данных. К счастью, в Apache Iceberg имеется множество утилит, позволяющих довольно просто перемещать такие объекты, как таблицы Hive, Delta Lake и Hudi.

Адаптация рабочих процессов

Исследуйте и скорректируйте свои рабочие процессы для поддержки Iceberg. Для этого нужно выяснить, как изменятся зависимости данных и как повлияет использование нового формата хранения на планирование и доступ к данным. Например, денормализуя несколько наборов данных в таблицу Apache Iceberg, вы должны убедиться, что таблицы обновляются и что любые направленные ациклические графы делают прием зависимым от любых операций с исходными таблицами, которые должны быть завершены до запуска задания приема.

Вам также необходимо решить, следует ли выполнять миграцию на месте или прибегнуть к теневой миграции. *Миграция на месте* использует существующие файлы данных для построения таблиц Apache Iceberg, тогда как *теневая миграция* предполагает создание копии набора данных в Apache Iceberg и последующий переход со старого набора данных на новый. В табл. 13.1 перечислены плюсы и минусы каждого подхода.

Таблица 13.1. Плюсы и минусы миграции на месте и теневой миграции

	Миграция на месте	Теневая миграция
Плюсы	Простота Потенциально низкие затраты на аренду хранилища	Безопасность, потому что сохраняет исходные данные и позволяет выполнить тестирование Минимальное влияние на существующие системы
Минусы	Повышенные риски, потому что данные переносятся напрямую и нет простой возможности выполнить откат в случае появления проблем Подходит только для таблиц в формате, поддерживаемом Iceberg	Повышенная сложность Потенциально высокие затраты на аренду хранилища

План миграции на месте в три этапа

Как вы увидите далее в этой главе, есть несколько утилит, помогающих выполнить миграцию на месте, включая процедуры `migrate` и `add_files`. Так как в этом случае нет необходимости записывать новые файлы, следует рассмотреть следующие вопросы:

- как переносить таблицу, за одну транзакцию или постепенно по разделам;
- когда следует переключить механизмы чтения и записи на использование новых таблиц вместо старых.

Наборы данных малого и среднего размеров можно легко перенести за одну транзакцию, но большие таблицы разумнее переносить постепенно, добавляя разделы в таблицу Apache Iceberg по одному. При выполнении постепенной миграции проверка числа записей и файлов между заданиями поможет убедиться, что новая таблица Iceberg является точной копией старой. С учетом вышесказанного план миграции на месте будет включать следующие шаги:

1. Определение количества файлов и записей в разделе старой таблицы.
2. Перенос файлов из существующего раздела в таблицу Apache Iceberg.
3. Определение количества файлов и записей в одном разделе таблицы Iceberg, чтобы убедиться в их совпадении.

Эти шаги нужно повторять, пока все разделы не будут добавлены в таблицу Iceberg. Затем можно переключить все операции чтения и записи на использование таблицы Apache Iceberg.

План теневой миграции в четыре этапа

При составлении плана теневой миграции данных в новую таблицу Apache Iceberg следует использовать многоэтапный подход, чтобы дать себе время на постепенную адаптацию новых систем. Ниже приводится обобщенный план такой миграции (см. схему на рис. 13.1).

Этап 1. Запись в старую систему, чтение из старой системы

На начальном этапе настройки таблиц Iceberg продолжайте записывать данные в существующую систему. Этот этап позволит вам создать инфраструктуру Iceberg, не затрагивая текущие операции. Однако в данный момент еще нельзя воспользоваться возможностями Iceberg. Также следует переписать все исторические данные из старой таблицы в новую и только потом переходить ко второму этапу записи новых данных в Iceberg.

Этап 2. Запись в старую и новую системы, чтение из старой системы

На этом этапе новые данные записываются как в старую, так и в новую системы. Такая избыточность обеспечит согласованность данных, но увеличит затраты на хранение.

Этап 3. Запись в старую и новую системы, чтение из новой системы

Как только вы будете уверены в правильности настроек Iceberg, переключите операции чтения на использование новых таблиц Iceberg. Убедитесь, что во время этого перехода между обеими системами поддерживается согласованность данных.

Этап 4. Запись в новую систему, чтение из новой системы

Постепенно откажитесь от старой системы и начните записывать данные исключительно в новые таблицы Iceberg. Внимательно следите за миграцией, чтобы выявить любые потенциальные проблемы.

Следуя этим рекомендациям, вы сможете свести к минимуму сбои, минимизировать риски и обеспечить успех миграции данных в Apache Iceberg, сохранив при этом целостность и доступность данных.

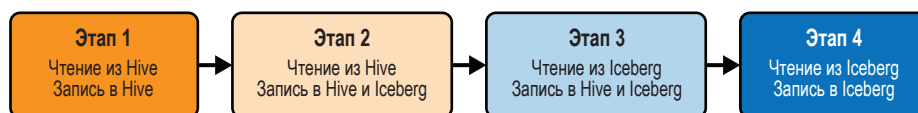


Рис. 13.1 ❖ План теневой миграции в четыре этапа

Миграция таблиц Hive в Apache Iceberg

Миграция с таблиц Hive в таблицы Apache Iceberg с использованием процедур, встроенных в Iceberg, дает лучшую управляемость и надежность. В этом

разделе мы рассмотрим процесс миграции таблиц Hive в Iceberg с помощью Spark и Dremio, что позволит пользователям использовать весь функциональный потенциал и надежность Iceberg. Расширения Iceberg Spark предлагают две процедуры, `snapshot` и `migrate`, каждая из которых служит определенной цели в процессе миграции таблиц Hive.

Процедура `snapshot`

При миграции из таблиц Hive в Apache Iceberg для создания временной копии таблицы в Iceberg с целью тестирования удобно использовать процедуру `snapshot`. Этот способ позволяет безопасно экспериментировать с данными, не затрагивая исходную таблицу. Снимок создается с использованием файлов данных из исходной таблицы.

Для демонстрации рассмотрим следующий пример кода:

```
-- Создать снимок таблицы 'db.tableA' и присвоить ему имя 'db.tableAsnapshot'
CALL catalog.system.snapshot('hive.db.tableA', 'db.tableAsnapshot')

-- Также есть возможность указать альтернативное местоположение снимка
CALL catalog.system.snapshot('hive.db.tableA', 'db.tableAsnapshot', 's3://
bucket/location')
```

В первом примере на основе таблицы Hive `tableA` создается снимок Apache Iceberg с именем `tableAsnapshot`. После этого у нас остается таблица Hive и появляется таблица Apache Iceberg, использующие одни и те же файлы данных. По умолчанию в пути будет создана папка для размещения метаданных и любых файлов данных, которые будут созданы позже. Это местоположение можно изменить, указав новое местоположение в третьем аргументе, как показано во втором примере.

Процедура `migrate`

Процедура `migrate` упрощает миграцию таблицы Hive в таблицу Apache Iceberg, сохраняя исходные файлы данных нетронутыми. Она копирует схему таблицы, спецификацию секционирования, свойства и местоположение из исходной таблицы в новую. Эта процедура – довольно мощный инструмент. Если файлы данных исходной таблицы имеют поддерживаемые форматы, такие как Avro, Parquet или ORC, то процесс выполняется особенно эффективно.

Вот как использовать процедуру `migrate`:

```
-- Мигрировать таблицу 'hive.db.sample' в таблицу Iceberg
-- с дополнительными свойствами
CALL catalog.system.migrate('hive.db.sample', map('foo', 'bar'))
```

Этот код мигрирует таблицу Hive `sample` в каталог Apache Iceberg. Метаданные новой таблицы создаются на основе существующих файлов данных,

но доступ к новой таблице через старую таблицу Hive исключается. Теперь к ней необходимо обращаться как к таблице Apache Iceberg. Если у вас есть нестандартные свойства, которые вы хотите перенести в новую таблицу, то можете передать словарь с этими свойствами во втором аргументе.

При использовании процедуры `migrate` следует уделить особое внимание совместимости схемы, решить, сохранять ли исходную таблицу, и передать процедуре все необходимые дополнительные свойства:

Совместимость

Убедитесь, что схема исходной таблицы совместима с требованиями Iceberg. Как отмечалось выше, в число поддерживаемых форматов файлов данных входят Avro, Parquet и ORC. Если схема исходной таблицы несовместима, то во время миграции могут возникнуть проблемы.

Создание резервных копий

При использовании процедуры `migrate` важно решить, следует ли сохранять исходную таблицу в качестве резервной копии. Сохранение резервной копии может пригодиться для отката или тестирования. Однако это может увеличить затраты на хранение. Установка `drop_backup` в значение `true` отменит сохранение исходной таблицы (это означает, что ссылка на таблицу в Hive Metastore в резервном пространстве имен не сохранится, но файлы данных уже будут использоваться новой таблицей Apache Iceberg).

Миграция из Delta Lake в Apache Iceberg

Миграция из Delta Lake в Apache Iceberg открывает новые возможности для управления данными, сохраняя при этом их историю и обеспечивая совместимость с надежным форматом таблиц. Хранилище Delta Lake, известное своей поддержкой формата файлов Parquet и функциями версионирования и доступа к историческим данным, предоставляет ценные возможности для управления данными.

При миграции из Delta Lake в Iceberg можно использовать действие `snapshotDeltaLakeTable` из модуля `iceberg-delta-lake`. Это действие позволяет создать копию существующей таблицы Delta Lake в Iceberg, которая будет использовать файлы данных исходной таблицы. Вновь созданная таблица Iceberg будет иметь ту же схему и секционирование, что и исходная таблица Delta Lake. Это действие обеспечивает плавный переход и гарантирует целостность данных.

Для миграции из Delta Lake в Iceberg вам понадобится минимальное число зависимостей, включая `iceberg-delta-lake`, `delta-standalone-0.6.0` и `delta-storage-2.2.0`. Процесс миграции поддерживает таблицы Delta Lake с определенными версиями протокола, обеспечивающими совместимость.

В следующем примере показано использование действия `snapshotDeltaLakeTable`. Как можно видеть, вызов действия включает передачу местоположения исходной таблицы Delta Lake, местоположения целевой таблицы, идентификатора новой таблицы Iceberg, используемого каталога, конфигурации Hadoop для доступа и дополнительных свойств таблицы:

```
import org.apache.iceberg.catalog.TableIdentifier;
import org.apache.iceberg.catalog.Catalog;
import org.apache.hadoop.conf.Configuration;
import org.apache.iceberg.delta.DeltaLakeToIcebergMigrationActionsProvider;

// местоположение исходной таблицы delta lake
String sourceDeltaLakeTableLocation = "s3://my-bucket/delta-table";

// местоположение новой таблицы Apache Iceberg
String destTableLocation = "s3://my-bucket/iceberg-table";

// Имя таблицы
TableIdentifier destTableIdentifier = TableIdentifier.of("my_db", "my_table");

// каталог iceberg, куда добавляется новая таблица
Catalog icebergCatalog = ...;

// настройки файловой системы для таблицы Delta Lake
Configuration hadoopConf = ...;

DeltaLakeToIcebergMigrationActionsProvider.defaultActions()
    .snapshotDeltaLakeTable(sourceDeltaLakeTableLocation)
    .as(destTableIdentifier)
    .icebergCatalog(icebergCatalog)
    .tableLocation(destTableLocation)
    .deltaLakeConfiguration(hadoopConf)
    .tableProperty("my_property", "my_value")
    .execute();
```

Действие `snapshotDeltaLakeTable` упрощает процесс миграции, гарантирует сохранность истории данных и одновременно предоставляет преимущества функций Iceberg.

Миграция из Apache Hudi в Apache Iceberg

Apache Hudi – это популярный формат хранения для озер данных, известный своей поддержкой транзакций ACID и эффективным управлением данными. Тем не менее давайте представим, что мы рассматриваем возможность миграции из Hudi в Apache Iceberg. Надеемся, вам будет приятно узнать, что когда мы писали эту книгу, для таблиц Hudi разрабатывалась процедура, аналогичная процедуре создания снимков Delta Lake (запрос № 6642). Эта

процедура позволяет быстро перенести данные, используя файлы данных существующей таблицы.

Процедура миграции таблиц Hudi в таблицы Iceberg включает использование класса `HudiToIcebergMigrationActionsProvider` из модуля `iceberg-hudi`. Ключевым действием для этой миграции является `snapshotHudiTable`, который позволяет сделать снимок существующей таблицы Hudi в новую таблицу Iceberg. Это действие предлагает надежный способ миграции из Hudi в Iceberg.

Вот пример использования действия `snapshotHudiTable` для миграции таблицы Hudi в таблицу Iceberg:

```
import org.apache.iceberg.catalog.TableIdentifier;
import org.apache.iceberg.catalog.Catalog;
import org.apache.hadoop.conf.Configuration;
import org.apache.iceberg.hudi.HudiToIcebergMigrationActionsProvider;

// Местоположение существующей таблицы hudi
String hudiTablePath = "hdfs://my-hudl-tablet";

// Имя новой таблицы iceberg
String newTableIdentifier = "my_db.my_table";

// Каталог новой таблицы iceberg
Catalog icebergCatalog = ...;

// Настройки файловой системы для таблицы Hudi
Configuration hadoopConf = ...;

HudiToIcebergMigrationActionsProvider.defaultProvider()
    .snapshot AudiTable(hudiTablePath)
    .as(TableIdentifier.parse(newTableIdentifier))
    .hoodieConfiguration(hadoopConf)
    .icebergCatalog(icebergCatalog)
    .execute();
```

В этом примере мы указываем путь к таблице Hudi, идентификатор новой таблицы Iceberg, конфигурацию Hadoop для доступа к файлам данных и каталог Iceberg.

Миграция отдельных файлов в Apache Iceberg

В некоторых сценариях наборы данных могут храниться в виде отдельных файлов, например наборы данных Parquet (в этих случаях Hive Metastore не поддерживает каталоги разделов, содержащие эти файлы), и требуется перенести эти файлы в таблицу Apache Iceberg для улучшения управления, увеличения производительности запросов и получения возможности эво-

люции схемы. Для таких сценариев Apache Iceberg предоставляет процедуру `add_files`, позволяя импортировать файлы в существующую таблицу Iceberg без создания новой таблицы. Кроме того, эту процедуру можно использовать для переноса данных из таблиц Delta Lake и Apache Hudi без сохранения истории.

Использование процедуры `add_files`

Процедура `add_files` – это универсальный способ добавления внешних файлов данных в таблицу Apache Iceberg. Она не анализирует схему файлов, поэтому вы должны сами убедиться, что файлы соответствуют схеме и секционированию целевой таблицы Iceberg, дабы предотвратить любые ошибки при чтении таблицы Apache Iceberg. Вот как можно использовать процедуру `add_files`:

```
CALL catalog.system.add_files(
  table => 'db.my_table',
  source_table => 's3://my-parquet-tables/tables',
  partition_filter => map('partition_col', 'partition_value'),
  check_duplicate_files => true
)
```

В этом примере мы сообщаем механизму, что все файлы данных, находящиеся в определенной папке, должны быть добавлены в таблицу `my_table`. Мы можем использовать аргумент `partition_filter`, чтобы добавить файлы только из определенного раздела, и включить проверку дубликатов файлов, дабы избежать многократного добавления одного и того же файла.

Эта процедура создаст метаданные для новых файлов и будет рассматривать их как часть набора файлов таблицы Iceberg (предполагается, что эти файлы соответствуют секционированию и схеме таблицы Iceberg). Стоит отметить, что последующие операции Iceberg могут физически удалять файлы, добавленные с помощью этого метода.

Миграция из Delta Lake или Apache Hudi без сохранения истории

Чтобы мигрировать данные из таблиц Delta Lake или Apache Hudi в таблицы Apache Iceberg без сохранения истории, выполните следующие действия:

1. Уничтожьте все предыдущие снимки в таблице Delta Lake или Hudi, чтобы сохранить только файлы в текущем снимке.
2. Используйте процедуру `add_files`, чтобы импортировать файлы из текущего снимка в целевую таблицу Iceberg, как показано в предыдущем примере.

Этот подход позволяет перейти на Apache Iceberg, сохранив только последнюю версию данных, что может быть полезно при сохранении упрощенной версии озера данных.

В лице процедуры `add_files` Apache Iceberg предоставляет гибкий и эффективный способ переноса отдельных файлов или данных из других форматов хранения в таблицу Iceberg.

Миграция откуда угодно путем копирования данных

Гибкость часто называется первостепенной целью. Когда дело доходит до миграции данных из различных источников в таблицу Apache Iceberg, на выбор есть два подхода. Перенести данные в новую таблицу можно с помощью оператора `CREATE TABLE...AS SELECT` (CTAS) в любом механизме, включая Spark, Dremio, Trino и Presto. Также есть возможность вставлять данные в существующие таблицы Iceberg из других источников, не являющихся таблицами, например из файлов JSON/CSV с помощью команды `COPY INTO` (Dremio/Snowflake) и из любой другой таблицы с помощью команды `INSERT INTO SELECT`.

Миграция данных в новую таблицу Iceberg

Миграция данных в новую таблицу Apache Iceberg с помощью оператора CTAS – мощный и универсальный метод по следующим причинам:

Эволюция схемы

CTAS позволяет адаптировать схему во время миграции. Вы можете отображать столбцы из исходной таблицы в целевую, переименовывать столбцы, изменять типы данных и применять выражения для вычисляемых столбцов.

Управление секционированием

Если исходные данные не секционированы нужным образом, то с помощью CTAS можно создавать разделы на основе определенных столбцов, дат или других критериев. Это поможет обеспечить лучшую организацию данных и производительность запросов.

Преобразование данных

В запросе CTAS можно осуществлять преобразование данных, очищать их, агрегировать и вообще выполнять любую предварительную обработку.

Этот подход обеспечивает детальный контроль над процессом миграции и дает возможность вносить изменения в схему и данные во время миграции. Ниже подробно описывается, как выполнить такую миграцию.

Сначала подключитесь к источнику исходных данных и к целевому каталогу Iceberg. Обычно это предполагает настройку механизма запросов (например, Dremios SQL Query Engine, Apache Spark, Trino, Presto) для доступа к исходным данным и целевому каталогу Iceberg.

После установки соединений можно выполнить миграцию с помощью оператора CTAS. Оператор CTAS создаст новую таблицу Iceberg и заполнит ее данными из источника. Вот пример использования Apache Spark SQL:

```
CREATE TABLE catalog.db.tableA
  USING iceberg
  PARTITIONED BY (month(ts_field))
  AS
  SELECT *,
    CAST(old_field AS <data_type>) AS updated_field
  FROM my_source_table;
```

В этом примере создается новая таблица Apache Iceberg в каталоге tableA. В таблицу копируются результаты запроса SELECT и попутно преобразуется тип данных старого поля. Здесь также определяется схема секционирования с использованием преобразования month, чтобы задействовать возможности секционирования Iceberg. Затем записываются новые файлы данных для создания этой таблицы.

При миграции больших наборов данных в новую таблицу Iceberg следует учитывать несколько рекомендаций, касающихся масштабируемости и производительности:

Выполняйте миграцию поэтапно

Подумайте о возможности поэтапного подхода вместо переноса всего набора данных за один раз. Начните с оператора CTAS, который выбирает один раздел или подмножество данных. После этого используйте инструкции INSERT INTO SELECT для постепенного добавления данных в разделы. Это снижает риск создания слишком большой нагрузки на ресурсы и улучшает управляемость. При этом обязательно проверяйте число записей по разделам, чтобы гарантировать полноту миграции данных.

Оптимизируйте для параллельной обработки

Производительность можно увеличить за счет параллельной обработки, если таковая поддерживается вашей системой запросов. Убедитесь, что параметры поддержки параллелизма в механизме запросов настроены соответствующим образом.

Осуществляйте мониторинг и журналирование

Внимательно следите за процессом миграции. Отслеживайте ход ее выполнения, потребление ресурсов и любые возникающие ошибки или проблемы. Журналирование миграции необходимо для устранения неполадок и аудита.

В целом использование оператора CTAS с механизмами запросов обеспечивает детальный контроль над процессом миграции в таблицу Apache Ice-

berg. Этот метод учитывает изменения схемы, стратегии секционирования и преобразования данных, что делает его пригодным для широкого спектра сценариев миграции. При работе с большими наборами данных подумайте о возможности поэтапной миграции и оптимизации производительности для большей эффективности процесса передачи данных с сохранением истории таблиц.

Миграция данных в существующую таблицу Iceberg

Для переноса данных в существующую таблицу Apache Iceberg требуются другие подходы, чем для CTAS, который создает новую таблицу. Здесь мы обсудим два распространенных пути:

- вставка данных из нетабличного источника (файлов) с помощью оператора `COPY INTO`;
- вставка данных из другой таблицы с помощью инструкции `INSERT INTO SELECT`.

COPY INTO

Оператор `COPY INTO` извлекает данные из источника, который не является каталогизированной таблицей, и вставляет их в целевую таблицу (создавая новые файлы данных). Использование `COPY INTO` дает множество преимуществ:

Приведение схемы

Оператор `COPY INTO` автоматически приводит данные из исходных файлов без схемы, таких как CSV, в соответствие со схемой целевой таблицы Iceberg, корректируя типы данных, имена столбцов и структуры по мере необходимости, что снижает риск несоответствия типов данных.

Эффективный прием данных

Оператор `COPY INTO` оптимизирован для эффективного приема данных, экономя ваше время и избавляя от необходимости вручную размещать данные в виде другой таблицы.

Инкрементный прием данных

Оператор `COPY INTO` можно также применять для приема дополнительных данных. При появлении новых файлов CSV, JSON или Parquet, которые нужно добавить в таблицу Iceberg, достаточно еще раз запустить оператор для переноса новых файлов.

Этот подход полезен тем, что избавляет от необходимости подготовки данных в виде таблицы, когда нужно просто вставить данные в другую таблицу. Рассмотрим пример его реализации:

```
-- Пример COPY INTO с использованием синтаксиса Dremio
COPY INTO catalog.db.my_iceberg_table
```

```
FROM '@my_dremio_source/folder'
FILE_FORMAT 'csv';
```

Цель в этом примере состоит в том, чтобы добавить данные из файлов CSV в таблицу с именем `my_iceberg_table`. Dremio прочитает все файлы CSV, применит схему из целевой таблицы и запишет новые файлы данных, которые будут добавлены в указанную таблицу.

Существует несколько соображений для успешной миграции с использованием `COPY INTO`:

Качество данных

Убедитесь, что данные в файлах CSV, JSON или Parquet соответствуют схеме целевой таблицы Iceberg. Несогласованные или искаженные данные могут привести к ошибкам приема.

Организация файлов

Организируйте файлы данных так, чтобы это имело смысл для вашего сценария. Например, если вы секционируете таблицу Iceberg по дате и выполняете копирование инкрементно, то организуйте файлы в папки по датам, это позволит явно указать, какие данные должны приниматься на каждом этапе.

Параметры формата файлов

В зависимости от характеристик данных можно указать дополнительные параметры в операторе `FILE_FORMAT`, такие как форматы даты и времени, символы-разделители и обработка значений `NULL`.

Оператор `COPY INTO` в Dremio способен выполнять инкрементальный прием данных, т. е. с его помощью можно добавлять новые данные в существующую таблицу Iceberg без загрузки всего набора данных.

Сначала нужно подготовить новые данные, организовав их в файлы, поместив файлы в исходное местоположение и добавив в их имена префикс с целевой датой. Затем следует повторно запустить команду `COPY INTO` и отфильтровать файлы, используя регулярные выражения, чтобы скопировать только файлы с префиксом, соответствующим определенной дате, указав ту же целевую таблицу Iceberg. Например:

```
-- Синтаксис DremioSQL
-- Использовать оператор COPY INTO с регулярным выражением
COPY INTO my_table
FROM '@my_storage_location'
REGEX '^2023-10-11_.*\.csv'
FILE_FORMAT 'csv';
```

В Snowflake оператору `COPY INTO` можно передать разные папки, представляющие разные точки приема:

```
-- Синтаксис SnowflakeDB
-- Использовать оператор COPY INTO для приема файлов
COPY INTO '@my_external_stage'
```

```
FROM '@s3://my-s3-bucket/path/tablea-2023-10-11'
FILE_FORMAT = (TYPE = 'CSV' FIELD_DELIMITER = ',' SKIP_HEADER = 1)
CREDENTIALS = (AWS_KEY_ID = '<your_aws_key_id>' AWS_SECRET_KEY =
'<your_aws_secret_key>')
VALIDATION_MODE = RETURN_ROWS;
```

Оператор `COPY INTO` предлагает простой и эффективный метод переноса данных в существующую таблицу Apache Iceberg. Он обеспечивает преобразование схемы и эффективный прием данных. Кроме того, он поддерживает инкрементные обновления, что делает его удачным выбором для поддержки истории таблиц при сохранении новых данных в таблице Iceberg.

INSERT INTO SELECT

Используя любой механизм запросов, можно вставлять данные из любой таблицы в таблицу Apache Iceberg с помощью `INSERT INTO SELECT`. Эта возможность может пригодиться при работе с механизмами, способными объединять данные из нескольких источников, такими как Dremio, Trino и Presto:

```
INSERT INTO tableB
SELECT * FROM tableA;
```

Для обновления данных можно использовать оператор `MERGE INTO`. Это более простое решение, если в исходной таблице есть поля `update_at` и `create_at`:

```
-- Синтаксис DremioSQL
MERGE INTO tableB AS target
Migrating from Anywhere by Rewriting Data | 281
USING (
    SELECT *
    FROM tableA
    WHERE created_at >= DATE_DIFF(CURRENT_DATE(), CAST(30 AS INTERVAL DAY))
        OR updated_at >= DATE_DIFF(CURRENT_DATE(), CAST(30 AS INTERVAL DAY))
) AS source
ON target.id = source.id
WHEN MATCHED THEN
    UPDATE SET
        target.column1 = source.column1,
        target.column2 = source.column2,
        target.created_at = source.created_at,
        target.updated_at = source.updated_at
WHEN NOT MATCHED THEN
    INSERT *
```

Запрос в этом примере объединяет данные, созданные или обновленные за последние 30 дней. Такой подход повышает производительность, потому что объединяется только ограниченное подмножество записей в обеих таблицах. При работе с таблицами использование оператора `INSERT INTO` позволяет легко выбирать данные из сторонних таблиц и вставлять их в таблицы Apache Iceberg. Более того, `MERGE INTO` упрощает выполнение комплексных обновлений на платформах, поддерживающих эту возможность.

Заклучение

В этой главе представлен подробный обзор стратегий и эффективных практик миграции данных в Apache Iceberg. Для начала мы заложили основу, выделив важные соображения и структурированные подходы, необходимые для успешной миграции. Понимание проблем и ловушек имеет первостепенное значение для успешной миграции.

Затем мы углубились в конкретные сценарии миграции, начав с таблиц Hive и показав, как использовать процедуры `migrate` и `snapshot`. Мы рассмотрели миграцию из Delta Lake и Apache Hudi, а для тех, кто работает без использования табличных форматов, рассмотрели миграцию отдельных файлов. Такая гибкость позволяет эффективно перемещать данные без ущерба для возможностей управления и производительности Iceberg.

Также мы рассмотрели возможность копирования данных из различных источников, в том числе с помощью оператора CTAS, `COPY INTO` и `INSERT INTO SELECT` в новые и существующие таблицы Apache Iceberg, независимо от исходного формата. Такая адаптивность позволяет беспрепятственно принимать данные из разных источников.

В главе 14 мы рассмотрим порядок использования Apache Iceberg в некоторых сценариях, включая информационные панели бизнес-аналитики и машинное обучение.

Глава 14

Примеры использования Apache Iceberg

В этой главе мы перечислим некоторые примеры использования Apache Iceberg в реальном мире и поделимся практическим опытом реализации различных сценариев, поддерживаемых архитектурой озера хранения данных. При рассмотрении сценариев мы будем акцентировать наше внимание на обеспечении качества данных в озерах данных, создании отчетов бизнес-аналитики и реализации критически важных процессов, таких как CDC. Другие сценарии построения аналитической архитектуры реального времени, реализации машинного обучения и медленно меняющихся измерений (Slowly Changing Dimensions, SCD) вы найдете в дополнительных главах 15 и 16. Эта глава представляет собой вводное руководство, которое показывает, как работать с Iceberg в реальных приложениях, а также подчеркивает его адаптивность и важность как основного элемента в любой архитектуре данных.

Обеспечение высокого качества данных в Apache Iceberg

Поддержание качества данных на высочайшем уровне имеет решающее значение для получения значимой информации. Если качество данных ухудшается на каком-то этапе рабочего процесса обработки данных, это может отрицательно повлиять на последующий анализ. Например, рассмотрим процесс извлечения, преобразования и загрузки (Extract, Transform, Load, ETL): он извлекает данные из операционной системы и передает их в аналитическую систему для использования в отчетах или специальном анализе. Если исходные данные имеют повторяющиеся значения или несоответствия или если такие проблемы возникают в процессе ETL и не устраняются до попадания

в аналитическую среду, то в результате могут быть сделаны ошибочные выводы, ведущие к неверным решениям. Это подчеркивает решающую важность качества данных в рабочих процессах обработки данных и необходимость обеспечивать точность, согласованность и надежность данных.

Обеспечить систематический подход к обеспечению хорошего качества данных помогает шаблон WAP (Write Audit Publish – записать, проверить, опубликовать). Давайте рассмотрим этот процесс более подробно.

1. **Запись:** данные сначала извлекаются из источников и записываются в промежуточное хранилище, благодаря чему промышленные данные изолируются от потенциальных несоответствий.
2. **Проверка:** после копирования данные подвергаются тщательной проверке типов данных, их целостности и удалению пустых или повторяющихся значений.
3. **Публикация:** после проверки данные атомарно копируются в промышленные таблицы. Атомарность гарантирует, что потребители увидят обновленный набор данных целиком или не увидят никаких обновлений, если в ходе копирования возникла какая-то проблема.

Давайте посмотрим, как можно реализовать этот процесс с помощью Apache Iceberg. Хотя Iceberg предлагает API и семантику для реализации WAP, бремя фактической реализации шаблонов ложится на вычислительный механизм. Apache Spark поддерживает реализацию WAP, поэтому мы будем использовать его.

WAP с использованием функции ветвления в Iceberg

Существует несколько способов реализации WAP в Iceberg, но мы сосредоточимся на подходе, использующем возможность ветвления на уровне таблицы Iceberg. Ветвление в Iceberg работает аналогично ветвлению в Git (<https://git-scm.com/docs/git-branch>) и позволяет создавать локальные ветви в промышленной таблице для выполнения изолированных действий с данными. Основным компонентом Iceberg, лежащим в основе ветвления, является снимок, который описывает состояние таблицы Iceberg в определенный момент времени. Ветви называются ссылками на эти снимки. Мы подробно обсудили ветвление в главе 10.

Давайте представим, что данные о продажах продуктов компании электронной коммерции регулярно извлекаются из систем управления реляционными базами данных и управления взаимоотношениями с клиентами, а затем загружаются в озеро данных Amazon Simple Storage Service (Amazon S3). Команда инженеров данных отвечает за создание и поддержку таблиц Iceberg в озере данных, предназначенных для обслуживания различных промышленных приложений, таких как аналитические отчеты и прогнозные модели. Как они могут гарантировать качество данных перед их отправкой в промышленную среду?

Первый шаг к реализации WAP в Iceberg – создание новой ветви на основе промышленной версии таблицы Iceberg, чтобы гарантировать, что она не оказала влияния на основную ветвь. После создания ветви в нее могут приниматься новые записи. После проверки достоверности данных и обеспечения соответствия данных стандартам качества их можно опубликовать в основной ветви, а если проверки не пройдены, то ветвь можно закрыть и еще раз попытаться выполнить задание, не влияя на производство. Этот процесс показан на рис. 14.1.

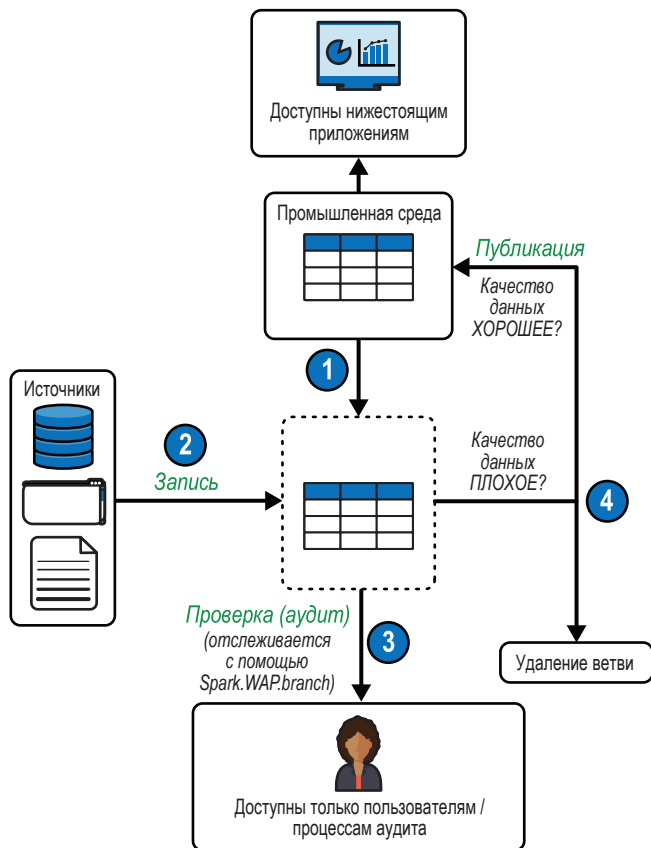


Рис. 14.1 ❖ Процесс WAP на уровне таблицы

Давайте попрактикуемся в реализации шаблона WAP для этого примера.

Создание ветви

Сначала создадим из существующей таблицы Iceberg новую ветвь с именем `ETL_branch`. Эта ветвь будет играть роль промежуточного хранилища для новых данных:

```
# Создать ветвь в таблице 'etl_branch'
spark.sql("ALTER TABLE catalog.db.table CREATE BRANCH etl_branch").show()
```

Затем выполним запрос к этой ветви и узнаем общее количество записей в наборе данных:

```
# Получить количество записей в таблице
spark.sql("SELECT COUNT(*) as total_records FROM catalog.db.table").show()
```

Этот запрос выведет общее количество записей в таблице. Это число пригодится для проверки задания приема после его запуска и поможет убедиться, что было добавлено правильное количество записей. Давайте также подтвердим существование созданной нами ветви с помощью следующего запроса, который возвращает список всех ссылок в таблице, включая ветви и теги:

```
# Вернуть список всех ссылок в таблице
spark.sql("SELECT * FROM catalog.db.table.refs").show()
```

В списке должны присутствовать основная ветвь main и созданная ранее etl_branch.

Запись данных

Прежде чем начать прием данных методом WAP, нужно выполнить несколько подготовительных шагов. Сначала настроим свойство таблицы Iceberg, добавив `write.wap.enabled=true`. Этот шаг подготавливает таблицу Iceberg к использованию шаблона WAP. Далее, чтобы гарантировать, что любые действия, будь то чтение или запись, будут применяться к этой конкретной ветви, укажем идентификатор ветви `etl_branch` в параметре `spark.wap.branch` конфигурации сеанса Spark:

```
# Включить поддержку WAP в Apache Iceberg
spark.sql("ALTER TABLE catalog.db.table SET TBLPROPERTIES ('write.wap.enabled='true')")
```

```
# Настроить таблицу для WAP
spark.conf.set('spark.wap.branch', 'etl_branch')
```

Теперь можно запустить задание ETL для приема новых записей в эту конкретную ветвь таблицы:

```
# Вставить новые записи в таблицу
spark.sql("INSERT INTO catalog.db.table SELECT * FROM new_data")
```

Если сейчас запросить количество записей в таблице, то вы увидите, что в таблице появились новые записи:

```
# Узнать количество записей в 'etl_branch'
spark.sql("SELECT COUNT(*) as total_records FROM catalog.db.table VERSION AS OF 'etl_branch']").show()
```

Напомним, что эта ветвь действует как самостоятельная версия промышленных данных. Любое действие, предпринятое в этой ветви, окажет влияние только на этот набор данных. Чтобы убедиться в этом, можно проверить количество записей в ветви `main`, оно должно совпасть с количеством, которое было получено предыдущим запросом:

```
# Вывести количество записей в ветви main
spark.sql("SELECT COUNT(*) as total_records FROM catalog.db.table VERSION AS OF 'main']").show()
```

Проверка данных

После записи новых данных в изолированную локальную ветвь `etl_branch` важно убедиться, что они соответствуют стандартам качества организации. Этап аудита действует как контрольно-пропускной пункт, на котором мы подвергаем новые данные тщательной оценке и убеждаемся в их соответствии имеющимся требованиям.

Для проверки можно написать свой код или прибегнуть к помощи сторонних инструментов, предназначенных для проверки качества данных. В этом упражнении мы выполним несколько простых проверок.

Значения NULL Сначала давайте используем библиотеку PySpark и с ее помощью проверим присутствие в таблице значений `null`:

```
# df = набор данных, извлеченный из таблицы "catalog.db.table"
# Проверить наличие пустых значений в каждом столбце
null_counts = df.select([count(when(col(c).isNull(), c)).alias(c) for c in df.columns])

# Вывести результат
null_counts.show()
```

Полученный набор данных `null_counts` будет содержать количество пустых значений в каждом столбце.

Повторяющиеся записи Для следующей проверки снова используем библиотеку PySpark и определим наличие в нашей таблице повторяющихся записей:

```
# df = набор данных, извлеченный из таблицы "catalog.db.table"
# Сгруппировать записи по всем столбцам и подсчитать
duplicates = df.groupBy(df.columns).count()

# Оставить только повторяющиеся записи, т. е. с 'count' > 1
duplicates = duplicates.filter(col("count") > 1)

# Дополнительно можно отбросить столбец count, если он не нужен
duplicates = duplicates.drop("count")

# Вывести результат (повторяющиеся записи)
duplicates.show()
```

Вывод покажет, какие записи повторяются (т. е. в которых все столбцы имеют одинаковые значения).

Согласованность дат Последняя проверка данных, которую мы покажем, – проверка согласованности дат. При работе с временными рядами или с записями, содержащими отметки времени, очень важно убедиться, что каждая дата в наборе данных действительна и находится в пределах определенного диапазона. Например, предположим, что принятые нами данные представляют период с 1 по 31 января 2024 года. Соответственно, количество записей со значениями `date_column`, попадающими в этот период, должно равняться количеству добавленных записей.

Давайте напишем код, выполняющий эту проверку:

```
# df = набор данных, извлеченный из таблицы "catalog.db.table"
# Определить диапазон дат
start_date = datetime(2024, 1, 1) # например, Jan 1, 2024
end_date = datetime(2024, 1, 31) # например, Jan 31, 2024

# Преобразовать date_column в тип данных "date"
df = df.withColumn("date_column", col("date_column").cast("date"))

# Отфильтровать DataFrame, чтобы получить записи в заданном диапазоне дат
within_range = df.filter((col("date_column") >= start_date) & (col("date_column") <= end_date))
# Подсчитать количество оставшихся записей
count_within_range = within_range.count()
```

Это число можно сравнить с разницей между количествами записей в ветвях `main` и `etl_branch`. Если они не совпадают, то следует проверить, в каких записях указаны неверные даты, вызывающие несоответствие.

Мы рассмотрели три примера проверок качества данных, которые можно выполнить в изолированной ветви. В период проверки новые данные еще недоступны конечным пользователям, обращающимся к ветви `main`. Возможность гибко выполнять проверки в изолированной ветви, не влияя на производство, является критически важной возможностью Apache Iceberg.

Используя этот подход, мы получаем пару преимуществ:

Повышение качества данных

Промышленные среды не подвергаются риску попадания в них непроверенных данных, что предотвращает неверные результаты и решения. Такой порядок исключает необходимость срочного исправления ошибок в данных и тем самым снижает риск внесения дополнительных ошибок в процессе исправления из-за спешки.

Эффективная обработка данных

По сравнению с традиционным способом использования промежуточной таблицы для проверки этот подход устраняет необходимость в копировании данных, позволяет экономить ресурсы и обеспечивает более высокую эф-

фективность. Он помогает с легкостью выявлять проблемы, например дублирование данных, которые часто упускаются при проверке новых данных.

Применение исправлений На этом этапе можно предпринять несколько действий, например сообщить об аномалиях заинтересованным сторонам, просмотреть задание ETL, определить причину аномалий и применить некоторые быстрые исправления. В качестве основного шага создадим два набора данных DataFrame – один с записями, требующими исправления, которые можно сохранить в другую таблицу или записать в файл для передачи заинтересованным сторонам, а другой с проверенными записями, которые можно сохранить в основной ветви таблицы:

```
# 1. Выявить записи с пустыми значениями
# Проверить каждый столбец и создать условие фильтрации
is_null_condition = [col(c).isNull() for c in df.columns]
combined_null_condition = is_null_condition[0]
for condition in is_null_condition[1:]:
    combined_null_condition = combined_null_condition | condition

# 2. Выявить повторяющиеся записи
# Сгруппировать по всем столбцам, подсчитать и отфильтровать
# записи, встречающиеся более одного раза
duplicates_condition = df.groupBy(df.columns) \
    .count() \
    .filter(col("count") > 1) \
    .drop("count") \
    .distinct()

# 3. Создать DataFrame для записей, требующих исправления
# Объединить условия поиска пустых значений и повторяющихся записей,
# чтобы отыскать все записи, нуждающиеся в исправлении
records_needing_remediation = df.filter(combined_null_condition)
    .union(df.join(duplicates_condition,
        df.columns, "inner"))
    .distinct()

# 4. Создать DataFrame для допустимых записей
# Использовать exceptAll для выбора допустимых записей
valid_records = df.exceptAll(records_needing_remediation)

# Вывести записи, нуждающиеся в исправлении
records_needing_remediation.show()

# Вывести допустимые записи
valid_records.show()

# Переписать в таблицу только действительные записи
valid_records.write.format("iceberg").mode("overwrite").save("catalog.db.table")
```

Теперь новые действительные записи сохранены в ветви main, а недействительные мы отправили заинтересованным сторонам, которые их исправят и вновь передадут нам.

Публикация изменений

Последняя операция в шаблоне WAP – публикация изменений и передача данных в промышленную среду, чтобы сделать их доступными для приложений. Операция публикации выполняется с помощью процедуры создания нового снимка (https://oreil.ly/416_7).

В Spark новый снимок создается на основе предыдущего процедурой `cherrypick_snapshot()`, которая сохраняет при этом оригинал без каких-либо изменений. В нашем случае мы можем выбрать конкретный снимок, ветвь (`etl_branch`), и на его основе создать новый снимок. Отличительная черта процедуры `cherrypick_snapshot()` состоит в том, что она воздействует только на метаданные, т. е. файлы данных остаются нетронутыми, а изменяются лишь ссылки в метаданных. В результате мы делаем новые данные доступными в промышленной таблице без фактического перемещения каких-либо файлов данных. Следует отметить одно ограничение: `cherrypick_snapshot()` выполняется за одну транзакцию.

Этой процедуре нужно передать идентификатор моментального снимка в качестве аргумента. Давайте узнаем идентификатор снимка, связанный с `etl_branch`, запросив таблицу метаданных `refs` (все таблицы метаданных, создаваемые с помощью Iceberg, обсуждались в главе 10):

```
# Запросить список ссылок для таблицы
spark.sql("SELECT * FROM catalog.db.table.refs")
```

Этот запрос вернет список ссылок (ветвей и тегов), и в нем мы сможем увидеть идентификатор текущего снимка для каждой ветви. Теперь выполним процедуру `cherrypick_snapshot()`:

```
# Создать снимок в 'main' на основе ветви 'etl_branch'
spark.sql("CALL catalog.system.cherrypick_snapshot('db.table',
2668401536062194692)").show()
```

После успешного выполнения операции текущим снимком основной ветви `main` станет текущий снимок ветви `etl_branch`. После этого записи, вновь вставленные в `etl_branch`, станут доступны в ветви `main`.

Если теперь запросить количество записей в `main` и `etl_branch`, то они должны оказаться равными:

```
# Посчитать записи в ветви 'main'
spark.sql("SELECT count(*) FROM catalog.db.table VERSION AS OF 'main']").show();

# Посчитать записи в ветви 'etl_branch'
spark.sql("SELECT count(*) FROM catalog.db.table VERSION AS OF
'etl_branch']").show();
```

В завершение этого конкретного сеанса WAP удалим конфигурационное свойство `spark.war.branch`, чтобы гарантировать, что все последующие операции чтения и записи будут выполняться из ветви `main`, а не из `etl_branch`:

```
# Выключить поддержку WAP  
spark.conf.unset('spark.wap.branch')
```

В этом примере мы показали вам, как использовать шаблон WAP в Apache Iceberg для решения проблем, связанных с качеством данных. В WAP перед передачей данных в промышленную среду применяется структурированный механизм для записи, оценки проблем с качеством и окончательной обработки или удаления данных. Этот метод помогает обеспечить надежность и гарантировать, что данные, влияющие на бизнес-решения, будут точными, согласованными и свободным от аномалий. Если вам потребуется изолировать изменения в нескольких таблицах, то, следуя тому же шаблону, можно создать ветвь на уровне каталога, используя каталог Nessie, описанный в главе 10.

Запуск аналитических рабочих нагрузок в озере данных

Панели отображения аналитической информации являются источником жизненной силы многих компаний, но зачастую очень сложно обеспечить высокую производительность таких панелей. Чем больше наборы данных, на основе которых создаются информационные панели, тем хуже их производительность и тем выше потребность в инженерных решениях и ухищрениях. Обычно эти решения имеют форму аналитических выборок и кубов, независимых структур данных, созданных из агрегатов по нескольким измерениям и показателям. Однако они имеют несколько проблем:

- аналитическую панель необходимо периодически обновлять вручную, чтобы отобразить свежие данные;
- аналитические панели могут стать очень большими и породить проблему нехватки памяти;
- происходит накопление исторических копий, увеличивая затраты на хранение и обслуживание;
- пользователь должен понимать, что его аналитическая панель работает с аналитической выборкой или кубом, а не с исходной таблицей;
- аналитические панели мешают достижению цели самообслуживания, потому что пользователям придется отправлять заявки для создания нужных им структур и ускорения работы панелей.

Использование Apache Iceberg, особенно в механизме Dremio SQL Query Engine, устраняет большую часть этих проблем благодаря поддержке агрегированных представлений. Агрегированные представления – это предварительно вычисленные агрегаты, хранящиеся в виде таблицы Apache Iceberg в озере данных пользователя и дающие ряд преимуществ:

- автоматическое обновление структуры данных с помощью Dremio;

- механизм запросов Dremio знает, как обрабатывать такие представления, чтобы избежать ошибки нехватки памяти;
- Dremio автоматически очищает исторические копии, чтобы избежать исчерпания памяти;
- аналитические панели можно создавать на основе исходной таблицы, а Dremio будет изменять агрегированное представление, когда аналитические инструменты будут отправлять агрегирующие запросы;
- их можно включать и выключать щелчком на переключателе или SQL-запросом, что обеспечивает ускорение самообслуживания;
- если в основе лежит таблица Apache Iceberg, то агрегированное представление обновляется постепенно, позволяя обновлениям данных распространяться практически в реальном времени.

Как видите, Apache Iceberg – это ядро, обеспечивающее ускорение работы аналитических панелей. Обычно рабочий процесс выглядит следующим образом:

1. Исходные данные помещаются в озеро в виде таблиц Apache Iceberg.
2. Создаются виртуальные витрины и продукты данных путем построения слоев логических представлений в этих таблицах.
3. Для наборов данных, на основе которых будет создана аналитическая панель, включается агрегирование. Сами агрегированные представления Apache Iceberg будут созданы без участия пользователя.
4. Получившаяся аналитическая панель будет работать быстрее, потому что агрегирующие запросы направляются к агрегированным представлениям, а не к исходным данным.

Давайте рассмотрим каждый шаг по очереди.

Помещение исходных данных в озеро

Первый шаг – помещение данных в озеро в виде таблиц Apache Iceberg. Это можно сделать с помощью многих инструментов, обсуждаемых в данной книге. Вот некоторые из возможных подходов:

- применение таких инструментов, как Apache Spark и Apache Flink, для приема пакетных или потоковых данных в таблицы Apache Iceberg в озере;
- использование операторов `CREATE TABLE...AS SELECT` (CTAS) с любым механизмом запросов для получения данных из источников, таких как базы данных, и записи их в таблицы Apache Iceberg в озере данных;
- использование оператора `COPY INTO` в Dremio для копирования разных файлов данных в существующую таблицу Apache Iceberg;
- применение инструментов интеграции данных, таких как Upsolver, Airbyte или Fivetran, для загрузки данных в таблицы Apache Iceberg с использованием их интерфейсов.

После помещения исходных данных в таблицы Apache Iceberg остается только подключиться к озеру (Amazon S3, Azure Data Lake Storage [ADLS], Minio) или каталогу Apache Iceberg (Hive, Nessie, Amazon Web Services [AWS] Glue) в Dremio, чтобы получить доступ к таблицам Apache Iceberg, как показано на рис. 14.2.

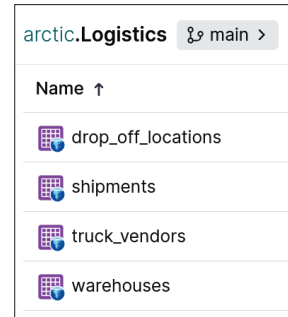


Рис. 14.2 ❖ Таблицы Apache Iceberg в пользовательском интерфейсе Dremio

Курирование виртуальных витрины и продуктов данных

В хранилищах данных вам часто придется выполнять очистку, проверку и обработку данных с применением слоев, организованных в наборы данных для разных бизнес-подразделений. Эти наборы известны как *витрины данных* (data marts). Dremio позволяет создавать виртуальные витрины данных на базе озера, поэтому вместо создания большого количества физических таблиц из исходных таблиц Apache Iceberg можно создать логические представления, которые инкапсулируют очистку, проверку и бизнес-логику, и организовать их в папки для каждого подразделения. Такой подход поможет вам организовать данные в витрины или продукты данных, как показано на рис. 14.3.

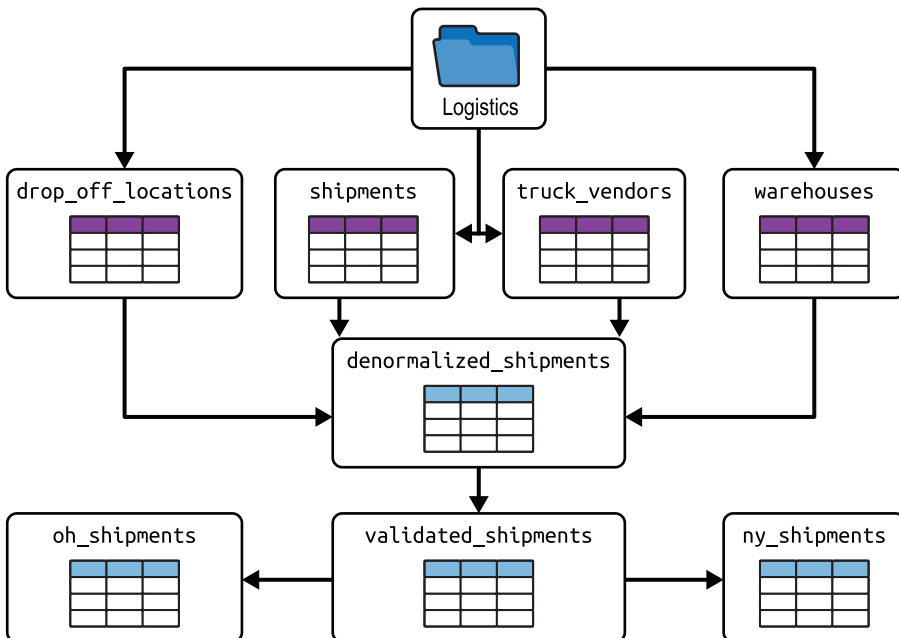


Рис. 14.3 ❖ Пример структуры виртуальной витрины или продукта данных в Dremio

Создание агрегированных представлений для ускорения аналитических панелей

Представление многих наборов данных в аналитических панелях будет осуществляться достаточно быстро без всяких ухищрений, но при работе с действительно большим набором данных может понадобиться создать агрегированное представление, чтобы обеспечить высокую производительность. Например, создавая аналитическую панель по поставкам в Огайо на основе `oh_shipments`, показанного на рис. 14.3, можно было бы включить агрегированное представление для этой таблицы, щелкнув несколько раз мышью в пользовательском интерфейсе Dremio или выполнив простой SQL-запрос:

```
ALTER TABLE arctic.logistics.oh_shipments
  CREATE AGGREGATE REFLECTION oh_shipments_agg
  USING
  DIMENSIONS (shipper_id, destination_city, shipping_method)
  MEASURES (shipment_id (COUNT), total_cost (SUM), delivery_time (AVG))
  LOCALSORT BY (shipper_id, destination_city);
```

Он создаст представление, оптимизированное для конкретных измерений и показателей, и таблицу Apache Iceberg с именем `oh_shipments_agg`, содержащую предварительно вычисленные агрегаты. Конечно, наши аналитики должны знать о существовании этого представления, но и без этого Dremio автоматически будет подставлять `oh_shipments_agg`, заметив поступление агрегирующих запросов к `oh_shipments` по тем же измерениям и показателям. Кроме того, поскольку наши исходные таблицы – это таблицы Apache Iceberg, то представления будут обновляться практически в режиме реального времени.

Подключение аналитических инструментов к представлению

Многие аналитические инструменты поддерживают интеграцию с Dremio, а некоторые, такие как Tableau и Power BI, даже встроены в пользовательский интерфейс Dremio. Это означает, что вам нужно лишь открыть свое представление (`oh_shipments`) в пользовательском интерфейсе Dremio и щелкнуть на кнопке **Tableau** или **Power BI** (как показано на рис. 14.4), чтобы немедленно установить соединение с Dremio для создания аналитической панели.

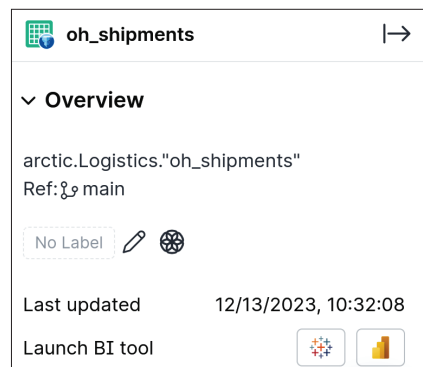


Рис. 14.4 ❖ Интеграция аналитических инструментов с Dremio

Преимущества выполнения аналитических рабочих нагрузок в озере данных

Объединив возможности Apache Iceberg и Dremio, мы создали высокопроизводительные аналитические панели, черпающие информацию непосредственно из хранилища данных. Это помогло нам сэкономить затраты на хранение и вычисления, связанные с перемещением данных в хранилище, а также созданием и обслуживанием аналитических выборок и кубов, и сохранить при этом простую модель самообслуживания для конечных пользователей. Экосистема Apache Iceberg наполнена инструментами, которые упрощают и масштабируют различные сценарии использования, позволяя работать только с одной копией данных в одном месте – в озере данных.

Реализация сбора измененных данных с помощью Apache Iceberg

Сбор измененных данных (Change Data Capture, CDC) – это неотъемлемый процесс в сфере аналитики. По своей сути CDC занимается сбором и отслеживанием изменений в исходных данных, позволяя последующим системам синхронизироваться эффективно и инкрементально. При традиционной пакетной обработке базы данных и хранилища данных часто выполняют массивные обновления, что может привести к потере информации и выполнению ресурсоемких операций. Такой способ обычно характеризовался использованием устаревшего представления данных, задержками в принятии решений и неэффективным использованием хранилищ и вычислительных ресурсов. Сосредоточив внимание на инкрементальных изменениях – вставки, обновления или удаления, – CDC гарантирует, что системы данных остаются синхронизированными без необходимости многократно обрабатывать обширные статические данные. На рис. 14.5 показано визуальное представление сбора измененных данных на высоком уровне.

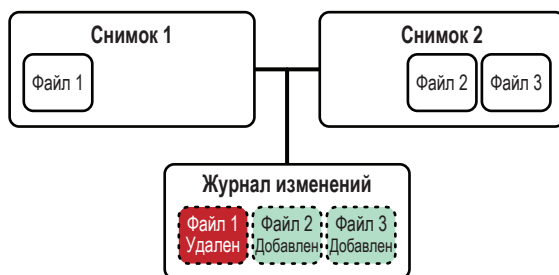


Рис. 14.5 ❖ Схема сбора изменений между двумя снимками

Представьте себе GreenMart – компанию розничной торговли с колеблющимся уровнем запасов и меняющимися ценами в динамичной розничной среде. Компания стремится получать информацию о наличии запасов в режиме реального времени для удовлетворения потребностей клиентов и делиться этой информацией с заинтересованными сторонами, такими как директора магазинов, через свою систему аналитической отчетности. Однако им потребовалось определить, как поддерживать обновленные отчеты, не оказывая большой нагрузки на систему пересчетом агрегатов для каждого изменения, учитывая, что количество транзакций очень велико. С этой целью в GreenMart приступили к поиску гибкой архитектуры данных, поддерживающей масштабируемые хранилище и вычисления. Архитектура должна обладать следующими характеристиками: возможность эволюции схемы и поддержка транзакций ACID (INSERT, UPDATE, MERGE INTO, DELETE), откатов и CDC. Возможное решение может включать использование облачной системы хранения, такой как Amazon S3, в сочетании с Apache Iceberg в качестве формата таблиц.

Один из подходов к решению этой задачи заключается в создании двух таблиц Iceberg, `inventory` и `inventory_summary`, в озере данных S3. В таблице `inventory` будут храниться данные из операционных систем, таких как базы данных и CRM, а в таблице `inventory_summary` – агрегированные и преобразованные данные для аналитических отчетов. Все обновления из транзакционных систем будут сохраняться в озере данных и впоследствии применяться к таблице `inventory` с использованием процесса ETL в Spark. Процесс CDC в Iceberg будет выявлять любые изменения в таблице `inventory` и сохранять их в журнале изменений с именем `inventory_changes`. При использовании этого журнала изменений будут обновляться только измененные данные в агрегированной таблице `inventory_summary`, что устраняет необходимость пересчитывать агрегаты для всего набора данных при каждом изменении. Такой подход гарантирует, что аналитические отчеты будут постоянно основаны на самых актуальных данных.

Весь этот процесс изображен на рис. 14.6. Давайте теперь посмотрим, как его реализовать.

Создание таблиц Apache Iceberg

Давайте создадим первую таблицу Iceberg `inventory` и запишем в нее некоторое количество тестовых данных:

```
spark.sql('''CREATE TABLE glue.test.inventory(
  product_id int,
  product_name string,
  stock_level int,
  price int,
  last_updated date) USING iceberg''')
```

```
spark.sql('''INSERT INTO glue.test.inventory VALUES
(1, 'Pasta-thin', 60, 45, '3/25/2023'),
(2, 'Bread-white', 55, 6, '3/10/2023'),
(3, 'Eggs-nonorg', 100, 8, '3/12/2023'),
(4, 'Sausage-pork', 72, 25, '3/29/2023'),
(5, 'Coffee-vanilla', 30, 45, '3/12/2023'),
(6, 'Maple Syrup', 20, 85, '3/29/2023'),
(7, 'Protein Bar', 120, 5, '3/15/2023')
''')
```

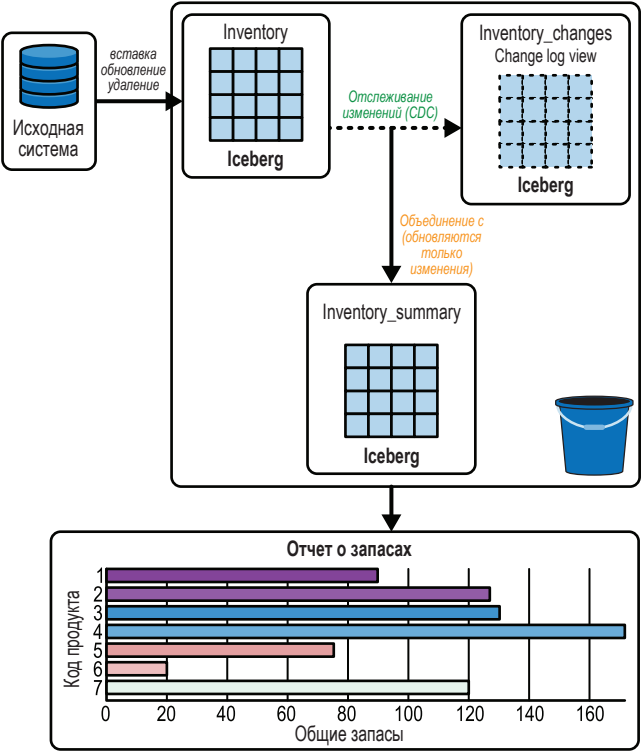


Рис. 14.6 ❖ Использование CDC для обновления агрегированных метрик

Выполним запрос и убедимся, что все данные сохранены в таблице правильно:

```
spark.sql("SELECT * FROM glue.test.inventory").toPandas()
```

Результат должен выглядеть примерно так:

product_id	product_name	stock_level	price	last_updated
1	Pasta	50	35	3/24/2023

6	Maple Syrup	20	85	3/29/2023
7	Protein Bar	120	5	3/15/2023
2	Bread-brown	87	8	3/25/2023
3	Eggs-organic	30	11	3/26/2023
4	Sausage-chicken	100	20	3/24/2023
1	Pasta-thin	60	45	3/25/2023
5	Coffee-arabica	45	60	3/18/2023
2	Bread-white	55	6	3/10/2023
3	Eggs-nonorg	100	8	3/12/2023
4	Sausage-pork	72	25	3/29/2023

Создадим вторую таблицу Iceberg `inventory_summary` – преобразованный набор данных с агрегированными значениями. Она будет служить основным источником данных для составления аналитических отчетов, которые помогут руководителям получить представление об уровнях запасов в конкретных магазинах:

```
spark.sql('''CREATE TABLE glue.test.inventory_summary(
    product_id string,
    total_stock string,
    avg_price string) USING iceberg''')
```

```
spark.sql('''INSERT INTO glue.test.inventory_summary
SELECT
    product_id,
    SUM(stock_level) AS total_stock,
    AVG(price) AS avg_price
FROM glue.test.inventory_new
GROUP BY product_id;
''')
```

Запросим таблицу, чтобы проверить правильность данных:

```
spark.sql("SELECT * FROM glue.test.inventory_summary").toPandas()
```

Результат должен выглядеть примерно так:

product_id	total_stock	avg_price
1	110.0	40.0
2	142.0	7.0

6	20.0	85.0

3	130.0	13.5

4	172.0	22.5

5	75.0	52.5

7	120.0	5.0

Применение обновлений из операционных систем

Чтобы применить все обновления из операционных баз данных, запустим задание ETL на основе Spark. Iceberg позволяет выполнять обновления на уровне записей с гарантиями транзакций. Вот простой запрос, моделирующий возможное обновление данных о запасах:

```
spark.sql('''UPDATE glue.test.inventory
SET stock_level = stock_level - 15
WHERE product_name = 'Bread-white' ''')
```

i Выборка изменений из операционной системы-источника и загрузка их в озеро данных не является частью этого сценария использования. Мы предполагаем, что изменения доступны в озере данных S3 в виде файла, и применяем их к соответствующим записям с помощью команды UPDATE в Iceberg.

Если теперь запросить таблицу inventory, мы должны увидеть обновленные данные:

```
spark.sql("SELECT * FROM glue.test.inventory").toPandas()
```

Обратите внимание, что запасы продуктов Pasta и Bread-white уменьшились:

product_id	product_name	stock_level	price	last_updated

1	Pasta	30	35	3/24/2023

6	Maple Syrup	20	85	3/29/2023

2	Bread-white	40	6	3/10/2023

2	Bread-brown	87	8	3/25/2023

1	Pasta-thin	60	45	3/25/2023

7	Protein Bar	120	5	3/15/2023

3	Eggs-organic	30	11	3/26/2023

4	Sausage-chicken	100	20	3/24/2023
5	Coffee-arabica	45	60	3/18/2023
3	Eggs-nonorg	100	8	3/12/2023
4	Sausage-pork	72	25	3/29/2023

Создание представления журнала изменений для сбора изменений

Далее мы создадим представление журнала изменений, чтобы собрать все изменения в таблице `inventory` и эффективно обновить `inventory_summary`, избегая полного пересчета всех агрегатов. Для решения этой задачи воспользуемся встроенной процедурой Spark Iceberg `create_changelog_view()`. Собрать изменения можно двумя способами: используя идентификаторы начального и конечного снимков или отметки времени начала и окончания. В этом примере в роли контрольных точек мы будем использовать снимки: начальный соответствует моменту сразу после инициализации таблицы `inventory`, а конечный – после выполнения задания ETL. Чтобы получить идентификаторы снимков, запросим таблицу метаданных `history`, поддерживаемую Apache Iceberg:

```
spark.sql("SELECT * FROM glue.test.inventory.history").toPandas()
```

Предположим, что по результатам, полученным из таблицы `history`, мы определили такие идентификаторы целевых снимков: 4816648710583642722 и 2557325773776943708 (у вас эти идентификаторы могут отличаться). Теперь запустим процедуру:

```
spark.sql("""CALL
glue.system.create_changelog_view(
    table => 'glue.test.inventory',
    options => map(
        'start-snapshot-id',
        '4816648710583642722',
        'end-snapshot-id',
        '2557325773776943708'
    )""")
```

Она создаст представление журнала изменений с именем `inventory_changes`, отражающее изменения, внесенные в таблицу в период между созданием первого и второго снимков. Давайте запросим изменения и посмотрим, как выглядит это представление:

```
spark.sql("SELECT * FROM inventory_changes").toPandas()
```

Результат должен выглядеть примерно так:

product_id	...	last_updated	_change_type	_change_ordinal	_commit_snapshot_id
2	...	3/10/2023	INSERT	0	2557325773776943708
2	...	3/10/2023	DELETE	0	2557325773776943708
1	...	3/24/2023	DELETE	1	2959510555509473926
1	...	3/24/2023	INSERT	1	2959510555509473926

Добавление изменений в агрегированную таблицу

Последний шаг включает обновление агрегированной таблицы `inventory_summary`, используемой аналитическими отчетами для извлечения информации об уровне запасов в магазине. Важно отметить, что мы стремимся избежать пересчета агрегатов для всего набора данных при каждом обновлении. Поэтому сосредоточимся на внесении только корректировок, имеющих смысл при изменении уровня запасов продукта. Такой подход обеспечивает эффективность вычислений, экономит время и гарантирует доступность для отчетов самых свежих данных без полного пересчета всех агрегатов.

Следующий фрагмент кода сначала создает представление на основе данных журнала изменений, состоящее из агрегированных изменений, и затем записывает эти изменения в таблицу `inventory_summary`, чтобы сделать их доступными для наших аналитических отчетов:

```
## Создать агрегированное представление
spark.sql("""
    CREATE OR REPLACE TEMPORARY VIEW aggregated_changes AS
    SELECT
        product_id,
        SUM(CASE
            WHEN _change_type = 'INSERT' THEN stock_level
            WHEN _change_type = 'DELETE' THEN -stock_level
            ELSE 0 END) AS total_stock_change,
        AVG(price) AS new_avg_price
    FROM
        inventory_changes
    GROUP BY
        product_id
""")

## Записать агрегированные изменения в inventory_summary
spark.sql("""
    MERGE INTO glue.test.inventory_summary AS target
    USING aggregated_changes AS source
    ON target.product_id = source.product_id
```

```

WHEN MATCHED THEN
    UPDATE SET
        target.total_stock = target.total_stock + source.total_stock_change
WHEN NOT MATCHED THEN
    INSERT (product_id, total_stock, avg_price)
    VALUES (source.product_id, source.total_stock_change,
source.new_avg_price)
"""
)

```

Обратите внимание, что в промышленном окружении эти операции обычно автоматизируются и постоянно контролируются, чтобы обеспечить оптимальную производительность и надежность. Для выполнения заданий CDC через регулярные промежутки времени и обновления данных практически в реальном времени можно использовать такие инструменты автоматизации, как Apache Airflow или задания cron. Также для регистрации операций и отправки уведомлений в случае сбоев или аномалий желательно использовать механизмы мониторинга и оповещения. Такая конфигурация обеспечит своевременное обновление данных и быстрое решение проблем, а также поможет поддерживать целостность и согласованность данных во всей системе.

Давайте теперь запросим агрегированную таблицу, чтобы увидеть изменения:

```
spark.sql("SELECT * FROM glue.test.inventory_summary").toPandas()
```

Как показано ниже, записи для продуктов с идентификаторами 1 и 2 изменились и отражают обновления, внесенные в исходную таблицу:

product_id	total_stock	avg_price

1	90.0	40.0

2	127.0	7.0

6	20.0	85.0

3	130.0	13.5

4	172.0	22.5

5	75.0	52.5

Аналитическая панель, использующая сводную информацию о запасах, теперь также обновляется, как показано на рис. 14.7.

Используя представление журнала изменений, мы решили важную проблему для GreenMart. Этот подход обеспечивает скорость и эффективность за счет вычисления только разностей, что сокращает время обработки и экономит вычислительные ресурсы. Он позволяет получать изменения в аналитических отчетах практически в реальном времени, предоставляя доступ к актуальной информации. Более того, по мере роста GreenMart и расширения

данных о запасах этот подход обеспечит масштабируемую и эффективную обработку данных.

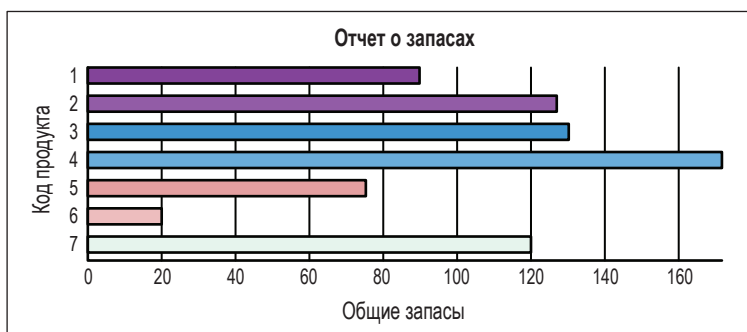


Рис. 14.7 ❖ Обновленная аналитическая панель, отражающая изменения в запасах

Заключение

В этой главе мы рассмотрели несколько примеров использования Apache Iceberg в качестве формата таблиц поверх озер данных и узнали, как практически реализовать эти важные приложения, от обеспечения высококачественных данных до создания информационных панелей и регистрации изменений.

На этом мы подошли к концу нашего исследования формата таблиц Apache Iceberg, и теперь вы сами можете создавать эффективные платформы данных.

Глава 15

Дополнительные примеры использования Apache Iceberg

Рабочие нагрузки реального времени, использующие AWS Kinesis и Apache Iceberg

В последнее время особую важность приобрели средства доступа к данным в реальном времени. Благодаря возможности доступа и анализа данных в реальном времени организации получают конкурентное преимущество, позволяющее быстро принимать решения, оптимизировать операционную эффективность, повышать качество обслуживания клиентов и выявлять проблемы на ранних этапах. По мере роста и расширения аналитических потребностей способность использовать рабочие нагрузки в реальном времени становится совершенно необходимой.

AWS Kinesis выступает в качестве надежного моста для потоковой передачи данных, облегчая прием, обработку и анализ данных в реальном времени и обеспечивая возможность мгновенной оценки ситуации и принятия решений, а также бесшовную интеграцию с различными аналитическими инструментами.

В отличие от пакетной обработки, когда данные собираются в течение определенного периода, а затем обрабатываются, инструменты потоковой аналитики обрабатывают данные практически мгновенно по мере их по-

явления, ускоряя процесс получения информации. Например, платформа потоковой передачи, предлагающая медиауслуги зрителям через интернет, благодаря аналитике в реальном времени может мгновенно рекомендовать видеоматериалы на основе текущих тенденций или привычек похожих пользователей. Такие приложения подчеркивают важность данных и аналитики в реальном времени.

Потоковые данные предоставляют много интересных возможностей для использования в аналитических сценариях, но они также создают препятствия с точки зрения масштабируемости, надежности и стоимости. При постоянном притоке данных нужна архитектура, способная масштабироваться и бесперебойно обрабатывать увеличивающийся объем данных, а также справляться с проблемой большого количества маленьких файлов, изменениями схем и многим другим. Кроме того, платформа должна поддерживать надежные транзакции и предоставлять эффективный способ их обработки.

Давайте посмотрим, как можно решить эту проблему с помощью AWS Kinesis и Apache Iceberg.

Рассмотрим в качестве примера авиакомпанию AeroWorld, для которой безопасность полетов и своевременность имеют архиважное значение. В своей работе компания в значительной степени зависит от прогноза погоды. Традиционных прогнозов погоды, обновляемых ежечасно или ежесуточно, не всегда достаточно. Авиационная отрасль требует более частых обновлений из-за быстрых изменений атмосферных условий, которые могут повлиять на маршруты полетов, безопасность пассажиров и расписание рейсов.

Инженерная группа AeroWorld стремится интегрировать данные из OpenWeather (<https://openweathermap.org/>) и других источников для получения потоковых данных об атмосферных изменениях и пытается решить сложную задачу: организовать эффективный прием, обработку и хранение больших объемов данных о погоде и, что еще важнее, предоставить эти данные команде аналитиков. Далее мы представляем наше решение этой проблемы.

Для приема и хранения больших объемов данных из OpenWeather API (<https://openweathermap.org/api>), нашего источника данных, мы используем AWS Kinesis Data Streams (<https://oreil.ly/HUBIT>). После сохранения метеорологических данных в Kinesis мы с помощью AWS Glue создадим и запустим задание ETL, осуществляющее прием данных в таблицу Apache Iceberg в озере данных Amazon S3. Iceberg как формат таблиц будет служить нам эффективным слоем хранения, предоставляя такие возможности, как гарантии согласованности, эволюция схемы, скрытое секционирование и доступ к историческим данным, которые имеют решающее значение для аналитических рабочих нагрузок. После сохранения данных в открытом формате Iceberg различные вычислительные системы смогут использовать их для своих нужд. Например, AeroWorld сможет создать панели мониторинга для слежения за погодными явлениями или построить модели машинного обучения.

На рис. 15.1 показана общая схема архитектуры нашего решения.

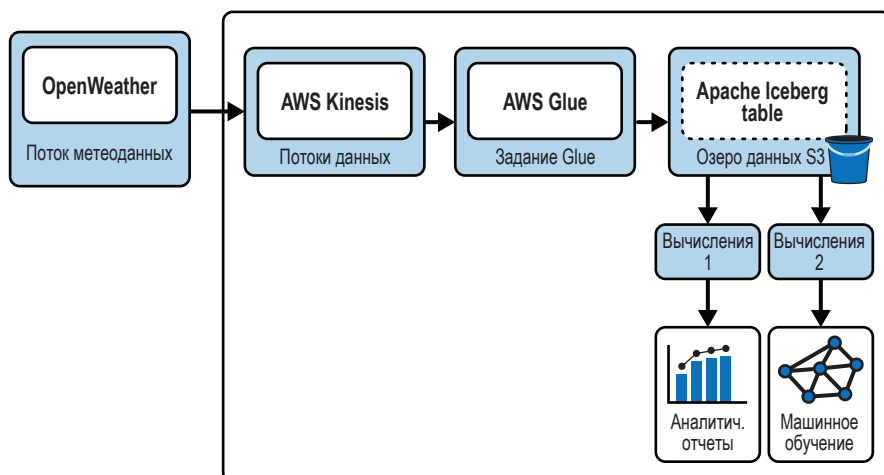


Рис. 15.1 ❖ Общая схема архитектуры решения для AeroWorld

Теперь посмотрим, как это реализовать.

Прием потоков с помощью AWS Kinesis

Первый шаг в этом процессе – получение потоковых данных из источника. В этом случае мы будем получать метеоданные из OpenWeather с помощью AWS Kinesis. Kinesis Data Streams упрощает сбор, обработку и хранение потоков данных в любом масштабе.

Итак, давайте создадим поток данных Kinesis. На главной странице AWS Kinesis создайте новый поток, а затем выполните следующие действия:

1. Дайте имя потоку (например, `example_stream_book`).
2. Установите режим емкости **provisioned** (зарезервировано).
3. Выделите один сегмент.

Теперь, создав свой поток, рассмотрим несколько важных аспектов.

Стратегия сегментирования и режим емкости

Сегмент (shard) в AWS Kinesis – это последовательность записей данных в потоке, выступающая в роли базового компонента пропускной способности в потоке данных Kinesis. Каждый сегмент в потоке Kinesis может записывать данные со скоростью 1 Мбайт в секунду, или 1000 элементов в секунду, и читать со скоростью 2 Мбайта в секунду. Вы должны выделить соответствующие сегменты в соответствии со своим вариантом использования и требованиями.

Kinesis предоставляет два режима емкости для сегментирования:

Режим по требованию

В режиме по требованию не нужно задавать количество сегментов – управление сегментированием будет осуществляться автоматически с учетом

объема поступающих данных, количество сегментов будет уменьшаться с уменьшением объема данных и увеличиваться с увеличением.

Зарезервированный режим

В зарезервированном (provisioned) режиме вы должны явно указать количество сегментов, которое должно быть выделено для потока. Каждый зарезервированный сегмент обеспечивает определенную емкость чтения и записи. Вы должны планировать и корректировать количество сегментов по мере изменения пропускной способности. Такой подход позволяет жестко контролировать расходы.

Для AeroWorld важно оценить ожидаемую скорость передачи данных и принять соответствующее решение в дни серьезных погодных аномалий или в определенные сезоны. Поэтому важно разумно распланировать сегменты, чтобы избежать выделения избыточных или недостаточных ресурсов, что может привести к росту расходов или к сбоям в приеме данных соответственно.

Щелкните на кнопке **Create data stream** (Создать поток данных), и поток `example_stream_book` должен быть запущен и готов к использованию производителем данных.

Производитель данных

В этом примере роль основного производителя данных играет OpenWeather API. Он передает метеоданные в реальном времени в поток Kinesis. Интегрировать источники данных с Kinesis можно с помощью SDK или коннекторов, обеспечивающих бесперебойную передачу данных. Мы будем использовать boto3 SDK. Код в следующем примере извлекает данные о погоде в реальном времени для различных городов и загружает их в поток Kinesis `example_stream_book`. Полный код с настройкой клиента и определениями функций вы найдете в репозитории книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg>).

Следующий блок кода перебирает список городов и пытается извлечь данные о погоде из API; если извлечение прошло успешно, данные передаются другой функции для отправки в поток Kinesis. После каждой итерации процесс приостанавливается, чтобы избежать отказов со стороны API:

```
## Настройка клиента выполнена перед этим фрагментом
## Полный код ищите в репозитории книги
if __name__ == "__main__":
    for city in cities:
        data = get_weather_data(city)
        if data:
            send_data_to_kinesis(data)
    time.sleep(5) # приостановиться, чтобы не вынудить API ограничить скорость
```


Преобразование данных и ETL с помощью AWS Glue

Следующий шаг в реализации архитектуры – настройка AWS Glue и Apache Iceberg. Идея заключается в том, чтобы использовать данные о погоде, хранящиеся в Kinesis, в качестве источника в Glue и создать таблицу Iceberg на основе этих данных. Перед тем как приступить к интеграции, необходимо выполнить несколько важных подготовительных шагов.

Создание базы данных Glue

Каталог данных AWS Glue отслеживает метаданные таблицы. Прежде чем мы сможем использовать этот сервис для операций ETL, нужно создать базу данных в этом каталоге. Перейдя в каталог данных, вы увидите раздел **Databases** (Базы данных) в панели слева. Выберите **Add Database** (Добавить базу данных), чтобы создать новую базу данных, и дайте ей имя «test».

Настройка ETL-задания Glue

AWS Glue предлагает визуальный интерфейс, упрощающий настройку заданий ETL. На странице **Jobs** (Задания) в AWS выберите шаблон для создания нового задания и используйте вариант **Visual with a Blank Canvas** (Визуальный редактор с чистым холстом). Перед вами откроется визуальный редактор с чистым холстом без ненужных узлов, которые вам нужно будет удалить или изменить, чтобы получить желаемый результат. Нажмите кнопку **Create** (Создать), чтобы создать холст задания ETL.

Напомним, что наша цель – создать таблицы Iceberg на основе данных из потока Kinesis. Для этого нам нужно добавить узел с Kinesis в качестве источника.

В настройках узла Kinesis убедитесь, что выбрали правильный поток в разделе **Stream Name** (Имя потока; в данном случае это `example_stream_book`), выберите в **Data Format** (Формат данных) формат JSON и в **Starting Position** (Начальная позиция) – **Latest** (Последняя). Установив в **Starting Position** (Начальная позиция) значение **Latest** (Последняя), гарантируем получение новых данных при каждом обращении к потоку.

В активный узел Kinesis добавьте узел SQL. Поток должен передаваться от узла Kinesis к узлу SQL.

Отношения узлов AWS Glue

В AWS Glue каждый узел является источником, приемником или преобразователем данных. Узлы связаны друг с другом отношениями родитель–потомок; никакой узел не приступит к решению своей задачи, пока все его родительские узлы не выполнят свою работу. Конечным результатом является создание ориентированного ациклического графа, определяющего порядок, в котором должны выполняться задачи в вашем задании, начиная с источников данных и, как правило, заканчивая приемниками, при этом каждый узел получает данные от своего родителя.

В настройках узла SQL укажите имя узла **Create Table** (Создать таблицу). Здесь же можно задать имя для входных данных из предыдущего узла Kinesis; оставьте имя по умолчанию **myDataSource**. Затем введите следующий код SQL, создающий таблицу на основе схемы потоков, если она еще не существует:

```
CREATE TABLE IF NOT EXISTS glue.test.my_streaming_data AS
(SELECT * FROM myDataSource LIMIT 0);
```

Добавьте еще один узел SQL с именем **INSERT INTO**, исходящий из узла Kinesis. Назначение этого узла – заполнять таблицу после создания и во время последующих обращений к потоку. Он должен иметь узел Kinesis в качестве родителя, чтобы получать данные из потока.

Вот SQL-запрос для этого узла:

```
INSERT INTO glue.test.my_streaming_data SELECT * FROM myDataSource;
```

На рис. 15.2 показано, как должен выглядеть поток AWS Glue.

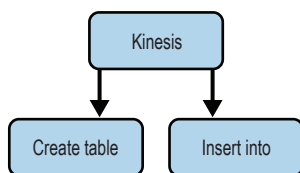


Рис. 15.2 ❖ Структура ETL-задания AWS Glue на данный момент

Теперь перейдите на вкладку **Job details** (Сведения о задании), чтобы завершить настройку соединения между Glue и Apache Iceberg.

Настройка свойств задания

Теперь настроим свойства задания. Сначала дайте имя заданию, например **Iceberg_ETL**, и выберите роль IAM, имеющую необходимые разрешения Glue. По умолчанию устанавливается тип задания **SparkStreaming**, потому что источником данных является Kinesis. Затем выберите **Glue 4.0** в качестве версии Glue и **Python 3** в качестве языка сценариев. Наконец, выделите количество рабочих процессов для обработки задания ETL; установим его равным 2, т. к. этого вполне достаточно для демонстрационных целей.

В разделе **Job parameters** (Параметры задания), внутри раздела **Advanced properties** (Дополнительные свойства) введем параметры для задания:

- `--datalake-formats`: мы будем использовать `iceberg`, чтобы сообщить Glue, какой формат таблицы использовать;
- `--conf`: этот параметр содержит список пар ключ–значение для настройки в Iceberg таких задач, как создание каталога, предоставление каталога хранилища и реализация настроек, специфичных для Spark SQL.

```
spark.sql.extensions=org.apache.iceberg.spark.extensions.
IcebergSparkSessionExtensions
--conf spark.sql.catalog.glue_catalog=org.apache.iceberg.spark.SparkCatalog
--conf spark.sql.catalog.glue_catalog.warehouse=s3://<your-warehouse-dir>/
--conf spark.sql.catalog.glue_catalog.catalog-impl=org.apache.iceberg.aws.glue.
GlueCatalog
--conf spark.sql.catalog.glue_catalog.io-impl=org.apache.iceberg.aws.s3.S3FileIO
```

Обратите внимание, что начальный параметр `--conf` отсутствует в блоке кода. Это связано с тем, что имя параметра – это начальный `--conf`. Будьте внимательны и не допускайте неожиданных переносов строк в настройках параметра `--conf`. Мы подробно обсуждали настройку этих параметров с использованием Spark в главах 5 и 6.

После определения требуемых настроек можно сохранить задание и перейти на вкладку **Script** (Сценарий). Ниже показан сам сценарий (за исключением кода начальной настройки; полный код можно найти в репозитории GitHub (<https://oreil.ly/supp-guide-apache-iceberg>)):

```
#####
# Сценарий, сгенерированный для узла Kinesis
#####

dataframe_AmazonKinesis_node1693319486385 = glueContext.create_data_frame.
from_options(
    connection_type="kinesis",
    connection_options={
        "typeOfData": "kinesis",
        "streamARN": "arn:aws:kinesis:us-east-1:xxxxxxx:stream/exam
ple_stream_book",
        "classification": "json",
        "startingPosition": "latest",
        "inferSchema": "true",
    },
    transformation_ctx="dataframe_AmazonKinesis_node1693319486385",
)

def processBatch(data_frame, batchId):
    if data_frame.count() > 0:
        AmazonKinesis_node1693319486385 = DynamicFrame.fromDF(
            data_frame, glueContext, "from_data_frame"
        )

#####
# Сценарий, сгенерированный для узла Create Table
#####

SqlQuery0 = """
CREATE TABLE IF NOT EXISTS glue.dip.my_streaming_data AS
(SELECT * FROM myDataSource LIMIT 0);
"""

CreateTable_node1693319510128 = sparkSqlQuery(
    glueContext,
```

```

        query=SqlQuery0,
        mapping={"myDataSource": AmazonKinesis_node1693319486385},
        transformation_ctx="CreateTable_node1693319510128",
    )

#####
# Сценарий, сгенерированный для узла Insert Into
#####

SqlQuery1 = """
INSERT INTO glue.dip.my_streaming_data SELECT * FROM myDataSource;
"""

InsertInto_node1693319537299 = sparkSqlQuery(
    glueContext,
    query=SqlQuery1,
    mapping={"myDataSource": AmazonKinesis_node1693319486385},
    transformation_ctx="InsertInto_node1693319537299",
)

glueContext.forEachBatch(
    frame=dataframe_AmazonKinesis_node1693319486385,
    batch_function=processBatch,
    options={
        "windowSize": "100 seconds",
        "checkpointLocation": args["TempDir"] + "/" + args["JOB_NAME"] + "/check
point/",
    },
)
job.commit()

```

Теперь запустим задание. В нашей таблице `my_streaming_data` должны появиться записи. По завершении задания панель управления Glue отобразит таблицу, указав текущее местоположение файла метаданных. Выходной каталог S3 будет иметь папку, специфичную для базы данных, где находятся отдельные папки для каждой таблицы, каждая из которых включает данные и метаданные. Обратите внимание, что это потоковое задание продолжит выполняться и записывать данные в таблицу Iceberg, и эти обновления будут доступны нижестоящим приложениям.

Анализ потока

Теперь, организовав получение потоковых данных и сохранение их в открытом формате (Iceberg), можно приступить к использованию этих данных в аналитических отчетах и машинном обучении. Архитектура с Apache Iceberg в качестве табличного формата дает широкие возможности управления данными по сравнению с использованием с форматами файлов данных, такими как Parquet. Одно из основных преимуществ заключается в способности Iceberg эффективно управлять эволюцией схемы, адаптировать и развивать ее, не прерывая текущие процессы. Это имеет решающее значение для

поточковых рабочих нагрузок, обеспечивая их непрерывными, бесперебойными данными для аналитики. Еще одним значительным преимуществом является эволюция разделов. В динамической потоковой среде шаблоны и объемы данных быстро меняются. Способность Iceberg развивать разделы без каких-либо побочных эффектов для существующей архитектуры делает его идеальным решением для таких сценариев. Кроме того, функция отката и доступность исторических данных бесценна для потоковых рабочих нагрузок. Они позволяют пользователям возвращаться к предыдущим состояниям данных, исправлять любые ошибки в потоках и включать запросы к историческим данным.

Машинное обучение с Apache Iceberg

В экосистеме машинного обучения решающее значение имеют алгоритмы и модели, но не менее важную роль играет также базовая архитектура данных. Надежная и адаптивная основа данных является ключом к эффективному обучению моделей и точности прогнозирования. Представьте себе модель в сфере здравоохранения, не прошедшую эффективного обучения на разнообразных наборах данных. Прогнозы такой модели вряд ли можно использовать для принятия решений. Аналогично в технологических сферах и бизнес-моделях схемы и структуры данных будут неизбежно изменяться с изменением поведения пользователей. На фоне этих постоянных изменений потребность в целостности и надежности данных становится критически важной, особенно для машинного обучения, основывающегося на непротиворечивых и согласованных шаблонах данных.

Apache Iceberg как формат таблиц решает некоторые ключевые проблемы табличных наборов данных в машинном обучении. Его функции, такие как принудительное применение схемы, гарантируют, что данные останутся чистыми и корректными. Эволюция схемы позволяет включать новые бизнес-требования, не нарушая существующие операции. Кроме того, с такими возможностями, как доступ к историческим данным и изолированное ветвление, специалисты по данным и аналитики могут повторно исследовать прошлые состояния данных, воспроизводить эксперименты и создавать изолированные среды для новых исследований, не влияя на промышленные данные.

В этом разделе мы рассмотрим несколько сценариев машинного обучения, чтобы показать важность хорошо спроектированной платформы данных, поддерживаемой Apache Iceberg. Мы используем архитектуру, показанную на рис. 15.3, как потенциальное решение для трех сценариев.

В этой архитектуре у нас будут задействованы пакетные и потоковые источники, размещенные в виде таблиц Apache Iceberg в нашем озере данных. Мы будем использовать эти источники для создания таблиц признаков в формате Apache Iceberg, которые затем можно будет использовать для передачи в модели прогнозирования.

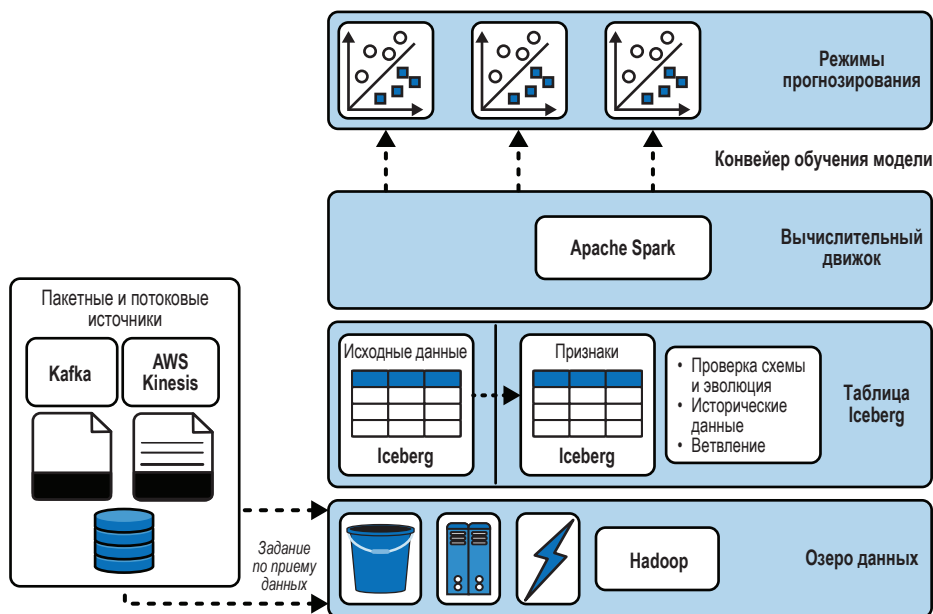


Рис. 15.3 ❖ Пример архитектуры

Сценарий 1: надежные конвейеры машинного обучения

У телекоммуникационной компании TelCan есть модель прогнозирования оттока, которая помогает выявлять и удерживать клиентов, которые склонны прекратить обслуживание. Существующая модель использует такие признаки, как продолжительность звонка, взаимодействие со службой поддержки клиентов и сведения о международных звонках. Рабочий процесс основан на Spark. TelCan регулярно обновляет свои конвейеры машинного обучения, добавляя новые данные из различных источников. Есть риск, что в процессе приема в таблицы попадут неверные типы данных, что приведет к сбоям в существующих конвейерах обучения. Кроме того, по мере добавления новых услуг, таких как VoLTE и 5G, начинают появляться соответствующие данные, такие как 5G_Usage_minutes и VoLTE_calls_made. Эти новые данные могут повысить точность модели прогнозирования оттока. Однако интеграция этих признаков в существующий конвейер обучения является сложной задачей. Компания сталкивается с двумя основными проблемами:

- Как обеспечить соблюдение схемы во время записи и уведомление заинтересованных сторон о любых сбоях?
- Как включить новые признаки в процесс обучения, не влияя на текущие наборы данных, используемые для обучения?

Чтобы конвейер обучения мог уверенно обрабатывать новые данные, нужна согласованная схема. Гарантия соответствия принимаемых данных предопределенной схеме дает уверенность в согласованности данных и надежности прогнозов. Таблицы Apache Iceberg имеют определенные схемы в своих метаданных, и механизмы вычислений могут использовать их для принудительного применения при записи новых данных и для приведения данных из старой схемы в текущую при чтении. Когда схемы принудительно применяются при записи, любые несоответствия в структуре данных немедленно отмечаются, благодаря чему последующие процессы, такие как конвейеры обучения, будут работать только с согласованными и заслуживающими доверия данными.

Ежедневное задание по приему данных в TelCan поставляет новые данные в конвейеры машинного обучения. Допустим, что задание по приему не выполняет никаких проверок схемы перед записью данных в таблицу Iceberg. В таком случае можно увидеть следующую ошибку (полный код с определениями функций и логикой обработки ошибок вы найдете в репозитории книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg>)):

```
# Принять новые данные
new_data_path = "churn_etl_new.csv"
iceberg_table_name = "glue.test.churn"
ingest_new_data(new_data_path, iceberg_table_name)

-----
# Ошибка приема данных в glue.test.churn: Невозможно записать несовместимые
данные в таблицу 'Iceberg glue.test.churn':
- Cannot safely cast 'Account_length': string to int
```

Как видите, Apache Spark генерирует ошибку, если схема входных данных не соответствует схеме существующей таблицы Iceberg.

В дополнение к собственным возможностям Iceberg по принудительному применению схемы TelCan может явно проверять входные данные, чтобы заранее выявлять любые ошибки, уведомлять ответственных лиц и избегать записи неверных данных в таблицы, используемые конвейером обучения. Предположим, что в настоящее время один из конвейеров машинного обучения использует набор данных Iceberg churn. Здесь мы используем пользовательскую функцию `validate_and_ingest`, чтобы сначала проверить схему и принять данные в случае успеха (полную реализацию `validate_and_ingest` вы найдете в репозитории книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg>)):

```
# Проверить и принять новые данные
new_data_path = "churn_etl.csv"
iceberg_table_name = "glue.test.churn"
validate_and_ingest(new_data_path, iceberg_table_name)
```

Предыдущий код сначала извлекает схему существующей таблицы Iceberg. Затем читает новые данные и сверяет их схему со схемой таблицы.

Если схемы совпадают, то новые данные добавляются в таблицу, иначе генерируется ошибка.

Давайте рассмотрим вторую часть проблемы: добавление новых признаков без прерывания обучения на текущих данных. TelCan хочет ввести два новых признака, `5G_Usage_minutes` и `VoLTE_calls_made`, в существующую таблицу Iceberg `churn`. Эта новая информация может иметь решающее значение для улучшения модели прогнозирования оттока, поскольку внедрение новых технологий может способствовать удержанию клиентов. Подобные изменения в бизнес-требованиях довольно распространены, поэтому важно иметь архитектуру данных, которая поддерживает быстрые и простые изменения в схеме без необходимости переписывать существующие таблицы (что может потребовать значительных затрат).

Для начала изменим схему таблицы `churn` с помощью Spark SQL:

```
# Добавить новые столбцы в таблицу Iceberg
spark.sql("""
    ALTER TABLE glue.test.churn
    ADD COLUMNS (5G_Usage_minutes FLOAT, VoLTE_calls_made INT)
""")
```

После добавления новых столбцов мы можем прочитать данные из нового источника, который включает эти столбцы, и добавить их в таблицу:

```
# Прочитать новые данные
df_evolved = spark.read.csv("newdatasource.csv")

# Добавить новые данные в таблицу Iceberg
df_evolved.writeTo("glue.test.churn").append()
```

Эти два шага гарантируют соответствие схемы базовой таблицы новым принимаемым данным.

Сценарий 2: воспроизводимость машинного обучения

Воспроизводимость в машинном обучении, подразумевающая достижение согласованных результатов с заданной версией модели на определенных наборах данных, важна как в академических исследованиях, так и в промышленности. Однако одна из проблем заключается в том, что крайне сложно воспроизвести эмпирические результаты определенной модели. Представьте, что вы специалист по данным в TelCan. Ваша модель прогнозирования оттока клиентов, построенная на основе исторических данных, достигла впечатляющих результатов. Теперь бизнес представил новый набор признаков (`5G_Usage_minutes` и `VoLTE_calls_made`) для построения новой модели. Через неделю вы замечаете падение качества прогнозирования новой модели. Вы подозреваете, что причиной могут быть новые данные или, возможно, из-

менные признаки. Для проверки вам нужно будет воспроизвести модель предыдущей недели, что означает возврат к старой версии набора данных. С традиционными архитектурами данных это может оказаться довольно сложной задачей. Как Apache Iceberg обеспечивает бесшовную воспроизводимость машинного обучения?

По умолчанию Apache Iceberg имеет две возможности для решения проблем воспроизводимости: доступ к историческим данным и маркировка. Давайте посмотрим, как использовать эти возможности для решения проблемы воспроизводимости в TelCan.

Исторические данные

Вместо создания нескольких копий данных, что часто практикуется в задачах воспроизводимости результатов машинного обучения, функция доступа к историческим данным в Iceberg позволяет получать доступ к более старым снимкам, используя либо отметку времени, когда снимок был зафиксирован, либо идентификатор снимка. Это приводит к более быстрой воспроизводимости, лучшему управлению версиями данных и снижению затрат на хранение. Iceberg позволяет находить все снимки, связанные с таблицей, с помощью таблицы метаданных `history`.

Давайте посмотрим на историю таблицы `churn` в TelCan:

```
spark.sql("SELECT * FROM glue.test.churn.history").toPandas()
```

Результат этого запроса выглядит следующим образом:

made_current_at	snapshot_id	parent_id	is_current_ancestor
2023-04-20 17:06:12.515	5889239598709613914	NaN	True
2023-05-03 19:24:24.418	7869769243560997710	5.889240e+18	True

В этой таблице присутствуют два снимка. Второй снимок (`snapshot_id` 7869769243560997710) – тот, в котором в таблицу были добавлены два новых признака, `5G_Usage_minutes` и `VoLTE_calls_made`. Мы можем определить его по отметке времени фиксации. Чтобы проверить, правда ли это, можно выполнить запрос Spark SQL к историческим данным:

```
spark.sql("SELECT * FROM glue.test.churn TIMESTAMP AS OF '2023-05-03 19:24:24.418'").toPandas()
```

Как видите, набор результатов включает два новых признака в виде двух столбцов:

State	...	Churn	5G_Usage_minutes	VoLTE_calls_made
LA	...	FALSE	5	1
IN	...	TRUE	3	0

```

-----
NY      ...  TRUE   3              1
-----
SC      ...  FALSE  10             0
-----
HI      ...  FALSE  12             0
-----
...     ...           ...           ...
-----
WI      ...  FALSE   6              2
-----
AL      ...  FALSE   5              3
-----
VT      ...  FALSE  15              0
-----
WV      ...  FALSE  13              1
-----

```

Для сравнения мы могли бы запросить первый снимок, с идентификатором 5889239598709613914, и посмотреть, присутствовали ли эти признаки в более ранней версии таблицы:

```
spark.sql("SELECT * FROM glue.test.churn VERSION AS OF
5889239598709613914").toPandas()
```

Если мы получим следующий набор результатов, то это будет означать, что более ранние снимки не содержат двух новых признаков:

```

State  ...  Total_intl_charge  Customer_service_calls  Churn
-----
LA     ...  2.35              1                          FALSE
-----
IN     ...  3.43              4                          TRUE
-----
NY     ...  1.46              4                          TRUE
-----
SC     ...  2.08              2                          FALSE
-----

```

Результаты подтверждают, что новые признаки были добавлены в более позднем снимке.

Теперь, лучше понимая историю таблицы, мы можем обучить модели на данных с ожидаемой схемой. Чтобы обучить модель на основе данных до добавления двух признаков, можно просто использовать эту более раннюю отметку времени, как показано в следующем фрагменте кода (полную версию кода вы найдете в репозитории книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg>)):

```
# Получить конкретный снимок таблицы Iceberg недельной давности
snapshot_id_of_interest = 5889239598709613914

df = spark.read \
    .option("snapshot-id", snapshot_id_of_interest) \
    .format("iceberg") \
```

```
.load("glue.test.churn")
```

```
# Преобразовать в набор данных Pandas для выполнения машинного обучения
pdf = df.toPandas()
```

Обучение различных версий модели на согласованных данных позволяет измерить фактическую выгоду от изменений в модели. Если новая модель работает лучше на новых данных с новыми признаками, но дает прогнозы того же качества, что и старая версия на старых данных, то можно считать, что улучшение обусловлено качеством данных, а не улучшением модели. Используя функцию доступа к историческим данным, специалисты по данным TelCan могут более полно тестировать свои модели на согласованных и воспроизводимых данных.

Теги

Теги – это метки для отдельных снимков. Так что технически метки – это просто имена версий, присвоенные снимкам. Подробнее о тегах можно прочитать в главе 11.

Ежедневное задание TelCan по приему данных теперь вставляет некоторые новые записи в таблицу Iceberg churn, как показано в следующем коде:

```
## Создать временное представление из новых данных
spark.sql(
    """CREATE OR REPLACE TEMPORARY VIEW new_train_data USING csv
        OPTIONS (path "newtrainingdata.csv", header true)"""
)

## Вставить новые данные в таблицу
spark.sql(
    "INSERT INTO glue.test.churn SELECT * FROM new_train_data"
)
```

Как специалист по данным вы решили использовать эту конкретную версию таблицы для обучения существующей производственной модели и сравнить ее с моделью, обученной на более поздних данных. Чтобы избежать необходимости постоянно искать идентификатор снимка, можно просто создать тег и обратиться к нему позже. Сделать это совсем несложно:

```
## Создать тег для указанного снимка
spark.sql("ALTER TABLE glue.test.churn CREATE TAG June_23")
```

Если теперь запросить таблицу метаданных refs, мы увидим все наши теги:

```
spark.sql("SELECT * FROM glue.test.churn.refs").toPandas()
```

Допустим, что этот запрос вернул следующие результаты:

```
name    type    snapshot_id    max_ref... min_snapshots_to_keep max_snap...
-----
```

main	BRANCH	2496871934354606665	NaN	NaN	NaN

June_23	TAG	2496871934354606665	NaN	NaN	NaN

Тег June_23 – это тот, который мы только что создали. Теперь мы можем сослаться на эту версию набора данных, используя данный тег, и воспроизвести модель. Следующий код показывает, как можно указать тег (полный код, реализующий процесс обучения модели, вы найдете в репозитории книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg>)):

```
df = spark.read \
    .option("tags", 'June_23') \
    .format("iceberg") \
    .load("glue.test.churn")
```

Используя такие функции Iceberg, как доступ к историческим данным и маркировка, можно эффективно решать некоторые из важнейших проблем с воспроизводимостью машинного обучения. Они дают большую гибкость и более простой способ воспроизведения моделей машинного обучения. Аналогичного результата можно достичь и с использованием нескольких таблиц, применяя маркировку на уровне каталога с помощью Nessie, как было показано в главе 11.

Сценарий 3: поддержка экспериментов с машинным обучением

TelCan наблюдает рост оттока клиентов премиум-класса, что приводит к существенной потере доходов. Команда специалистов по данным предполагает, что интеграция отзывов клиентов и заявок на поддержку может помочь выявить новые закономерности для прогнозирования оттока. Внедрение такого изменения непосредственно в существующий набор данных несет в себе значительные риски. Этот набор данных управляет бизнес-решениями в реальном времени и программами по работе с клиентами, а также вводится в другие продукты данных. Нарушение его целостности, даже кратковременное, может привести к выработке ошибочных стратегий и дальнейшим потерям доходов. В настоящее время единственный вариант – это создание нескольких копий данных для экспериментов, что довольно дорого и сложно и может привести к таким проблемам, как дрейф данных. Поэтому задача заключается в следующем: получить возможность безопасно и гибко экспериментировать с новыми данными, не влияя на промышленный набор данных и не создавая несколько копий.

В этом может помочь возможность ветвления на уровне таблиц Iceberg. Ветви в Iceberg – это изменяемые именованные ссылки на снимки, которые можно обновлять и связывать с новыми снимками. Таким образом, создав ветвь из существующей таблицы Iceberg, можно изолированно записывать

данные, относящиеся к этой ветви, и сделать эту ветвь доступной для экспериментов или удалить ее. Ветвление подробно обсуждается в главе 11.

Ветви обеспечивают изолированный способ взаимодействия с наборами данных, что может принести пользу в таких ситуациях, как эксперименты с машинным обучением. Для начала команда инженеров TelCan решила создать изолированную ветвь `Churn_PremiumExp`:

```
## Создать ветвь
spark.sql("ALTER TABLE glue.test.churn CREATE BRANCH Churn_PremiumExp")
```

Затем они интегрировали новые записи для эксперимента, которые извлекаются из опросов клиентов, форм обратной связи и других внешних источников.

Новые данные, хотя и потенциально ценные, являются экспериментальными. Поэтому они записываются изолированно в ветвь `Churn_PremiumExp`:

```
# Загрузить новые данные
df_exp = spark.read.csv("churndata.csv", header=True, inferSchema=True)

## Записать данные в ветвь
df_exp.write.format("iceberg").mode("append").save("glue.test.churn.branch_
Churn_PremiumExp")
```

Помещение новых данных в изолированную ветвь гарантирует, что они не окажут никакого влияния на основную таблицу. Таким образом, приложения или пользователи, использующие основную версию таблицы, не заметят этих изменений. Это дает команде возможность поэкспериментировать с данными и построить модели.

Давайте убедимся, что основной набор данных об оттоке остался нетронутым, а новые данные были добавлены в ветвь `Churn_PremiumExp`:

```
# Запросить основной набор данных
spark.table("glue.test.churn").toPandas()

# Выведет: 1240 записей × 20 столбцов
# Запросить экспериментальную ветвь
spark.read.format("iceberg").load("glue.test.churn.branch_Churn_
PremiumExp").toPandas()

# Выведет: 2175 записей × 20 столбцов
```

Эти результаты подтверждают, что вновь добавленные записи доступны только в изолированной ветви, но недоступны в основной таблице.

Используя эти данные, команда может создавать и тестировать новые модели в рамках своих экспериментов. Следующий код показывает, как указать, что данные должны извлекаться из ветви. Полный код с процессом обучения модели вы найдете в репозитории книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg>):

```
df_prem = spark.read \
    .option("branch", 'Churn_PremiumExp') \
```

```
.format("iceberg") \  
.load("glue.test.churn_new")
```

Если после эксперимента команда посчитает новый набор данных бесценным, то они могут сделать эти данные доступными из основной таблицы, объединив новую ветвь с основной. С другой стороны, если эксперимент не показал существенных выгод, ветвь можно без опаски удалить, оставив основную таблицу в неприкосновенности:

```
## Удалить ветвь  
spark.sql("ALTER TABLE glue.test.churn DROP BRANCH Churn_PremiumExp")
```

Благодаря поддержке ветвления в Iceberg специалисты из TelCan могут экспериментировать и вводить новые данные, не влияя на основные данные, и тем самым гарантировать, что решения по-прежнему будут приниматься на основе целостных и качественных данных. То же самое можно организовать с использованием нескольких таблиц, осуществив ветвление на уровне каталога Nessie, как обсуждалось в главе 11.

Согласованность данных, возможность проведения экспериментов и воспроизведения результатов имеют решающее значение в машинном обучении. Функции Apache Iceberg можно использовать для защиты целостности и качества данных и предоставления специалистам по машинному обучению возможности экспериментировать и откатывать изменения без особых хлопот, тем самым повысить эффективность и надежность рабочих процессов машинного обучения.

Работа с историческими данными и медленно меняющимися измерениями

SCD (Slowly Changing Dimensions – медленно изменяющиеся измерения) необходимы для отслеживания эволюции информации в базах данных OLAP. Особенно это относится к описательным атрибутам, привязанным к фактическим данным, таким как характеристики продукта. SCD обеспечивает точность исторических данных, предотвращая сбои в аналитических рабочих нагрузках. Среди шести типов SCD тип SCD Type 2 (SCD2) сохраняет полную историю значений, добавляя новые записи для изменения атрибутов измерений, отмечая старые как ставшие недействительными. Он использует такие столбцы, как Effective Start Date, Effective End Date и Is_Current Flag. SCD – это независимая от технологии практика, применимая к различным архитектурам данных.

Представьте себе, что Nova Trends, розничный продавец, торгующий товарами для дома и имеющий как обычные магазины по всей Северной Америке, так и платформу электронной коммерции, использует архитектуру озерного хранилища с Apache Iceberg на Amazon S3 для аналитики, получая выгоду от открытой структуры данных, эволюции схем, путешествий во времени и транзакционных гарантий.

В какой-то момент Nova Trends столкнулась с трудностями в управлении историческими данными. Обновление адреса клиента непосредственно в таблице Customer, когда клиент переезжает из Чикаго в Париж, может нарушить точность аналитических отчетов, которые имеют решающее значение для анализа продаж, инвентаризации и маркетинга. Это прямое обновление также сотрет любой исторический контекст, например переход клиента от онлайн-покупок в Чикаго к посещениям магазина в Париже, что приведет к потере ценных сведений о поведении клиента.

Чтобы решить эти проблемы с историческими данными, компания приняла подход SCD2 для таблицы Customer, включив такие поля, как `eff_start_date`, `eff_end_date` и `is_current`, помогающие эффективно отслеживать изменения. Команда создала две таблицы Iceberg, Sales (факт) и Customers (измерение, управляемое методом SCD2), ввела суррогатный ключ (`customer_dim_key`) в таблицу Sales для сохранения точности исторических данных, обработала изменения адресов клиентов с помощью логики SCD2 с соответствующими датами вступления в силу и добавила новые данные о продажах для проверки правильности обработки этих изменений.

На рис. 15.4 показано визуальное представление этого подхода.

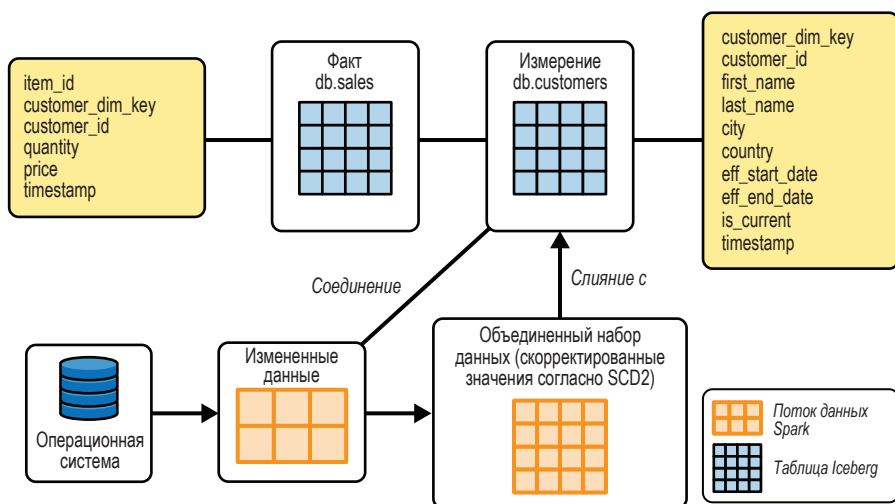


Рис. 15.4 ❖ Обработка исторических данных с использованием шаблонов SCD2

Давайте посмотрим, как все это работает.

Создание таблиц фактов и измерений для подготовки к применению SCD2

Чтобы воспроизвести сценарий Nova Trends, создадим таблицу фактов Iceberg с именем Customers. Ниже приводится код, создающий схему (полный код, создающий таблицу и внедряющий данные, вы найдете в репозитории книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg>)):

```
dim_customer_schema = StructType([
    StructField('customer_id', StringType(), False),
    StructField('first_name', StringType(), True),
    StructField('last_name', StringType(), True),
    StructField('city', StringType(), True),
    StructField('country', StringType(), True),
    StructField('eff_start_date', DateType(), True),
    StructField('eff_end_date', DateType(), True),
    StructField('timestamp', TimestampType(), True),
    StructField('is_current', BooleanType(), True),
])
```

Выполним запрос к таблице, чтобы узнать, что она содержит:

```
spark.sql("SELECT * FROM glue.dip.customers").toPandas()
```

Результат должен выглядеть так:

cmr_	...	...	...	...	eff_	eff_	timestamp	is_	customer_
id					start_date	end_date		crnt	dim_key
1	...	...	...	...	2020-09-27	2999-12-31	2020-12-08 09:15:32	True	1694797722535090
2	...	...	...	...	2020-10-14	2999-12-31	2020-12-08 09:15:32	True	1694797722510645

Поле `customer_dim_key` служит мостом к таблице измерений Customer. Используя это поле в таблице Sales, можно связать каждую продажу с определенной версией записи о клиенте. Это особенно полезно в таких сценариях, как изменение в данных клиента (в данном случае переезд из одного города в другой) и когда желательно сохранить контекст продаж по старым и новым профилям клиента. Это также помогает различать исторические и текущие записи для одного и того же клиента.

Теперь давайте поработаем с таблицей фактов Sales. Следующий код выводит схему (полный код, создающий набор данных DataFrame с информацией о продажах под именем `sales_fact_df`, можно найти в репозитории книги на GitHub (<https://oreil.ly/supp-guide-apache-iceberg>)):

```
from pyspark.sql.functions import to_timestamp

fact_sales_schema = StructType([
```



```

StructField('item_id', StringType(), True),
StructField('quantity', IntegerType(), True),
StructField('price', DoubleType(), True),
StructField('timestamp', TimestampType(), True),
StructField('customer_id', StringType(), True)
])

```

Чтобы реализовать SCD2, важно определить правильную версию клиента (например, Париж или Чикаго) и гарантировать, что каждая продажа связана с соответствующим клиентом, и, следовательно, сохранность исторического контекста. Это достигается с помощью LEFT OUTER JOIN с определенной логикой условий.

Сначала сопоставим продажу с правильным клиентом на основе его уникального идентификатора (`sales_fact_df.customer_id == customer_ice_df.customer_id`). Затем убедимся, что отметка времени продажи приходится на эффективную дату начала срока действия записи клиента или на более позднее время (`sales_fact_df.timestamp >= customer_ice_df.eff_start_date`). Также проверим, что продажа произошла до эффективной даты окончания срока действия записи клиента. Эти условия вместе гарантируют связывание продажи с активной и действительной записью о клиенте (`sales_fact_df.timestamp < customer_ice_df.eff_end_date`):

```

from pyspark.sql.functions import when

join_cond = [sales_fact_df.customer_id == customer_ice_df.customer_id,
             sales_fact_df.timestamp >= customer_ice_df.eff_start_date,
             sales_fact_df.timestamp < customer_ice_df.eff_end_date]

customers_dim_key_df = (sales_fact_df
                        .join(customer_ice_df, join_cond, 'leftouter')
                        .select(sales_fact_df['*'],
                              when(customer_ice_df.customer_dim_key.isNull(), '-1')
                              .otherwise(customer_ice_df.customer_dim_key)
                              .alias("customer_dim_key") )
                        )

# Создать таблицу Sales
customers_dim_key_df.writeTo("glue.dip.sales").create()

```

Теперь, получив соединение в виде DataFrame и добавив ключ измерения клиента к каждой записи, получившаяся таблица должна выглядеть примерно так:

```

+-----+-----+-----+-----+-----+-----+
|item_id|quantity|price|timestamp|customer_id|customer_dim_key|
+-----+-----+-----+-----+-----+-----+
| 111|    40| 90.5|2020-11-17 09:15:32|    1|1694797722535090|
| 112|   250|80.65|2020-10-28 09:15:32|    1|1694797722535090|
| 113|   10|600.5|2020-12-08 09:15:32|    2|1694797722510645|
+-----+-----+-----+-----+-----+-----+

```

Обратите внимание, что `LEFT OUTER JOIN` гарантирует сохранение в результирующей таблице всех записей о продажах, даже если соответствующая запись измерения клиента не найдена. Для несуществующих записей о продажах мы присваиваем полю `customer_dim_key` значение по умолчанию (-1), чтобы легко выявить их и исправить.

Теперь попробуем получить информацию о количестве продаж по странам, выполнив агрегирующий запрос:

```
spark.sql(
    'SELECT ct.country, '
    'SUM(st.quantity) as sales_quantity, '
    'COUNT(*) as count_sales '
    'FROM glue.dip.sales2 st '
    'INNER JOIN glue.dip.customers ct on st.customer_dim_key = ct.customer_
dim_key '
    'group by ct.country').show()
```

```
+-----+-----+-----+
|country|sales_quantity|count_sales|
+-----+-----+-----+
| Canada|          290|           2|
|   US  |           10|           1|
+-----+-----+-----+
```

Обработка изменений в данных клиентов и реализация SCD2

В этом разделе мы рассмотрим еще один гипотетический сценарий. Представьте, что клиентка Angie Keller (`customer_id=2`), проживающая в Чикаго, переехала в Париж. Также предположим, что из операционной базы данных поступила запись о новом клиенте Sebastian White. Следующий код создает `DataFrame` с информацией об этих клиентах:

```
new_customer_dim_df = spark.createDataFrame([(3, 'Sebastian', 'White',
    'Rome', 'IT',
    datetime.strptime(datetime.today().strftime('%Y-%m-%d'), '%Y-%m-%d'),
    datetime.strptime('1999-12-31', '%Y-%m-%d'),
    datetime.strptime('2020-12-09 09:15:32', '%Y-%m-%d %H:%M:%S'), True),
    (2, 'Angie', 'Keller',
    'Paris', 'FR',
    datetime.strptime(datetime.today().strftime('%Y-%m-%d'), '%Y-%m-%d'),
    datetime.strptime('1999-12-31', '%Y-%m-%d'),
    datetime.strptime('2020-12-09 10:15:32', '%Y-%m-%d %H:%M:%S'), True)],
    dim_customer_schema)

new_customer_dim_df = new_customer_dim_df.withColumn("customer_dim_key", random_udf())
```

В соответствии с подходом SCD2 с появлением новой записи клиента предыдущая запись должна быть отмечена как устаревшая. Для этого мы выполним следующие шаги.

Во-первых, установим условие соединения `join_cond` для сопоставления записей из существующего набора данных клиентов с новыми входными записями на основе `customer_id`. Используя определенное условие соединения, мы выявляем записи в существующем наборе данных, которые имеют соответствующие записи в новом наборе данных.

Для этих выявленных записей сохраним большую часть их исходных атрибутов, но скорректируем значение `eff_end_date`, чтобы оно соответствовало значению `eff_start_date` новой записи, и запишем в `is_current` значение `false` (указывающее, что для данного клиента эта запись больше не является текущей). Затем объединим обновленные записи с информацией о клиентах с совершенно новыми записями и получим консолидированный набор данных (`merged_customers_df`):

```
from pyspark.sql.functions import lit

# Определить соответствие записей из существующего и нового наборов данных
# по полю customer_id
join_cond = [customer_ice_df.customer_id == new_customer_dim_df.customer_id,
customer_ice_df.is_current == True]

## Найти запись о клиенте для обновления
customers_to_update_df = (customer_ice_df
    .join(new_customer_dim_df, join_cond)
    .select(customer_ice_df.customer_id,
            customer_ice_df.first_name,
            customer_ice_df.last_name,
            customer_ice_df.city,
            customer_ice_df.country,
            customer_ice_df.eff_start_date,
            new_customer_dim_df.eff_start_date.alias("eff_end_date"),
            customer_ice_df.customer_dim_key,
            customer_ice_df.timestamp)
    .withColumn('is_current', lit(False))
)

## Объединить с новыми записями о клиентах
merged_customers_df = new_customer_dim_df.unionByName(customers_to_update_df)
```

Наконец, используем команду `MERGE INTO` для обновления существующих и вставки новых записей. Iceberg позволяет обновлять записи измерений клиентов в озере данных с транзакционными гарантиями:

```
# Преобразовать merged_customers_df в SQL-представление
merged_customers_df.createOrReplaceTempView("merged_customers_view")

# Инструкция MERGE INTO
merge_sql = """
MERGE INTO glue.dip.customers AS target
USING merged_customers_view AS source
ON target.customer_dim_key = source.customer_dim_key
WHEN MATCHED THEN
```

```

UPDATE SET
    target.first_name = source.first_name,
    target.last_name = source.last_name,
    target.city = source.city,
    target.country = source.country,
    target.eff_start_date = source.eff_start_date,
    target.eff_end_date = source.eff_end_date,
    target.timestamp = source.timestamp,
    target.is_current = source.is_current
WHEN NOT MATCHED THEN
    INSERT *
"""

```

```

# Выполнить инструкцию SQL
spark.sql(merge_sql)

```

Теперь посмотрим, что получилось после обновления:

```
spark.sql("SELECT * FROM glue.dip.customers").toPandas()
```

Как видите, данные о клиентах обновились:

cmr_ id	...	eff_ start_date	eff_ end_date	timestamp	is_ crnt	customer_ dim_key
1	...	2020-09-27	2999-12-31	2020-12-08 09:15:32	True	1694797722535090
2	...	2020-10-14	2023-09-15	2020-12-08 09:15:32	False	1694797722510645
2	...	2023-09-15	2999-12-31	2020-12-09 10:15:32	True	1694798226033799
3	...	2023-09-15	2999-12-31	2020-12-09 09:15:32	True	169479822594

Мы сократили эту таблицу, чтобы уместить по ширине книжной страницы. Полную таблицу можно увидеть в дополнительном репозитории (https://oreil.ly/apache-ice_more-content). Как и ожидалось, теперь у нас есть две записи для Angie: одна историческая со старым городом, Чикаго, а другая – текущая с новым городом, Парижем. Запись для Чикаго теперь имеет в поле `eff_end_date` значение 2023-09-15, совпадающее с датой активации записи для Парижа (обратите внимание на `eff_start_date`). Кроме того, в таблицу измерений добавились данные о совершенно новом клиенте (Sebastian White). Реализовав технику SCD2, компания Nova Trends теперь может отслеживать исторические изменения в данных о своих клиентах.

Добавление новых данных о продажах в новом месте

Давайте сделаем еще один шаг и добавим новые данные о покупках Angie в Париже:

```
sales_fact_df = spark.createDataFrame([('103', 300, 15.8, datetime.
    strptime(datetime.today().strftime('%Y-%m-%d')+' 12:15:42', '%Y-%m-%d
    %H:%M:%S'), '2'),
                                     ('104', 10, 800.5, datetime.
    strptime(datetime.today().strftime('%Y-%m-%d')+' 06:35:32', '%Y-%m-%d
    %H:%M:%S'), '2')], fact_sales_schema)

sales_fact_df.show()
```

```
+-----+-----+-----+-----+-----+
|item_id|quantity|price|          timestamp|customer_id|
+-----+-----+-----+-----+-----+
|   103|      300|  15.8|2023-09-15 12:15:42|          2|
|   104|       10|800.5|2023-09-15 06:35:32|          2|
+-----+-----+-----+-----+-----+
```

```
# Загрузить новые данные о продажах
customer_ice_df = spark.sql("SELECT * FROM glue.dip.customers")

join_cond = [sales_fact_df.customer_id == customer_ice_df.customer_id,
sales_fact_df.timestamp >= customer_ice_df.eff_start_date, sales_fact_df.
timestamp <= customer_ice_df.eff_end_date]

customers_dim_key_df = (sales_fact_df
    .join(customer_ice_df, join_cond, 'leftouter')
    .select(sales_fact_df['*'],
            when(customer_ice_df.customer_dim_key.isNull(),
                '-1').otherwise(customer_ice_df.customer_dim_key).alias("customer_dim_key") )
    )

customers_dim_key_df.writeTo("glue.dip.sales").append()
```

Если теперь прочитать таблицу Sales, то можно будет увидеть все записи о продажах вместе с обновленными данными:

```
spark.sql("select * from glue.dip.sales").toPandas()
```

Результаты должны выглядеть так:

```
item_id quantity price timestamp          customer_id customer_dim_key
-----
103      300      15.80 2023-09-15 12:15:42 2          1694798226033799
-----
```

104	10	800.50	2023-09-15 06:35:32	2	1694798226033799

111	40	90.50	2020-11-17 09:15:32	1	1694797722535090

112	250	80.65	2020-10-28 09:15:32	1	1694797722535090

113	10	600.50	2020-12-08 09:15:32	2	1694797722510645

Наконец, запустим агрегирующий запрос и проверим, были ли зафиксированы изменения, как ожидалось:

```
spark.sql("""
    SELECT ct.country, SUM(st.quantity) as sales_quantity, COUNT(*) as
count_sales
    FROM glue.dip.sales st
    INNER JOIN glue.dip.customers ct on st.customer_
dim_key = ct.customer_dim_key group by ct.country
""").show()
```

```
+-----+-----+-----+
|country|sales_quantity|count_sales|
+-----+-----+-----+
|    US|          10|          1|
|    FR|         310|          2|
| Canada|         290|          2|
+-----+-----+-----+
```

Теперь, как и ожидалось, мы видим две покупки, сделанные Angie, и ее актуальный адрес (country = FR).

Глава 16

Библиотеки на Java и Python для работы с Iceberg

Эта глава демонстрирует практические приемы работы с таблицами, схемами и файлами данных Apache Iceberg из программного кода на Java и Python. Она охватывает такие аспекты, как создание таблиц, схем и спецификаций сегментирования, а также чтение и обновление данных. Цель главы – дать читателям навыки эффективного взаимодействия с Apache Iceberg их программного кода.

Java API

Java API позволяет программно управлять метаданными и данными таблиц Iceberg из приложений на Java. В этой части главы мы рассмотрим такие операции, как создание таблиц и схем, а также сегментирование, чтение и обновление данных, чтобы вы могли быстро начать работу с этим API.

Создание таблицы

Как и при использовании любого другого вычислительного механизма, прежде чем начать создавать таблицы Apache Iceberg и взаимодействовать с ними, нужно создать каталог. В общем случае в Java API таблицы создаются с использованием реализации интерфейса `Catalog` или `Table`. В этой главе вы увидите, как использовать два встроенных каталога для создания таблиц Iceberg. Однако вы можете реализовать и использовать собственный каталог.

Инициализация каталога Hive включает определение имени и свойств каталога. Вот пример создания таблицы Iceberg с использованием каталога Hive:

```
import org.apache.iceberg.hive.HiveCatalog;
import org.apache.iceberg.Table;
import org.apache.iceberg.catalog.TableIdentifier;

HiveCatalog catalog = new HiveCatalog();
catalog.setConf(spark.sparkContext().hadoopConfiguration());
Map<String, String> properties = new HashMap<String, String>();
properties.put("warehouse", "...");
properties.put("uri", "...");

catalog.initialize("hive", properties);

TableIdentifier name = TableIdentifier.of("hr", "employee");
Table table = catalog.createTable(name, schema, spec);
```

Предыдущий код настраивает каталог Hive, который следит за таблицами Iceberg в Hive Metastore. Он выполняет необходимые настройки Hive и инициализирует каталог свойствами, такими как warehouse и URI. Затем вызывает метод `createTable()` для создания новой таблицы Iceberg с именем `employee` в пространстве имен `hr` с определенной схемой и спецификацией секционирования.

Каталог Hadoop можно использовать с файловыми системами, которые поддерживают атомарное переименование, например Hadoop Distributed File System (HDFS). Вот как создать таблицу вместе с каталогом Hadoop:

```
import org.apache.hadoop.conf.Configuration;
import org.apache.iceberg.hadoop.HadoopCatalog;
import org.apache.iceberg.Table;
import org.apache.iceberg.catalog.TableIdentifier;

Configuration conf = new Configuration();
String warehouseLoc = "hdfs://host:8020/path";
HadoopCatalog catalog = new HadoopCatalog(conf, warehouseLoc);

TableIdentifier name = TableIdentifier.of("hr", "employee");
Table table = catalog.createTable(name, schema, spec);
```

Создание схемы

Чтобы создать схему таблицы, нужно создать объект класса `Schema` в Iceberg. Вот пример определения схемы:

```
import org.apache.iceberg.Schema;
import org.apache.iceberg.types.Types;

Schema schema = new Schema(
    Types.NestedField.required(1, "id", Types.LongType.get()),
```



```
Types.NestedField.required(2, "department", Types.StringType.get()),
Types.NestedField.required(3, "role", Types.StringType.get()),
Types.NestedField.required(4, "salary", Types.LongType.get()),
Types.NestedField.required(5, "region", Types.StringType.get())
);
```

Этот код создаст схему для таблицы с пятью полями.

Создание спецификации секционирования

Спецификация секционирования в Java API определяется с помощью шаблона построителя. В Iceberg процесс построения спецификации секционирования запускает вызов `PartitionSpec.builderFor(schema)`. Секционирование основывается на схеме таблицы. Затем с помощью построителя нужно указать, как секционировать данные на разделы. Например:

```
import org.apache.iceberg.PartitionSpec;

PartitionSpec spec = PartitionSpec.builderFor(schema)
    .day("hire_date")
    .build();
```

Здесь стратегия секционирования определяется использованием метода `.day("hire_date")`. Это означает, что Iceberg создаст отдельный раздел для каждого отдельного дня.

Чтение таблицы

Для взаимодействия с таблицей Iceberg и получения информации о схеме, секционировании, метаданных и файлах данных в Java API используется интерфейс `Table`. Вот несколько наиболее часто используемых методов:

`schema()`

Возвращает текущую схему таблицы.

`spec()`

Сообщает информацию о секционировании таблицы.

`currentSnapshot()`

Извлекает текущий снимок таблицы.

`location()`

Возвращает базовое местоположение таблицы.

Сканирование на уровне файлов

Чтение из таблицы Iceberg начинается с создания объекта `TableScan` вызовом метода `newScan()`. `TableScan` возвращает список задач (файлы данных, файлы

удаления и т. д.), которые связаны с желаемым сканированием. Это понадобится, если вы захотите, чтобы ваш вычислительный движок поддерживал Apache Iceberg, так как с помощью TableScan можно составить список файлов для сканирования и обработки движком запросов без повторного выполнения процесса чтения метаданных Iceberg. Например:

```
TableScan scan = table.newScan();
```

Также можно использовать метод `filter()` объекта `TableScan` для фильтрации данных:

```
TableScan filteredScan = scan.filter(Expressions.equal("role", "Engineer"));
```

Предыдущий код прочитает метаданные таблицы и вернет список всех файлов в текущем снимке, содержащих записи с `role = Engineer`.

Для извлечения определенных данных служит метод `select()` объекта `TableScan`:

```
TableScan scan = table.newScan()
    .filter(Expressions.equal("role", "Engineer"))
    .select("id", "salary");
```

Сканирование на уровне записей

Сканирование на уровне записей в Java API начинается с создания объекта `ScanBuilder`, возвращающего отдельные записи, вызовом `IcebergGenerics.read()`. Для больших наборов данных это может быть непрактично, и предпочтительнее использовать механизм запросов, который, в свою очередь, скорее всего, будет использовать объект `TableScan`, представленный выше. Например:

```
ScanBuilder scanBuilder = IcebergGenerics.read(table);
```

Затем можно использовать метод `where()` объекта `ScanBuilder` для фильтрации записей:

```
CloseableIterable<Record> result = IcebergGenerics.read(table)
    .where(Expressions.lessThan("id", 5))
    .build();
```

В примере выше выражение `Expressions.lessThan("id", 5)` определяет записи, которые должны быть включены в результат.

Обратите внимание, что `ScanBuilder()` предоставляет более прямой способ извлечения записей из таблицы Iceberg, заключающийся в применении фильтров, подобных показанному в примере, тогда как `TableScan()` позволяет получить список задач/файлов для сканирования и передать их в выбранную вычислительную систему.

Обновление таблицы

Интерфейс `Table` предоставляет методы для обновления таблицы. Они также следуют шаблону строителя. Например, чтобы обновить схему таблицы, сначала нужно вызвать `updateSchema()`, затем добавить обновления в строителя и, наконец, вызвать `commit()`, чтобы зафиксировать изменения. Например:

```
table.updateSchema()
    .addColumn("location", Types.StringType.get())
    .commit();
```

Здесь вызовом метода `addColumn()` мы добавляем в таблицу Iceberg новый столбец с именем `location` типа `String`. В числе других доступных методов можно назвать: `updateLocation`, `updateProperties`, `newAppend`, `newOverwrite` и `rewriteManifests`.

Создание транзакции

Транзакции в Iceberg поддерживают атомарную фиксацию нескольких изменений в таблице. Отдельные операции в транзакции создаются с использованием фабричных методов, и все эти операции фиксируются вместе с использованием метода `commitTransaction()`.

Например, в следующем примере в рамках одной транзакции мы сначала перезаписываем некоторые файлы, соответствующие выражению фильтра, затем заменяем их новым файлом данных и обновляем местоположение таблицы с помощью `updateLocation()`. В конце мы выполняем атомарную фиксацию:

```
Transaction t = table.newTransaction();

// Перезаписать файлы
t.newOverwrite()
    .overwriteByRowFilter(Expressions.equal("id", 5))
    .addFile(newDataFile)
    .commit();

// Обновить местоположение таблицы
t.updateLocation()
    .setLocation("new-location-path")
    .commit();

// Зафиксировать все произведенные изменения
t.commitTransaction();
```

Python API

Python API для Apache Iceberg позволяет взаимодействовать с таблицами Iceberg из приложения на Python без использования стороннего движка (библиотек на Rust и Go). Он также дает возможность работать с каталогами Apache Iceberg и метаданными и выполнять такие операции, как создание таблиц и планирование сканирования. В этом разделе мы рассмотрим несколько практических примеров, чтобы вы могли увидеть, как настраивать каталоги, создавать и загружать таблицы и запрашивать данные.

Установка PyIceberg

Последнюю версию PyIceberg можно установить из каталога PyPi:

```
pip3 install "pyiceberg[s3fs,hive]"
```

Эта команда установит PyIceberg с s3fs в качестве реализации FileIO с поддержкой Hive Metastore. Обратите внимание, что при установке PyIceberg можно указать в скобках [] различные зависимости, включая AWS Glue, Amazon Simple Storage Service (Amazon S3), PyArrow и многое другое. Также для доступа к файлам вам нужно будет установить s3fs, adlfs или PyArrow.

Настройка каталога

Настройка каталога – первый шаг к использованию API для доступа к таблицам Iceberg. В настоящее время PyIceberg поддерживает каталоги AWS Glue, Hive и REST. Существует несколько способов настройки каталога: с помощью файла `~/.pyiceberg.yaml`, с помощью переменных окружения или путем передачи конфигурации данных напрямую через CLI или из кода на Python.

Давайте посмотрим, как настроить различные каталоги с помощью PyIceberg. Для определения конфигураций мы будем использовать файл `pyiceberg.yaml`. Поместите этот файл в свой домашний каталог, местоположение которого может отличаться для разных операционных систем и настроек переменных окружения.

Каталог Hive

Для настройки каталога Hive нужно указать URI в Hive Metastore и сведения о системе хранения. Например, если в качестве хранилища вы используете Amazon S3, то вам нужно указать настройки, специфичные для S3 (конечную точку, идентификатор ключа доступа и секретный ключ доступа). Вот как может выглядеть содержимое такого файла конфигурации:

```
catalog:
  default:
    uri: thrift://localhost:9083
  s3:
    endpoint: http://localhost:9000
    access-key-id: admin
    secret-access-key: password
```

Каталог Glue

Чтобы использовать каталог Glue, нужно напрямую передать учетные данные AWS через Python API или настроить их в файле конфигурации. Вот пример конфигурации:

```
catalog:
  default:
    type: glue
    aws_access_key_id: <ACCESS_KEY_ID>
    aws_secret_access_key: <SECRET_ACCESS_KEY>
    aws_session_token: <SESSION_TOKEN>
    region_name: <REGION_NAME>
```

Каталог REST

Конфигурация каталога REST включает URI сервера REST и учетные данные, необходимые для аутентификации. Вот пример конфигурации:

```
catalog:
  default:
    uri: http://rest-catalog/ws/
    credential: t-1234:secret
```

Также в конфигурации каталога REST можно указать дополнительные параметры, такие как токен носителя для заголовка `Authorization` и `rest.sigv4-enabled`, указывающий на необходимость использования AWS Signature v4 (SigV4) для аутентификации запросов.

Загрузка таблицы

Загрузить таблицу с помощью PyIceberg можно либо из каталога, либо напрямую из файла метаданных. Давайте посмотрим, как это сделать на практике.

Из каталога

Чтобы загрузить таблицу из каталога, сначала нужно подключиться к этому каталогу. Следующий пример показывает, как подключиться к каталогу Glue:

```
from pyiceberg.catalog import load_catalog
```

```
catalog = load_catalog("glue_catalog")
catalog.list_namespaces()
```

Последний вызов вернет доступные пространства имен. В этом примере возвращается пространство имен `company`.

Чтобы вывести список всех таблиц в пространстве имен `company`, можно вызвать метод `list_tables()`:

```
catalog.list_tables("company")
```

Этот вызов вернет список таблиц, существующих в пространстве имен, в виде списка кортежей, например `[("company", "employees")]`.

Наконец, чтобы загрузить таблицу `employees`, можно использовать метод `load_table()`:

```
catalog.load_table("company.employees")

# Альтернативный вариант:
catalog.load_table(("company", "employees"))
```

Из файла метаданных

Загрузка таблицы напрямую из файла метаданных может пригодиться в ситуациях, когда нужно сделать это в обход каталога для выполнения одnorазовых транзакций только для чтения. Любые обновления таблиц всегда должны выполняться только через каталог, чтобы обеспечить их согласованность. Загрузить таблицу можно с помощью класса `StaticTable`:

```
from pyiceberg.table import StaticTable

ice_table = StaticTable.from_metadata( "s3a://my-bucket/test.db/salesnew/
metadata/00001-91773d96-d1f4-490e-a894-9bc0652b6500.metadata.json"
)
```

Метод `from_metadata()` принимает местоположение файла метаданных в хранилище и загружает его в виде таблицы.

Создание таблицы

Чтобы создать таблицу с помощью `PuIceberg`, нужно определить схему, спецификацию секционирования или порядок сортировки, а также каталог, в котором должна быть создана таблица. Затем передать всю эту информацию методу `create_table()`, который создаст требуемую таблицу. Вот пример создания таблицы `employee` с пятью полями:

```
from pyiceberg.catalog import load_catalog
from pyiceberg.schema import Schema
from pyiceberg.types import DoubleType, StringType, NestedField
```

```
# Определить схему
schema = Schema(
    NestedField(required=False, field_id=1, name="id", field_type=DoubleType()),
    NestedField(required=False, field_id=2, name="role", field_type=String
Type()),
    NestedField(required=False, field_id=3, name="salary", field_type=Double
Type()),
    NestedField(required=False, field_id=3, name="department", field_type=String
Type()),
    NestedField(required=False, field_id=3, name="region", field_type=String
Type())
)

# Загрузить каталог
catalog = load_catalog("glue_catalog")

# Создать таблицу
catalog.create_table(
    identifier="company.employee", location="/Users/user/Desktop/docker-spark-iceberg/wh/
employees/", schema=schema
)
```

Обновление свойств таблицы

PyIceberg имеет ограниченную поддержку операций записи. Однако обновить свойства таблицы можно через Transaction API. Например:

```
# настроить свойство
with table.transaction() as transaction:
    transaction.set_properties(prop="value")
assert table.properties == {"prop": "value"}
```

Этот блок кода запускает транзакцию в таблице, настраивает свойство, а затем проверяет, правильно ли настроено свойство. Если процесс завершается неудачей, вся транзакция откатывается, и таблица останется в неприкосновенности.

Запрос данных

Чтобы запросить таблицу с помощью PyIceberg, вам понадобится объект, аналогичный объекту TableScan в Java API, который вернет список файлов. Методу `table_scan()` нужно передать несколько аргументов, включая фильтры, ограничения и идентификаторы снимков. Вот пример, показывающий, как загрузить таблицу `employees` из `glue_catalog` с фильтром, оставляющим записи о сотрудниках с зарплатой от 50 000 долларов и выше:

```
from pyiceberg.catalog import load_catalog
from pyiceberg.expressions import GreaterThanOrEqual
```

```
catalog = load_catalog("glue_catalog")
table = catalog.load_table("employee")

scan = table.scan(
    row_filter=GreaterThanOrEqual("salary", 50000.0),
    selected_fields=("id", "role", "salary"),
    limit=100
)
```

После выполнения сканирования таблицы необходимо передать список задач/файлов чему-то, что сможет обработать и прочитать список файлов. В PyIceberg имеются средства для передачи плана сканирования в PyArrow, DuckDB и Ray.

Заключение

В этой главе рассматриваются Java и Python API для работы с таблицами Apache Iceberg. С их помощью можно создавать таблицы, планы сканирования для вычислительных движков, транзакции и многое другое. Разрабатываются новые API для других языков, таких как Rust и GO, что, безусловно, способствует расширению охвата Apache Iceberg разными языками и инструментами, реализованными на этих языках.

Предметный указатель

A

- Actions, пакет, 89
- add_files, процедура, миграция в Iceberg, 295
- all_data_files, таблица метаданных, 212
- all_manifests, таблица метаданных, 215
- ALTER TABLE, 98
 - ветвление в Iceberg, 225
 - Dremio, 168
 - Spark SQL, 148
- ALTER TABLE...ADD COLUMN, 149
- ALTER TABLE...ADD COLUMNS, 168
- ALTER TABLE...ADD PARTITION FIELD, 152
- ALTER TABLE...ALTER COLUMN, 150, 169
- ALTER TABLE...DROP COLUMN, 151, 169
- ALTER TABLE...DROP PARTITION FIELD, 152
- ALTER TABLE (Flink), 189
- ALTER TABLE...MODIFY COLUMN, 168
- ALTER TABLE...RENAME COLUMN, 150
- ALTER TABLE...RENAME TO, 148
- ALTER TABLE...REPLACE PARTITION FIELD, 152
- ALTER TABLE...SET/DROP IDENTIFIER FIELDS, 153
- ALTER TABLE...SET TBLPROPERTIES, 149
- ALTER TABLE...WRITE DISTRIBUTED BY PARTITION, 152
- ALTER TABLE... WRITE ORDERED BY, 152
- Amazon MCK, 257
- Amazon DynamoDB, 123
 - и каталог Hadoop, проблемы атомарности, 123
- Amazon EMR, 125
- Amazon Kinesis Data Analytics, 257
- Amazon Kinesis Data Firehose, 257
- Amazon Kinesis Data Streams, 257
- Amazon S3
 - как каталог Iceberg, 64
 - управление и безопасность, 264
- Amazon Simple Storage Service (Amazon S3), 122
- Amazon Web Services (AWS), потоковая обработка данных, 256
- Apache Avro, формат файлов данных, 52
- Apache DataSketches, библиотека, 63
- Apache Flink, 182
 - запись данных, 191
 - настройка, 182
 - операции DDL, 185
 - потоковая обработка данных, 243
 - чтение данных, 189, 245
 - DataStream и Table API, 193
- Apache Hudi, 293
- Apache Iceberg, 24, 43
 - архитектура, 45, 51
 - история разработки, 30
 - ключевые особенности, 46
 - озера данных, 32
 - озерные хранилища данных, 37
 - потоковая обработка, 237
 - таблицы метаданных, 201
 - хранилища данных, 29
 - Flink DataStream API и Flink Table API, 245
 - Spark Structured Streaming API, 239
- Apache Iceberg Sink Connector (Kafka), 253
- Apache Kafka Connect, 252
- Apache ORC, формат файлов данных, 52
- Apache Parquet, 91

Apache Parquet, формат файлов данных, 52
 Apache Spark, 138
 запись данных, 157
 и миграция каталогов, 133
 настройка для работы с Iceberg, 138
 оконные функции, 156
 операции DDL, 145
 поточковая обработка, 238
 процедуры обслуживания таблиц Iceberg, 161
 чтение данных, 154, 239
 Actions пакет, 89
 ASSIGN (Nessie), 235
 AVG(), агрегирующие запросы, 155, 170
 AWS CloudTrail, 285
 AWS Glue, 176
 настройка, 176
 настройка с помощью Spark, 142
 создание таблиц данных, 177
 AWS Glue, каталог, 126, 283
 AWS Glue Data Catalog, 64, 180
 AWS Glue Studio, 258
 AWS Key Management Service, 264
 AWS Lake Formation, сервис, 283
 AWS SDK, пакет, 144
 Azure Data Lake Storage (ADLS), 122, 268
 Azure Key Vault, 268

B

binpack, стратегия уплотнения, 94
 по умолчанию, 174
 binPack, SparkActions, 90
 blob, puffin-файлы, 62

C

Catalog, интерфейс (Spark), 142
 cherrypick snapshot, 234
 COPY INTO
 миграция данных в Apache Iceberg, 298
 Dremio, 172
 COUNT()
 агрегирующие запросы, 155, 170
 Dremio, 170
 CREATE CATALOG (Flink), 185
 CREATE DATABASE (Flink), 187
 create_data_frame.from_catalog(), 180

CREATE (Iceberg), 69
 CREATE TABLE, 97
 Dremio, 166
 Flink, 188
 Spark SQL, 145
 CREATE TABLE ... AS SELECT (CTAS), 98
 CREATE TABLE...AS SELECT (Dremio), 166
 CREATE TABLE...AS SELECT (Spark SQL), 147
 CREATE TABLE...LIKE (Flink), 188
 CREATE TABLE...PARTITIONED BY (Flink), 188
 current-snapshot-id, 79
 Customer-Managed Encryption Keys (CMEK), 272

D

DataFrame, преобразование в Glue DynamicFrame, 179
 DataFrame API, операции DDL, 145
 DataStream API (Flink), 193, 245
 DataStreamWriter (Spark), 240
 DEFINER (Trino), 278
 DELETE (Dremio), 173
 DELETE FROM (Spark), 160
 Delta Lake, 292
 Dremio, 164
 Dremio Cloud, 164
 Dremio Lakehouse, 164
 Dremio SQL Query Engine, 68, 164
 настройка, 164
 обслуживание таблиц Iceberg, 173
 операции DDL, 166
 Dremios SQL Query Engine
 запись данных, 171
 чтение данных, 169
 DROP TABLE, 154
 Dremio, 169
 Hadoop, 124
 DROP TABLE (Flink), 189
 DynamicFrame (AWS Glue), 179
 DynamoDB, 123

E

entries, таблица метаданных, 218
 execute (SparkActions), 90
 expire_snapshots, 161

expire_snapshots, процедура, 115
 EXPIRE SNAPSHOTS (Dremio), 174
 Expressions, библиотека, 92

F

files, таблица метаданных, 206
 filter (SparkActions), 90
 FlinkSink.forRowData(), метод, 249
 FlinkSource, класс (Java), 247, 248
 Flink SQL, клиент, 185

G

GlueContext, объект, 180
 GlueContext.write_data_frame.from_catalog(), 181
 Google Cloud Storage (GCS), 122, 271

H

Hadoop, каталог, 122, 141, 185
 Hadoop Common (библиотека), 183
 history, таблица метаданных, 201
 Hive
 каталог, 125, 186
 оригинальный формат таблиц, 40
 фреймворк, миграция таблиц в Apache Iceberg, 290
 Hive Metastore, 125, 141
 как каталог Iceberg, 64
 HudiToIcebergMigrationActionsProvider, процедура (Iceberg), 294

I

iceberg-catalog-migrator, команда, 132
 iceberg-delta-lake, модуль, 292
 iceberg-hudi, модуль, 294
 iceberg-spark-runtime, пакет, 139
 INSERT INTO, 71
 настройка порядка сортировки, 99
 Dremio, 172
 Flink, 191, 249
 Spark SQL, 157
 INSERT INTO SELECT, миграция данных в Apache Iceberg, 300
 INSERT OVERWRITE
 Flink, 191, 249
 Spark SQL, 158
 INVOKER (Trino), 278

J

Java (Flink), 193
 JDBC, каталог, 129
 плюсы и минусы, 129
 JobManager (Flink), 184

K

Kafka Connect, 252
 kafka-connect-iceberg, модуль, 255

M

manifests, таблица метаданных, 208
 Maven, проект, создание задания Flink, 193
 MAX(), агрегирующие запросы, 156
 Dremio, 171
 max-commits, 92
 max-concurrent-file-group-rewrites (SparkActions), 90
 max-file-group-size-bytes (SparkActions), 90
 MERGE INTO, 74
 миграция данных в Apache Iceberg, 300
 Dremio, 74, 172
 Spark SQL, 74, 158
 metadata_log_entries, таблица метаданных, 203
 migrate
 команда, 132
 процедура (Iceberg), 291
 MySQL, 129

N

Nessie, каталог, 127
 плюсы и минусы, 127
 поддержка парадигмы «данные как код», 127

O

OPTIMIZE (Dremio), 94
 option (SparkActions), 90
 options (SparkActions), 90
 org.apache.iceberg.spark.
 SparkCatalog, 141
 org.apache.iceberg.spark.
 SparkSessionCatalog, 142

P

partial-progress-enabled (SparkActions), 90
 partial-progress-max-commits (SparkActions), 91
 partitions, таблица метаданных, 211
 partition-spec-id, 80
 PostgreSQL, 129
 Project Nessie, маркировка и ветвление на уровне каталога, 227
 puffin-файлы, 62
 PyFlink, 182
 PySpark, настройка для работы с Iceberg, 140

R

RANK(), оконные функции, 156, 171
 refs, таблица метаданных, 217
 register, команда, 132
 register_table() (Spark SQL), 134
 remove_orphan_files, процедура, 116
 remove_orphan_files (Spark), 162
 REPLACE PARTITION, 107
 REST, каталог, 128
 плюсы и минусы, 128
 REWRITE DATA (Dremio), 174
 rewrite_data_files (Spark), 162
 rewriteDataFiles (SparkActions), 90
 rewrite-job-order (SparkActions), 91
 REWRITE MANIFESTS (Dremio), 175
 rollback_to_snapshot (Iceberg), 232
 rollback_to_timestamp (Iceberg), 233

S

SELECT, 79
 SELECT *, запрос, 154
 SELECT, запрос
 Dremio, 170
 Flink, 245
 SELECT * FROM WHERE (Spark), 155
 set_current_snapshot (Iceberg), 233
 SET REF (Nessie), 235
 SHOW LOG (Nessie), 235
 snapshot, процедура (Iceberg), 291
 snapshotDeltaLakeTable, процедура (Iceberg), 292
 snapshotHudiTable, процедура (Iceberg), 294

snapshots, таблица метаданных, 204
 snapshot() (Spark SQL), 135
 Sort, стратегия уплотнения, 94
 SortOrder, свойство таблиц, 117
 Sort (SparkActions), 90
 SparkActions, 89
 SparkCatalog, 141
 SparkSession.builder (PySpark), 140
 SparkSessionCatalog, 142
 Spark SQL
 изменение таблиц с помощью расширений Iceberg, 151
 миграция каталогов, 133
 настройка для работы с Iceberg, 139
 операции DDL, 145
 создание таблицы, 68
 ALTER TABLE, 148
 CREATE TABLE...AS SELECT, 147
 DROP TABLE, 154
 Spark Structured Streaming API, 239
 spark.table() (DataFrame API), 154
 SSE-C (SSE с ключами, предоставленными клиентом), 264
 SSE-KMS (SSE со службой управления ключами AWS), 264
 SSE-S3 (SSE с ключами, управляемыми S3), 264
 SUM()
 агрегирующие запросы, 171
 Dremio, 171
 SUM(), агрегирующие запросы, 155

T

Table API (Flink), 193, 245
 TableLoader, класс (Java), 247
 Tabular, 282
 target-file-size-bytes (SparkActions), 90
 TaskManager (Flink), 184
 Theta Sketch, 63

U

UDF (User Defined Function – функции, определяемые пользователем), 167
 UPDATE
 Dremio, 173
 Spark SQL, 160
 UPSERT (Flink), 192
 UPSERT/MERGE INTO, 74

Dremio, 74
Spark SQL, 74

V

VACUUM, метод (Dremio), 174

W

WAP с использованием функции
ветвления в Iceberg, 303
WAP (Write Audit Publish – записать,
проверить, опубликовать), 226, 303
write.delete.mode, 112
write.merge.mode, 112
write.update.mode, 112

Z

z-упорядочение как стратегия
уплотнения, 101
zOrder
 стратегия уплотнения, 94
 SparkActions, 90

A

Автоматизация уплотнения, 96
Агрегирующие запросы, 155, 170
Архитектура (Apache Iceberg), 51
Атомарные операции
 многотабличные транзакции, 230
 обновление текущего указателя на
 метаданные, 122
 указатель на текущие метаданные, 63
Аудит и журналирование, 261

Б

Безопасность на уровне файлов, 261

В

Ветвление
 каталога, 228
 таблиц, 224
 изоляция изменений, 223
 мониторинг, 221
 на уровне каталога, 227
Восстановление данных (таблица
метаданных history), 201
Выборка всех записей, 154

Вычисление
 среднего значения, 155
 суммы, 155

Д

Двойные кавычки (фильтр where
в синтаксисе Spark SQL), 93
Динамический режим перезаписи, 159
Добавление столбца (ALTER
TABLE), 149, 168

Ж

Журналы и журналирование
(metadata_log_entries, таблица
метаданных), 203

З

Запись данных, 67
 запрос слияния, 74
 запросы вставки, 71
 каталоги, 66, 71
 слой метаданных, 67
 создание таблиц, 68
 списки манифестов, 76
 файлы манифестов, 74, 76
 Dremio, 171
 Flink, 191, 248
Запросы
 агрегирующие, 155, 170
 взаимодействие с компонентами
 Iceberg, 66
 вставки, 70
 информации о состоянии
 в прошлом, 82
 процесс записи, 68
 процесс чтения, 79
 слияния, 74
 слой данных как источник ответов, 52
 с фильтрацией записей, 154
 SELECT, 170
Запуск аналитических рабочих
нагрузок в озере данных, 310
Зоны шифрования, 263

И

Извлечение, преобразование и загрузка
(Extract, Transform, Load, ETL), 302

Изменение

столбца (ALTER TABLE), 150, 168
таблиц с помощью расширений
Iceberg Spark SQL, 151

Изоляция изменений с помощью
ветвления, 223

К**Каталоги, 121**

ветвление, 223, 227
маркировка, 229
миграция, 131
настройка в Spark, 140
обновление файла каталога, 70
операции DDL, 185
слой, 66
сравнение, 122
требования, 121
чтение данных, 83
чтение и запись, 66, 71, 79
AWS Glue, 126
Hadoop, 122, 185
Hive, 125, 186
JDBC, 129
Nessie, 127
REST, 128

Каталоги (Apache Iceberg), 63

архитектура, 63
Amazon S3, 64
AWS Glue Data Catalog, 64
Hive Metastore, 64

Коллекция метрик, 113**Копирование при записи, 109**

и слияние при чтении, 108

Копирование при записи (сору-on-write, COW), 54**Копирование при записи (COW), 76**

процесс записи, 76

**Курирование виртуальных витрины и
продуктов данных, 312****М****Маблицы метаданных**

доступ из Flink SQL, 190

Маркировка таблиц, 226**Метаданные таблицы, 82****Миграция**

в новую таблицу Apache Iceberg, 296

в существующую таблицу Iceberg, 298

в Apache Iceberg, 287

вопросы планирования, 288

из отдельных файлов, 295

из таблиц Hive, 291

из Apache Hudi, 293

из Delta Lake, 292

путем копирования данных, 296

между каталогами Iceberg, 131

на месте, 289

Многотабличные транзакции, 230**Многофакторная аутентификация, 261****Мониторинга и оповещения**

система, 262

Н**Настройка**

параметров (AWS Glue), 177

распределения операций записи
(ALTER TABLE), 152

частоты фиксаций, потоковые
данные, 241

О**Обработка транзакций в реальном
времени, 26****Обслуживание таблиц Iceberg, 173****Объединение при чтении (MOR), 76**

процесс записи, 76

Объектное хранилище, 118

в слое данных, 52

Озера данных, 54

плюсы и минусы, 34

Оконные функции, 156, 171**Оперативная аналитическая обработка
(OLAP), 26, 52****Операции на языке определения
данных (DDL), 166****Оптимизация**

производительности, 87

коллекция метрик, 113

перезапись манифестов, 114

секционирование, 103

фильтр Блума, 119

процесса чтения, 80

хранилища, 115

Откат изменений, 202, 231

П

Пакетное чтение, ветвление каталога, 228
 Пакетный режим (Flink), 245
 Перезапись
 манифестов, 114, 175
 файлов данных, 174
 Переименование столбца (ALTER TABLE), 150, 169
 Подключение аналитических инструментов к представлению, 313
 Подсчет записей (агрегирующие запросы), 155
 Поиск максимального значения, 156
 Политика удаления и хранения данных, 262
 Политики корзины
 Amazon S3, 265
 Google Cloud Storage, 272
 Получение информации о состоянии в прошлом
 файлы манифестов, 84
 файлы метаданных, 84
 Помещение исходных данных в озеро, 311
 Порядковые номера, файлы удаления, 56
 Порядок сортировки по умолчанию, 95
 Поточковая обработка данных, 237
 Amazon Web Services (AWS), 256
 Apache Flink, 243
 Apache Kafka Connect, 252
 Поэтапное выполнение, 92
 Проблема большого количества маленьких файлов, 88
 Производственные практики (изоляция изменений с помощью ветвления), 223

Р

Размер файла и размер группы записей (Parquet), 91
 Распределенная файловая система
 в слое данных, 52
 Hadoop, 262

С

Сбор измененных данных с помощью Apache Iceberg, 314

Секционирование и разделы, 103
 скрытое, 105
 Семантический уровень, 273
 Скрытое секционирование, 68, 105
 Слияние при чтении (merge-on-read, MOR), 54, 109
 и копирование при записи, 108
 Слой
 данных, 52, 67
 метаданных, 56, 66
 архитектура, 56
 чтение и запись, 66
 Снимки
 доступ из Flink SQL, 191
 создание, 70
 Создание
 агрегированных представлений для ускорения аналитических панелей, 313
 таблиц, 68
 с доступом по строкам и маскирование столбцов (Dremio), 167
 с секционированием, 146
 с секционированием и сортировкой (Dremio), 166
 Соответствия между каталогами Apache Iceberg и источниками данных Dremio, 165
 Сортировка как стратегия уплотнения, 97
 Списки
 манифестов, 58
 процесс записи, 74, 76
 процесс чтения, 80
 управления доступом, 262, 267, 270, 273
 Amazon S3, 267
 Статический режим перезаписи, 158
 Столбчатые структуры, 52
 Схемы
 создание таблиц, 69
 файлов манифестов, 58
 файлов метаданных, 60

Т

Таблицы метаданных, 201
 совместное использование, 220

- all_data_files, 212
- all_manifests, 215
- entries, 218
- files, 206
- history, 201
- manifests, 208
- metadata_log_entries, 203
- partitions, 211
- refs, 217
- snapshots, 204

Теневая миграция, 289

Технологические практики, 200

У

Удаление

- осиротевших файлов, 162
- столбца (ALTER TABLE), 151, 169

Указатель на текущие метаданные, 63

Уплотнение, 88

- автоматизация, 96
- обзор стратегий, 94
- сортировка как стратегия, 97
- стратегия binpack, 174
- Actions пакет для Spark, 89
- binpack стратегия, 94
- Sort стратегия, 94
- z-упорядочение как стратегия, 101
- zOrder стратегия, 94

Управление

- аутентификацией и идентификацией, 261, 266, 272
- доступом
 - на основе ролей, 269
 - на основе тегов (Tag-Based Access Control, TBAC), 283
 - политики корзин (Amazon S3), 265
 - принцип минимальных привилегий, 261
 - строгая аутентификация и управление идентификацией, 261
 - управление аутентификацией и идентификацией, 266, 272
- Dremio, 275
- Nessie, 281
- RBAC, 276
- Tabular, 282

- Trino, 278
- и безопасность, 260
 - на уровне каталога, 280
 - семантический уровень, 273
- Amazon S3, 264

Условные запросы, 154

Установка порядка сортировки (ALTER TABLE), 152

Установка/сброс полей идентификаторов (ALTER TABLE), 153

Ф

Файлы

- данных, 52
 - безопасность, 261
 - совместное использование таблиц метаданных, 220
 - all_data_files таблица метаданных, 212
- манифестов, 57
 - запись, 74
 - перезапись, 162
 - получение информации о состоянии в прошлом, 84
 - процесс записи, 76
 - процесс чтения, 80
 - схема, 58
 - all_manifests таблица метаданных, 215
- метаданных, 60
 - процесс записи, 69, 72
 - процесс чтения, 79
- позиционного удаления, 55, 111
- удаления, 54
 - удаления по равенству, 55, 111
- Фиксированные и переменные накладные расходы, 88
- Фильтрация записей (Dremio), 170
- Фильтр Блума, 119
- Форматы таблиц озер данных, 42
- Функции, определяемые пользователем (User Defined Function, UDF), 167

Х

Хранилища данных, плюсы и минусы, 31

Ч**Чтение данных**

- каталоги, 71, 79
- процесс выполнения запросов, 79
- слой метаданных, 66
- списки манифестов, 80
- файлы манифестов, 80
- файлы метаданных, 79
- AWS Glue, 180

Dremio, 169

Flink, 189

Ш

Шифрование, 261

Э

Эволюция разделов, 106

Томер Ширан, Джейсон Хьюз и Алекс Мерсед

Apache Iceberg. Полное руководство

Главный редактор *Мовчан Д. А.*
dmkpress@gmail.com

Перевод *Киселев А. Н.*

Корректор *Синяева Г. И.*

Верстка *Чаннова А. А.*

Дизайн обложки *Мовчан А. Г.*

Гарнитура PT Serif. Печать цифровая.

Усл. печ. л. 29,9. Тираж 100 экз.

Веб-сайт издательства: **www.dmkpress.com**

Традиционные архитектурные шаблоны хранения данных сильно ограничены. Чтобы использовать их, приходится применять довольно дорогостоящие процессы ETL для загрузки данных в каждый инструмент, отключающий доступ к функциям хранилища данных. Отсутствие гибкости в этих шаблонах вынуждает замыкаться на некотором наборе инструментов и форматов, что вызывает дрейф данных. Данная книга демонстрирует более удачное решение.

Apache Iceberg предлагает высокую производительность, масштабируемость и экономичность — главные преимущества, свойственные открытым озерам данных.

Прочитав книгу, вы узнаете:

- как организована архитектура таблиц Apache Iceberg;
- что происходит за кулисами, когда вы выполняете операции с таблицами Iceberg;
- как еще больше оптимизировать таблицы Iceberg, чтобы добиться максимальной производительности;
- как использовать Iceberg с популярными движками данных, такими как Apache Spark, Apache Flink и Dremio.

«Непревзойденное руководство по Apache Iceberg!»

Махди Карабибен,
инженер по данным,
Zendesk

Томер Ширан — основатель и директор по разработке Dremio, открытой платформы для хранения данных, позволяющей компаниям выполнять аналитику в облаке.

Джейсон Хьюз — директор по технической поддержке в Dremio. Ранее работал директором по продукту, техническим директором и старшим архитектором решений.

Алекс Мерсед — работал разработчиком и инструктором в таких компаниях, как GenEd Systems, Crossfield Digital и CampusGuard.

Книга адресована специалистам, занимающимся обработкой и анализом данных, а также администраторам, обслуживающим озера данных.

Узнайте, почему Apache Iceberg является одной из ведущих технологий для реализации открытых озерных хранилищ данных.

ISBN 978-5-93700-289-1



9 785937 002891 >

ЭТОТ КОНТЕНТ КУПЛЕН НА САЙТЕ **SKLADCHIK.ORG**

ЧТО ТАКОЕ КЛУБ «СКЛАДЧИК»?



Платформа

Это платформа, где каждый день тысячи людей собираются вместе, чтобы общими усилиями находить, приобретать и изучать курсы интересные именно им.



Сообщество

Это сообщество из 500 000 людей, открывших для себя выгодный способ быть в тренде самых актуальных и получать ценные знания по минимальной цене.



Библиотека

Это крупнейшая библиотека инфопродуктов в которой можно найти практически любой курс или тренинг продававшийся за последние 10 лет, а также уникальные авторские инфопродукты, которые не получится найти больше нигде.



**ТЫ ЕЩЕ
НЕ В КЛУБЕ?**
**ПРИСОЕДИНЯЙСЯ
И НАЧИНАЙ ЭКОНОМИТЬ!**

